Learning What to Change: Toward Failure-Grounded Co-Induction of Representations and Executable Reasoning Languages

Chimdi Walter Ndubuisi and Toni Kazic
University of Missouri

8 November 2026

Abstract

When a reasoning system's explanation fails, it must decide what to change: search longer, look at the problem differently, build an operation its language lacks, change how its program refers to objects, ask for more evidence, or decline to answer. A model can predict without an executable explanation; a symbolic system can search without knowing whether its representation or language is adequate; a verifier can reject a program without saying what should replace it. This paper studies the decision that remains, learning what to change, as three requirements: *capability availability* (can the language express the needed explanation), *capability discovery* (can the system find and execute it within its resources) and *capability selection* (can evidence decide which representation, operation or strategy to use). ARC-AGI-2 is the test bed, because each task demands a new skill from a few demonstrations and every explanation can be executed and checked exactly. CORA (Constructive Operator Reasoning Architecture), a typed object-program reasoner that verifies or abstains, is the instrument. *Official accuracy:* the CORA-only Kaggle entry (submission 57902131) scored 0.83%; the neural comparison entry (submission 57867631), credited to NVARC's method, scored 24.03%; both are public-leaderboard scores on 120 hidden ARC-AGI-2 tasks, and no later mechanism ran in either. *Tested capabilities and measured improvements:* the CORA-only entry's verified answers were right on 81.5% to 95.8% of the outputs it answered on three related, previously exposed benchmarks; once a bounded search failed, typed holes computed from the demonstrations recovered 64 of 95 controlled generated tasks, against 48 from the demonstrations alone, 38 for static search in the starting language, which used 3.2 times the CPU, and 16 with holes taken from another task's demonstrations (independently audited); in a held-back-half test on 464 related-benchmark tasks (624 query outputs, all scored once before), reused learned object operations got 227 query outputs right against 118 for static search in the original language with matched extra CPU, a gain entirely on 1D-ARC, while static search won on ConceptARC (independently audited); and a learned search order got 320 against 234 query outputs right at 500 counted operations per hole (search CPU 115 against 106 seconds), though a table of success rates (318) and a linear model (323) did about as well. *Transfer of the pipeline:* CORA's verify-or-abstain acceptance design and its step of asking for informative evidence, implemented separately for published Boolean gene-regulation models, reconstructed 101 against 66 of 595 published rules with inferred regulators (every exact recovery a one-input rule) and, in a separate protocol on the same reused cohort, 320 against 291 with informatively chosen observations, at first with fewer correct held-input predictions; the one revision there was made by researchers, and an order learned on ARC did not help (177 against 194 for a hand-set simplicity order). Through separately written adapters the loop was then tested on held items in new domains: on 217 held real benchmark functions recovered exactly from 25% random probes, it recovered 45 against 12 for the same loop with a shuffled failure description (reconstructions of known functions; 153 of the 217 were recovered by no method); on 200 held enzymatic reactions, given 3 examples per family, it gave the exact product for 76, against 33 for the best fixed published rule and 75 for a classical nearest-neighbour template method (reconstruction within families, not discovery); spreadsheets gave a null and a world-model probe on public ARC-AGI-3 games did not pass its pre-registered test. *The frontier:* selection is not yet learned. In a pre-registered test on 50 held controlled generated tasks, failure evidence beat a random order of the same changes (30 against 20 tasks; 10 gained, none lost; Holm-adjusted $p = 0.006$) but not the best fixed order; across that test's two revisions, where one change can check cheaply whether it applies, the best fixed order matched the oracle and choosing added nothing, and where a change cannot, no fixed order of the bank reached the oracle (best 44 against 55 of 84) and the pre-registered router did, though its advantage over that best order, chosen after seeing the held results, is not established; a fixed hand-written diagnosis controller

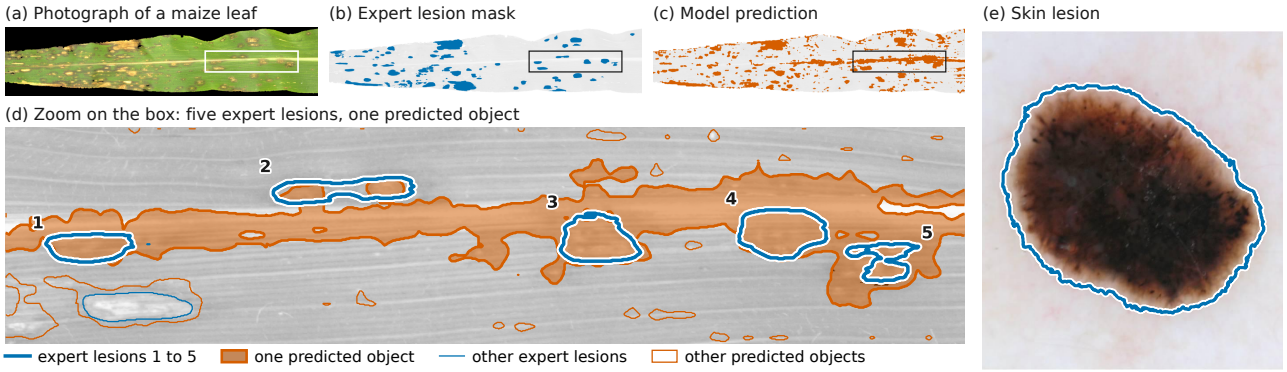


Figure 1: Two lesions pose the question. (a) A maize leaf, (b) its expert lesion mask, (c) a segmentation model's prediction; (d) one predicted object (orange) merges five expert lesions (blue) along the central vein. (e) A skin-cancer lesion with its expert boundary (ISIC Archive image ISIC_0000013 and its ISIC 2018 mask, both CC0; the ISIC record documents a histopathology-confirmed invasive melanoma, which the image itself does not establish). Motivation only: not CORA data and not a diagnosis. Panels (a) to (d) show a validation leaf of the first author's unpublished lesion study, a qualitative example chosen to illustrate the failure, not a representative sample; it is not one of the reserved images of that study. In the first author's unpublished work [121, 122], an atlas check flags images outside its reference set, and a pilot found what to change: seeding (where objects start), not geometry; cheap methods, using about 900 times less CPU, matched a geometric solver where it finished (3 of 21 windows).

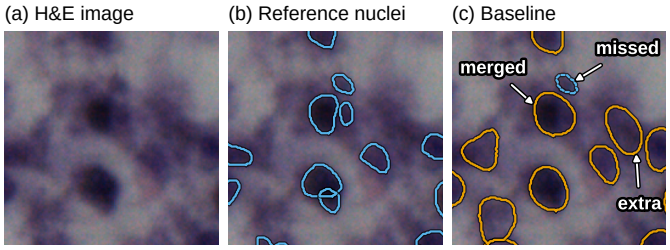


Figure 2: Melanoma nuclei, cropped from a NuInsSeg calibration image [108] (CC BY 4.0): (a) the H&E image, (b) reference nuclei, (c) the baseline segmenter's objects, with merged, missed and extra errors marked. CORA's loop ran on these images (Section 6.7). Its strict check admitted no change, so it made no held-out edit; a looser, tolerant check made 32 accepted edits on held NuInsSeg images, 10 beneficial, 16 harmful and 6 neutral, and no target passed its control gates. A secondary run that skipped operator admission made 4,000 edits on NuInsSeg: 1 helpful, 7 harmful, the rest neutral.

recovered 219 of 256 authored Boolean functions, and a fixed build-then-ask schedule recovered the same 219; conditional construction reached 433 of 595 published rules, which a classical learner equals by design; and no real ARC-AGI-2 case yet shows a correct answer caused by choosing the right revision after a failure. Post hoc, on the exposed public evaluation and not as an official score, the neural entry was right on 41 query outputs that CORA does not answer, and there ten times CORA's budget finds no program while diagnostics read from the demonstrations name a missing action in 35 of 36 tasks: the language, not the budget, falls short. We contribute a taxonomy of eight failure types, a decision framework with elementary conditional results, controls that separate a failure's content from the act of changing something, and a falsifiable research programme, failure-grounded co-induction of representations and executable reasoning languages, and we label every claim by its evidence.

1 The missing decision in artificial reasoning

Two lesions, one question. A lesion on one of the first author's maize leaves and a skin-cancer lesion (Figure 1) pose the question of this paper. Before any rule about a lesion can be right, the system must represent the lesion as an object, and one predicted object that merges five lesions defeats every rule over it. When the bioimage study of Section 6.7 ran CORA's frozen policies on 16 of the first author's maize leaves (a private evaluation target, with gold-corrected lesion masks and every decision frozen before any evaluation mask was read), the cell-trained baseline as shipped scored a pooled F1 of 0.0000, CORA abstained on all 16 leaves, so it did no harm and gained nothing, and plain search over the same operations made 120 edits, 103 of them helpful, while F1 stayed near zero (0.0113). *What it taught us:* local edits cannot rescue a representation that is wrong for

the domain; a system must recognise that its representation is inadequate, not only patch its outputs. *What it opens:* detecting a representation failure, such as a collapsed baseline, and changing the representation itself, for example to a lesion-trained model, before editing. The leaf images are the first author's unpublished data, so these counts can be reproduced only with them. On tumour nuclei the loop's steps carried over (Figure 2), but its strict check admitted no change; and a calibrated acceptance rule, tested afterwards on fresh melanoma images, failed its single test (Section 6.7).

Reasoning, in the sense this paper studies, means learning from a few observations: working out the rules, a grammar of operations, that explain them; noticing when the evidence shows those rules fall short; changing them; and carrying the same way of reasoning to a new domain. The question is our synthesis of the authors' research discussions (Section 1.1), not a quotation of them. An ARC task's demonstrations are a few observations of a hidden rule. In biology each observation is an experiment, a measured state change or a response to a perturbation, and each is costly, so a reasoner must get the most from little evidence and know which observation to ask for next; with the same number of observations, choosing them well recovered 320 against 291 of 595 published gene rules exactly, on a reused cohort (Section 6.2). Doing all of this together, transfer included, is close to what one would want from real reasoning. We do not claim to have achieved it; we make parts of it executable, measure them, and say which part is still missing.

The core idea fits in one sentence: when reasoning fails, the failure is evidence, and CORA uses it to find what is missing, builds that missing piece, checks it against the observations, and keeps it only if it helps. Every section returns to this sentence (Figure 4; Algorithm 1). The results state how far the evidence goes. On ARC-style tasks a failed search says when to build, and the demonstrations say what: the typed holes are computed from the demonstrations, and the failure triggers their construction. In a held test on logic functions, the comparison with a shuffled failure description, which is the test that the failure description itself carries the gain, favoured the real description (Section 6.11).

Figure 3 shows a puzzle from the public evaluation set of ARC-AGI-2 [41]. Each object either moves up until it is blocked or moves down to the border, depending on whether another object has the same size. In five configurations of the system (recorded runs on one task, drawn as an illustration) the outcome was the same: no program that reproduces both demonstrations and answers the test. The reasons were different each time. While the background was assumed to be colour 0, no view of the grid produced objects on which the rule fits both demonstrations. Once the background was taken to be each grid's majority colour, the rule could be written but lay far beyond the enumerator's reach. With every object bound to the same action, nothing fitted; without moves by destination, nothing fitted either. Each cause calls for a different intervention: change the representation, construct instead of enumerating, revise the binding of objects to actions, or build the missing operation. The observation alone does not say which, and acting on the wrong one wastes the budget or, worse, admits a program that fits for the wrong reason. A later constructor, frozen before its single pre-registered run, automatically reached a program that gives the same answers on this puzzle after its own first search failed (Section 5.8); because the puzzle shaped the design, that shows the mechanism working, not generalization.

Where large language models and neural systems fall short, stated fairly. Large language models and other neural systems are strong at proposing answers, at recognising patterns and at spending computation at test time; their record on ARC is summarised below. What the tested neural route, which predicts output grids directly, does not give is an executable explanation that can be checked, unlike neural systems that write programs [101, 138]; and neural systems do not report which part of their reasoning to change after a failure. Our own evidence makes this concrete. On the exposed public evaluation, the neural comparison entry, our re-implementation of NVARC's method, was right on 43 of 172 query outputs, while the CORA-only entry answered only 3 and all 3 were right; and CORA-CAST's programs reproduce the demonstrations and give the answer for only 2 of the neural entry's 344 attempts, both right (Section 5.4; post hoc, exposed public evaluation, not an official score). A neural system can be right without an explanation, and a symbolic system can be precise and narrow. A symbolic system can also search a fixed language without knowing whether its representation or language is adequate, so a long failed search does not say whether more search would help, and a verifier can reject a candidate without saying what should replace it. Each is a real strength with a missing step after it. The deeper problem is learning what to change. The comparison also says where to start. On the 41 query outputs that the neural entry gets right and CORA does not answer, ten times CORA's search budget finds no program, and automatic diagnostics, run on the demonstrations alone, name what CORA's language lacks: an action in 35 of the 36 tasks, in 14 of them also a view of the objects, and a guard in 1. The neural system's right answers thus mark, task by task, what a grammar learned from a few observations would have to acquire (Section 5.4;

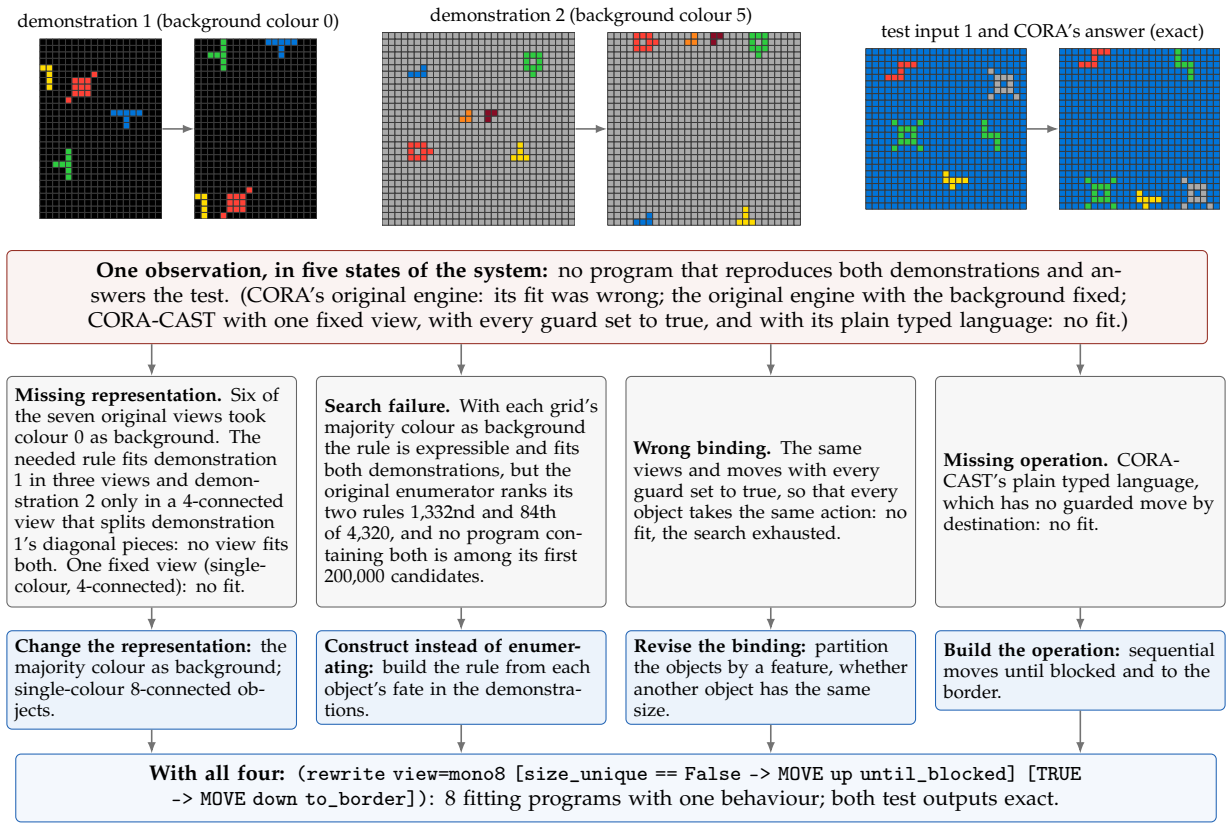


Figure 3: Why the same observed failure can need different interventions: a real case from CORA's records. In public evaluation puzzle 62593bfd of ARC-AGI-2, objects whose size another object shares move up until blocked and the others move down to the border. In every state of the system the outcome was the same, no correct program, and the failure record (no fit, or a fit that is wrong on the test) did not say why; the four causes were told apart only by different controls: a change of view, a ranking analysis, a guard ablation and an operator ablation. The puzzle was inspected while building and shaped the move constructor, so this is development evidence, not accuracy. Grids are drawn from the puzzle file and from CORA-CAST's recorded first attempt by `paper/v2_neural_cora/figures/make_same_failure_grids.py`; the controls' exhausted searches (0 fits each) are in `experiments/cora_cast_20261026/runs/chain_v2/m2/ARC_eval_dev/decisions.jsonl` and the plain typed language's in `experiments/cora_cast_20261026/M2_REWRITE_REPORT.md`; the original engine's views and enumeration ranks are the independent review's analysis (`records/CORA_SYNTHESIS_REVIEW_20261027.md`, sections 2.3 and 3). Puzzle data: ARC Prize Foundation.

post hoc, exposed public evaluation, not an official score).

This paper is about that choice. When an explanation fails, a learner can search longer or in a different order, revise how its program binds variables to objects, change its representation, construct an operation its language lacks, change its grammar or library, change the rule by which it accepts answers, seek more evidence, or keep its uncertainty and abstain. We call the choice among these the *missing decision*. It is missing in two senses. In most systems it is made outside the system, by a researcher who reads the failures and edits the language. And it is missing from most evaluations, which score how often a system answers correctly rather than how efficiently it identifies and acquires the capability an unfamiliar task needs, the skill-acquisition efficiency by which Chollet defines intelligence [26].

What current systems already do well. The claim of this paper is not that current models cannot reason. Language models write and repair programs, reason in natural language and adapt their weights at test time; on ARC, deep-learning-guided program synthesis and test-time training raised the ARC-AGI-1 private-set state of the art from 33% to 55.5% in 2024 [40], test-time training of a transductive model won the 2025 competition [87, 155], an evolving library of Python programs written with a language model and evolved natural-language instructions reached 26.0% and 29.4% on ARC-AGI-2's semi-private set [9, 132], and compositional neuro-symbolic filtering of a language model's proposals raised its ARC-AGI-2 public evaluation score from 16.0%

to 24.4% [50]. Program synthesis fills holes in sketches and prunes by deduction [59, 60, 154]; library learning compresses solved programs into reusable abstractions [16, 57, 66]; formal verification refines an abstraction when a counterexample proves spurious [43]. These systems keep improving at producing candidate answers, searching for executable programs and checking proposed solutions, and several also change their language: libraries grow from solved programs, tools are built from a few checked examples [22], and abstraction refinement and representational repair change a representation after a failure. What these results do not test is whether a system can choose among kinds of revision (search more, change the view, build an operation, ask for evidence) from typed failure evidence, under matched controls. That is the gap this paper studies: the difference between learning to produce a solution and learning which part of the reasoning process must change to make new solutions possible. We do not claim that these systems have reached a ceiling: refinement loops, which improve a program or a model's weights from a feedback signal, were the defining theme of 2025 [42], and the question here is narrower, whether the choice of revision can itself be made from evidence, and measured.

1.1 The central question, and ARC as its test bed

The question grew out of the authors' research discussions on grammar induction in biology, mathematics and other domains. What follows is our synthesis of those discussions, not a quotation and not an endorsement: *how can an intelligent system identify when its current representation, executable language or explanation is inadequate, determine what needs to change, construct and verify the necessary capability, and transfer that learning to unfamiliar problems?* The discussions separated four layers: inducing a representation of data whose syntax and semantics are unfamiliar, in a form that supports reasoning; the reasoning mechanism itself; learning from failure when each semantic type has only a few instances; and transfer, in the sense of generalising over the reasoning and not only over the data. They treated grammars, deterministic, stochastic or probabilistic, as explicit generative rules that may also be concealed inside neural models, with rule induction as the hard problem. They contrasted mathematics, where each field's grammar and definitions are explicit, with biology, where the rules are not understood at a deep level and stochasticity and evolutionary history shape what is observed. And they proposed a validation ladder: reproduce what is known, generate plausible ideas that can be verified, then verify them. Learning what to change is this work's research question; the experiments test parts of it, and no learned revision policy has yet beaten its controls.

An inspiration from plant phenotyping. The first author's unpublished work on maize leaf lesions, which does not use CORA, shaped the same question from another side; apart from the one leaf in Figure 1, only aggregate statements of it are used here, and none is a CORA result, a discovery or evidence of transfer (the CORA trial on 16 of the first author's leaves, above, is a separate run). Two hypothetical leaves with the same lesion area, one with a hundred small lesions and one with five large ones, differ in how their lesions are organised, so a representation that records area alone cannot tell them apart. A phenotype atlas of lesion organisation records when a new image lies outside the range its reference images cover, as 18,066 of 19,741 image records from a larger collection did (whether those judgements are right was not evaluated) [121]. A separate pilot on splitting merged lesions localised what had to change: an expensive geometric solver gave the same scores as cheap shortest-path methods on every window where it finished (3 of 21), at about 900 times the CPU, while 239 of 309 automatically placed seed points, where splitting starts, fell outside every expert-outlined lesion, so seeding, not geometry, was the part to change; and on visual review many visible spots had no expert outline, so some apparent errors may be failures of the specification rather than of the model [122].

Why ARC-AGI-2 is a good test bed. The question scores a process, constructing, checking and revising explanations and choosing what to revise, not ARC accuracy, and ARC-AGI-2 exposes that process well.

- *It measures acquiring a skill, not recalling one.* Each task gives a few demonstrations of a transformation the solver has not seen, so success measures how efficiently a new skill is acquired from little data [26], and the tasks are designed to resist memorisation and brute force [41].
- *Answers are exact, and an explanation can be a program.* An output is right only if every cell matches, so a candidate explanation can be executed on the demonstrations and checked without a human judge.
- *Failures are observable and attributable.* Because every hypothesis is executed, a failure leaves a record: which demonstration failed, which cells were wrong, whether the search ran out of budget or exhausted its space.

- *Scoring is honest.* Two attempts are allowed per query output, and the scoring tasks are hidden from participants, so a reported score cannot be tuned on the scoring tasks [41, 42, 87].

The biology studies then test whether the same process carries to data with different meaning (Section 6). CORA tests one concrete version of the grammar theme: failure-triggered extension of a typed grammar, checked by execution. On grids and lists that is the construction test (once a bounded search fails, typed holes computed from the demonstrations, then symbolic construction) and its list-function replication (Section 5.5.2); in biology it is the literature-grounded bioimage grammar (Section 6.7), where in calibration the strict rule admitted no operator and the tolerant rule admitted one (Section 5.2), and on the held-out targets the strict rule made no edit (Section 6.7). This taught us that in the imaging and Perturb-seq studies a uniform admission bar was either too strict or not selective; whether noise is the cause is an untested hypothesis. It opens acceptance tied to the target domain: a calibrated rule set on one dataset has since failed on a new source (Section 6.7), and a bar keyed to prior knowledge is untested.

Terms used throughout, in plain words. A *task* (a puzzle) gives a few *demonstrations*, pairs of an input grid and its output grid, and one or more *queries*: test inputs whose outputs, the *query outputs*, must be produced. In biology a query is something else, and we always say which: a requested simulated observation (a training label), or a *held input*, an input state of a simulated model withheld from training whose next state is predicted afterwards. A *program* is an executable explanation: a short sequence of typed operations that turns an input into an output. A *grammar* (or language) is the set of operations and the rules for combining them, so it fixes which programs can be written. A *typed hole* is a missing part of a program sketch, together with the input and output pairs that part must produce, worked out from the demonstrations. A *representation* is how raw data are described before reasoning starts, for example a grid as objects with colours, sizes and positions. In a gene network, a *regulator* of a gene is another gene whose state can change its next state, and the *support* of a rule is the set of inputs it actually depends on. An *adapter* is the translator that turns one domain's observations into the entities and operations a shared reasoning core understands. A *cofactor* split builds a rule over three or more inputs from two simpler rules, one for each value of a chosen input (Shannon decomposition). The identity $f = (\neg x \wedge f_{x=0}) \vee (x \wedge f_{x=1})$ holds for every Boolean function. It goes back to Boole [15] and Shannon [148] and underlies binary decision diagrams [20]. It is established mathematics, not a CORA invention; CORA's question is when failure evidence justifies using it. *Verification* executes a program on every demonstration and, in CORA, rebuilds it with each demonstration left out; *abstention* is declining to answer when no program passes. *Exact recovery* means reconstructing a published rule exactly, for every input state. *Held out* means kept away from a study's development; *exposed* means the data had been seen, used in development or scored before, so held-out exposed data are never unseen. *Pass at two* means an output counts as correct if either of two attempts matches exactly. *Counted operations* are candidate applications counted by the search, a machine-independent budget, and *CPU-hours* measure processor time. The *starting language* is a study's language before any operation is added, and *static search* searches it with no new operation. An *arm* is one configuration compared in a test, the method or one of its controls, run on the same tasks under the same limits.

The investigated sequence. CORA investigates one sequence of steps: observe, represent, construct, execute, diagnose, revise, verify and reuse (Figure 4). Algorithm 1 states it in plain words: answer only with a program that fits every observation and, rebuilt without each, still ranks that observation's output among its top two predictions, and otherwise turn the failure into a specification of the missing piece. Table 1 marks which steps are implemented, which have shown a measured improvement, and which remain goals.

Three requirements organise the evidence. *Capability availability*: can the language express the explanation the task needs? *Capability discovery*: can the system find and execute it within its resources? *Capability selection*: can evidence decide which representation, operation or strategy to use? The results show progress on the first two, inside supplied spaces. Once a bounded search fails, typed holes computed from the demonstrations find missing operations reliably inside a supplied space (64 of 95 in all three seeded repeats, against 48 from the demonstrations alone and 3 for blind enumeration of the same structures in all three); post hoc, of the 31 tasks that construction solved and matched static search did not, 13 have no equivalent program in the starting language at any depth searched (not proof that none exists deeper), 17 have one that static search would have needed an estimated 1 to 135 times its budget to reach, and 1 was a static-search process failure (Section 5.5.3). A learned order converts a binding budget into answers (320 against 234); reused operations get held tasks right

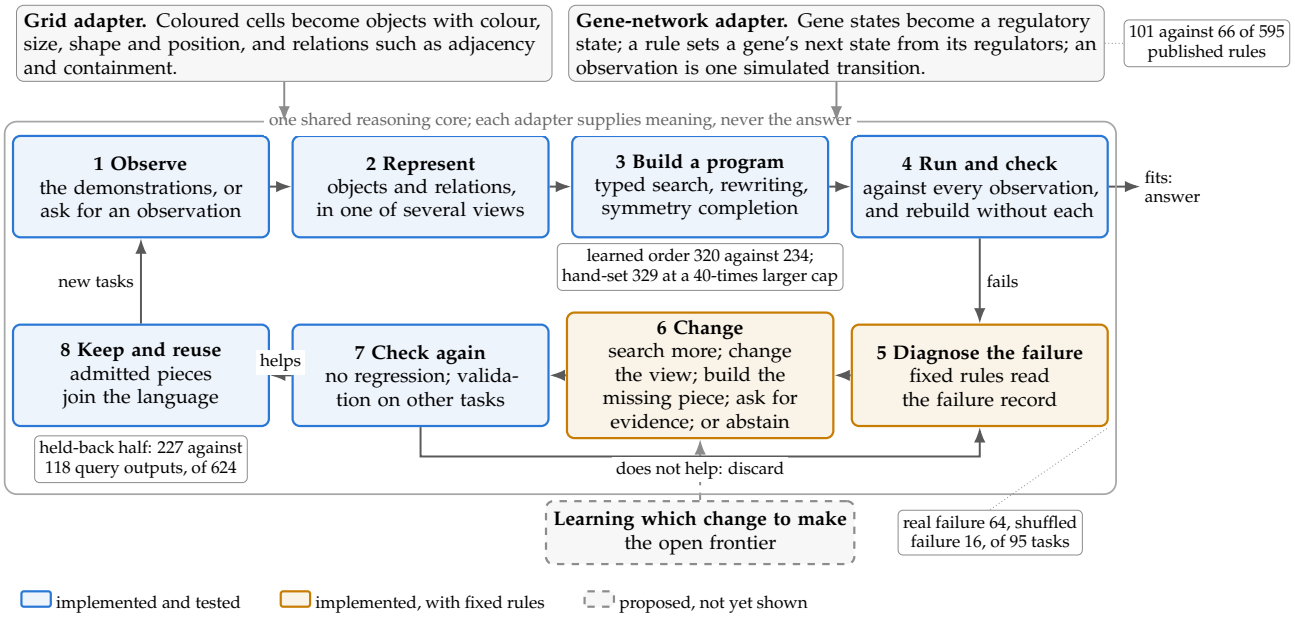


Figure 4: CORA turns a failed explanation into a missing piece. When no program fits the observations, the failure is evidence: CORA works out what the missing piece must produce, builds it, checks it on the task and on other tasks, and keeps it only if it helps. Each badge gives the control that tests that step. The scored Kaggle entry runs steps 1 to 4; the other steps ran in separate research studies. The same core serves grids and gene networks; only the adapter changes. Colours give each step's evidence status.

inside their family (227 against 118 of 624, all of the gain on 1D-ARC); and a larger gene-rule language raised coverage (433 against 321). The third is the open frontier: where a fixed alternative had the same capabilities and evidence, it matched the diagnosis-driven controller (219 against 219 authored Boolean functions, for a fixed build-then-ask schedule); a router that reads CORA's failure record gave the same answers as a fixed order of the same two changes, and in the test's second revision, where a change cannot cheaply check whether it applies, no fixed order of the bank reached the oracle (best 44 against 55 of 84) while the pre-registered router did, an advantage not established against that best order (Section 5.9); the 433 against 433 tie between conditional construction and a classical learner holds by design; and no learned selector has beaten its controls. The negative results fit the same frame. A learned library changed no prediction because it made nothing new available; grown representations failed because none made the answer available; and operations learned on one-dimensional families did not carry to two-dimensional concepts, where static search won (9 against 27).

Thesis. Reasoning progress depends on telling failures of knowledge, representation, program construction and verification apart, and on choosing the intervention that the available evidence justifies. Put as a research claim, the next challenge in reasoning is not only learning to solve problems but learning what must change in a system's representation, language or evidence when its existing reasoning fails. We investigate this claim through executable mechanisms, conditional theory, measured results and falsifiable experiments, and we keep the claim apart from what has been shown: a proposed research paradigm is not an empirically demonstrated breakthrough, and nothing here demonstrates unrestricted universality or solves the general problem of acquiring a reasoning language.

CORA. CORA (Constructive Operator Reasoning Architecture) is the instrument of the investigation. It is execution-grounded program synthesis: typed object programs, a search over them, exact execution, verify-or-abstain (no program prediction unless the program passes its checks; the entry then emits its fallback, a copy of the query), and failure-triggered construction of missing operations. Because every hypothesis is a program, every failure leaves a typed record: which demonstration failed, which cells were fixed, broken or left wrong, whether the search was stopped by its budget or exhausted its fragment, and which hole would have to be filled for a sketch to fit. That record is the evidence on which a decision about what to change can be based, and the

Algorithm 1 CORA in plain words: answer only with a checked program, and turn a failure into a missing piece.

Require: a few observations D (the demonstrations), the queries X^*, a language $\mathcal{L}$, a budget
describe each observation as objects and relations ▷ the domain adapter
$P \leftarrow$ the programs in $\mathcal{L}$ that reproduce every observation in D, within the budget
if the top $p \in P$, rebuilt without each observation (x_j, y_j) in turn, still ranks y_j among its top two predictions for x_j **then**
    return $p(X^*)$ ▷ answer: the CORA-only entry's rule; the neural entry's side requires the top prediction
end if
record the failure: which observation failed, which cells, a budget stop or an exhausted space
diagnose with fixed rules: more search, another view, a missing piece, more evidence, or abstain
if a missing piece is diagnosed **then**
    work out from D what the piece must map (a typed hole); build candidates in a declared space
    keep a candidate only if it is new, fits the task, breaks no solved task and passes other tasks
    add the kept piece to $\mathcal{L}$ and go back to line 2
end if
return abstain ▷ no checked program: decline to answer

paper asks how far it goes.

Contributions, and how each differs from the closest prior work. Each item names what earlier work did, what is new here, the evidence and its limit. Section 2.2, Table 4 and Appendix E give the full comparison; no item claims priority beyond the work we read.

1. *Controls that separate the content of what is built from the act of building.* Growing a representation or a language where analysis fails is an old idea. Berwick's acquirer built one new grammar rule where parsing stopped [14]; Utgoff treated a shift of bias as part of inductive concept learning [160]; inverse resolution invented new predicates [116], and Stahl showed for which language biases such invention is a useful shift of bias [157]; learning from failures turns failed hypotheses into constraints that prune a fixed hypothesis space, and a later version invents predicates while it learns [46, 47]. In the work we read, none reports a pre-registered control that keeps the growth machinery fixed and replaces only what the task or the failure supplies. We report such controls. On 95 controlled generated tasks, typed holes computed from the task's own demonstrations once search failed recovered 64, against 16 with holes taken from another task's demonstrations and 32 for blind enumeration of the same 211 structures in a majority of repeats (Section 5.5.3; independently verified). On 217 held real benchmark logic functions, the loop recovered 45 against 12 for the same loop with a shuffled failure description, which is the test that the failure description itself carries the gain (Section 6.11; held items; every recovery is a reconstruction of a known function, not a discovery). Limits: on ARC-style tasks the holes come from the demonstrations and the failure only triggers construction, so that control shows that the task's own data matter, not that failure analysis does; and 153 of the 217 logic functions were recovered by no method.

2. *Construction that makes operations reachable, inside a declared space.* Library learning compresses solved programs into abstractions, which leaves what a language can express unchanged and raises what a bounded search reaches [16, 57, 66]; witness functions and λ^2 deduce what a sub-program must compute inside a fixed language [60, 137]; and DeepCoder widens its active set of existing functions when search fails [11]. Here a hole computed from the demonstrations selects a new, persistent operation from a declared space, admitted only after regression and cross-task checks whose compute is charged to it. A post hoc reachability analysis says what that bought: of the 31 tasks that construction solved and matched static search did not, 13 have no equivalent program in the starting language at any depth searched, and 17 have one 1 to 4 cost levels deeper than static search reached, which it would have needed an estimated 1 to 135 times its budget (median about 5 times) to find (Section 5.5.3). Limits: every answer lay in a space declared in advance, so this is selection within a known space, not open invention; no equivalent at any depth searched is not proof that none exists deeper; and since construction and compression both act on what a bounded search reaches, no advantage over library learning is claimed until the two are compared on held tasks at matched total compute.

step	what is implemented	measured improvement, with its label	still a goal
observe	reads demonstrations; in biology, requests simulated labels	choosing observations: 320 against 291 of 595 exact rules (published-model reconstruction, reused cohort)	evidence chosen by a learned policy
represent	object views (three in the entry; a supplied bank of 22)	the bank solved 9 of 9 against 4 of 9 (nine cases written by the engine's author, five built to need an added view)	choosing the view from evidence: a fixed hand-written view score solved 3 of 9
construct	typed search, guarded rewriting, symmetry completion, construction from typed holes	64 against 38 of 95 (controlled generated tasks); 343 against 118 of 624 (held-back-half test, exposed tasks)	construction beyond a supplied structure space
execute	a deterministic interpreter runs every candidate	every reported answer is an executed program	none
diagnose	typed failure records; explicit routing rules	a fixed controller recovered 219 of 256 against 38 (authored functions); a fixed build-then-ask schedule also 219	a learned diagnosis of which change to make
revise	new operations, cofactor splits and view switches inside supplied spaces	a researcher-made query-policy revision gained 310 with no loss at 20 labels (authored functions), below a fixed expressive policy	learned selection of the revision
verify	fit to every demonstration, rebuilding without each, regression and validation gates	81.5% to 95.8% precision on three related benchmarks; shuffled demonstrations: at most 2.4%	certifying correctness, which no check can do
reuse	promoted operations kept in a registry	227 against 118 of 624, all of the gain on 1D-ARC (held-back-half test)	reuse across families: 23 against 28 of 60, and 9 against 27 on ConceptARC

Table 1: The investigated sequence, step by step. Every number is a row of the claim registry, with its own denominator and label; none ran in a scored entry except verification.

3. *Three requirements measured apart, and the measured condition under which choosing pays.* Rational metareasoning chooses computations by their expected value [143], explanation-based learning discards control rules whose cost exceeds their benefit [112], and a recent ARC system learns a meta-controller over proposals within a fixed language [173]. Here availability, discovery and selection are measured apart, and the decisive comparison varies only the order of the same changes at equal cost. In a pre-registered test on 50 held controlled generated tasks, failure evidence beat a random order of the same changes (30 against 20 tasks; 10 gained, none lost; Holm-adjusted $p = 0.006$) but not the best fixed order (Section 5.9). Its audit states the lesson: "Choosing what to change can pay only when a change cannot cheaply check whether it applies, or cannot finish within the budget when it does; a proposed second revision would add an object-view search that runs to its cap when it does not apply." That second revision has run (Section 5.9), and its audit states the condition across both: "Where one change can check cheaply whether it applies, the best fixed order matched the oracle and choosing added nothing (revision 1); where a change cannot, no fixed order of the bank reached the oracle (best 44 against 55 of 84), and the pre-registered router did (revision 2)." Against the best fixed order chosen after seeing the held results, the router's advantage is not established (11 tasks against 0; interval 0.0 to $+26.1$ points). Post hoc analyses of development data say why learned guidance tied a plain success-rate table: the failure features were informative, but the table already tried a working edit first in almost every hole that had one (Section 8.2, item 4).

4. *One loop design carried to new domains through adapters, with held tests and negatives.* Learning Boolean update rules from state transitions [75, 102], choosing perturbations to discriminate consistent networks [74], applying generalized enzymatic reaction rules [125] and fitting signalling logic models to data [144] are established. What is new is one loop design, build an executable rule, check it, keep it only if it holds up, otherwise abstain, carried to each domain through a declared adapter and tested on held items against static, shuffled and classical comparators, with every negative reported. Published gene-network models gave 101 against 66 and, in a separate protocol, 320 against 291 of 595 exact rules (published-model reconstruction); held real benchmark logic functions, 45 against 12 of 217 for a shuffled failure description (every recovery a reconstruction of a known function); held enzymatic reactions, 76 of 200 against 33 for the best fixed published rule, while a classical nearest-neighbour method recovered 75, about as many (difference +0.005; 95% interval -0.110 to 0.115), as reconstruction of known transformations on held substrates of the same families, not discovery and not transfer across families; spreadsheet text tasks, a null on both primaries (22 of

53 against 17 for each control); synthetic signalling tasks, a result limited to reconstructing one removed prior edge on one network and design; and public ARC-AGI-3 games, a probe whose pre-registered test did not pass (probe results on public games, not a competition score) (Section 6.11). Limits: each domain's code was written separately, so the procedure's design transferred, not its code; the measured-data studies admitted no rule or gave exploratory signals only; and nothing learned on ARC helped in biology (177 against 194).

5. *A taxonomy of reasoning failures* (Section 3): search failure, missing representation, incorrect variable or object binding, restricted executable expressivity, invalid composition, insufficient evidence, overconservative verification and incorrect semantic specification, each defined over an executable system, mapped to measured CORA cases, and marked by whether a failure record identifies it or further assumptions are needed.
6. *A theory of learning what to change* (Section 4): a decision model in which the learner keeps competing explanations of its failure and chooses among costed interventions using only what it has observed; elementary conditional results that bound what search, ranking, verification and language growth can do and that account for the measurements (Table 8); and predictions, each with what would falsify it. The results are elementary and conditional, and no priority is claimed for them; the learned decision policy is a conjecture.
7. *Official results, attributed exactly* (Section 5.1). The CORA-only entry (submission 57902131), with no pretrained model, scored 0.83%, against 0.00% for every earlier CORA entry. The neural comparison entry (submission 57867631) scored 24.03%; its first attempt is our re-implementation of NVARC's published method, credited to NVARC, and CORA added no answer on the 172 inspected public query outputs (43 of 172 right with or without it). No later research mechanism is attributed to either frozen entry.
8. *A falsifiable research program* (Sections 7 and 8): failure-grounded co-induction of representations and executable reasoning languages, what would distinguish it experimentally from earlier neural-symbolic and metareasoning approaches, and the experiments that could refute it.

What this means for the field, with the gaps. The organizers of the ARC Prize have said in public posts what they think is missing, and our evidence bears on most of it. After the 2024 competition they concluded that new ideas are needed [86], and they present the benchmarks as a compass for research, not a finish line [33, 34, 39]. Chollet describes reasoning as checking, step by step, that a candidate is exactly correct, after a learned intuition has proposed a few candidates [27]. He defines the goal as human-level efficiency in acquiring skills, which current systems do not yet reach, and ARC-AGI-3 scores the efficiency of actions in environments seen for the first time [29, 31, 32]; he expects future systems to move toward symbolic learning, where the meaningful difference is efficiency [36]. Knoop describes neural networks and symbolic programs as complementary: programs generalize from a few examples and can be verified, but they are hard to find as they grow, so the two must be merged, and the lab the two founded works on program synthesis guided by deep learning [28, 88, 89]. In 2026 they reported that the strongest systems on the interactive ARC-AGI-3 benchmark write executable models of each environment on the fly, under a symbolic outer loop that drives learned models and tests explicit hypotheses [35, 37, 38, 83, 90], and that open-ended invention is unsolved [91]. They also ask for care with claims: tasks that brute-force search solves do not point toward general intelligence [80], a perfect score on public demonstration games is not a benchmark score [38], and they have praised open, thorough program-synthesis and small-model work [7, 30, 81]. Measured against these, our evidence and its gaps are as follows. *Symbolic synthesis on the fly from a few examples:* CORA builds operations from a task's own demonstrations, inside a declared space (64 of 95 generated tasks), but on the public evaluation set it answered 3 of 172 query outputs, all 3 right, so reach on real tasks is the gap. *Explicit hypothesis testing:* every answer is an executed program checked against every demonstration and rebuilt without each one, and 81.5% to 95.8% of the answered outputs were right on three related benchmarks; the check establishes consistency, not correctness, and it cannot tell the intended task from a rival the demonstrations do not separate (Proposition 5). *Data efficiency:* in the generated-task test each admitted operation was built from one task's own demonstrations and then reused within its family, though the number of demonstrations per built operation was not tallied; across families it did not help (23 against 28 of 60). *Learned guidance for synthesis:* a learned order changed which answers a fixed budget found (320 against 234 correct query outputs at 500 counted operations per hole, with search CPU 115 against 106 seconds; at 20,000 the counts were close, 327 against 329, inconclusive), but a fixed table of success rates (318) and a linear model (323) did about as well, so no advantage specific to that guide is shown; a later specification-reading guide, frozen before any held task existed, solved 1,083 of 1,128 held tasks generated from ARC-AGI-2 public training input grids at 125 counted operations each, against 953 for the

hand-set order, 881 for a frequency table and 1,018 for a linear model, search efficiency at a binding budget, not new reach (Section 5.6.1); and a router that read the failure record gave the same answers as a fixed order where every change could check cheaply whether it applies; where a change cannot, the pre-registered router reached the oracle and no fixed order of the bank did (best 44 against 55 of 84), an advantage not established against that best order (Section 5.9). *Open-ended invention:* construction builds operations only inside a constructor language we supplied, so it is not open-ended invention; letting failures extend that constructor language is an open direction. The common gap is selection: choosing, from the evidence of a failure, which change to make.

How claims are graded. Every result carries one of six evidence labels: *official competition score* (the hidden Kaggle test), *independently held evaluation* (items held out from a study's development; on the ARC-style benchmarks always previously exposed, so never unseen), *synthetic demonstration* (controlled generated tasks or authored functions we built, with a planted cause), *source development* (exposed training or source-half tasks), *published-model reconstruction* (simulations of published models) and *prediction on real measurements* (reused measured cohorts); no CORA-generated biological hypothesis has been tested by a designed intervention, and there is no laboratory confirmation. *Independently audited* means re-checked after scoring, from the saved rows and records, by a separate process in this project that did not design the study (an automated reviewer working under the first author, not an outside referee); *independently verified* means that our own recomputation reproduced the result; *companion-reviewed* means checked only by the companion study's own separate automated review of saved aggregates. A number labelled *post hoc* comes from an analysis chosen after the outcome was seen, and *exploratory* marks a comparison that was not pre-registered. We also distinguish six kinds of statement and label each where it is made: *implemented and tested capabilities*, *measured improvements* over named controls, *conditional theoretical results* (true under stated assumptions), *supported interpretations* of the evidence, *conjectures*, and *open directions*. Table 2 lists the principal claims in these terms. Results on public training puzzles are development evidence, never accuracy; results on the 120 public evaluation puzzles are descriptive, since we inspected them while building; the only clean ARC accuracy is the hidden Kaggle test.

Table 2: The principal claims and their status. Every number is a row of the shared claim-to-evidence registry and keeps that row's status, denominator and comparison. Every held task of the ARC-style benchmarks had been run and scored once on 2 November 2026 by the submitted entries, so none is unseen.

status	claim	where
tested capability	verify-or-abstain typed program construction runs end to end on hidden tasks; the linked CORA-only entry scored 0.83% (submission 57902131); its answers were right on 81.5% to 95.8% of the outputs it answered on three outside benchmarks, and at most 0.024 with another task's demonstrations	Sections 5.1 and 5.2
tested capability	typed holes computed from the demonstrations once search fails select a missing operation inside a declared space: 8 of 11 tasks, against 0 with another task's holes (at family level 4 of 4 families, $p = 0.125$); against matched search, 8 against 5, inconclusive	Section 5.5.2
measured improvement	learned search order: 320 against 234 correct query outputs on 464 held tasks at 500 counted operations per hole (search CPU 115.25 against 106.35 seconds; every held task scored once before by the submitted entries); a learned reordering that a fixed table of success rates also captures	Section 5.6.1
measured improvement	specification-reading guide: 1,083 of 1,128 held tasks generated from ARC-AGI-2 public training input grids solved at 125 counted operations each, against 953 for the hand-set order, 881 for a frequency table and 1,018 for a linear model; a guide reading only the constructor's own block features 1,076, not significantly fewer; search efficiency at a binding budget, not new reach	Section 5.6.1
measured improvement	typed holes at scale: 64 of 95 controlled generated tasks, against 38 for static search in the starting language, which used 3.2 times the CPU charged to growth, 48 from the demonstrations alone and 16 with another task's holes; independently verified	Section 5.5.3
supported interpretation	of the 31 tasks that only construction solved, 13 have no equivalent starting-language program at any depth searched (not proof that none exists deeper), 17 one that static search would have needed an estimated 1 to 135 times its budget to reach, and 1 was a static-search process failure (post hoc, descriptive)	Section 5.5.3
measured improvement	reuse of learned object operations on held halves: 227 against 118 of 624 query outputs, all of the gain on 1D-ARC (ConceptARC 9 against 27); task-local construction 343 against 118; independently audited; every task had been scored before	Section 5.8
tested capability	a fixed diagnosis controller recovered 219 of 256 authored Boolean functions against 38 for the two-input language, which is all it can express; a fixed build-then-ask schedule recovered the same 219; not ARC	Section 5.5.3

status	claim	where
measured improvement	inferred regulator support: 101 against 66 of 595 published rules exactly, prospectively, inside a supplied rule language; every exact recovery a one-input rule (the cohort had been parsed for an inventory, and 33 models were readmitted)	Section 6.2
measured improvement	informative observations: 320 against 291 of 595 exact rules, with held-input accuracy lower (1,586 against 1,747 of 2,380, unanswered inputs counted wrong) until a completion vote fixed before it ran (2,200 against 2,153); reused cohort, post hoc	Section 6.2
tested capability	conditional (cofactor) construction on published models: 433 against 321 of 595 exact rules; a classical learner equals it by design	Section 6.2
measured improvement	held real benchmark logic functions: 45 against 12 of 217 for the same loop with a shuffled failure description (held items; reconstructions of known functions, not discoveries; 153 recovered by no method)	Section 6.11
measured improvement	held enzymatic reactions: 76 of 200 against 33 for the best fixed published rule; a classical nearest-neighbour method 75 (reconstruction within families, not discovery)	Section 6.11
tested capability, null	spreadsheet text tasks: 22 of 53 against 17 for each control, neither pre-registered comparison significant; synthetic signalling tasks: a result limited to one removed prior edge; ARC-AGI-3 games: a probe that did not pass its pre-registered test (probe results on public games, not a competition score)	Section 6.11
tested capability, outcome none	in a pre-registered test on 50 held controlled generated tasks, failure evidence beat a random order of the same changes (30 against 20 tasks; 10 gained, none lost; Holm-adjusted $p = 0.006$) but not the best fixed order	Section 5.9
tested capability, pre-registered outcome met against the static order	revision 2, on 84 controlled generated tasks (not ARC puzzles): a fixed router reading one fact from CORA's failure record, whether every demonstration keeps its grid size, solved 55 against 29 for the static full bank in its pre-registered order (26 tasks against 0); against the best fixed order chosen after the results (44 of 84) the advantage is not established (11 against 0)	Section 5.9
tested capability	on the exposed public evaluation (post hoc), CORA-CAST programs explain 2 of the neural comparison entry's 344 attempts, both right, and none of the 301 wrong ones; where CORA has its own answer (3 query outputs) its check accepts both right attempts and rejects all 4 wrong ones	Section 5.4
supported interpretation	on the 41 query outputs the neural entry gets right and CORA does not answer, the language falls short, not the budget: no program at ten times the budget; a missing action first in 35 of 36 tasks, a missing guard in 1; diagnostic rules refined on these tasks (post hoc)	Section 5.4
conditional theory	solved $\leq$ generated $\leq$ expressible; finite consistency does not determine the intended output; verification establishes consistency, not intended correctness; selective acceptance can raise precision while lowering coverage; under agreement of all fitting programs, counting a program undefined at the query as disagreeing, language growth can only remove answers on tasks already fitted; demonstrations carry a bounded amount of information; under noise, a likelihood bar reduces to a count of net corrections	Section 4.3
supported interpretation	coverage comes before ranking; on natural tasks the bottleneck is constructibility, not localisation; at a binding budget the search order changes which answers a fixed budget finds	Section 5.10
conjecture	a learned intervention policy that chooses among search, representation change, construction, evidence acquisition and abstention can beat fixed rules at matched cost	Section 4.6
open direction	capability selection: a learned choice of revision that beats strong static access to the same capabilities; representation revision on natural tasks; learned cross-domain transfer of a revision policy, which has not been shown	Section 8

Table 3: What the headline numbers mean. Each row names the unit and denominator, the comparison, a plain reading and the evidence label; the sections cited in Table 2 give the limits. Every held task of the ARC-style benchmarks had been run and scored once on 2 November 2026 by the submitted entries.

number	what is counted	against what	why it matters, in one sentence	label
0.83%	the public-leaderboard score on 120 hidden ARC-AGI-2 tasks, either of two attempts	0.00% for every earlier CORA entry	an official score; local tests on exposed tasks suggest that coverage, not precision, is CORA's main engineering limit, and they do not explain this hidden score	official competition score
24.03%	the same, for the neural comparison entry	CORA added 0 of 172 local query outputs	the neural method is NVARC's; CORA's second attempt replaced nothing in either rehearsal, and its effect on the hidden score is unknown	official competition score
81.5% to 95.8%	answered outputs that were right, on ConceptARC, Mini-ARC and 1D-ARC	at most 2.4% with another task's demonstrations	when CORA answers it is usually right, and its answers depend on the task's own evidence	related benchmarks, exposed

number	what is counted	against what	why it matters, in one sentence	label
64 against 38 of 95	generated tasks recovered in at least two of three seeded repeats	static search in the starting language, which used 3.2 times the CPU charged to growth; 48 from the demonstrations alone; 16 with another task's holes	typed holes computed from the demonstrations once search fails find missing operations reliably inside a supplied space	controlled generated tasks, independently verified
13 and 17 of 31	tasks only construction solved: no equivalent static program at any depth searched; one 1 to 4 cost levels deeper	static search's own record and its estimated budget (1 to 135 times)	construction made some operations available and shortened deep compositions for the rest	post hoc, descriptive
227 against 118 of 624	correct held query outputs on three related benchmarks	matched static search with the same extra CPU	operations learned on one half are reused on the other, but only within 1D-ARC	held evaluation, exposed tasks (scored once before by the submitted entries)
320 against 234 of 624	correct held query outputs at 500 counted operations per hole	the hand-set search order (search CPU 115 against 106 seconds)	a learned order spends a small budget better; the hand-set order reached 329 only at a 40-times larger per-hole cap, a sampled cap, not end-to-end compute and not a proven minimum	held evaluation, exposed tasks
2 of 172	public ARC-AGI-2 evaluation query outputs right by task-local construction	0 for every other arm	construction reaches a real puzzle, from one task; both are the CORA-only entry's own answers there; inconclusive	exposed public tasks
2 of 344	neural comparison entry attempts on the public evaluation that a CORA-CAST program reproduces from the demonstrations	none of its 301 wrong attempts	CORA can check a neural answer only where its language reaches; there it separated right from wrong	exposed public tasks, post hoc
41 in 36 tasks	public evaluation query outputs the neural entry got right and CORA did not answer	no CORA-CAST program at ten times the budget	the language, not the budget, falls short: a missing action first in 35 of the 36 tasks	exposed public tasks, post hoc
101 against 66 of 595	published gene rules reconstructed exactly	no inferred regulators, and shuffled ones	knowing which gene matters helps rebuild one-input rules; every exact recovery was a one-input rule	published-model reconstruction; pre-registered replication, the first scoring of this cohort, whose models had been parsed before; 33 readmitted models
320 against 291 of 595	exact rules after 20 chosen observations each	20 uniformly chosen observations	informative observations identify rules better, at first at a cost in prediction	published-model reconstruction, reused cohort
2,200 against 2,153 of 2,380	held inputs of the simulated models predicted correctly	uniform selection with the same completion vote	a completion rule, specified after the overall result but frozen before the per-target diagnosis was decoded, recovered the prediction loss; with a fill of zero, active selection stays below uniform (2,002 against 2,028)	published-model reconstruction, reused cohort, post hoc
433 against 321 of 595	exact rules at 40 observations, conditional construction	the two-input language on the same observations; a classical learner also 433	a richer language recovers rules with three or more inputs, and a classical method equals it by design	published-model reconstruction, reused cohort; independently audited
219 against 38 of 256	authored Boolean functions recovered exactly	the two-input language, which can express 38; a fixed build-then-ask schedule also 219	the gain comes from the supplied larger language and one bought label per built function, not from diagnosis	synthetic demonstration (authored functions), not ARC
177 against 194 of 595	exact rules with an operator order learned on ARC	a hand-set simplicity order	what was learned on ARC did not help in biology	published-model reconstruction, reused cohort
45 against 12 of 217	held real benchmark logic functions recovered exactly from 25% random probes	the same loop with a shuffled failure description	the comparison with the shuffled description tests whether the failure description itself carries the gain; 153 of the 217 were recovered by no method	held items; reconstruction of known functions, not discovery
76 against 33 of 200	held substrates of 40 enzyme families given the exact product, from 3 examples per family	the best of 1,224 published generalized rules, chosen on the same examples; a classical nearest-neighbour method 75	cutting a rule from the observed change helps over the published rules, as the classical method also does	held items; reconstruction of known transformations within families
22 against 17 of 53	held spreadsheet text tasks recovered	static search at four times the budget, and pieces from another task's failure (17 each)	neither pre-registered comparison was significant; the gain is mainly faster search	held items; a null

Organisation. The paper follows one order. This section states the broader question and why ARC-AGI-2 tests it. Sections 2 to 4 make the question precise: prior work in program synthesis, grammar induction, bias shift and predicate invention, library learning, neural guidance and verification; the failure types and what can identify them; and the decision model with its conditional results. Section 5 shows what CORA and its constructors achieve on ARC-style tasks, positives first and limits beside them, including where a neural system and CORA each fall short (Section 5.4) and the pre-registered test of failure-guided choice (Section 5.9). Section 6 asks what carried to other domains, and what did not, including the held tests in new domains (Section 6.11). Sections 7 and 8 state the research direction this points to, the decisive experiments, the falsifiable predictions and the limitations. The appendices hold the submitted system, the experiment records in full, the cross-domain records, the studies completed after the reviewed package, the related work in detail, the ARC Prize Paper Track mapping, the reproducibility map, the exact commands and the claim-to-code table.

2 From program synthesis to language revision

In terms of the core idea: earlier work builds, ranks and checks candidates very well, mostly inside a fixed language; this section asks where each line of work stops before building the missing piece from the failure itself.

2.1 What a fixed language buys, and what it costs

A domain-specific language gives a reasoner three things that a free-form answer does not: every hypothesis has exact semantics, every hypothesis can be checked against the evidence, and hypotheses compose. It also fixes, in advance, which explanations exist. Two properties of a language must therefore be kept apart. *Expressibility* is whether some program of the language produces the intended output; *discoverability* is whether the search finds that program within its budget. A program can be expressible and still undiscoverable, because enumeration cost grows exponentially with program length, and a learned policy changes the order, and so the cost, of reaching a program, not the set of programs that a complete search could reach at unlimited budget [129]; and a larger language can make the intended program shorter while adding rivals that fit the same demonstrations and mislead the selection [131]. A fixed language is thus both the strength and the limit of program synthesis: it makes reasoning checkable, and it decides in advance what can be learned. The question of this paper, which part of the system to change when reasoning fails, only arises once the language is allowed to change.

Program synthesis, and the ideas behind ARC. Chollet defines intelligence as skill-acquisition efficiency over a scope of tasks, with respect to priors, experience and generalization difficulty; ARC's priors are designed to be as close as possible to Core Knowledge (objectness, goal-directedness, numbers and counting, basic geometry and topology), and ARC can be seen as a program synthesis benchmark [26]. ARC-AGI-2 preserves the input-output pair task format of its predecessor [41], and the ARC Prize 2024 report credits the rise of the ARC-AGI-1 private-set state of the art from 33% to 55.5% to techniques including deep learning-guided program synthesis and test-time training [40]. CORA is execution-grounded program synthesis: typed object programs, a search over them, exact execution, verify-or-abstain, and failure-triggered construction of missing operations. It is designed as the discrete program-search half of a neuro-symbolic system, in which neural components would propose and the symbolic engine verify and decide; that is a design, and neither entry does it yet (the CORA-only entry has no neural component; in the neural comparison entry attempt 1 is always the neural answer and CORA only proposes its own grid for attempt 2, `cast_selective_v2/api.py` lines 320 to 344). Its parts map to these ideas concretely. Object-centric typed programs encode the objectness and geometry priors (objects under three connectivity rules, symmetry completion, geometric operators). Verify-or-abstain addresses overfitting to a few demonstrations (Section 4). Failure-triggered construction acquires, from a failure, a skill the language lacks (Section 5.5.2). Contract pruning makes the discrete search efficient (Proposition 11). The neural parts are measured findings about where neural guidance helps a verifier and where it does not: the neural comparison entry, whose neural core is NVARC's published method, with CORA as a verified second attempt, scored 24.03%, with a CORA increment of 0 measured on the 172 inspected public-evaluation outputs (its effect on the hidden Kaggle test is unknown), and neither our language-model inventor nor the neural proposers of a companion study of this project (called the companion study below) beat static or failure-blind controls (Section 5.6). These results taught us that a verifier can add only outputs missing from its proposer's pool, and

that unaided proposers did not exceed what static search reached. They open construction aimed at the missing outputs.

2.2 Learning inside a fixed language, and changing it

Feedback, program synthesis, abstraction learning and neural-symbolic combination are not new, and a living survey covers many more approaches to ARC than we discuss here [161]. We group the nearest work into six themes and end each with what CORA adds; Table 4 then states, for each claim of this report, what prior work had already established, CORA's difference, the control that isolates it, the result and its limit.

Program search for ARC. ARC can be read as a program synthesis problem (Section 2.1), and much recent work searches for programs with a neural model's help, combining a neural model with a program search [101, 138]; object-centric neuro-symbolic program synthesis for ARC, with vector symbolic algebras representing objects and guiding the search, is also published [76], and so is compositional neuro-symbolic reasoning that extracts object-level structure, lets a language model's priors propose transformations from a fixed domain-specific language and keeps only hypotheses consistent across the demonstrations, raising a language model's ARC-AGI-2 public evaluation score from 16.0% to 24.4% [50]. Filtering proposals by consistency with the demonstrations is therefore not CORA's; its scored entry differs in using no pretrained model at all: typed object programs, executed exactly, with an answer given only when the program passes its checks.

Neural test-time training. Test-time training of a transductive model, one that predicts the output grid directly instead of writing a program [2, 62], won the 2025 competition [87, 155], and ARC Prize named refinement loops that year's theme [42]. Small recursive networks [77, 163] and description-length minimization [103] need no pretraining. CORA is not a neural method but is designed to check such a model's answers; in our neural entry, which re-implements NVARC's method, it added no answer on the inspected public outputs (Section 5.3).

Neuro-symbolic and library learning. Inventing functions [22, 118, 164], learning libraries [16, 57, 66], learning to search [150], learning a program language end to end with a neural interpreter [107], repairing a representation after reasoning failures [21], choosing which hole to fill by abstract interpretation in top-down synthesis [98] and topological features are all established. CORA adds a specific sequence: a failed search, then typed holes computed from the demonstrations, then bounded selection of an operation for them from a pre-registered space, then admission only after regression and cross-task checks whose compute is charged to it (the failure-to-admission sequence, row A of Table 4), while our own compression of solved programs shortened programs without extending reach (Appendix B.5), one test of our compression and not of DreamCoder's.

Verification and selective prediction. Accepting only programs that fit every demonstration is standard in ARC program search; selective prediction abstains when a model is unsure [56, 63]; Joffe and Eliasmith hold out each demonstration in turn to choose which object properties a rule reads [76]; and jackknife+ turns leave-one-out refits into prediction intervals with a coverage guarantee when the data are exchangeable [12]. CORA uses rebuilding without each demonstration to decide whether to answer at all, inside a strict policy that also requires agreement among fitting programs and across rounds and the same answer on a restart (stability-gated abstention, row C of Table 4). It is measured against accepting every fitting program and against matched search (Section 5.2); the precision gain belongs to the whole strict policy, leave-one-out alone was not significant, and unlike jackknife+ it gives no coverage or precision guarantee.

Learning from failure in grammar induction, bias shift and predicate invention. Adding a rule where analysis fails is an old idea in language learning. Berwick's acquirer built one new grammar rule at the point where parsing stopped [14], a chart-based learner abduced the edges that a failed parse lacked and generalised them into grammar rules [48], transformation-based learning adds the rewrite rule that removes the most errors of a baseline's output [17, 18], and a meta-interpretive learner switched from regular to context-free grammars when the regular hypothesis failed [117]. In concept learning, Utgoff treated the language a learner uses to describe concepts as its bias and shifted that bias when it proved too strong: when no description in the current language was consistent with the examples, his program added a new term to the language [160]. In inductive logic programming, inverting resolution let a learner invent predicates that were not in its vocabulary [116], and Stahl showed for which language biases inventing predicates is a useful way to shift bias [157]. Soar compiles the reasoning that resolved an impasse into new rules [96]. In program synthesis, failed hypotheses and conflicting partial programs become constraints that prune a fixed hypothesis space [46, 59]; a predicate-invention version of learning from failures adds predicates while it learns one task [47]; and conflict-driven synthesis already pairs a statistical model that chooses what to try with deduction that rules out what cannot work [59]. CORA's

growth step follows this pattern. It differs in computing typed holes from the demonstrations once search fails, in selecting an operation for them from a declared space instead of pruning, in keeping that operation across tasks only after checks on other tasks, and in charging the compute of its admission checks; we compare it with matched search, with another task's holes, with the demonstrations alone and with blind enumeration. Grammar induction also separates naming repeated structure, which compresses without generalising [123], from merging items into classes or variables, which generalises [136, 158]. The operations CORA promotes are still stored without parameters, and our null result for our own library compression (Appendix B.5) is one test of that compression, not a general result about library learning. Partial structure supplied as supervision helped grammar induction more than better fitting did [134]; the regulator sets of the Boolean study (Section 6.2) are different, a prior inferred from the same demonstrations rather than added supervision. One induction model ran unchanged on English, German and Chinese [85], coupled priors improved induction across eight languages [13], and parsers reduced to part-of-speech tags transferred directly between languages [109]; between unrelated domains, what transferred was second-order structure [51]. In these studies, learned content transferred over a shared abstract alphabet. Our transfer test had one in place: a later adapter mapped ARC grids and gene states onto the same binary truth tables and found 22 exact relations in ARC training tasks, and the order counted from them still did not help (177 against 194 exact rules for a hand-set simplicity order and 182 for the same counts randomly reassigned; Section 6.6). Learning Boolean update rules from state transitions is established in logic programming [75], and decomposing each ARC example into input and output pixel facts let a general-purpose inductive logic programming learner solve 22% of the 400 public ARC-AGI-1 training tasks with one hour per task on one CPU [70].

Changing the representation, and deciding what to compute. Two further lines of work bear on the missing decision. Counterexample-guided abstraction refinement checks a coarse model and, when a counterexample proves spurious, refines the abstraction where it failed [43]: a failure changes the representation, not only the search. Representational repair revises a logical theory's signature after reasoning failures [21], and a meta-interpretive learner can switch grammar class when the simpler class fails [117]. Rational metareasoning treats computation itself as an action chosen by its expected value [143], and a recent ARC system learns a meta-controller over proposal refinement within a fixed language [173]. Utility estimates for learned control knowledge, and the discarding of rules whose cost exceeds their benefit, go back to explanation-based learning [112]. What this paper adds to these lines is not a new mechanism of any one kind but a joint, measured treatment: the failure types that call for different changes (Section 3), a decision model over representation, binding, construction, search, acceptance and evidence (Section 4), and controls that separate the content of a failure record from the act of changing something (Section 5).

2.3 Where CORA extends or tests these ideas

Table 4: Each claim of this report against what prior work had already established, with the control that isolates it, the result and its limit (prior-art review: `records/NOVELTY_PRIOR_ART_REVIEW.md`; our theory and novelty audit: `records/audit_20261108/THEORY_AND_NOVELTY_AUDIT.md`). Negative comparisons (library learning, learned search control, learned executors, pruning from failed hypotheses, neural program writers) are reported where they were run (Section 5.3; Appendix B.4; Appendix B.5); Appendix E compares ten lines of work in more detail.

claim	what prior work established	CORA's difference	isolating control	result	limit
A. an operation selected for typed holes computed once search fails, then promoted	deduced hole examples [60, 137], on ARC [3]; widening the active set of functions when search fails [11]; hole filling ordered by abstract interpretation [98]; requested, verified library components [165]; libraries from solved programs [57, 66]	an unsolved task's typed holes select a new, persistent operation from a pre-registered structure space, admitted only after regression and cross-task checks	another task's holes; demonstrations only; blind enumeration; matched search	8 of 11 tasks against 0 ($p = 0.0078$; at family level 4 of 4 families, $p = 0.125$), 4 ($p = 0.125$) and 5 ($p = 0.375$); enumeration covers as many at five times the compute; at scale 64 against 16 and 48 of 95	selection within a declared space that contains the answers; descriptively lower CPU on 11 constructed tasks, not coverage; no entry runs it

claim	what prior work established	CORA's difference	isolating control	result	limit
B. a failed bounded search triggers growth	growth on failure [11, 22, 120]; bias shift when no description is consistent [160]; chunking at impasses [96]; sound unrealizability proofs [72]; a learned controller deciding compute and stopping [173]	a budget heuristic: a four times deeper search must fail first	decoys that need only deeper search	kept searching on all 4 decoys; invented on 19 of 20 family tasks	no control isolates the trigger itself
C. stability-gated abstention (strict policy)	exact-fit acceptance in ARC program search [101, 167, 170]; the consistent selective strategy, which rejects every point the version space does not classify unanimously [56]; selective deep networks [63]; holding out each demonstration to choose which object properties a rule reads, on ARC [76]; leave-one-out intervals with a coverage guarantee [12]	the whole module is re-derived without each demonstration and must predict it, as an abstention check (top 1 in CORA-CAST version 2, with agreement across modules and restart stability)	the first complete fitting candidate, unvalidated, on 176 tasks from public training puzzles; accepting every fitting program on 1,244 query outputs	13 of 13 accepted right (Wilson lower bound 0.77); the 2 rejected were the 2 wrong; strict policy +0.105 precision at lower coverage; leave-one-out alone not significant	stability, not truth; leave-one-out not isolated; no coverage or precision guarantee; fails when the rule is inexpressible
D. guarded object rewriting	object-graph filters and transformations [170]; decision lists [142]	construction from each object's fate under typed guards, exactly verified	every guard set to true; one fixed view	development 10 against 3 of 176 ($p = 0.016$); sealed 8 against 6 of 207 ($p = 0.5$)	the guard effect is not confirmed
E. general symmetry completion	symmetrization in the second-place solution of the 2020 ARC Kaggle competition [52]	any axis or centre; complete predictions preferred, but CORA-CAST version 1 can return a partial one on a multi-query puzzle	none	fits 981571dc, already the neural answer	not isolated
F. contract-pruned typed search	pruning partial programs by component specifications or abstract semantics [58, 60, 68, 168, 172]	an ARC grid abstract domain (shape, palette, counts) with forward and backward contracts	exact execution at equal depth	0 lost fits; 44,485 instead of 1,269,443 executions; budget stops 32 to 0	a speed gain only; sound on a finite scope; no real-task answer changed
G. symbolic answers beside a neural model	induction with transduction [101]; the 2024 ensembles [40]	an order and a check: a symbolic attempt 2 only when accepted	the neural entry alone	43 of 172 either way; 0 gained, 0 lost	a policy, not a method
H. solved $\leq$ generated $\leq$ expressible	consistent programs kept apart from their ranking [137]	used as a measurement order: reachability before ranking	none	no program in memory fit in 113 audited puzzles	an accounting identity
I. the content of a failure, tested apart from the act of building	learning from failures prunes with constraints from failed hypotheses [46, 47]; bias shift and predicate invention add terms when a language fails [116, 157, 160]	pre-registered controls that keep the growth machinery and replace only what the task or the failure supplies	holes from another task; a shuffled failure description; blind enumeration	64 against 16 and 32 of 95 generated tasks; 45 against 12 of 217 held logic functions (reconstructions of known functions)	on ARC-style tasks the holes come from the demonstrations; 153 of the 217 recovered by no method
ARCANA [173] (comparison)	object-centric perception, program proposals over 47 typed primitives, failure feedback that refines a latent prior, a learned meta-controller; 32.5% reported on ARC-AGI-2's semi-private set	CORA's failure step adds an operation to the language rather than reshaping a proposal distribution, and CORA abstains; neither runs in a submitted entry	none (no released code)	not compared	author claims; a fixed language, no abstention
CrossBeam [150] (comparison)	learned bottom-up program composition, guided by the search history and partial executions	CORA's construction test extends the language after a failure and admits the new operation conservatively; CORA's learned search order gained at a binding budget (320 against 234 query outputs, search CPU 115 against 106 seconds) but matched frequency and linear baselines (Section 5.6.1)	none	not compared	guiding search within a language, not extending it
DreamCoder, LILO, Stitch [16, 57, 66] (comparison)	compressing solved programs into reusable abstractions, with neural guidance, language-model naming or fast corpus-guided compression; this leaves expressibility unchanged and raises budgeted reach	CORA grows operations from unsolved tasks' typed holes and admits them only after regression and cross-task checks; both act on what a bounded search reaches	compression against construction (Section 5.8)	CORA's own compression shortened programs from a median of 28 nodes to 27 and added no reach	one test of CORA's own compression; their systems were not run, so no advantage is claimed

claim	what prior work established	CORA's difference	isolating control	result	limit
Popper [46] (comparison, executed)	learning logic programs by turning failed hypotheses into constraints	an executed baseline on CORA's object facts, with a constraint-logic completion backend beside CORA's enumerator	same object encodings, unequal budgets	uninformative: unequal budgets on small development samples, so no count is reported as a result; the constraint-logic backend agreed with enumeration on 1,082 sketches	small development samples; no superiority either way
REVEAL, Best-Fit Extension [94, 102] (comparison, executed)	inferring a gene's inputs from state transitions, and the best-fitting rule on a chosen input set	CORA's biology procedure infers support inside verify-or-abstain and adds cofactor construction after a certified failure	a classical support-table learner on identical observations	433 against 433 published rules and 219 against 219 authored functions, both equal by construction	a consistency check, not a comparison of methods; the classical method is as good on these endpoints

What CORA adds. The general neuro-symbolic architecture is not new, and every ingredient has a nearest predecessor. Relative to DreamCoder and LILO, which grow libraries from solved programs [57, 66], CrossBeam, which learns to guide bottom-up search [150], ARCANA, which refines proposals within a fixed language [173], Popper, which prunes a fixed hypothesis space with constraints from failed programs [46], counterexample-guided synthesis, which refines candidates within a fixed space of holes [154], witness functions, which decompose a specification within a fixed language [137], self-repair by language models, which edits programs and keeps nothing for later tasks [25, 128], and selective prediction, which abstains on low confidence [56], the claims are narrow, each paired with the experiment that isolates it. (i) *Controls on the content of a failure* (row I of Table 4): pre-registered controls that keep the growth machinery and replace only what the task or the failure supplies, run in two domains (another task's holes on ARC-derived generated tasks, 64 against 16 of 95; a shuffled failure description on held real logic functions, 45 against 12 of 217, reconstructions of known functions). (ii) *A failure-to-admission sequence* (row A): a failed search, then typed holes computed from the demonstrations, then bounded selection of a persistent operation from a pre-registered structure space, then admission only after regression and cross-task checks whose compute is charged to it, measured against shuffled, demonstrations-only, blind-enumeration and matched controls; post hoc, 13 of the 31 tasks it alone solved have no equivalent program in the starting language at any depth searched (not proof that none exists deeper). Holes, analysis-guided hole filling [98], object-centric neuro-symbolic synthesis for ARC [76] and neural proposals from a fixed language filtered by cross-example consistency [50] are prior art, so only this sequence and its controlled measurement are claimed; its benefit is selection within a space that contains the answers, and no entry runs it. (iii) *Stability-gated abstention* (row C): synthesis reruns without each demonstration, and the strict policy answers only if every round predicts its held-out output, the fitting programs agree and a restart gives the same answer; in a controlled follow-up on public ARC-style tasks the whole strict policy, not leave-one-out alone, raised observed precision among its emitted answers, at lower coverage and with no guarantee (Section 5.2). (iv) *Contract-pruned typed search* (row F): sound contracts over an ARC grid abstraction complete a depth-2 search at a fixed budget with 28 times fewer executions. The report's contribution is the controlled, executable measurement of these mechanisms, with their negative results reported as findings.

Scope. Every ingredient has a nearest predecessor (Section 2): inventing functions, learning libraries, filling program holes, shifting a language's bias, inventing predicates, object-centric neuro-symbolic synthesis, using topology and combining neural and symbolic methods are established, so this report adds specific mechanisms and controlled measurements of them (rows A, C, F and I of Table 4), not a new architecture. The capability tests recover removed operations; invention of unforeseen capabilities is open. No revision policy is trained, so learned transfer between domains is open, and CORA discovered no biology. Attempt 1 of the neural comparison entry is NVARC's published method [155], and the 27.64% of NVARC's own notebook, which we reran with compatibility and deployment patches only as a reference, is NVARC's.

3 A taxonomy of reasoning failures

In terms of the core idea: a failure is useful evidence only if it is read correctly, so this section lists the eight kinds of failure a record can point to, and what each one needs.

To decide what to change, a learner must first say what can go wrong. We describe a reasoning system by five parts, each of which can be changed: a *representation* that turns raw data into typed objects, features and relations (for grids, CORA's views); an *executable language* of typed programs over that representation; a *binding* scheme by which a program's selectors, guards and variables refer to objects and values; a *search* that, within a budget, generates the candidate programs whose predictions it retains; and an *acceptance rule* that decides whether to answer. A task gives demonstrations and queries; the intended query outputs are read only by the evaluator. A failure is a task on which the system gives no answer or a wrong one.

Three causes of a wrong answer. The language cannot express the answer (*representation*); a correct program exists but is not generated within the budget (*construction*); or it is generated but not chosen (*selection*). The three nest (Proposition 2), so no ranking recovers an answer that the language cannot express. Our earlier work located CORA's failures at the outermost level (Section 5.3). The question of this report is therefore how a system can learn from failure *which* level to change, its own language included, and whether doing so helps on unfamiliar problems.

The three causes are too coarse to choose an intervention: a representation can fail because its objects are wrong or because its programs cannot refer to the right object; construction can fail for want of budget or of an operation; and the demonstrations themselves can leave the answer open or point to a different task. We distinguish eight failure types (Table 5).

1. *Search failure.* A correct program is expressible at reachable cost, but the search does not generate it within its budget. Remedy: more budget, or a better order.
2. *Missing representation.* The objects, features or relations that the rule refers to are not produced by the representation, so no program over it can state the rule. Remedy: a different view or a new relation.
3. *Incorrect variable or object binding.* The objects and operations exist, but a program binds the wrong objects or values to an action, or the language cannot bind a value read from one object to an action on another. Remedy: revised guards or bindings.
4. *Restricted executable expressivity.* No program of the language, under any available representation, produces the intended output. Remedy: a new operation or a larger grammar.
5. *Invalid composition.* The parts exist, but their composition is ill-typed, undefined on some demonstration, or relies on a decomposition the task does not have. Remedy: a different decomposition.
6. *Insufficient evidence.* The demonstrations do not determine the intended output within the language: fitting programs disagree at the query, or the information needed exceeds what the demonstrations carry. Remedy: more or better-chosen evidence, a prior, or abstention.
7. *Overconservative verification.* The acceptance rule rejects correct answers that the evidence supports. Remedy: a less uniform admission bar.
8. *Incorrect semantic specification.* The evidence or the system's objective specifies a different task from the intended one, so a program fits and is wrong for a reason that no check on the evidence can see. Remedy: outside knowledge, or evidence that separates the intended task from its rivals. An example outside ARC may come from the first author's unpublished leaf-lesion work, which does not use CORA (Section 1.1): on visual review many visible spots had no expert outline, so some detections scored as errors may be failures of the specification rather than of the model; this is a visual-review observation, not a measured rate.

3.1 The same observation, different causes

The types are not observations. A failure record reports what happened, and several types can produce the same record. Figure 3 illustrates this with recorded runs of different configurations of the system on one task. The outcome, no correct program, arose from a missing representation, a search failure, a wrong binding and a missing operation in turn, and the record said no fit under all four. It separated one of them, since the search failure stopped at its budget while the view and guard controls exhausted their fragments, but it did not separate the others, which needed a change of view, a guard ablation and an operator ablation. The second case shows the same ambiguity between insufficient evidence and restricted expressivity.

failure type	what shows it in CORA (measured)	what can identify it
search failure	a deeper search at four times the budget fitted 34 source tasks against 7 at the base budget; at 500 counted operations per hole a learned order got 320 held outputs right where the hand-set order got 234; on 62593bfd the needed rule ranked 1,332nd and 84th of 4,320 (Figure 3)	the record shows a budget stop; only matched extra search confirms it
missing representation	six of seven views assumed background colour 0, while 96 of the 120 public evaluation puzzles have a non-zero majority colour in some grid; a Perturb-seq record typed 1,842 of 2,508 failed attempts as no relation the language can state, and a follow-up that added pair relations admitted none either, because even on training data no candidate neared the admission bar	a change of view that turns no fit into a fit; one record suggests, but does not show, it
incorrect binding	on 97d7923e selectors compare a feature only with a literal, so the legend cannot be read (98 and 162 fits, none right); the most frequent missing capability in 46 reviewed failures is a cross-object reference (13); executable bindings solved 120 of 120 generated legend puzzles and no real one	a guard or binding ablation; typed mismatch records
restricted expressivity	a frozen grammar excludes the answer for 754 of 842 training tasks by grammar-specific conditions; none of 425 natural construction units has a hole mapped by the 211 whole-grid structures; 190 of 656 Boolean targets need more than three inputs; reviewers put the first break at expressivity in 29 of 46 failures	a proof for an explicit grammar, or an exhaustive check of a declared space; a capped search is not proof (113 audited puzzles)
invalid composition	on list functions the deletion left the identity as the only decomposition, so the hole specification equalled the demonstrations alone (5 and 5 of 9); the neural inventor's compress, downscale and trim proposals agreed with the references on 0 to 10% of probe grids	type errors and undefined executions are in the record; a missing decomposition shows as holes that no structure maps
insufficient evidence	ten binary demonstrations carry at most ten bits, below the 10.2 needed to name three regulators among twenty variables; with exactly two demonstrations every leave-one-out round is a one-demonstration synthesis; all 256 Perturb-seq demonstrations were similar, and only 32 of 16,320 held responses were directional	counting fitting programs that disagree at the query, where the version space can be enumerated; otherwise a prior
overconservative verification	the strict policy gave up 31 correct answers to exclude 21 wrong ones; in a LIVECell calibration the requirement that all near-top fits agree blocked every correction, and strict CORA made 0 edits on three held-out image datasets; a validation pool of factor-2 tasks quarantined factor-3 instances that would have solved the other 4 tasks of their family	only labelled held outcomes; the cost of abstaining is invisible in the record
incorrect semantic specification	with another task's demonstrations precision fell to 0.03 or less while answers persisted; demonstration windows that held only debris let an erase-everything program survive leave-one-out (10 of 11 accepted deletions removed real nuclei); ranked by raw wrong cells, 26 of 98 near-miss attempts equal the query	not from the evidence alone (Proposition 5); shuffled-evidence controls and outside knowledge

Table 5: Eight failure types, the CORA measurements that show each, and what can identify it. The measurements are those of Sections 5 and 6, with their own denominators and status; public-evaluation counts are descriptive, training-task counts are diagnoses and never accuracy.

Task-local ambiguity, and two sources of rival fits. Write $A(D, x^*) = \{[\![p]\!](x^*) : p \in V_{\mathcal{L}}(D)\}$ for the predictions of the version space at a query. The task is ambiguous at x^* when $|A(D, x^*)| > 1$: two attempts cover at most two members, and no rule that reads only D can tell which member is right. Rival fits arise from true ambiguity: when the only blue object is also the leftmost, "recolour blue objects" and "recolour the leftmost object" both fit, and a query whose leftmost object is red separates them; only more evidence, a prior or a second attempt resolves this. They also arise from *inexpressibility*: when the true rule is outside $\mathcal{L}$, every fit is spurious, and no selection rule repairs it. On the public evaluation puzzle 97d7923e, CORA's typed rule language versions 4 and 5 (the rule language has numbered versions, each adding rule forms: version 2 recovers CORA's own program classes, version 3 adds construction from residual masks, version 4 adds rules over parts of objects and lines, and version 5 is the latest; Appendix B.5) found 98 and 162 fitting programs in the recorded runs, 64 of the latter using rule language version 5's rules, and none predicts the query output. An earlier reading called the puzzle ambiguous; that reading was wrong. Read over all three demonstrations the rule is determined (for each colour in the top-left legend, whose marker is k cells tall, fill the k-th tallest column capped by that colour), but CORA's selectors compare a feature only with a literal, so the rule is inexpressible and every fit is spurious. (The rule language version 5 count may also vary with the process hash seed, because a tied sort key meets a cap of 64 programs; it has not been reproduced under a pinned seed.)

A typed failure record that locates the needed revision: a real-data case. The companion study ran one frozen, audited construction on real Perturb-seq perturbation-response profiles (changes relative to control cells) [140]: across 448 audited gene-by-configuration records (64 genes, seven compared configurations), no rule other than an alias of the fixed reference (a rule with the same outputs as the reference on every demonstration and probe input) was admitted for any of the 64 genes. The typed failure record says why. All 256 demonstration labels were `similar`; of the 2,508 failed attempts of the expanded static search, 1,842 were typed `no_relation` (no relation the language can state explains the change), 480 `insufficient_support` (not enough supporting evidence) and 186 `binding_mismatch` (a mismatch in how a candidate binds its inputs); and the candidate bank varies only thresholds, so more candidates or another search order cannot help. The record therefore locates the

needed revision at the level of representation and grammar (the source of a relation and the graph hop), not search: a concrete case in which a typed record pointed to a candidate revision. The scope is narrow. Choosing that repair is a researcher's diagnosis from the trace, not a demonstrated autonomous policy; it is not biological transfer, discovery or a capability-growth result; and the original run did not reach the predictor stage: with no rule admitted, the six configurations other than the domain's own baseline predictor would all fall back to the same prediction, so their comparison could not show a gain.

Because all 256 demonstration labels were similar, insufficient evidence explains the record equally well. A frozen follow-up was designed to separate the candidate causes with a two-by-two design: scalar thresholds against comparisons between two genes' imputed responses (estimated from pathway-neighbour perturbations, never from the held measurement), crossed with exact fit against a fixed likelihood-based fold vote, beside label-shuffled, pair-shuffled, exhaustive static, ridge, mean and constant controls. All 326,400 predictions (20 compared configurations, 255 held source-validation perturbations of the same reused cohort, 64 readout genes) were sealed before one scoring. No configuration admitted a rule, and each scored zero on directional macro-F1 (the mean F1 over the higher and lower response classes): only 32 of 16,320 held responses were directional, every method missed all of them, and a constant "similar" predictor was right on 16,288. Even on training data no candidate gained more than one net correction over the baseline, against the four the fold vote requires, so the run could not tell the representations apart; the record had pointed away from search, and too little informative evidence per gene is a testable explanation (Section 6.9).

3.2 What can be identified, and what needs further assumptions

Three tiers follow from the cases above, and Section 4 makes the first two precise. *From the record alone* a learner can read whether the search stopped at its budget or exhausted a declared fragment, which compositions were ill-typed or undefined, which demonstrations and cells fail (the residual masks), whether a partial fit exists, and which typed holes a sketch would need. These facts localise a failure; they do not name its cause. *With a control*, an intervention run as an experiment, a learner can identify a search failure (matched extra search), a missing representation (a change of view), a wrong binding (a guard or binding ablation), a missing operation (construction that succeeds where matched search fails) and a wrong specification (another task's evidence as a negative control). Each control costs compute, so identifying the cause is itself a decision with a cost. *Only with further assumptions or data* can a learner establish restricted expressivity beyond an explicit, exhaustively checked space (a capped search that fails proves nothing about the language), separate insufficient evidence from a wrong specification (which needs a prior or new evidence), or detect overconservative verification (which needs labelled outcomes). The decision of what to change therefore has to be made under uncertainty about why the failure occurred, and that uncertainty is the starting point of the theory.

4 A theory of learning what to change

In terms of the core idea: theory answers, under stated assumptions, when the missing piece can exist at all (availability), when a search can find it (discovery) and when the evidence can choose it (selection).

This section gives a decision model for the missing decision and the results that constrain it. Three kinds of statement are kept apart and labelled. *Definitions* state the model. *Propositions* are mathematical results; each is elementary, and each holds only under its stated assumptions, which are checked against the code where the code allows it. *Conjectures* are claims about learning that the experiments test and could refute. Nothing in this section is an empirical result except where a results section is cited, and no statement here is a universal impossibility theorem.

4.1 Setting

Setting. A task is $T = (D, X^*)$ with demonstrations $D = \{(x_j, y_j)\}_{j=1}^{m}$ and queries X^* (the queries); query outputs are read only by the evaluator. A language $\mathcal{L}$ is a set of typed programs with an exact executor $[\![p]\!]$. For a demonstration (x, y) and a candidate output $\hat{y} = [\![p]\!](x)$, write $\mathit{diff} = \mathbb{1}[x \neq y]$ and $\mathit{wrong} = \mathbb{1}[\hat{y} \neq y]$; the residual masks are

$$F = \mathit{diff} \wedge \neg \mathit{wrong} \text{ (fixed)}, \qquad B = \neg \mathit{diff} \wedge \mathit{wrong} \text{ (broken)}, \qquad R = \mathit{diff} \wedge \mathit{wrong} \text{ (remaining)},$$

and $r(p, D)$ collects them over D. A program *fits* when $B = R = 0$ on every demonstration; the version space is $V_{\mathcal{L}}(D) = \{p \in \mathcal{L} : p \text{ fits } D\}$. Fitting is consistency with the demonstrations, not correctness on the test. Write q for a proposer (enumerator or language model), π for the revision choice that maps $(D, p, r(p, D))$ and the remaining budget to the next action, and $\mathcal{A}$ for the action menu. The version space is Mitchell's [113], here over programs; CORA never computes it, and inspects only the fitting programs that its bounded search reaches.

Typed executable hypotheses and demonstration constraints. A hypothesis is a typed program; a type error rejects a composition before any execution, and every well-typed candidate has exact semantics under $[\![\cdot]\!]$. Each demonstration is a constraint, $[\![p]\!](x_j) = y_j$, and $V_{\mathcal{L}}(D)$ is the intersection of the m constraints. The constraints are the only information about the rule: nothing in D separates two programs that agree on every x_j.

4.2 The decision model

Definition 1 (Reasoning system and failure record)**.** A reasoning system is $S = (\rho, \mathcal{L}, \beta, \sigma, b, \kappa)$: a representation ρ that maps raw data to typed objects, features and relations; a language $\mathcal{L}$ of typed programs over ρ; a binding scheme β by which selectors, guards and variables refer to objects and values; a search σ that, within budget b, retains the predictions $K_t(b)$ of the programs it generates; and an acceptance rule κ. Its *failure record* on a task is $\mathrm{rec}(S, D)$: the residual masks $r(p, D)$ of the executed candidates, the search status (stopped by the budget, or a declared fragment exhausted), the closest partial fits, the typed holes (for each sketch that would fit if one hole were filled, the input and output pairs that the hole must map on every demonstration) and the acceptance outcome. The record is a function of S, D and the queries; it never reads the query outputs. When an output's size differs from its input's, the residual masks are not defined cell by cell, and a fit is checked by exact equality of grids.

Definition 2 (Failure hypotheses, interventions and their value)**.** Let Θ be the failure types of Section 3. An intervention $a \in \mathcal{A}$ changes one part of the system: search longer or in another order (σ, b); revise a binding (β); change the representation (ρ); construct an operation from a typed hole $(\mathcal{L} \to \mathcal{L} \cup \{o\})$; change the grammar or library $(\mathcal{L})$; change the acceptance rule (κ); acquire evidence $(D \to D \cup D'$, where the environment permits); or abstain. Each has a cost $c(a)$ in counted operations or CPU, including the cost of its own checks. A *revision policy* π maps $(D, S, \mathrm{rec}(S, D))$ and the remaining budget to an intervention. Over a distribution of tasks its value is

$$V(\pi) = \mathbb{E}_t\big[\,\mathrm{correct}_t(\pi) - \mu\,\mathrm{wrong}_t(\pi) - \lambda\,\mathrm{cost}_t(\pi)\,\big], \tag{1}$$

where $\mathrm{correct}_t$ and wrong_t count accepted query outputs that are and are not exact, abstention scores zero, $\mu \geq 0$ prices a wrong answer and $\lambda \geq 0$ prices compute. *Learning what to change* is learning π on source tasks so that $V(\pi)$ is high on new ones, under one information constraint: π reads demonstrations, executions, failure records and admitted capabilities, and never a withheld output.

A policy that keeps a belief $P(\theta \mid \mathrm{rec})$ over failure hypotheses (each θ a set of failure types, since causes can act together), and predicts the change $\Delta_\theta(a)$ in correct and wrong answers if θ is the cause, can act by maximising

$$g(a \mid \mathrm{rec}) = \sum_\theta P(\theta \mid \mathrm{rec})\,\big[\Delta_\theta^{\mathrm{correct}}(a) - \mu\,\Delta_\theta^{\mathrm{wrong}}(a)\big] + \mathrm{VOI}(a \mid \mathrm{rec}) - \lambda\,c(a), \tag{2}$$

where $\mathrm{VOI}(a \mid \mathrm{rec})$ is the expected gain of the decisions that follow from the change in the belief that observing a's outcome brings. This is the value-of-computation view of rational metareasoning [143], applied to changes of the reasoner itself rather than to steps of a fixed search, and the information term is theirs: without it a pure diagnostic action, a control that adds no answer, would always score below zero and never be run, although such controls are how causes are told apart. What is specific here is that an intervention that changes ρ, $\mathcal{L}$, β or κ persistently changes every later task, so its adoption needs evidence beyond the task that motivated it (Section 4.4), the utility problem of learned control knowledge [112] in another form.

Proposition 1 (A record does not identify its cause)**.** *Let a cause θ range over sets of failure types, and let tasks, with their demonstrations, queries and answers, be drawn from $P(D, X^*, Y^* \mid \theta)$. Suppose two causes θ_1 and θ_2 induce the same distribution of what the learner has observed so far: the demonstrations, the queries and the record. Then every policy whose next action depends only on what it has observed chooses the same distribution of actions under both. A policy can separate them only by an action a whose observation afterwards, the new record of the changed system together with any new evidence $D' \supseteq D$ that a acquires, differs in distribution under θ_1 and θ_2.*

Proof. The policy's input has the same distribution under both causes, and so has its output; only an input that differs can change it. □ The statement concerns a distribution of tasks: for one fixed task the record is a deterministic function of the system and the data, so the uncertainty is about which kind of task, with which cause, produced it. Figure 3 illustrates the situation with recorded runs of different configurations on one task, not one system under two hypotheses: the record no fit arose under four causes, of which only the search failure's differed (a budget stop rather than an exhausted fragment), and all four controls were needed together to separate them. The proposition makes the cost of identification explicit: a policy that must tell causes apart pays for the controls that do so.

4.3 Mathematical results

The results below bound what search, ranking, verification and language growth can achieve, and say which quantity to measure first. Each is elementary; their value is that they account for the measured pattern of Section 5 and name the experiment that could contradict it. They are conditional results, true under the assumptions stated with each, and no priority is claimed for them. The nesting result was stated on 27 October, before the construction tests; the others were added on 6 November, after some of the measurements, so Table 8 shows what the tests account for, not predictions made in advance.

Five results in simple words, each with the experiment it explains.

- *Many explanations* (Proposition 4). A few observations can fit many rules that disagree on the next input. Intuition: two points fit many curves. It explains why fitting programs can be wrong, and why CORA checks stability by rebuilding without each observation (Section 5.2).
- *Build before you rank* (Proposition 2). If the right program is not among the candidates, no ranking can find it: solved is at most generated, which is at most expressible. Intuition: you cannot pick a card that is not in the deck. It accounts for why the best possible choice among the neural comparison entry's candidates could add at most 3 of 172 query outputs, and why construction, not ranking, moved the controlled results (Section 5.5).
- *Bigger language, more ambiguity* (Proposition 7). When every fitting program must agree before an answer is given, enlarging the language can only remove answers on tasks the smaller language already fits, unless a selection rule or a prior breaks the tie. Intuition: more candidate explanations means more chances to disagree. It says why growth needs a selection rule, the open requirement; it has not been tested directly.
- *A missing input blocks exact recovery* (Proposition 8). If a rule depends on an input outside the variables considered, no search over those variables recovers it exactly, however good the search. Intuition: a recipe that needs an ingredient you never bought. It accounts for why the two-input rule language recovered none of the 274 published rules with three or more essential inputs, and for the losses at the regulator shortlist (Section 6.2).
- *A counting limit* (Proposition 9). A few binary observations can single out only a bounded number of rules. Intuition: ten yes-or-no answers distinguish at most 1,024 cases. It accounts for the design simulation, in which no design singled out more than 3% of three-input rules at ten observations.

The nesting. Let $K_t(b)$ be the predictions retained for task t within budget b, y_t its answer and $a_t \subseteq K_t(b)$ its two attempts. Coverage is $\mathrm{cov}(b) = \frac{1}{T}\sum_t \mathbb{1}[y_t \in K_t(b)]$ and two-attempt accuracy is $\mathrm{acc}_2 = \frac{1}{T}\sum_t \mathbb{1}[y_t \in a_t]$.

Proposition 2. *If attempts are drawn from the retained predictions, no selection rule, however learned, raises* acc_2 *above* $\mathrm{cov}(b)$*, and coverage cannot exceed the share of tasks whose answer is* expressible *in* $\mathcal{L}$*, that is, predicted at* x^* *by some program of* $\mathcal{L}$ *that fits* D*.*

So solved $\leq$ correct candidate generated $\leq$ expressible. The decomposition is an accounting identity, not a law; its use is operational: it says which quantity to measure first, and why work on selection cannot help where coverage is zero. *Corollary (selection headroom).* Under the same condition, a change of the selection rule alone can add at most $\mathrm{cov}(b) - \mathrm{acc}_2$ to two-attempt accuracy. Over the neural comparison entry's candidate pool that headroom was 3 of 172 outputs (Section 5.3).

Reachability. Let H be a solver's hypothesis memory and $R_k(H)$ the programs reachable from H in at most k edits of a fixed edit menu (in the audit of Section 5.3, the harness's menu: fill a parameter, selector, transformation, output or representation slot; compose; mutate; propose). If no program in $H \cup R_1(H)$ is correct on the test, then no ranking of H and no one-step revision policy over that menu, however trained, yields a correct answer in one step. This is the condition the reachability audit measures.

A larger language can lower accuracy. Let $\mathcal{L} \subset \mathcal{L}'$. At unlimited budget $\mathrm{cov}_{\mathcal{L}'} \geq \mathrm{cov}_{\mathcal{L}}$, but accuracy need not grow.

Proposition 3. *If $\mathcal{L}'$ adds programs that fit the demonstrations, predict differently on the query and outrank the correct program under the selection score, two-attempt accuracy can fall while coverage rises.*

Two tasks suffice. On the first, the correct program is ranked first in $\mathcal{L}$, and $\mathcal{L}'$ adds two programs that fit equally, outrank it and predict two different wrong outputs, so both attempts become wrong. On the second, no program of $\mathcal{L}$ fits, and $\mathcal{L}'$ adds the correct program together with two higher-ranked fitting programs that predict two other wrong outputs, so coverage rises while neither task is answered correctly. This is the program-synthesis form of the finding that synthesisers can degrade as their grammars grow more expressive [131]. Any procedure that grows its own language must therefore guard against admitting operations that fit by chance.

Proposition 4 (Demonstrations underdetermine programs)**.** *Write $A(D, x^*) = \{[\![p]\!](x^*) : p \in V_{\mathcal{L}}(D)\}$ for the version space's predictions at a query (Section 3.1). Let D be a finite demonstration set with $V_{\mathcal{L}}(D) \neq \varnothing$, and let x^* differ from every demonstration input. Suppose that $\mathcal{L}$ can represent a constant output z for every allowed output grid z, test exact equality to x^*, and form the conditional program that returns z on equality and otherwise runs any $p \in \mathcal{L}$, evaluating only the branch it chooses (or that all programs are total). Then every allowed output grid is a possible prediction at x^* among the programs in $V_{\mathcal{L}}(D)$; in particular, the query is ambiguous if at least two output grids are allowed.*

Proof. Choose $p \in V_{\mathcal{L}}(D)$. For any allowed z, let q_z be the program with $[\![q_z]\!](x) = z$ if $x = x^*$ and $[\![q_z]\!](x) = [\![p]\!](x)$ otherwise. Since every demonstration input differs from x^*, q_z agrees with p on all demonstrations, so $q_z \in V_{\mathcal{L}}(D)$ and $[\![q_z]\!](x^*) = z$. □ The statement is conditional on that expressive language; it does not claim that CORA's bounded grammar can build these programs, and with only a restricted set of output constants its conclusion concerns that set, not all grids. It is the program-language form of the classical point that a learner without inductive bias cannot classify any unseen instance, and classifies only where its version space votes unanimously [114]. Finite consistency alone does not determine the intended query output; language restrictions, priors and selection rules supply additional inductive assumptions. CORA's guards may test only features with values seen in the demonstrations, a restriction that is an inductive bias and not a neutral search setting; real ambiguity remains even so.

Verification and abstention. For a module M and query x^*, the acceptance rule answers the top complete prediction $\hat{y}$ of $M(D)$ only if $\hat{y} \neq x^*$ and, for every j, the module rebuilt on $D \setminus \{(x_j, y_j)\}$ predicts y_j at x_j among its top k predictions ($k = 2$ in CORA-CAST version 1; $k = 1$ in CORA-CAST version 2, which also requires every leave-one-out round (one rebuild with one demonstration held out) to predict $\hat{y}$, no disagreement across modules, and the same result on a restart); otherwise it abstains, and the entry's fallback applies (Algorithm 2). Puzzles with fewer than two demonstrations are never answered.

Certification is stability, not truth. Rebuilding without each demonstration asks whether the same prediction follows from every subset of $m - 1$ demonstrations. It cannot certify a rule whose crucial evidence occurs once, and it can certify a wrong rule that every subset supports. Execution-grounded verification establishes consistency and stability under declared tests; it does not certify unseen correctness.

Proposition 5 (Verification establishes consistency, not intended correctness)**.** *Let two tasks share their demonstrations D and a query x^* but have different intended outputs. Any acceptance rule that reads only D and x^* answers both tasks or neither, and if it answers, it is wrong on at least one.*

Proof. The rule's decision and answer are functions of (D, x^*), which the tasks share, and one answer cannot equal two different outputs. □ Leave-one-out rebuilding is such a rule, and it is weak when $m = 2$. The shuffled-demonstration results are the measured face of Proposition 5: another task's demonstrations are

consistent, programs fit them, and the programs are wrong on the test (Section 5.2). The proposition separates two questions that are easily conflated: whether a program is correct relative to a specification, which execution can check exactly, and whether the demonstrations identify the intended specification, which no check on them can settle.

Proposition 6 (Selective acceptance trades coverage for precision)**.** *Let a strict rule accept a subset S' of the answers S accepted by a looser rule. Then the strict rule's coverage is at most the looser rule's, and its precision is higher exactly when the removed answers $S \setminus S'$ are less precise than S as a whole.*

Proof. The precision of S is a weighted mean of the precisions of S' and $S \setminus S'$. □ The strict policy removed 31 correct and 21 wrong answers, a set far less precise than fit-only acceptance as a whole, so precision rose while coverage and the count of correct answers fell (Section 5.2); whether the trade is worth making depends on the price μ of a wrong answer in (1). Abstaining wherever the version space is not unanimous is the consistent selective strategy [56], and learners that answer only when every surviving hypothesis agrees can bound their abstentions [100]; CORA requires agreement only among the fitting programs found within budget, adds leave-one-out, and has no such bound.

Proposition 7 (Language growth under agreement of all fitting programs)**.** *Let the acceptance rule answer z at x^* exactly when $V_{\mathcal{L}}(D) \neq \varnothing$ and every program of $V_{\mathcal{L}}(D)$ returns z at x^*, a program that is undefined at x^* counting as disagreeing, and abstain otherwise, where the version spaces are exact (or, for a bounded search, where the fits retained under $\mathcal{L}'$ include those retained under $\mathcal{L}$). Let $\mathcal{L} \subseteq \mathcal{L}'$. On every task with $V_{\mathcal{L}}(D) \neq \varnothing$: if the rule answers z under $\mathcal{L}'$, it answers z under $\mathcal{L}$. Hence on tasks that the smaller language already fits, growing the language never adds an answer and never changes one; it turns an answer into an abstention exactly when $\mathcal{L}'$ adds a fitting program that predicts differently at x^*. New answers can arise only on tasks with $V_{\mathcal{L}}(D) = \varnothing$.*

Proof. Fitting depends only on the program, so $V_{\mathcal{L}}(D) \subseteq V_{\mathcal{L}'}(D)$. If every program of $V_{\mathcal{L}'}(D)$ returns z at x^*, so does every program of the non-empty set $V_{\mathcal{L}}(D)$. □ Without the convention on undefined programs the statement is false: if every fitting program of $\mathcal{L}$ is undefined at x^*, a larger language can add an answer. Two consequences shape the experiments. Under agreement-based acceptance, the measure of a grown language is the set of *newly reachable* tasks, those it fits where the old language fitted nothing, not its total number of fits; and growth that adds rival fits to reachable tasks can only add abstentions. Selection assumptions can change this: a rule that requires agreement only among fitting programs whose prior or description-length score lies within a margin of the best is not monotone in this way, and growth can then add answers on fitted tasks when its programs displace disagreeing ones from the margin. That benefit comes from the prior, an inductive assumption that fit alone does not supply [141]. The proposition has not been tested directly. The bioimage calibration of Section 5.2 only illustrates disagreement leading to abstention inside one fixed language: 152 programs fit the closing operator's demonstrations, the near-top fits disagreed about which objects to edit, and agreement admitted no edit.

Proposition 8 (An omitted essential input cannot be recovered by search)**.** *Let the intended rule f depend on an input v: some two inputs that differ only in v have different values of f. If every candidate in a set C is invariant to v, then no member of C equals f; and if the demonstrations or the check set contain such a pair, no member of C fits them all.*

Proof. An invariant candidate gives the same value on the pair, and f does not. □ No search order, ranking or budget within C can then recover f; only a change of what C reads can. This is an expressibility obstruction, separate from the identification ambiguity of Proposition 4. Its measured face is the Boolean shortlist: for 59 of 182 source tasks and 176 of 356 prospective tasks a regulator proven essential lay outside the eight-variable shortlist chosen from the ten demonstrations, so no candidate in the shared bank could be correct and no ranking rule could repair it (Section 6.2).

Proposition 9 (Demonstrations carry a bounded amount of information)**.** *Let a learner observe the binary outputs of a rule from a class H on m inputs, chosen in advance or one after another in the light of earlier outputs. At most 2^m rules of H can be* singled out*, that is, left as the only member of H consistent with what was observed. In particular, singling out every member of H needs $m \geq \log_2 |H|$ [1, 6].*

Proof. The observations form a binary decision tree of depth at most m, with at most 2^m leaves; a rule is singled out only if it is alone at its leaf. □ Three consequences bound the biology studies. Naming three regulators among twenty candidates needs $\log_2 \binom{20}{3} \approx 10.2$ bits, more than ten binary demonstrations carry. In a

23-gene model, at most 1,024 of the 388,656 rules with up to three inputs, about 0.26%, can be singled out by any ten demonstrations. And ten transitions show at most ten of the sixteen input rows of a four-input rule, so at least 64 functions of its true inputs fit them before any restriction of the rule language or any prior narrows them; in the worst case the number of transitions needed grows as $\Omega(2^K + K \log n)$ for K inputs among n candidates [1]. The bound counts rules; it does not limit a particular rule. Eight states that cover every combination of a rule's three inputs, with all other genes at 0, single out any rule of at most three inputs that depends on all three: a consistent rival that omitted one of them would be constant along it on these states, while the rule is not, and a rival that reads all three agrees with the rule on every combination of them, so it is the rule. Evidence chosen for the hypothesis at hand can therefore do far better than random evidence, which is why evidence acquisition is an intervention of the decision model and not an afterthought.

Proposition 10 (Exact acceptance under noise, and the bar as a count)**.** *(i) If each of n observations disagrees with the true rule independently with probability at least $\varepsilon > 0$, the true rule passes a zero-error check with probability at most $(1-\varepsilon)^n$. (ii) If an observed class equals the predicted class with probability q and each of the other $K-1$ classes with probability $(1-q)/(K-1)$, independently, then $\log P(D \mid r)/P(D \mid h) = (C-H)\log\frac{(K-1)q}{1-q}$, where C and H count the observations on which r corrects and harms h; so the bar* (3) *admits r exactly when $C-H$ reaches a fixed count. (iii) No revision is then admitted for a unit on which h makes fewer errors than that count, whatever the representation, language or search.*

Proof. (i) Independence. (ii) Observations on which r and h predict the same class contribute equal factors; a correction contributes q against $(1-q)/(K-1)$, a harm the reverse, and an observation on which both are wrong contributes equally. (iii) C cannot exceed the errors of h. □ With the constants of the Perturb-seq follow-up ($K = 3$, $q = 0.8$, $\tau = \log 20$, prior odds 48) the count is four net corrections per gene; with $q = 0.95$ it would be two. A likelihood with an uncalibrated q is a count threshold under another name, and the event floor of (iii) can be checked on training data before any run; for the Perturb-seq follow-up the number of correctable events per gene has not been counted, so the proposition does not say that the floor caused its null.

When acquiring evidence is the right intervention. Proposition 9 says when evidence acquisition can help and when it cannot: below the counting floor no design can identify the rule, and above it the choice of states decides how much of the possible identification is realised. Choosing perturbations to discriminate among the Boolean networks consistent with the data goes back to Ideker et al. [74], and choosing where a committee of consistent hypotheses disagrees to query by committee [147]. A design simulation on invented random rules, which is a simulation and not biology evidence, measured both sides (`experiments/boolean_design_simulation_20261106/sim/RESULTS.md`; an independent review found it sound with fixes, `experiments/boolean_design_simulation_20261106/REVIEW.md`). With ten variables and fifteen transitions, choosing each state where the surviving rules disagree most left the target as the only surviving rule for 97% of three-input rules, against 8% with random states. At ten transitions no design identified three-input rules (at most 3% in any cell), as the bound requires, although designed states still raised two-input rules from 44% to 92% when read with a fixed tie-break. With sixty variables no design identified more than 13% of three-input rules even at twenty transitions, while random states with the true inputs handed over reached 54%: there, finding the variables is the bottleneck. Perturbation pairs chosen without labels hurt rather than helped. And once the states were designed, three other parts of the system became the bottleneck: the shortlist of candidate variables, strict leave-one-out acceptance, which needs every assignment of the support seen twice, and rules outside the language, on which halving inside a three-input language converged on a single wrong rule far more often than random states did. In the terms of the decision model, seeking evidence is the right intervention when the budget is above the counting floor and the variables are already found; below the floor, or with the variables unknown, it is not; and after it the limit moves to the shortlist, the acceptance rule or the language. The simulation's labels are noise-free, and its active design needs each label before it chooses the next state, which a real study would have to provide.

Conservative structural contracts. Let α map a grid to an abstract state (shape, palette and colour counts, with bounds), and let each operator f carry a forward contract F_f on abstract states. The contract is *sound on a scope* S if, for every abstract state s that the search can form and every grid $g \in S$ that s stands for, $\alpha(f(g)) \sqsubseteq F_f(s)$; soundness must hold on abstract inputs, because the search merges states. The search prunes a partial program when the summary of all its completions, built from these contracts, is incompatible with $\alpha(y_j)$ for some demonstration and a backward check finds no predecessor state from which $\alpha(y_j)$ is reachable; incompatibility must cover every completion, since a small prefix can still reach a larger output, for example by scaling.

Proposition 11. *If every operator's contract is sound on S in this sense, every grid the search meets lies in S, and both searches run to completion, search that prunes by the contracts returns the fitting programs of exact execution, up to its merging of observationally equivalent programs and its cap of 64 recorded fits, and never executes more programs.*

The proof is immediate from soundness. What the code lets us check is soundness on a declared finite scope only: every grid up to 3 by 3 over three colours, every 4 by 4 grid over two, random abstract inputs, and structured random grids up to 20 by 20 (`experiments/cora_cast_20261026/CONTRACT_SOUNDNESS_REPORT.md`); the exhaustive tests already check the stronger, abstract-input notion. That is finite-model validation and property testing, not a proof for arbitrary grids, so the proposition is a guarantee only inside S. On the tasks run, pruning lost no fitting behaviour against exact execution, and it let a depth-2 search finish within a fixed budget of 20,000 executions on every task of a development set (budget stops 32 to 0; 44,485 instead of 1,269,443 executions); it saves CPU only for expensive operators.

4.4 Validation criteria: when to adopt a change

Capability promotion. A proposed operation o is promoted (admitted into the language) from $\mathcal{L}$ to $\mathcal{L} \cup \{o\}$ only if (i) it is new: no program of $\mathcal{L}$ reproduces its behaviour on probe grids (a fixed set of test grids) within a fixed number of executions; (ii) it solves its motivating task; (iii) it breaks none of the 14 solved regression tasks; and (iv) it passes validation on separate tasks of its family. Promoted operations are versioned and can be rolled back. Promotion can change expressivity, the budgeted search order or the ranking; it adds a correct answer only when the selected prediction for a query becomes exactly right where the baseline's was not, and fit alone establishes none of these. Check (iv) is the held-out test of prediction 4 below; checks (i) to (iii) only keep the library clean.

Logical and contextual consistency. CORA's verification tests logical consistency: fit to the declared tests and stability under leave-one-out rebuilding. It has no term for contextual consistency, agreement with established knowledge. Comments on the discussion draft argued that a revision which conflicts with established knowledge should face a higher bar than one that agrees with it; the idea is Bayesian in spirit (prior odds on a revision) and is not new: informative structural priors in network inference [119] and literature-derived prior networks calibrated to data [144] precede it. It gives the bioimage calibration a precise reading, as a hypothesis for further investigation: unanimity is a uniformly high bar and the tolerant rule a uniformly lower one, and neither differed from its controls (the strict rule and its failure-blind and shuffled controls admitted no operator; the tolerant rule admitted the low-threshold operator, as did most of its failure-blind and shuffled control runs). A bar keyed to consistency with prior knowledge is the testable next mechanism. Published Boolean-network models [133] carry their regulator wiring, and pre-registered tests on 44 of them tested a support prior, not this bar: the inferred wiring is a prior built from the task's own demonstrations, and for exact fits the likelihood ratio is 1 or infinite, so the bar reduces to a prior threshold and remains untested. In those tests a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code, inferred each gene's regulators from the task's own demonstrations and used them as its prior; it exactly recovered 99 of 333 published single-gene rules, against 85 with no wiring and 87 with the inferred wiring shuffled (+4.20 points; that study's bootstrap resampled models rather than the 30 publication lineages, so we rely on the replication for inference), and in a frozen replication on 75 further models in 50 lineages it recovered 101 of 595 against 66 for each of three controls, which succeeded on the same 66 targets; every exact recovery there was a one-input rule (Section 6.2). With the correct wiring supplied instead it recovered 148 of 333, a reference ceiling, in which no condition separated the acceptance bar from the prior's own preference. It is recovery of published computational models within biology, with the rule language supplied, not ARC-to-biology transfer and not wet-lab discovery. Keeping hypotheses and tests executable also makes a prior-knowledge check inspectable, which answers a concern raised in those comments about language-model systems mixing up context; that is motivation, not evidence.

For a current explanation h and a revision r, such a bar accepts r when

$$\log \frac{P(D \mid r)}{P(D \mid h)} \;\geq\; \tau + \log \frac{P(h)}{P(r)}, \tag{3}$$

so a revision with low prior odds, because it conflicts with what is known, needs more evidence. The threshold τ sets the uniform part of the bar; the prior odds set the part keyed to knowledge. Unanimity and the tolerant

agreement rule are both uniform bars, which is the precise sense in which neither could tell a change that agrees with established knowledge from one that does not. The bar keyed to prior knowledge has not been tested; for noisy observations, Proposition 10 shows that it reduces to a count of net corrections.

The three requirements in the terms of the model. *Availability* is expressibility: the intended answer is predicted at x^* by some program of the current language and representation that fits D. *Discovery* is generation within budget: the answer lies in what the search generates at budget b. Proposition 2 orders the two, since a task is solved only if its answer is generated and generated only if it is expressible. *Selection* is the choice, among candidate interventions and among fitting programs, that the decision model makes from evidence; Propositions 4 and 1 say why it cannot be read off the demonstrations alone. An experiment speaks to selection only when every arm has the same capabilities available and the same budget to discover them, so that the arms differ only in how they choose.

Which claim a result supports. We apply one rule to every ARC claim about choosing revisions and use only the row the evidence reaches (Table 6). On real answers the evidence reaches at most the second row, and only narrowly. No result reaches the third row for a choice among revisions; the closest, on generated tasks, concerns construction: typed holes computed from a task's demonstrations once its search failed beat another task's holes (Section 5.5.3). Because the cause was planted and the kind of revision fixed in advance, that is evidence about construction, not about choosing among revisions on real answers. Two revisions of the one test that varied only the order of the same changes at equal cost, on generated tasks, established no advantage over the best fixed order; the second beat its pre-registered static order and its shuffled and random controls where a change cannot cheaply check whether it applies (Section 5.9).

what the experiment shows	what may be claimed	reached so far?
more fitting programs, but no more correct real answers	improves candidate coverage	yes: the 22-view bank on authored cases; object-level proposals on source halves
more correct real answers, while a strong fixed expansion ties	new capabilities help; selection unresolved	on exposed or constructed data only: the held-back-half construction and reuse, conditional construction in biology, the bounded loop; on real ARC-AGI-2 only narrowly: one exposed task, examined while CORA was designed, whose two outputs the CORA-only entry already answers
beats shuffled routing but ties full static search	failure information improves efficiency under the tested constraints	not reached for a choice among revisions; closest, on generated tasks and for construction: typed holes computed from the task's demonstrations recovered 64 of 95, against 48 from the demonstrations alone, 38 for static search in the starting language (which used more CPU) and 16 from another task's holes; in the bounded Boolean loop the 12.9% saving at equal accuracy is the fixed schedule's one failed build attempt per built function, and a feature-blind schedule tied; in the pre-registered order test, failure evidence beat a random order but not the best fixed order; in its second revision, where a change cannot cheaply check whether it applies, the pre-registered router beat the static full bank in its pre-registered order (26 tasks against 0) and the shuffled record (18 against 0), but its advantage over the best fixed order chosen after the results is not established (11 against 0), and the fact it reads comes from the demonstrations rather than from the failed search; not on real answers
beats strong static and shuffled controls on real answers	evidence for failure-conditioned revision	no
a learned router does the above on held tasks	evidence for learning which revision to make	no

Table 6: The decision rule for claims about choosing revisions. A result supports only the row it reaches.

4.5 Predictions, with what would falsify each

The model and the propositions make five predictions about the mechanisms CORA implements; Predictions 2 to 5 were written before the construction test (27 October against 1 November). Sections 5 and 6 test them; we never count a prediction as a result.

Prediction 1. *Execution gives observable constraints.* Every executed hypothesis yields $r(p, D)$ and a fit or no-fit per demonstration; any correct program satisfies all of them, so sound partial-program contracts can prune by them without losing a correct program. Falsified by a lost correct program under pruning. A failure to find a fit proves nothing beyond the fragment enumerated: it is not a proof that no program exists.

Prediction 2. *Typed diagnosis narrows the causes.* If the failure report names the failing level correctly (search further, revise a part, or add a capability) and the needed change is reachable, acting on the report beats acting without it at equal compute. Falsified if failure-blind proposals, or another task's report, do as well.

Prediction 3. *Capability invention changes the reachable space.* When no program of the whole language fits the demonstrations, only a new operation, relation or representation can raise coverage (Proposition 2); when only $H \cup R_k(H)$ holds no fitting program, more search or a new capability can. Falsified if the same tasks are recovered by searching the old language with the invention's compute.

Prediction 4. *Independent verification limits overfitting.* By Proposition 3, each admitted operation adds programs that can fit by chance. Admitting an operation only if it also fits tasks it was not made for, and breaks no solved task, is a held-out test of the operation itself; if chance fits on v validation tasks were independent, each with probability at most ϵ, one pre-specified spurious operation would pass with probability at most ϵ^v, but a constructor that chooses among N candidates can admit a spurious one with probability up to $N\epsilon^v$, which for 211 structures at $\epsilon = 0.1$ and $v = 2$ is no bound at all; validating on tasks held out after selection restores it (an idealization either way: real tasks are not independent). Falsified by regressions, or by promoted operations that fail on excluded tasks of their own family.

Prediction 5. *Reuse makes later synthesis cheaper.* If a correct program of cost k in $\mathcal{L}$ has cost $k' < k$ in $\mathcal{L}'$, enumeration with branching factor b in $\mathcal{L}$ and $b + \delta b$ in $\mathcal{L}'$ needs about $(b + \delta b)^{k'}$ rather than b^k candidates, so a shorter program pays only if the wider search does not cost more than it saves, the utility problem of learned control knowledge [112]; on the public evaluation a learned library raised execution-budget stops from 9 to 43 (Section 5.8). For a genuine expansion, k is beyond any budget. Falsified if promoted operations do not solve later tasks that need them at the base budget.

Predictions 2 and 3 are the ones the self-extension tests are designed to break: failure-blind invention and search at matched compute are their controls. For the neural inventor neither prediction survived them (Section 5.5.1); for the symbolic constructor, the specification deduced after a failed search beat a shuffled control at task level, its edge over the demonstrations alone was directional but not significant, and its edge over matched search was inconclusive (Section 5.5.2). This taught us that the evidence was usable, since a symbolic constructor used it, and that the neural inventor's limit is writing the operation. It opens neural proposers that hand the typed specification to a symbolic constructor.

4.6 Conjectures

The central claim of the paradigm is a conjecture about learning, not a theorem, and the experiments so far have not established it.

Conjecture 1 (Learned intervention choice)**.** On a family of tasks whose failures have several causes, a revision policy that reads failure records and chooses among search, binding revision, representation change, construction, evidence acquisition and abstention achieves a higher value (1) on held tasks than the best fixed intervention order, selected on source tasks, and than spending the same compute on search, at matched cost. *Falsified* if, at matched counted work on held tasks, the policy does not beat the best fixed order, or does not beat its own failure-blind and shuffled-record variants.

Conjecture 2 (Co-induction)**.** On natural tasks, changes of representation chosen from failure records raise the number of newly reachable tasks per unit of compute more than growth of the operation library alone, because a representation change shortens many programs at once. Changes diagnosed by researchers from failure records count as instances, so the one tested so far, a diagnosed extension that fitted 18 training tasks against 17 for the base language and 17 for a random extension of the same size, is a failed instance. *Falsified* if, at matched work, representation changes chosen from failure records add no more newly reachable tasks than random representation changes of the same size, or than library growth.

Conjecture 3 (Transfer of procedure, not content)**.** What a revision policy learned on grids can carry to a domain with a different alphabet is its procedure and its second-order structure (which level to revise, when to seek evidence, when to abstain), not its operations. *Falsified* if a policy learned on grids helps on biological rule inference only through transferred operations, or if no part of it helps at all against a policy learned in the target domain at matched data.

The record so far bears on these only indirectly. Inside fixed languages, static search matched learned control; the only learned component that beat its hand-set comparator, a search order, changed which answers a binding budget found, while at 20,000 counted operations per hole the counts were close (327 against 329; inconclusive) (Section 5.6.1); the first pre-registered test of failure-guided choice found the outcome it had pre-registered, none (Section 5.9); and even with a shared binary alphabet in place, an order counted from ARC did not help on gene rules (177 against 194; Section 6.6).

5 CORA as an empirical investigation

In terms of the core idea: this section shows it working on ARC-style tasks. The scored solver checks, construction from failures builds the missing piece, reuse keeps it, and a learned search order spends effort where it pays.

This section uses CORA's accumulated experiments as evidence about seven causal questions, not as a chronology. Does execution-grounded verification make answers trustworthy, and at what cost (Section 5.2)? Where does reasoning fail first: generation, search or selection (Section 5.3)? Can a failure specify the capability that is missing (Section 5.5)? When does learned guidance help, and when does it hurt (Section 5.6)? Can the representation or the language itself be revised (Section 5.7)? Are acquired capabilities reused (Section 5.8)? Does reading the failure choose the right change (Section 5.9)? Between Questions 2 and 3, Section 5.4 reads the neural comparison entry and CORA side by side on the same outputs. Section 5.10 puts the answers in order. Each negative finding is stated as an observation, what it establishes about the mechanism tested, and the experiment it motivates. Engineering details of the entries are in Appendix A, and the full records of the experiments in Appendices B and C.

What works, in brief. The positives come first, each with its label and limit. The scored solver is verify-or-abstain: on three related ARC-style benchmarks the CORA-only entry's answers were right on 81.5% to 95.8% of the outputs it answered, so on those exposed local tests coverage, not precision, looks like its main engineering limit; they do not explain its separate official 0.83% (Sections 5.1 and 5.2). Once a bounded search failed, typed holes computed from the demonstrations recovered 64 of 95 controlled generated tasks against 38 for static search in the starting language, which used 3.2 times the CPU charged to growth, 48 from the demonstrations alone and 16 with another task's holes; our own recomputation verified the result (Section 5.5.3). In the held-back-half test, operations learned on one half of three related benchmarks and reused on the other got 227 of 624 outputs right against 118 for static search in the original language with matched extra CPU, and task-local construction got 343 right against the same 118; our audit passed it, the whole reuse gain is on 1D-ARC, and static search won on ConceptARC (Section 5.8). A learned search order beat the hand-set order at the small budget (320 against 234 correct; a table of success rates, 318, and a linear model, 323, did about as well) and, at a 40-times smaller per-hole cap, got nearly as many held outputs right as the hand-set order at its largest sampled cap (320 against 329; the answer sets differ; the cap ratio is not end-to-end compute and not a proven minimum) (Section 5.6.1). On the exposed public ARC-AGI-2 evaluation, task-local construction got 2 of 172 query outputs right where every other arm of that study got none, from one task, an inconclusive result; both are the CORA-only entry's own answers on that task (Section 5.8).

Results are kept in four classes that are never pooled: official Kaggle scores (Section 5.1); the permitted public evaluation, 120 puzzles with 172 query outputs, which is development evidence (Section 5.3); and two

statement	kind	assumptions	measured face
solved $\leq$ generated $\leq$ expressible (Proposition 2)	accounting identity	attempts drawn from retained predictions; expressible means predicted by a fitting program	reachability audit, neural pool headroom
a larger language can lower accuracy (Proposition 3)	elementary result	added programs fit, differ at the query and outrank; two tasks	none yet (97d7923e had no correct fit in either language version, an inexpressibility case)
demonstrations underdetermine programs (Proposition 4)	elementary result	non-empty version space; constants, equality test and lazy conditional expressible	rival fits (Section 3.1)
consistency is not intended correctness (Proposition 5)	elementary result	the rule reads only D and x^*	shuffled-demonstration controls
selective acceptance (Proposition 6)	elementary result	strict accepts a subset	strict policy against fit-only
growth under all-fitting agreement (Proposition 7)	elementary result	agreement of every fitting program, an undefined program counting as disagreeing; exact or nested version spaces	untested; the bioimage calibration illustrates disagreement leading to abstention
omitted essential input (Proposition 8)	elementary result	candidates invariant to the input	Boolean shortlists; 0 of 274 rules with three or more inputs in the two-input language
bounded information (Proposition 9)	counting bound	binary outputs, m observations	the design simulation
exact acceptance under noise (Proposition 10)	elementary result	independent errors; a class-conditional noise model	the Perturb-seq bar of four net corrections; the per-gene event floor not counted
sound contracts prune exactly (Proposition 11)	conditional guarantee	soundness on abstract inputs over the declared scope; complete searches; merging and the fit cap	contract audits
records do not identify causes (Proposition 1)	elementary result	equal distributions of what has been observed	an illustration (Figure 3)
coverage before ranking; constructibility, not localisation	supported interpretation	the tests run	Section 5.10
Conjectures 1 to 3	conjectures	as stated	Conjecture 1: one pre-registered test, outcome none; the others not yet tested

Table 7: The theory's statements by kind. Elementary results are true under their assumptions and are not claimed as new learning theory; interpretations rest on the tests run; conjectures are open.

classes of companion experiments, developed separately from the entries: development-only task studies on tasks built from public training puzzles, and synthetic research (Sections 5.5 to 5.8). Public training-set results are never presented as an independent benchmark: accuracy evidence means the Kaggle scores and the public evaluation only, never performance on training puzzles.

Evidence classes. We never pool these. *Generated*: puzzles drawn from families written in CORA's own language; engineering evidence, not ARC ability. *Source*: public ARC-AGI-2 training puzzles that no design choice used; development evidence, and the ARC Prize guidance on training-set results applies to them, so we report them only as such. *Development*: the 120 public evaluation puzzles (172 query outputs), which we inspected while building the methods; for the neural grid model they are also contaminated: its authors report that descriptions of them leaked into its training data, and its published training recipe uses them, answers included, as the validation set that ranks and keeps checkpoints [155], an exposure through validation and checkpoint selection, not gradient training on their answers (whether the hosted checkpoint followed that recipe exactly is not attested). *Capability recovery*: constructed task families over real input grids that need deliberately removed operations. *Hidden*: the official Kaggle test, the only clean ARC evaluation. *Biology*: retrospective or fresh public biological data. A query output is right if either of two attempts is exact; a puzzle is *strictly* solved

result	test	measured	reading
Many explanations (Proposition 4)	answer whenever a program fits every demonstration, or also require the strict rule's rebuilds without each demonstration, on 1,244 query outputs of three public ARC-style benchmarks scored once before	32 of the 134 answers from fitting programs were wrong; the strict rule removed 21 of those 32 and 31 right answers. In the first run on these benchmarks, given another task's demonstrations, the CORA-only entry still answered 307 of the 1,244, and 6 were right	supported
Build before you rank (Proposition 2)	the best possible choice among the neural comparison entry's candidates, 172 public evaluation query outputs (inspected during development)	a right answer was among the candidates for 46 and chosen for 43, so any ranking could add at most 3; a frozen ranking by CORA's contracts added 0 and lost 0	supported
Bigger language, more ambiguity (Proposition 7)	not yet run: grow a language under agreement of all fitting programs and count, on tasks the smaller language fits, answers turned into abstentions	no test; in one fixed imaging language, 152 programs fitted an operator's demonstrations and agreement admitted no edit (an illustration)	untested
A missing input blocks exact recovery (Proposition 8)	rules reading at most two inputs, on 595 published gene rules with 40 simulated observations each (reused cohort; simulations)	0 of the 274 rules with three or more essential inputs; conditional construction, which reads up to four, recovered 112 of them (81 of 90 three-input); a classical learner given the same observations also reached 433	supported
Counting limit (Proposition 9)	designs of observations for invented random rules over 10 variables (a simulation, not biology)	with 10 observations no design singled out more than 3% of three-input rules (the bound allows 3.9%); with 15, observations chosen where the surviving rules disagree singled out 97%, random ones 8%	supported
A typed hole gives the missing piece an exact target (Section 5.5.3)	95 generated tasks, each missing one operation of a declared space of 211 structures; three seeded repeats; pre-registered	the task's own holes 64; demonstrations alone 48 (16 gains, 0 losses); another task's holes 16, all in the one world whose removed operations compose into one another (0 of 59 in the other two); static search 38 with 3.2 times the CPU charged to growth; blind enumeration 3 in every repeat, 74 in some	supported on generated tasks: discovery in a supplied space
Construction finds answers that static search at the same CPU does not (Section 5.8)	464 held-back tasks of ConceptARC, Mini-ARC and 1D-ARC, 624 query outputs, held out from development but scored once before	reused operations, then construction where their search failed, 343; static search with the same CPU allowance 118; construction from another task's demonstrations 10; on the two-dimensional benchmarks 57 against 30 of 315	supported on exposed related tasks
A learned order changes what a capped search finds, not what exists (Section 5.6.1)	learned against hand-set search order on the same 624 query outputs, at four caps	320 against 234 at 500 counted operations per hole; 327 against 329 at 20,000, where the cap still bound on all 11 tasks whose attempts differed; a frequency table 318, a linear model 323	supported at a binding cap

Table 8: Each theoretical statement and the test that bears on it. Rows 1 to 5 are elementary conditional results, proved in Section 4.3 under their stated assumptions; rows 6 to 8 are mechanism statements tested in pre-registered studies. Proposition 2 was stated on 27 October, before these tests; the others were added on 6 November, after some of them, so the table shows what the tests account for, not predictions made in advance. A fitting program reproduces every demonstration; agreement of all fitting programs answers only when every fitting program returns the same output; an essential input is one whose value can change the rule's output; static search searches the starting language with no new operation; blind enumeration adds the declared structures one at a time and searches the task with each. The strict-rule test of row 1 and rows 2, 7 and 8 use previously exposed tasks; rows 4 and 5 use simulations.

when all its outputs are.

Apart from the official scores, every study below is a companion experiment, developed separately from the entries and run in no Kaggle entry: development-only task studies on tasks built from public training puzzles, synthetic research on generated or authored tasks, and studies on outside benchmarks. All inference is at task level, since seeds replicate within a task (`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`).

5.1 The system and its official results

CORA's executable layer. A grid is an integer array with colours 0 to 9. CORA segments it into objects under seven *views*: same-colour components with 4- or 8-connectivity; multicolour components with 4- or 8-connectivity; same-colour components against the most frequent colour as background; one object per colour; and proximity groups. Each object carries 36 built-in features (colour, size, bounding box, holes, line and border flags, ranks and others) and takes part in 12 relations (above, adjacent, contains, inside, same shape and others). A rule pairs a *selector*, which objects, with an *action*, what to do (recolour, move, delete, copy, draw, edit parts of objects, draw lines); a program is an ordered rule list with a default action and an output specification. Programs are typed S-expressions compiled to CORA's own interpreter, so every candidate has exact semantics. Figure 5 shows one rule CORA built, as an illustration. Six of the seven views assumed background colour 0

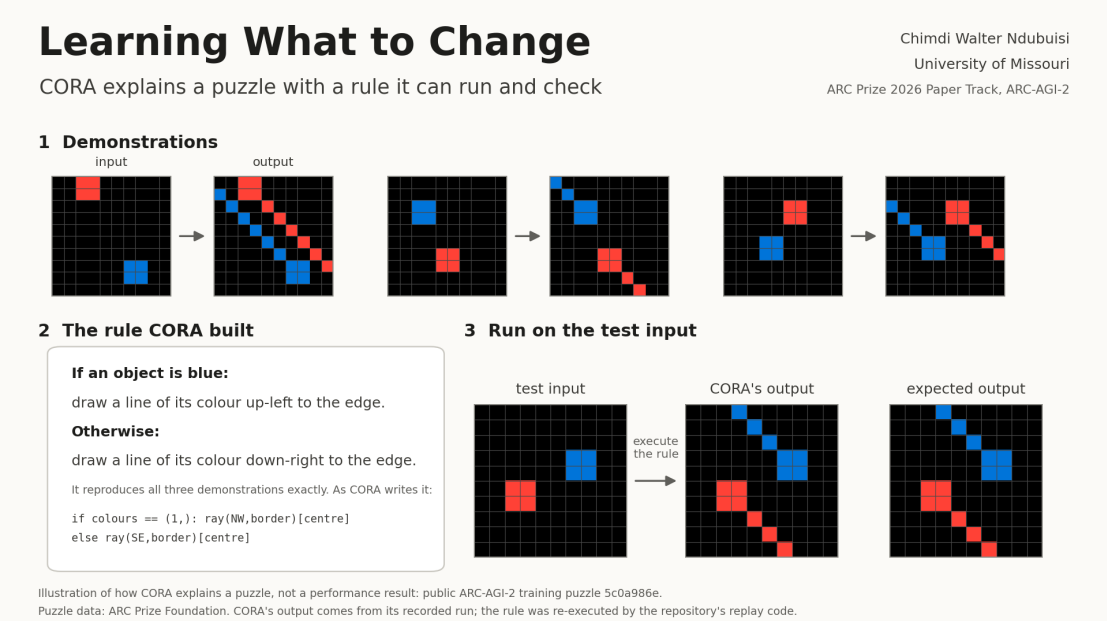


Figure 5: How CORA explains a puzzle: an illustration, not a performance result. The puzzle is task 5c0a986e of the public ARC-AGI-2 training set [8]; the rule is the one CORA built for it ("if an object is blue, draw a line of its colour up-left to the edge; otherwise down-right"), and CORA's output is from its recorded run, with the rule re-executed by the repository's replay code. Puzzle data: ARC Prize Foundation.

(Appendix B.7). CORA-CAST, the hand-written solver that both entries run, adds sound structural contracts, general symmetry completion and guarded relational rewriting over background-adaptive objects; nothing in the scored entry is learned (Appendix A gives its modules, Algorithm 2 its steps, and Table 25 its caps).

Three components, kept distinct. *CORA-CAST*: typed guarded relational transformations and structural verification, written by hand and run by both entries. *Capability growth*: failure-conditioned invention and reuse of executable operations, a research experiment that no entry runs. *The neural hybrid*: the deployed process that combines a neural model's predictions with CORA-CAST's executable candidates.

The two entries. The CORA-only entry (submission 57902131, 2 November 2026) runs CORA-CAST version 1 and a second reviewed CORA solver, on CPU, with no neural model, no GPU and no external data; every non-fallback answer is a program prediction executed on the puzzle, and every other output is a copy of the query. The neural comparison entry (submission 57867631, 1 November 2026) answers attempt 1 with our re-implementation of NVARC's published test-time-adaptation method [155], using NVARC's checkpoint, and offers CORA-CAST version 2's answer for attempt 2 when its strict acceptance policy accepts it. The paper links the CORA-only entry. Neither entry runs failure-triggered construction or grammar growth; those are research experiments, and a separate public research notebook replays the growth prototype (`https://www.kaggle.com/code/chimdiwalter/cora-grammar-growth-research-replay?scriptVersionId=365118951`).

Context for the CORA-only entry's score. The CORA-only entry's 0.83% is CORA alone, offline on Kaggle's CPUs, with verify-or-abstain. The results below come from other partitions and conditions, so they are context for it, not a direct comparison: the winner of the 2020 ARC Kaggle competition (icecuber), whose entry relied entirely on brute-force program search [40], scored 1.6% on ARC-AGI-2's semi-private evaluation set [41, Table 1], and CompressARC, with no pretraining or external data, 4% on ARC-AGI-2 [42]. The ARC-AGI-2 paper treats accuracies below 5% as generally not meaningful, since they are likely to come from noise-level heuristics or incidental pattern fits [41]; every system without pretraining named here, the CORA-only entry included, sits below that level, so this comparison is context, not a ranking. The large scores come from language models proposing programs or natural-language instructions, on frontier-model API budgets outside Kaggle's offline limits. On the ARC Prize community leaderboard, whose semi-private results ARC Prize runs and

entry	submission	public score
the neural comparison entry: NVARC's method, re-implemented, with CORA-CAST version 2 on attempt 2 (Appendix A)	57867631, 1 November 2026	24.03%
the CORA-only entry: CORA only, on CPU	57902131, 2 November 2026	0.83%
CORA's original engine	57144804, 13 October 2026	0.00%
CORA's original engine with failure-driven construction of missing growth operators	57354496, 18 October 2026	0.00%
an older engine	54617562, July 2026	0.00%
NVARC's own notebook, another team's published method, rerun with compatibility and deployment patches only as a reference; not ours	57160429	27.64%

Table 9: Official public-leaderboard scores on the hidden ARC-AGI-2 test. Private scores are revealed after the competition. Four quantities are kept apart: NVARC's own notebook (27.64%, not ours); the neural comparison entry's score, our re-implementation of NVARC's method (attempt 1 is always the neural answer), 3.6 points below it; the CORA-only entry's score, which is CORA alone; and CORA's increment inside the neural comparison entry, 0 gained and 0 lost on the public evaluation and unmeasured on the hidden set (Section 5.3). No pretrained model's accuracy is CORA's.

verifies [9], an evolving library of Python programs written with a language model [DreamCoder-inspired; 132] scored 26.0% for $476, and evolved natural-language instructions, not executable programs (Berman), 29.4% for $3,648, both on ARC-AGI-2's semi-private set. Offline and compute-limited on Kaggle, the 2025 leaders scored 24.03% (NVARC), 16.53% (the ARChitects) and 12.64% (MindsAI) on the private set [42, Table 1]; NVARC's 2025 private score is a different quantity from the neural comparison entry's 2026 leaderboard score, which happens to be the same number. On the 2026 ARC-AGI-2 public leaderboard itself, downloaded on 3 November 2026, the top team had reached 88.06% and the median team 29.31%, of 3,627 teams; this is context for the neural comparison entry's 24.03%, not a ranking claim. The sets and budgets differ from ours, so this places the CORA-only entry rather than ranks it. That neural half is the one CORA's verifier is designed to sit beside; our measured attempts at the integration have not yet helped (Section 5.3), and the work continues (Section 8). Each figure is checked against its source in `paper/REFERENCE_CHECK.md`.

No late submission was possible. On 6 November the first author authorized one late CPU submission: the CORA-only system with the object-level step of Section 5.5.3 running only where that system abstains, frozen and independently reviewed before the attempt. Kaggle refused it, because submissions have been disabled for this competition, and nothing was submitted (`experiments/late_submission_cpu_20261106/records/SUBMISSION_REFUSED.md`); no new system can be scored on the hidden test, and 0.83% remains the CORA-only accuracy. In the package's outcome-free rehearsal on the 120 public-evaluation inputs (never scored), the step ran on the 118 tasks where the CORA-only system abstains, built candidates for one of them, none of which fitted on retry, and changed no answer. The rehearsal taught us that the object step does not yet reach these tasks. It opens construction for harder, multi-rule tasks.

5.2 Question 1: verification, and what it costs

Verify-or-abstain answers only when a program reproduces every demonstration and passes its stability checks. If verification works, answers should be precise on problems the system was not built on, precision should collapse when the evidence belongs to another task, and stricter acceptance should buy precision with coverage (Proposition 6). All three were measured.

The submitted systems on similar problems (pre-registered; run with this project's own evaluation harness). The three submitted CORA systems, byte-identical (the CORA-only entry; the neural comparison entry's CORA component under the strict policy, with input-copy stand-ins for its neural part; and the plain CORA-only configuration), ran on three public ARC-style benchmarks unused in development: ConceptARC [115]

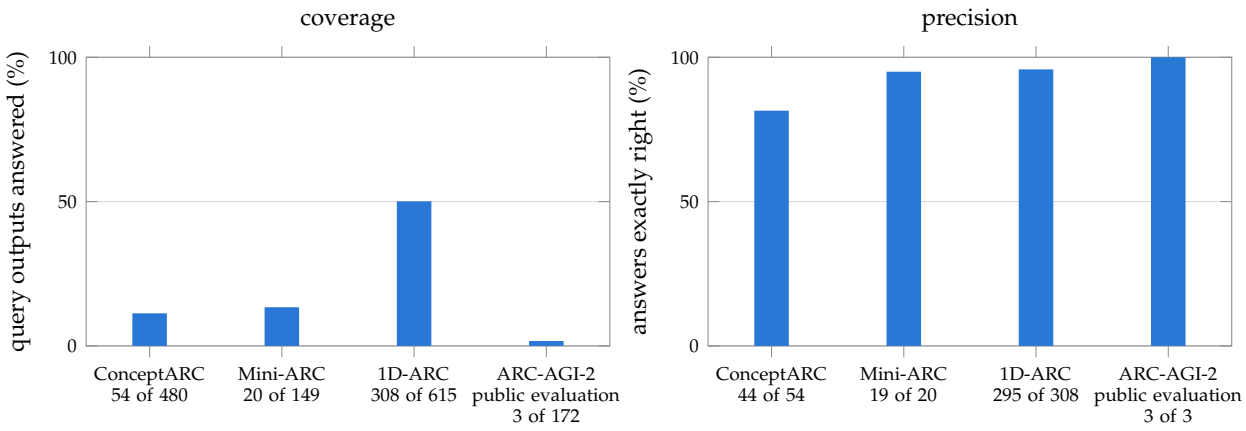


Figure 6: Coverage and precision of the scored CORA-only system, unchanged, on three public ARC-style benchmarks unused in development and on the ARC-AGI-2 public evaluation set (inspected while building; descriptive). Coverage is the share of query outputs answered by a program rather than an input copy; precision is the share of those answers that are exactly right. As measured: ConceptARC 54 of 480 answered, 44 right (81.5%); Mini-ARC 20 of 149, 19 right (95.0%); 1D-ARC 308 of 615, 295 right (95.8%); public evaluation 3 of 172, 3 right (100%). With another task's demonstrations its precision fell to at most 0.024 on each outside benchmark. The benchmarks are in the record's order, which claims no ranking of difficulty. Data-derived from the audited records (`records/NOV2_UNIVERSALITY_SIMILAR_PROBLEMS_RESULT.md`; the public-evaluation count from the learned-library test's no-library condition, Table 27); no new run.

(160 tasks, 480 query outputs), Mini-ARC [84] (149) and 1D-ARC [171] (615 within the entries' input limit of 30 rows and 30 columns, which for its single-row grids is 30 cells; 286 wider tasks are outside it). No input and output pair equals one of ARC-AGI-2, and no task was excluded for overlap; the control gives each task another task's demonstrations (`records/NOV2_UNIVERSALITY_SIMILAR_PROBLEMS_RESULT.md`; harness `experiments/universality_similar_problems_20261102/`). The two entries' components answered 3% to 50% of query outputs, with precision 0.815 to 1.000 (Wilson intervals over outputs); the CORA-only entry's two-attempt accuracy (an output counts if either attempt is exact) was 0.117 on ConceptARC, 0.141 on Mini-ARC and 0.480 on 1D-ARC. For context, the winner of the 2020 ARC Kaggle competition, with three guesses per query, solved 51% of ConceptARC (the mean of the per-concept accuracies in Table 1 of Moskvichev et al. [115], 30 queries per concept); the entries use two attempts. With another task's demonstrations precision fell to 0.00 to 0.03 in every pairing, but the answer count often did not halve, because a donor task's demonstrations are themselves consistent: verification guards against programs that do not fit the given evidence, and cannot tell that the evidence belongs to another task. By the pre-registered rule, 2 of the 9 system and benchmark pairs (the plain CORA-only configuration included) reached verified transfer. It opens transfer criteria judged by precision against the shuffled arm. Limits: no control with CORA's original engine and, until the follow-up below, no matched-search control; intervals not clustered by type or concept, a non-copy proxy for answering, and NVARC's training data include ConceptARC and Mini-ARC (which bears on the neural comparison entry's neural part, not on CORA).

A pre-registered follow-up on the same 924 tasks and 1,244 query outputs added the controls that the first test lacked, declared before their runs: accepting the top program that fits every demonstration, the leave-one-out variants, and blind enumeration in CORA-CAST's own language at the CPU that the neural comparison entry's component actually used (Table 26, Appendix B.1). The pre-declared primary, precision of the strict selective policy minus that of accepting every fitting program, is +0.105 (95% CI 0.029 to 0.234, one-sided $p = 0.0017$): higher observed precision among the strict policy's emitted answers on these public tasks. The gain belongs to the whole policy (module choice, top-1 leave-one-out, agreement among fitting programs and across rounds, and restart stability); all 82 of its answers came from the module that also supplied the top fitting candidate, with no grid mismatch. Its cost sits beside it: coverage fell by 0.042 (Holm-corrected $p = 0.0026$), from 134 to 82 answers, giving up 31 correct answers to exclude 21 wrong ones, so the total of correct answers fell from 102 to 71 and this is not an accuracy gain. Genuine top-1 leave-one-out without the strict policy's other checks pointed the same way (+0.087) but was not significant after correction (Holm-corrected $p = 0.176$), so no claim is made for leave-one-out alone. It opens tests that add the strict policy's checks one at a time. Under the CPU budget that the neural comparison entry's CORA-CAST component actually used, blind enumeration in

CORA-CAST's language solved 1.4% of outputs and CORA-CAST's construction and search 8.2% (0.082 against 0.014, Holm-corrected $p = 0.003$). The submitted CORA-only entry, which also runs a second solver under a larger budget, solved 28.8%, mostly through that solver; that comparison is descriptive, not compute-matched. These are correct-answer rates on non-copy outputs, not two-attempt scores. With another task's demonstrations, precision fell to 0.000 to 0.026 for every condition. On 176 unselected source puzzles the validated policies of CORA-CAST version 2 accepted 13 answers, all correct, and rejected exactly the 2 wrong candidates, abstaining on 93% of the queries (Appendix B.1).

Counterexamples: programs that fit and are wrong. Five measured cases. On the sealed source puzzles (held out and unopened until one final run), 3 of the 11 answers of CORA-CAST's core without its acceptance filter fit every demonstration and were wrong (Section 5.7.1). On the public evaluation, all 7 of the companion study's verified symbolic candidates were wrong (Section 5.3). On BIG-bench list functions [156], for 2 concepts the full language, before any deletion, fits the demonstrations and mispredicts the queries (`experiments/capability_growth_20261027/reports/TEST_U3_REPORT.md`). On fresh nuclei images, 22 of the 33 deletions that CORA-CAST's fitting programs proposed without the leave-one-out check removed real nuclei; under the neural comparison entry's rule the check accepted none of the 33, the 11 right ones included (Section 6.7). On a fresh fluorescence image set, an erase-everything program fitted demonstration windows that held only debris and survived leave-one-out, so the neural comparison entry's rule accepted 11 deletions, 10 of them removing real nuclei (Section 6.7): here the check itself passed a fitting but wrong program. Each program is consistent with its demonstrations and wrong on the test. They taught us that stability under rebuilding cannot tell a rule from a rival the demonstrations do not separate. They open acceptance that also weighs consistency with prior knowledge, the bar proposed above.

Strictness was selective on symbolic benchmarks and suppressed corrections in an imaging calibration. On symbolic benchmarks a stable fit is usually a right one: CORA's frozen solvers were right on 81% to 100% of their answers on three ARC-style benchmarks, and precision fell to 0.03 or less under shuffled demonstrations (above; Figure 6 shows coverage and precision per benchmark as measured: coverage falls from 1D-ARC to the ARC-AGI-2 public evaluation while precision stays between 81.5% and 100%). In noisy scientific imagery the same strictness can suppress useful corrections. In the bioimage study's LIVECell [55] calibration (calibration only; `experiments/universality_bio_grammar_20261103/WATERFALL_livecell_close.json`), the induction specified 10 operators and admitted none. For the morphological closing operator, 152 programs fit the demonstrations and 52 passed exact re-execution, yet the strict agreement policy admitted 0 edits: the fitting programs together select 5,139 of the 23,127 candidate units (objects an edit can act on), only 6 are selected by all of them, and none of those 6 is the top choice of a leave-one-out round; no round failed and no program conflicted. Near-optimal programs disagree about which objects to edit, so the requirement that all near-top fits agree (unanimity), not the harm check, blocks the corrections. A tolerant agreement rule was the next test; in the same calibration it admitted one operator, adding low-threshold objects (46 validation edits, 5 of 6 validation parents better, validation F1 0.4622 to 0.4625; `records/NOV5_BIO_GRAMMAR_TRANSFER_RESULT.md`). That is calibration evidence, not a result. On the held-out targets the strict rule made no edit at all; the tolerant rule made one harmful edit and no beneficial one on LIVECell, the same harmful edit that the failure-blind and shuffled control seeds admitting its operator made, and 32 accepted edits on NuInsSeg, 10 beneficial, 16 harmful and 6 neutral; no target passed its control gates (Section 6.7).

What Question 1 established. *Observation:* verified answers were precise on fresh ARC-style problems (81.5% to 95.8% of the CORA-only entry's answers right on the three outside benchmarks) and collapsed to 0.03 or less with another task's demonstrations; the strict policy showed higher observed precision than accepting every fitting program (+0.105, 95% CI 0.029 to 0.234; 71 of 82 against 102 of 134), at a cost in coverage (-0.042): it gave up 31 correct answers to exclude 21 wrong ones, and leave-one-out alone was not significant (+0.087, $p = 0.176$ after Holm correction for multiple comparisons); in noisy images the same strictness blocked every correction. *Explanation:* verification establishes consistency and stability under declared tests, which on exact symbolic tasks usually coincides with correctness, but it cannot tell the intended task from a rival that the demonstrations do not separate (Propositions 5 and 6). *Research implication:* acceptance should weigh evidence beyond the demonstrations, as the bar (3) does; it is falsified if a bar keyed to prior knowledge admits no more correct changes than a uniform bar at matched precision. A calibrated bar set on one dataset has since failed on a new

source (Section 6.7), so any such bar must be tied to the target domain.

5.3 Question 2: where reasoning fails first

Proposition 2 orders the measurements: selection cannot help where the correct candidate is never generated, and generation cannot exceed what the language expresses. The public evaluation set, inspected while building and therefore descriptive, was measured in that order.

Symbolic CORA. Every CORA-only configuration we ran on Kaggle (CORA's original engine alone; the original engine followed by rule language version 3 with 3,000 or 10,000 candidates, rule language version 4 or rule language version 5; two attempt orders) solved 0 of the 120 public evaluation puzzles (172 queries). In the submission configuration, 273 of 344 attempts were the fallback, nothing was certified, and rule language version 3's search (113 within the CPU limit, 7 stopped at it) kept no fitting program. The original engine's 71 attempts on 33 puzzles were all wrong, 45 returned the query unchanged, and the median attempt got none of the cells that had to change. Rule language version 3 with 3.3 times the candidates fit none of the 119 puzzles it searched; no global colour map, rotation or reflection, integer scaling, tiling or crop to one object reproduces the demonstrations of any puzzle; 81 puzzles keep the input's size and 39 change it, where rule language versions 3 and 4 construct nothing by design; in 55 of the 81 same-size puzzles an output object spans several input objects (`records/EVAL120_CAPABILITY_GAP_ANALYSIS_20261022.md`). These zeros taught us that the failures lie in what the languages can build: size changes and output objects that span several input objects. They open constructors for size-changing outputs, and representations whose objects can span input objects.

Reachability. The architecture campaign's harness gave CORA a hypothesis memory of up to 32 executed programs per puzzle and a menu of typed actions (fill a parameter, selector, transformation, output or representation slot; compose; mutate; new proposals; commit; stop). In all 113 evaluation puzzles it could audit (7 of the 120 were cut by its 120 CPU-second guard), no program in the memory even fit the demonstrations, and none in any one-step successor is correct on the test (`experiments/architecture_sweep_20261026/runs/episodes_v1/reach_eval_dev.json`). On source puzzles the same audit found 89 already correct, 1 construction-reachable and 482 unreachable of 572 training states, and 27, 0 and 159 of 186 selection states; on generated faults about a quarter were reachable. So on real puzzles, within the measured budget, the gap is in generation, actions and representation, not in ranking, and it is not proven inexpressibility: the condition under which no selection or revision policy can help (Section 4). An independent review of 46 failed puzzles, labelled by reading the demonstrations against CORA's typed signatures (judgments, not measurements), put the first break at expressivity in 29, the output canvas in 13 and the object representation in 4, and at search or selection in none; 27 lack two capabilities and 13 lack three or more, and the most frequent missing one is a cross-object reference (13 of 46) (`records/CORA_SYNTHESIS_REVIEW_20261027.md`, section 2). This taught us that missing capabilities, not search or selection, bind on these puzzles. It opens adding those capabilities, cross-object reference first.

Table 27 (Appendix B.2) collects the reachability, gap type and answers of every mechanism by dataset.

Selection over a neural pool. Where a strong neural model supplies the candidates, the question becomes how much a verifier can add.

selection of the neural pool	neural alone	with CORA-CAST as attempt 2	losses
votes, then likelihood	17	19	0
likelihood only (as in the entry)	22	24	0
pool oracle (any candidate right)	27	29	n/a

Table 10: Query outputs right, of 172, on the 120 public evaluation puzzles (development evidence; the checkpoint is contaminated for these puzzles). The neural pool is the grid model's no-adaptation run with 16 decoding and 16 scoring augmentations. CORA-CAST supplies attempt 2 only where it has a complete prediction; the gain is construction, not selection, since 62593bfd's answer is absent from the pool. Research runs, without the deployed solver's stricter acceptance rule, which can remove correct candidates as well as wrong ones; with that rule, on the neural entry's own predictions, the neural comparison entry gained 0 and lost 0 (Appendix B.2).

Neural-symbolic attempts after the submission (companion analyses of 1 and 2 November; public development). Four further attempts to add CORA to the neural entry gave nothing. (i) *Headroom.* The neural entry's candidate pool holds a correct answer for 46 of the 172 outputs, 3 more than the neural comparison entry answers, ranked 3rd to 5th in pools of 6 to 8, so no selector over this pool can add more than 3 (`kaggle/score_first_2026/configs/h2/results/headroom_eval120.json`). (ii) *A frozen contract-based selector* over the pool changed 8 attempt-2 outputs and equalled the neural comparison entry: 43 of 172, 0 gained, 0 lost (`kaggle/score_first_2026/configs/h2/results/cast_contract_select_v1.score.json`). (iii) *Model-assisted repair.* NVARC's model without per-puzzle training answered 162 outputs, 8 of them right at attempt 1; the companion study's repairs of its residuals corrected none and broke none (`experiments/independent_scoring_20261102/neural_repair_v1.score.json`). (iv) *Symbolic candidates.* The companion study's 7 verified symbolic candidates were all wrong: merged as attempt 2 they left the CORA-only entry's 3 unchanged and would have cost the neural comparison entry one correct neural second attempt, 43 to 42 (`experiments/independent_scoring_20261102/single_action_v1.score.json`). Each was scored once, in a separate process. Together they taught us that the pool holds few answers that selection or repair could add, and that the verified symbolic candidates fit without generalizing. They open construction of outputs outside the neural pool, judged on held queries.

The finished companion tests. Each ran once under a frozen protocol and passed a postscore or postrun audit (Appendix B.2). CORA's earlier construction grammar turned 0 of 172 frozen neural first guesses into a program that passes every demonstration, as with no target, while the neural first attempt alone was right on 33 of 172; all 172 of its searches stopped at their execution budget, so that zero is bounded by budget in that grammar. CORA-CAST's own modules explain 2 of the neural entry's 344 attempts, both right, and on the other 41 right outputs they find no program at ten times the budget (Section 5.4). For the companion study's one-step constructor language, a reachability analysis of 842 ARC training tasks (a diagnosis of the language, not accuracy) found that for 754 of 842 tasks no program in the language reproduces even the two demonstrations the solver was given, that perfect search inside the language would reach at most 22 to 88 tasks, and that a grammar covering the published reference solvers would need a search reaching programs of about 29 nodes where bottom-up enumeration reaches 4 to 6. And on the outside benchmarks the companion constructor answered 20 against 120 of 1,244 queries for full-budget static search, pooled, and 0 of the 172 public evaluation queries in every arm.

What Question 2 established. *Observation:* within the measured budget no demonstration-fitting program was generated for any of 113 audited public evaluation puzzles, reviewers put the first break at expressivity in 29 of 46 sampled failures, and over the neural comparison entry's pool no selector could add more than 3 of 172 outputs. *Explanation:* the binding limit is reach, the programs the language and its search can construct, not ranking; for one explicit grammar this is proved for 754 of 842 training tasks. *Research implication:* work on selection should wait for reach; broader grammars with deeper search should be varied one at a time to separate representation from search, and the claim is falsified if a better ranker over the same candidates raises exact answers where no correct candidate was generated.

5.4 Where neural and symbolic reasoning each fall short

In terms of the core idea: a query output that a neural system gets right and CORA cannot explain is a failure with a known answer, and the failure says what CORA's language is missing.

The neural comparison entry, a language model specialized for grids and trained briefly on each task (Appendix A), and CORA fail in opposite ways, and reading them side by side on the same outputs shows what each lacks. Every number here is post hoc, on the exposed public evaluation set (120 tasks, 172 query outputs), and not an official score: both entries and the held-back-half study had scored these tasks, CORA-CAST was designed with them in view, and the neural checkpoint is contaminated for them (Section 5). Table 11 gives the counts, and Appendix B.9 the definitions, budgets and controls.

The neural entry answers more, but returns grids, not programs that can be checked (tested capability, post hoc). With two attempts per query output, the neural entry was right on 43 of the 172 query outputs, and the CORA-only entry answered 3, all right. The neural entry returns grids, not programs. We call a neural attempt

on the public evaluation (120 tasks, 172 query outputs)	count
neural comparison entry right, either of two attempts	43 query outputs (33 on the first attempt); every output right in 27 tasks
CORA-only entry answered, all right	3 query outputs in 2 tasks
right in both entries; CORA right where both neural attempts are wrong	2; 1
right in neither entry	128 query outputs
neural attempts that a CORA-CAST program explains	2 of 344, both right; none of the 301 wrong
neural entry right, CORA no answer	41 query outputs in 36 tasks
a CORA-CAST program at ten times the budget, or in the wider check	none
first gap a missing action: no operation in CORA's language produces the change	35 tasks (40 query outputs)
object views also fail (scene parsing)	14 of those tasks (18 query outputs)
first gap a missing guard: no rule list over CORA's guards assigns the actions	1 task (1 query output)
depth as the first gap	none; in 3 tasks an object needs two actions on the part CORA can write

Table 11: Where the neural comparison entry and CORA each fall short. Post hoc, exposed public evaluation, not an official score. The neural checkpoint is contaminated for these tasks, and all of CORA's answers come from two design-exposed tasks. *Explained*: a CORA-CAST program reproduces every demonstration and outputs the attempt; it does not mean right. *Ten times the budget*: the execution budgets, kept-list budgets and CPU caps multiplied by ten, with CORA's language fixed; the *wider check* also raised tenfold three bounds that CORA-CAST's code treats as part of its language. The diagnostics read only the demonstrations and queries, and their rules were refined while being run on these tasks. Counts re-derived by our own check (`paper/final_push_20261108/neural_vs_cora/CHECK.md`).

explained when a CORA-CAST program reproduces every demonstration exactly and outputs that attempt; explained means consistent with the demonstrations, not right. CORA-CAST's five modules, the three that both entries ship and two research modules, explain 2 of the neural entry's 344 attempts, both right and both among CORA's own 3 answers: the first attempt on task 981571dc, by symmetry completion (Figure 7), and the second attempt on the second query output of task 62593bfd, by guarded rewriting. None of the 301 wrong attempts is explained, and ten times the execution budgets, kept-list budgets and CPU caps, with CORA's language fixed, explains nothing more. Inside the neural comparison entry, CORA's checked second attempt answered only 981571dc, with the neural first attempt's grid, so it replaced nothing.

CORA explains everything it answers, but answers little (tested capability, post hoc). Each of CORA's 3 answers comes from a program that reproduces every demonstration and is rebuilt without each one, and on no other query output does CORA have a complete answer of its own. On those 3 its check accepts both right neural attempts and rejects all 4 wrong ones, and on the first query output of 62593bfd, where both neural attempts are wrong, CORA's own answer is right. On the other 169 it can neither confirm nor reject a neural attempt, so for CORA most neural answers remain guesses. As a filter, explanation is exact but narrow: keeping only explained attempts would cut the entry from 43 right outputs to 2. The limit is reach, not judgement, so the filter becomes useful only as the language grows. A partial program can already help: on 0934a4d8 a symmetry completion program fixes 19 of the 27 cells of the query output, and both neural attempts are wrong only in the 8 it leaves open. All of CORA's answers here, and both explained neural attempts, come from two design-exposed tasks (62593bfd and 981571dc), and 0934a4d8 is design-exposed as well.

Why CORA missed what the neural entry got right (measured diagnosis, post hoc). That leaves 41 query outputs, in 36 tasks, that the neural entry got right and CORA did not answer. Automatic diagnostics, which read only the demonstrations and queries and use CORA-CAST's own object views, guards, actions and move rules, sorted them by cause.

- *Not the budget.* No CORA-CAST program reproduces the demonstrations of any of the 36 tasks, at the CORA-only entry's budget (45 CPU-seconds per task) or at ten times it, even in a wider check that also raised three bounds that CORA-CAST's code treats as part of its language. No task used more than 9.4 CPU-seconds, and typed search, given more executions, exhausted every two-step program.

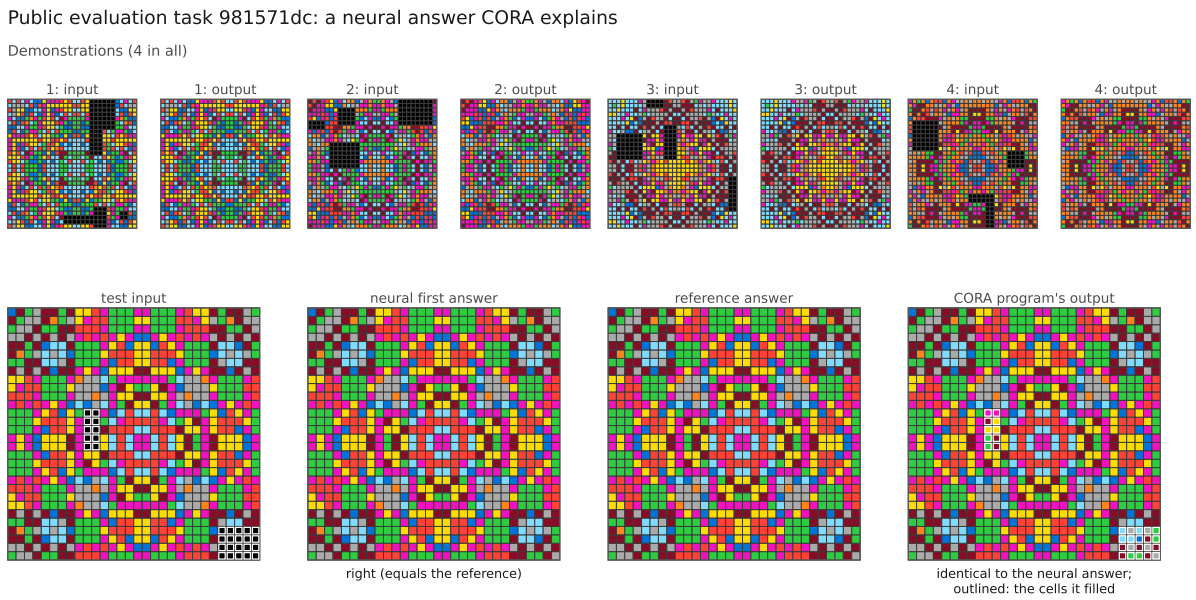


Figure 7: A neural attempt that CORA explains. Task 981571dc of the public evaluation set: the neural comparison entry's first attempt is right, and one symmetry completion program, found from the four demonstrations alone, reproduces every demonstration and outputs exactly that attempt. Outlined cells are the black cells it filled from their symmetric partners. The program line, in CORA's own syntax, reads in words: fill colour 0 (black) from the grid's own symmetries, without cropping. CORA's demonstration-consistent programs give one distinct output here, so the demonstrations and CORA's language pin the answer down. The task is design-exposed. Post hoc, exposed public evaluation, not an official score. Puzzle data: ARC Prize Foundation.

- *The language.* Guarded rewriting's actions write only an object's own cells, its holes, a one-cell ring of background around it, or a straight slide. In 35 of the 36 tasks CORA's language (all three shipped modules) has no operation that produces the change: some changed cells cannot be written by any action (21 tasks), the output of a size-changing task is built rather than cut out of the input (13), or objects move by distances that CORA's stopping rules do not produce (1). In the last task (d35bdbdc) every change can be written, but no rule list over CORA's guards assigns the actions.
- *Scene parsing, in 14 of those tasks:* no CORA-CAST object view makes the objects correspond in every demonstration; on inspection, these tasks act on panels, repeating patterns or parts of objects.
- *Depth is never the first gap,* though in 3 tasks an object needs two actions at once even on the part of the change that CORA can already write.

Figure 8 shows the most common cause on one task.

What the gap says CORA must learn (interpretation and open direction). *Observation:* the neural entry was right on 41 query outputs for which CORA has no program at any budget we tried. *Explanation:* what is missing is operations, not search: mostly drawing on the background, editing part of an object and building a new output grid (Appendix B.9 lists each task), then views of panels, patterns and parts, guards that relate one object to another, and several actions per object. *Research implication:* CORA must acquire actions first, then views, guards and depth. More budget alone closes none of the 41, and neural guidance would help most by proposing new actions and views, not by ranking what the language already has (Proposition 2). Each of these outputs is a fully specified failure: a few demonstrations, a right answer (known here only post hoc) and a record that the

language has no program for it. Building the missing operations from such failures, keeping one only when a program using it reproduces every demonstration, and judging it on held tasks against static access to the same operations and against shuffled failures, is the test of learning what to change on real tasks (Section 8.1); it has not been run. In the terms of the central question (Section 1.1), the neural entry holds whatever rules it uses, or has absorbed from exposure, in a form that cannot be run or checked, while CORA states its rules exactly but has too few. Inducing the missing rules from a few demonstrations, and carrying the same procedure to new domains, is where the two must meet.

Why this differs from the earlier zero. The earlier neural-guesses test (Appendix B.8) aimed CORA's earlier construction grammar at the first attempts of the same neural run and explained none of the 172, but all 172 of its searches stopped at their budget of 150,000 executions, so that zero is bounded by budget in a different grammar. It searched first attempts only, so of the two attempts explained here only 981571dc's was within its scope. CORA-CAST's symmetry completion and guarded rewriting express both programs directly and find them in a few CPU-seconds.

Limits. Counts only, post hoc, on exposed tasks; no accuracy, generalisation or hidden-set claim follows. The diagnostic rules were refined while they ran on these same tasks (the move check and the guard test were corrected after a first pass misread b5ca7ac4 and d35bdbdc, and the line and border labels were tightened), and the first-pass outputs were not saved. Each task gets one verdict, so a missing action can hide a guard or depth gap; readings marked "on inspection" are not diagnostic output; and finding no program within these bounds does not prove that none exists in a larger language. The neural entry's right answers may reflect exposure, since its checkpoint is contaminated for this set, so they do not reveal how it reasons. Every count was re-derived by our own check (`paper/final_push_20261108/neural_vs_cora/CHECK.md`).

5.5 Question 3: can a failure specify what is missing?

A failed search leaves a residue: the demonstrations it could not fit and the holes a sketch would need. If that residue carries the content of the missing capability, a constructor given it should succeed where the same constructor given another task's residue fails, and should beat more search in the old language at matched compute. The three tests below ask this of a neural inventor, of a symbolic constructor, and of natural tasks.

5.5.1 A neural inventor given the failure report

The self-extension test (Appendix B.3) removed four operations of the arc-dsl language [71] (compress, downscale, trim and upscale), built 11 tasks per operation over real source input grids, and asked a 4B language model to write the missing operation from the task's own failure report, from no report (failure-blind), from another family's report (shuffled), by writing whole programs, or by searching 20 times longer. The controller's decisions matched the evaluator's truth: it chose invention on 19 of 20 family tasks and kept searching on all 4 decoys. Own reports recovered 4 of the 11 motivating tasks in some seed, against 1 failure-blind, 1 for whole-program writing and 0 for shuffled reports and longer search (Table 28), but no task-level comparison reached $p < 0.05$, and the pre-registered follow-up was inconclusive: 2 against 2 tasks for own reports and failure-blind invention, while search in the starting language at matched compute recovered 5 (Table 29). Given each removed operation's exact input and output behaviour on the task's own grids, the proposer recovered 1 of 11 tasks. Promoted operations broke none of 14 solved tasks and were reused within their family, and a symbolic merge lifted two promoted instances to one parametric operation; none transferred across families. The binding limit was the proposer's ability to implement a grid operation, not CORA's evidence or checks, which is why the next test handed the specification to a symbolic constructor.

5.5.2 A symbolic constructor given a typed hole specification

Because the neural inventor's coding proved the binding limit (Section 5.5.1), this test separated what the missing operation must do from how it is written (`experiments/capability_growth_20261027/reports/TEST_S1_REPORT.md`; protocol frozen before any outcome, sha256 in `experiments/capability_growth_20261027/partitions/TEST_S1_FREEZE.json`; development evidence on the same constructed tasks over public training grids). For each motivating task, a failed search (even four times deeper) triggers a *hole specification*, which is deduced from

the demonstrations through the starting language's invertible operations (`cgrow/controller.py`, `sketches`): a sketch that would fit if one hole were filled (the identity, or an invertible operation applied before or after the hole), with the inputs and outputs that hole must map on every demonstration. The failure only triggers the deduction; it does not supply the specification. A symbolic constructor then searches a pre-registered schema language of 211 structures (selections, maps and block operations) for a structure that satisfies the specification; the result goes through the same novelty, regression, validation and promotion checks and CORA-CAST's contract checker. Conditions: the hole specification; the same plus the model's proposed form (the primary condition, *specification and form*); construction from the demonstrations alone; another task's hole specification (*shuffled*); blind enumeration of the schema language through the search (at most 60 CPU seconds); and the starting language's search at matched compute (*matched search*, 150 CPU seconds).

family (tasks)	specification and form	hole specification	demonstrations alone	shuffled	blind enumeration (any seed, majority)		matched search
compress (3)	3	3	2	0	2	2	1
downscale (3)	2	2	1	0	3	3	0
trim (2)	1	1	0	0	2	1	2
upscale (3)	2	2	1	0	3	3	2
all, tasks of 11	8	8	4	0	10	9	5
runs of 33 (task by seed)	24	24	12	0		22	15
mean CPU seconds per run	9.6	9.6	4.8	5.4		48.7	141.1

Table 12: The construction test: motivating tasks recovered. Every condition except blind enumeration gave the same outcome on all three seeds of every task, so the task is the unit of inference; for blind enumeration, tasks recovered in at least one seed and by majority. Primary comparison, specification and form against matched search: 8 against 5 tasks, discordant 4 to 1 (tasks where only one of the two succeeded), exact McNemar $p = 0.375$: inconclusive. The unit-level tests first reported over the 33 runs counted each task three times and are withdrawn (`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`).

Results, at task level. *Every comparison.* The hole specification recovered 8 of 11 tasks: against the pre-registered primary, matched search, 8 against 5 (below, $p = 0.375$); against the demonstrations alone 8 against 4 (4 to 0, $p = 0.125$), one task more in each family, directional and not significant; against blind enumeration 8 against 10 in any seed, at a fifth of its compute; and against another task's specification 8 against 0 (8 to 0, exact $p = 0.0078$). The last, the only significant task-level comparison, shows that the constructor uses the specification. The positive pattern holds in all four families (3 of 3, 2 of 3, 1 of 2 and 2 of 3 tasks against 0 in each), but the 11 tasks share four operations: with the family as the unit, a two-sided sign test over the four paired directions gives $p = 0.125$, so the result does not show generalization across unseen operation families (the companion study's records; available from the authors on request); by design a wrong specification cannot help, so it is not evidence of a causal gain over the alternatives. *For each of the four removed capabilities a reference-equivalent structure was promoted*, in a controlled, pre-generated ablation: for each removed family, a reference-equivalent structure from a fixed 211-structure language (equal to the removed reference at every parameter value) was promoted, solved 6 of 6 evaluator-only tasks of its family, whose original ARC input puzzles were absent from the motivating and promotion-validation tasks (one evaluator puzzle also occurs in the regression set), and passed CORA-CAST's contract checker (compress and downscale as expansions relative to the starting language's one-million-execution classification check; upscale, classed an expansion by that check, is a deep macro (a long composition of existing operations) of the starting language, as the matched search of the self-extension follow-up showed; trim through two structures that the starting language reaches as macros); the neural inventor never recovered compress or trim in the earlier tests. This is controlled executable reuse, not fresh transfer: the evaluator tasks were generated on 27 October from the removed reference operations and kept only where the reference succeeded, and their reference-positive outcomes were seen that day in the self-extension test's positive control, before this test's protocol was frozen on 1 November (the companion study's records; available from the authors on request). *Against matched search the primary comparison is inconclusive*: 8 against 5 tasks (4 to 1, $p = 0.375$); trim favours the search (2 of 2 against 1 of 2), which reaches trim's macro at high cost; the specification condition used about a fifteenth of the compute (9.6 against 141 CPU seconds per run). *The model's proposed form added nothing*: it answered "index map, no parameter" on all 33 runs, so the primary condition equals the hole specification on every task. *Blind enumeration* of the schema language recovered 10 of 11 tasks in at least one seed (9 by majority) at five times the compute, so in a 211-structure language the specification used descriptively less CPU on these 11 constructed tasks, without more coverage. It

opens the same comparison in structure spaces too large for blind enumeration to cover. No promoted structure was rolled back, and the whole test cost 0.24 CPU hours and 0.02 GPU hours.

What the construction test establishes. Established: the specification CORA deduces after a failed search selects reference-equivalent members of a known, pre-registered space, with descriptively lower CPU on 11 constructed tasks, and the constructor uses the specification's content (8 to 0 tasks against another task's specification). Open: invention outside that space (all four capabilities are structures of the schema language), an edge over the demonstrations alone (4 to 0 tasks, not significant) and over the old language's search at matched compute (prediction 3). For the predictions, prediction 2 holds in the narrow sense that a wrong specification fails, and controlled reuse within each family holds, on evaluator tasks examined in earlier experiments (prediction 5). An independent replication from a shared written specification (`records/NOV1_S1_SHARED_SPEC.md`) is not reported here; our own replication on list functions did not replicate the primary test (below). The list-function result taught us that construction needs an invertible operation to decompose the demonstrations. It opens tests in which each deleted operation leaves an invertible decomposition.

The construction test on list functions (companion experiment, pre-registered). The construction rule, unchanged, on BIG-bench list functions [156] with four operations deleted (9 motivating tasks, 4 families): the hole specification recovered 5 tasks against 0 with another task's specification, exact $p = 0.0625$, so the pre-declared primary test did not replicate; it equalled the demonstrations alone (5), because the deletion removed the only invertible unary operation and left the identity as the only decomposition (`experiments/capability_growth_20261027/reports/TEST_U3_REPORT.md`). It opens tests in which each deleted operation leaves an invertible decomposition.

5.5.3 Natural tasks: failures localise, but holes are not constructible

The natural-task growth test, phase 1: failures localise, but no hole is constructible (pre-registered; source halves only; a negative). To test the construction loop of Section 5.5.2 on natural tasks, we split ConceptARC, Mini-ARC and 1D-ARC by task hash before any search into a source half (460 tasks, 620 query outputs) and a held half (464 tasks), and ran the unchanged controller with the full starting grammar as its base on the source halves only (`experiments/natural_growth_20261105/PROTOCOL.md`). The base search fitted 7 source tasks and the deeper-search probe at four times the budget 34; of 453 base-search failures, 436 (96%) localised, and the controller sent 425 to construction with three typed holes each. No hole of any of the 425 units was mapped exactly by any of the 211 pre-registered structures (12 near misses); growth from the holes, from shuffled holes and from the demonstrations alone produced 0 candidates and 0 promotions, and a blind-enumeration sample (60 units, 947 structure retries) fitted none, while the same candidate step finds candidates for 14 of the 19 construction units of the construction test's own world. The held halves were held out from the test's development, although every task of the three benchmarks had been run and scored once on 2 Nov by the submitted entries and once by the companion study's constructor; phase 2 as frozen was not run, because its pre-registered expectation from these counts was that growth adds nothing over matched static search, and a follow-up test on the same held halves is designed separately (`experiments/natural_growth_b_20261105/`). The bottleneck this locates is constructibility: object-level holes against whole-grid structures (below). It taught us that coverage comes before ranking. It opened the object-level construction step below.

Where the hole step stops on natural tasks: localisation, then constructibility. The growth machinery derives a typed hole from a partial fit: the search fits some demonstrations or some cells, and the residue names what is missing. On the constructed tasks exactly one operation was missing, so every failure localised and every hole was constructible. Natural tasks stop the step at one of two points. In the companion study's multi-object layout grammar the search stopped before any partial fit: most of the 842 eligible training tasks ended at the first step with no shape template, and on the public evaluation set no executable candidate existed (Appendix B.2); there the representation must change before a failure can be localised, and localising the failure is how the representation would be chosen. In our pre-registered natural-task growth test, with the full starting grammar as the base, the failures did localise: 96% of the base-search failures on the source halves of ConceptARC, Mini-ARC and 1D-ARC had a partial fit with at least one cell right, because the input grid itself is a right-shaped program whenever the output keeps the input's size, and so are many cheap edits of it. Each of the 425 construction units then received three typed holes, and none of the 425 was mapped exactly by any of the 211 pre-registered

structures (12 near misses, where a structure matches at least half of some hole's pairs): the holes ask for object-level transformations (moving or recolouring a segment, filling between pixels, extracting an object) and the structures are whole-grid index maps, selections, block reductions and colour maps (Section 5.5.3). That exhaustive check is the natural-task counterpart of the Boolean omitted-regulator case: within the pre-registered structure space the construction is unreachable by construction, not by a budget. So the bottleneck moves from localisation to constructibility, and the count of localisable failures and the count of constructible holes are the two measurements that say how far natural tasks are from the regime in which failure-grounded growth can work at all. The zero taught us that coverage comes before ranking. It opened object-level construction, which then found proposal-accepted candidates for 306 of the 425 units (development only; below).

The object-level construction step (ours; development only, no held data). Taking the holes at their own level, a hole-directed object-level structure space was added to the same growth machinery, with the controller's holes, the search and the construction test's gates unchanged (`experiments/object_growth_20261105/REPORT_DEV.md`): each input grid is segmented into objects under 12 views, every changed object's fate is bound to a typed edit read off the target, a candidate is one or two first-match rules or a whole-output form, kept only if it maps every hole pair exactly and is accepted by the declared proposal heuristic, which is not re-induction with a pair withheld: an edit must be bound in at least two pairs, whole-output forms (crop, token, count and symmetry completion) are flagged unconditionally, and colour tables count any pair with a constant recolouring. Accepted candidates are then gated by novelty, retry, regression and validation; the regression check rolls a promotion back if a solved task loses its fit or its old first prediction drops out of the two attempts (the old prediction need not stay in the first slot). On the 425 natural failures of the three source halves it found a full object correspondence for 378 units and, for 306, a candidate exact on all demonstrations and accepted by the declared proposal heuristic (ConceptARC 24 of 74, Mini-ARC 9 of 68, 1D-ARC 273 of 283). Gated, with no regression roll-back anywhere: on Mini-ARC 23 task-local operations fitted their motivating task, 0 were promoted (no other source task of the benchmark fitted any of them), 6 of the 68 failures were recovered with exact test answers, and 3 fitting rules mispredicted the test (for example, every object except the largest gravitating to its nearest object); on ConceptARC 2 operations were promoted (crop to the topmost object; delete every object but the bottommost) and 15 of the 74 failures were recovered with exact test predictions; on 1D-ARC operations were promoted in 179 of the 283 units (19 distinct operations in the registry: moves by one to three cells and until blocked, connect, rays, hollow, recolour the largest, and a shape-keyed colour table) and 246 of the 283 failures were recovered, 172 of them through a promoted operation. On an 80-task sample of ARC training tasks (a development check on exposed training tasks, never an accuracy figure), rerun under the frozen constructor, the starting grammar fitted 4; of its 76 failures, 29 had a full correspondence, 5 a candidate accepted by the heuristic, 3 fitted on retry, and the same 3 predicted every query output exactly. This is development evidence on source halves that the holes live at the object level, not an accuracy figure and not a held result; the held-half test and the ARC-AGI-2 check have since run (Section 5.8).

Construction at scale: an independently verified controlled positive. A separate frozen, pre-registered study built 99 generated transformation tasks from public ARC training inputs in three constructed worlds, each task missing an operation representable in the declared 211-structure constructor space; 95 were eligible (one slot unfilled, three sent elsewhere by the controller). It ran once on private Kaggle CPU kernels, and our own recomputation, with the input binding checked, agreed with the frozen output. Growth from typed holes computed from the demonstrations once search failed recovered 64 tasks in at least two of three seeded repeats (all 64 held in all three): 60 through promoted candidates and four through a quarantined fallback that fitted when none was promoted, so recovery and admission are separate endpoints. The pre-declared comparator, static search in the starting language, which received more processor time, recovered 38, and growth from holes taken from another task's demonstrations 16: 31 gains and 5 losses, a difference of 27.4 points, exact paired $p = 1.29 \times 10^{-5}$ (which treats the 95 tasks as independent) and an eight-cluster 95% interval of +5.1 to +40.6 points, with 6 of the 8 task groups favouring construction. The pre-registered decision is that growth from typed holes beats matched static search. Compute favoured the comparator: static search used 8.47 measured CPU-hours on the motivating tasks; the three growth repeats together were charged 2.61 CPU-hours under the protocol's complete-cost rule (their measured growth was about 0.31 CPU-hours, plus 902 CPU-seconds of shared diagnosis), so static search used 3.2 times the charged amount. The typed holes add what the demonstrations alone do not: used as the hole, the demonstrations alone recovered 48, all inside the 64, so the holes added 16 tasks and lost none, while blind enumeration of the same space reached 74 in some seed,

32 in at least two of three (the criterion growth used) and only 3 in all three. The 16 recovered with another task's holes all fall in the one world whose removed operations compose into one another (0 of 59 in the other two), so that control shows that the task's own data matter, not that failure analysis does. The result is fragile. Per world the counts are 23 against 15, 20 against 7 and 21 against 16; leaving out the second or third world gives intervals that include zero, and eight clusters are few. Reuse on excluded tasks of the same families, 51 against 28 of 60, is descriptive only, because its pre-registered condition failed: 17 excluded sources are shared across worlds, and the family label routed the capabilities. On other families construction recovered 23 against 28, so nothing transferred. Deviations are disclosed: two memory errors raised inside the memory handler escaped the rerun rule, 36 searches were rerun for memory and 23 stopped near the memory limit; a post hoc check that excludes every memory-affected motivating search still holds (27 against 4 of 84). All 93 output files match their manifests byte for byte. A separate companion audit of the saved traces connects each stage of the loop, from hole to promotion to later reuse, without rescoring. No neural model ran in this study, and none of it ran in a scored entry.

Why construction works here. When static search of the starting language fails, CORA forms typed holes: undoing one invertible starting-language operation on the demonstrations, or none, gives the exact input and output pairs the missing piece must produce. The failure triggers this step; the demonstrations supply its content. All 211 declared structures are checked against these pairs in about 0.02 CPU-seconds, and only matches enter a search. This is reliable discovery in a supplied space, not open invention, and it does not reach natural tasks: no hole from the 425 natural failures of Section 5.5.3 matched a structure.

Was it faster search, or a missing operation made available? A post hoc, descriptive reachability analysis of the 31 tasks that construction solved and matched static search did not (`records/CONSTRUCTION_REACHABILITY_ANALYSIS_20261108.md`) found both. Post hoc and descriptive: of the 31 tasks that construction from typed holes computed from the demonstrations solved and matched static search did not, 13 have no equivalent program in the static language at any depth searched, and 17 have one 1 to 4 cost levels deeper than static search reached, which it would have needed an estimated 1 to 135 times its budget (median about 5 times) to find; the last task is a static-search process failure, not a reachability result. For the 13, static search completed cost levels 14 to 16 on each task, after 17 to 79 million executions, without a fit. Five caveats bound this: the static programs are the shortest known, not proven shortest; the matched searches capped the function values they could build, which the routes used here do not need; no static program at any depth searched is not a proof that none exists at a greater cost; the execution factors assume a constant growth per cost level, taken from two points of each task's own record; and all 11 operations are members of a space fixed in advance, four of them known when that space was written, so this is selection within a known space, not open invention. *What it taught us:* the typed-hole specification makes finding operations reliable, and inside a space that already holds the answers it both reaches operations that bounded static search does not reach and replaces deep compositions with one admitted operation. *What it opens:* removing operations from the 211-structure language so that invention itself is tested, disjoint worlds for confirmatory reuse, repaired memory handling, and natural tasks.

A bounded diagnosis-and-revision loop on authored Boolean functions (not ARC). The companion study ran one frozen test of a whole bounded loop. On an authored test of all 256 Boolean functions of three inputs, each hidden among eight inputs with seven of its eight truth-table rows shown, a fixed hand-written controller that detects that no two-input rule fits, buys the one unseen row and builds a three-input table recovered 219 functions exactly (38 two-input rules accepted at once plus 181 built; 1,003 of 1,024 held-out bits), against 38 for the two-input language alone, which is all that language can express. A fixed build-then-ask schedule with the same language and evidence rule recovered the same 219 functions, so the gain comes from the supplied larger language and one bought label per built function rather than from diagnosis, and 37 two-input rules that fitted all seven shown rows but were wrong were accepted unchanged by every policy (19 of them still got all four held bits right, and the design produces about 38 such fits). A fixed random action order also recovered 219, a control routed by shuffled features recovered 173, and the classical readout on the same observations equals the controller's by construction. In every built case the controller took one pass: the two-input search failed, it paid one label for the unseen row, then construction, replay and prediction followed; its paid search for hidden relevant inputs never ran. At that equal final accuracy the fixed controller used 12.9% less counted work than a fixed build-then-ask schedule (363,390 against 417,147 units; equivalently, the schedule used 14.8% more), all of it the schedule's one failed build attempt before it asked for the unseen row, and 53.9% less than one fixed random action order (787,678 units), which asked one extra label per built function (362 against 181 paid labels). This is descriptive and post hoc, limited to these authored functions and one hand-written router that is fixed,

not learned; it is not evidence of learned or causal selection, the counted work is an implementation-specific count of native operations rather than CPU time, and it is not a statistical speed claim; a control routed by shuffled features lost 46 functions, but because a feature-blind schedule tied, the test cannot show that authentic failure features help (our audit, `records/audit_20261108/AUTONOMOUS_REASONING_LOOP_RESULTS.md`, section 2; the companion's separate automated review of the saved aggregates passed 68 checks). These are 256 authored Boolean functions, not ARC-AGI-2 puzzles, and never an ARC score. *What it taught us:* the gain came from the supplied language, not from diagnosis, and the loop diagnoses a failure to fit, not ambiguity inside a fit that succeeds. *What it opens:* an uncertainty or counterexample policy for explanations that fit, tested against a strong fixed evidence-gathering comparator.

What Question 3 established. *Observation:* given the failure report, a 4B neural inventor did not beat failure-blind invention (2 against 2 of 11 tasks) and could implement a specified behaviour for 1 of 11 tasks; given a typed hole specification, a symbolic constructor rebuilt 8 of 11 tasks against 0 with another task's specification, but not more than matched search (5) or blind enumeration (10 in some seed); at scale, typed holes computed from the task's own demonstrations beat static search in the starting language, 64 against 38 of 95 generated tasks (independently verified); on 256 authored Boolean functions a fixed controller recovered 219 against 38, and a fixed build-then-ask schedule recovered the same 219; on natural tasks 96% of failures localised and none of 425 holes was constructible from whole-grid structures, while object-level construction found accepted candidates for 306 (development only). *Explanation:* typed holes computed from a task's demonstrations once its search failed carry the content of the missing operation in these controlled tasks, since a symbolic constructor given them succeeded and one given another task's holes did not; the failure itself only triggered construction; writing the operation, not the evidence, limited the neural inventor; and on natural tasks the bottleneck moves from localisation to constructibility (Proposition 2). *Research implication:* the decisive tests are in structure spaces too large for blind enumeration, and on held natural tasks; the mechanism is falsified there if construction from the task's own typed holes adds no task over matched search and over another task's holes. The one real ARC-AGI-2 answer of failure-triggered construction (Section 5.8) used the failure only as a trigger: the constructor computed its holes from the demonstrations, not from the failed search, and no step chose among revisions, so it shows reach on an exposed task, not learning what to change.

5.6 Question 4: when learned guidance helps, and when it hurts

Learned control can act on a search in two ways: by reordering what the search tries, or by removing what it would try. Proposition 2 predicts that neither can add reach; reordering can only change how much of the reach a fixed budget uses, and removing can only lose answers that the static search would have found.

5.6.1 A learned search order, held and pre-registered

Learned guidance, a small network trained only on the object-level step's own successes and failures on the source halves, replaced the step's hand-set search order and was scored once on 464 held tasks with 624 queries (held out from this study's development; the submitted entries had run and scored these tasks once on 2 November) (`experiments/guided_construction_20261106/runs/held_run/RESULTS.json`; protocol, amendments and freeze in the same folder). The guide is a two-layer network over hole, view and block features that orders blocks by predicted chance of success per predicted operation; nothing is removed from the search, and every candidate is still verified exactly. An independent post-score audit recomputed every reported number from the committed rows and passed (`experiments/guided_construction_20261106/reviews/INDEPENDENT_POST_SCORE_AUDIT.md`); the companion study's own evidence read was still pending when this version was written.

counted operations per hole	125	500	2,000	20,000
hand-set order, correct query outputs of 624	207	234	296	329
learned guidance, correct query outputs of 624	320	320	320	327

Table 13: The held test of learned search order: correct query outputs on 464 held tasks (624 queries) by cap. The pre-registered primary is the column at 500 counted operations per hole.

At 500 counted operations per hole, the pre-registered primary, learned guidance got 320 query outputs right against 234 for the hand-set order: 87 tasks gained and 1 lost, +86 with a family-clustered 95% interval of 41 to 136, exact sign test $p = 5.75 \times 10^{-25}$. The gain is on 1D-ARC, 275 against 191 (+84 of the +86); ConceptARC (33 against 32) and Mini-ARC (12 against 11) barely change. At that equal budget the guided search used slightly more work: search CPU 115.25 against 106.35 seconds, constructor CPU 113.78 against 102.61, and counted operations 295,993 against 289,225. Secondary comparisons, Holm-adjusted: a linear model (323) and a frequency order built from a fixed table of success rates (318) were inconclusive against the guide, so the gain is a learned reordering that such a table also captures, and we call it learned guidance, not neural guidance; guidance computed from another task gave 305 (inconclusive, interval 0 to 32); the hand rule gave 278, which the guide beat (interval 8 to 89); and a guide trained without the test's benchmark gave 312, which beat the hand-set order (interval 34 to 126).

The comparison across caps needs care. With a 40-times smaller per-hole cap (the ratio of two sampled caps, not end-to-end compute and not a proven minimum for the hand-set order), learned guidance reached 320 correct outputs, against 329 for the hand-set order at 20,000 counted operations per hole, but the two answer sets are not the same: 598 of the 624 query attempt sets are identical and 26 differ, with 4 tasks correct only under guidance and 6 only under the hand-set order. In aggregate the hand-set order at 20,000 used 5,438,628 counted operations and 629.8 seconds of search CPU, against 295,993 and 115.3 seconds for learned guidance at 500: ratios of 18.4 and 5.5 between the two caps. These are aggregate ratios across budgets, not savings at matched answers, and not end-to-end savings. Guidance added no reach: at 20,000 the two orders gave 327 against 329, inconclusive (4 against 3 discordant tasks, interval −11 to 6; the protocol had predicted a tie). On the ARC-AGI-2 public evaluation both orders gave 2 correct outputs on 1 task at 20,000, with identical attempts on all 120 tasks; at 500 the hand-set order solved that task and the guide did not. That task is the one construction answered in the held-back-half test (Section 5.8). In two dimensions (ConceptARC and Mini-ARC, 315 outputs) at 500 counted operations per hole, guidance computed from another task did at least as well as the guide (48 against 45, with 43 for the hand-set order), so the guide's advantage over another task's guidance is one-dimensional (275 against 257) (descriptive). Two of the three pre-registered predictions were refuted. *Observation:* at a binding budget a learned order used much more of the step's reach. *Explanation:* the search order decides how much reach a fixed budget uses, and the learning is shallow. *Research implication:* spend the budget the order frees on a larger edit and selection language under the learned order; the claim that order matters only at binding budgets is falsified if the gain persists when the cap no longer binds. Why the learned guide tied a plain success-rate table is analysed post hoc in Section 8.2 (item 4). The guide's coarse features may not tell operation families apart, so a guide that reads a task's input and target grid pairs, together with the constructor's own analysis of each search block, is pre-registered against the hand-set order, the frequency table, the linear model, the earlier guide and a shuffled-specification control at the same budget, on generated tasks from operation families held out of training. It has since run once: at 125 counted operations it solved 1,083 of 1,128 held tasks, against 953 for the hand-set order, 881 for a frequency table and 1,018 for a linear model, and a guide reading only the constructor's own block features solved 1,076, not significantly fewer (pre-registered; audited; `experiments/spec_guide_20261108/reviews/POST_SCORE_AUDIT.md`): search efficiency at a binding budget, not new reach. Feeding the guide another task's specification cost 33 tasks (52 against 19). The loss was concentrated in a few edit types, and the interval clustered by edit type included zero (−6 to +81), so the pre-registered test that the guide uses the specification's content was not met. A linear probe on the encoder's summary recovered the generating edit type for 64.6% of held tasks, against 46.1% for the same architecture with random weights. The encoder learned edit-type information, but that did not make the block order measurably better than the step's own features. This is a step toward neural guidance for program synthesis, after DeepCoder [11], RobustFill [53], BUSTLE [127], CrossBeam [150], LambdaBeam [151] and neural-guided deductive search [79]. It is not a transformer that synthesises programs.

5.6.2 Strong static baselines and learning inside a fixed language

A parallel series of controlled pilots, verified by a read-only review, tested learned controllers against static baselines on generated, authored and public training tasks (Table 30, Appendix B.4). In every pilot a strong static search or constraint baseline matched or beat each learned controller, and failure-blind controls tied failure-conditioned ones; learned components helped only when they re-ranked inside a fixed program bank, never when they pruned search. Five further architecture families for learning inside a fixed language advanced none, with 0 on the public evaluation throughout (Table 31, Appendix B.5); blind repair matched evidence-directed

repair on 40 fresh source puzzles; a guide trained on frozen features of a 4-billion-parameter open language model trailed the hand-designed order at equal budgets (198 against 306 at 500 candidates); failure-derived pruning lost nothing and saved no CPU (ratio 0.98); and the companion study's learned revision ranker lost its pre-registered primary comparison to streaming static search (8 against 12 of 16 held grid tasks).

Learned hard routing removes branches that strong static search needs. A learned failure controller that could only prune relative to the static condition solved 33 of 128 grid queries against static search's 87, and 14 of 128 graph queries against 72 (Table 34); in every controlled pilot a strong static baseline matched or beat the learned controllers (Appendix B.4). Pruning can only remove what the static search would have explored. It opens learned guidance that reorders rather than removes branches, the only way learned controllers helped in the pilots.

Neural proposers did not beat static or failure-blind controls. Our language-model inventor's failure reports did not beat failure-blind invention (2 against 2 of 11 tasks), matched search in the starting language recovered more (5 against 2), and given each removed operation's exact input and output behaviour it recovered 1 of 11 tasks: the binding limit is the 4B proposer's ability to implement a grid operation (Section 5.5.1). No proposer produced a program that reproduces the demonstrations of any of the first 20 public evaluation puzzles (Appendix B.6). The companion study's learned revision ranker lost its pre-registered primary comparison to streaming static search (Appendix B.5), and its source-only neural proposals for the bioimage grammar failed their source criterion (the companion study's records; available from the authors on request). Where neural guidance meets the verifier, verification is safe but finds little: over the neural comparison entry's neural pool a frozen contract-based selector changed 8 attempt-2 outputs and kept 43 of 172, 0 gained and 0 lost, because the pool holds at most 3 more correct outputs (Section 5.3). These taught us that writing the operation is the neural proposers' binding step, and that verification adds little when the pool lacks the answer. They open proposers that hand a typed specification to a symbolic constructor, and construction of outputs outside the pool.

Logic-programming baselines, executed (development samples). Because CORA's typed programs are tree-shaped, like Lisp expressions, we also compared it with executed logic programming rather than a simulation. Popper [46], run in SWI-Prolog on a restricted encoding of CORA's own object features and relations, solved 1 of 5 encodable exposed public training puzzles against CORA's 3 of 5, and 9 of 24 and 25 of 59 generated samples (18 of 24 and 26 of 59 with a relation added after the fact). A constraint-logic completion backend produced the same solution lists as CORA's enumerator on all 1,082 sketches both completed, and all 263 solutions it returned fitted in CORA's executor, but on a harder replay it added no new demonstration fit and no new exact prediction. These are small development samples with unequal budgets and encodings; they show that the language is executable in logic programming and give neither learner a general advantage. CORA's typed abstract syntax is Lisp-like, which is not a literal Lisp benchmark.

What Question 4 established. *Observation:* the only learned component that beat its hand-set comparator, a search order, did so at a binding budget and added no reach; in every controlled pilot a strong static baseline matched or beat the learned controllers, a controller that could only prune solved 33 of 128 queries against 87 for static search, and no learned revision ranker or neural proposer beat its static or failure-blind controls. *Explanation:* inside a fixed language, guidance can only reorder or remove what the static search would explore, so it helps where the budget binds and hurts where it prunes. *Research implication:* learned guidance should reorder, never remove, and should be tested where static search cannot cover the space; the claim is falsified if a learned controller that removes branches beats static search at matched work on held tasks.

5.7 Question 5: revising the representation or the language

Figure 3 showed a failure whose repair needed a new representation, a new binding and a new operation at once. The tests below ask how far such revisions have been made, by hand and by the system, and what they bought.

One exact new executed-program prediction (on a puzzle that shaped the design). Guarded relational rewriting answers both query outputs of puzzle 62593bfd with `(rewrite view=mono8 [size_unique == False -> MOVE up until_blocked] [TRUE -> MOVE down to_border])`, in words: on single-colour 8-connected objects, move up until blocked each object whose size another object shares, and move every other object down to the border. No CORA-only configuration had answered it, the neural model's whole candidate pool misses both outputs, and the controls with one fixed view, with every guard set to true, or with the plain typed language find no fit. The trace credits the representation (8-connected single-colour objects on the majority colour; background-0 views merge the background), the guard (a boolean feature partitions the movers) and the schedule (sequential moves with destination modes). The 8 fitting programs share one behaviour; the attempts are identical under two hash seeds; and rebuilding without each demonstration in turn reproduces it (2 of 2). It is development evidence on an inspected puzzle that also shaped the design: the move constructor implements a recommendation that the review derived partly from this puzzle. General symmetry completion fits 981571dc (correct, but already the neural model's answer) and 0934a4d8 (8 of 27 test cells undetermined): no new exact output. On Kaggle's slower CPUs CORA-CAST version 1 does not reach the 62593bfd answer within its cap (Appendix A). This taught us that a time cap makes an answer depend on the machine's speed. It opens budgets counted in program executions rather than seconds.

A representation defect found by review. An independent read-only review found that six of CORA's seven views, and its original symmetry completion, assumed background colour 0, while 96 of the 120 public evaluation puzzles have a non-zero majority colour in some grid (Appendix B.7, item 1). Taking each grid's majority colour as background is the representation that the 62593bfd program needs, yet alone it gave no new fit: a representation fix enables programs without creating fits by itself.

5.7.1 Guarded rewriting on source puzzles, in development and in one sealed confirmation

Development sets. On two sets of unselected source puzzles (88 each), guarded rewriting answered 10 of 176 puzzles, the same actions with every guard set to true 3 (7 gained, 0 lost; exact two-sided McNemar $p = 0.016$), one fixed view 7 (3 gained, 0 lost), and CORA-CAST's plain typed language 3 (9 gained, 2 lost: different puzzles). Answered puzzles include a size table, crops to the smallest or the unique entity, deletion of the smallest, recolouring of holed objects, recolouring from the adjacent object and sequential moves. Contracts pruned without losing a program in any audit and cut executions 28 to 36 times; at a fixed execution budget they answered 37 against 29 of 60 generated queries, and equally on real puzzle sets. Executable bindings solved 120 of 120 generated legend puzzles against 0 for frozen tables, but reached no real puzzle. This taught us that the generated legend puzzles did not capture what real legend puzzles need. It opens bindings built and tested on real puzzles. All CORA-CAST screens together cost 3.40 CPU hours and no GPU (`experiments/cora_cast_20261026/M2_REWRITE_REPORT.md`, `experiments/cora_cast_20261026/ABLATIONS_AND_COSTS.md`).

The sealed confirmation. One run, authorized in advance, opened the campaign's 207 sealed source puzzles after an integrity audit (partition hash verified; no ledger row or run log had read it; the frozen snapshots reproduce their recorded decisions exactly) under a protocol frozen before any outcome (`experiments/cora_cast_20261026/protocols/CONFIRMATION_PROTOCOL.md`, sha256 357c9379).

condition	puzzles solved, of 207	query outputs, of 228	CPU s
original-CORA baseline (CORA-CAST's plain typed language, depth 2)	4	4	230.8
CORA-CAST without guarded rewriting (guards set to true)	6	6	240.8
frozen CORA-CAST core (symmetry completion, guarded rewriting, typed search)	8	8	253.7
matched extra search (no guards, plus depth-3 search at the core's compute or more)	7	7	452.6

Table 14: The single confirmation on 207 sealed source puzzles. Primary comparison: the core against no guards, 2 gained and 0 lost, exact McNemar $p = 0.5$, bootstrap 95% interval of the difference, clustered by puzzles that share a grid, 0 to 2.6 points. Against the baseline: 4 gained, 0 lost, $p = 0.125$. Against matched extra search: 2 gained, 1 lost, $p = 1.0$. Verdict under the frozen rule: inconclusive, since the gain did not reach the pre-declared significance level.

The pre-declared rule required more gains than losses against no guards with $p < 0.05$ and a strict count at least that of matched extra search; with every discordant puzzle in its favour, $p < 0.05$ needs at least 6 of them. The direction held (Table 14), but the effect is smaller than in development and the test has little power at this size, so the guard mechanism is *not* confirmed. It taught us that the development effect was smaller on sealed puzzles than this test could detect. It opens a sealed test sized to that smaller effect. Both primary gains are hole-count programs (a table keyed by holes; a recolouring guarded by `has_hole`). Matched extra search answers one puzzle that the core misses. Under the confirmation procedure the core made 3 fitting but wrong attempt-1 predictions (11 puzzles attempted, 8 right). The deployed solvers apply stricter acceptance rules (Appendix A); we do not assume, and do not claim, that they remove these errors. The whole run cost 0.13 CPU hours. Nothing was tuned or rerun afterwards.

Representation growth and a diagnosed extension. The companion study grew eight families of representation (object relations, typed crops, canvas filters, object maps, role and palette maps, object correspondence, crop-and-palette roles and multi-object layouts); no grown representation passed its sequence of gates, and none beat its shuffled, random or static control (Appendix B gives the counts).

Growing expressibility from a diagnosed gap (the third step; its first test is complete and null). The companion study's diagnosis of the natural-task nulls found that the fixed construction language has no identity geometry, so it has no structure for an unchanged shape followed by a general shared palette map, and a constructor for *identity geometry, then palette map* was built (the companion study's records; available from the authors on request). The test, a one-use comparison that passed independent review and a post-run audit, added this extension to the language and pre-registered: *reachability*, the share of support-only folds and tasks in which some program reproduces every support demonstration, before and after the extension; the *gap type*, shape-feasible against colour-binding conflict; a matched control, a random extension of the same size at equal work; and only then held-query correctness on the newly reachable tasks. *Pre-declared success criterion:* a reachability gain from the diagnosed extension beyond the random extension, which would support the third step of the loop; it would not be learned-library transfer. *Result:* on the 120 public evaluation tasks (358 support-only folds) all 13 language variants, the extension included, fitted 0 tasks and 0 folds; on the 1,000 training tasks (999 eligible, 3,222 folds; development only, reachability only) the base language fitted 17 tasks, the extension 18 and the random extension 17, and the one task the extension added lies in an excluded development partition, so the criterion, at least six task wins, failed. No query was predicted. This is a natural-task coverage null for this hand-supplied extension, with one development-only diagnostic positive: it did not close the measured reachability, and other extensions and binding rules remain untested (the companion study's records; available from the authors on request). It taught us that the measured gap is wider than one supplied structure can close.

A wider view bank, a coverage sweep and an early-stop defect. A supplied bank of 22 reversible object views solved 9 of 9 authored cases against 4 of 9 for the three original-style views, and the full bank in one fixed shuffled order also solved 9 of 9; the engine's author wrote all nine cases, five of them built to need an added view, so this checks that the added views run, not how often they help. A fixed hand-written score that searched only its two best-scoring views solved 3 of 9, and it chose the same views when the demonstration outputs were shuffled. A frozen sweep then compared the three original views, the full bank in a fixed order and the full bank in random order on the demonstrations of all 1,000 exposed training tasks, an endpoint of demonstration coverage only, never accuracy: the full bank fitted the demonstrations of fewer tasks (2 against 8 of 1,000, with six losses and no gains). Source analysis showed that this comparison measured a defect, not the views. The search ended the whole search of that configuration whenever one view of one grid held more than 64 objects, a limit of that representation, as if the shared computing allowance were spent. Because the view that treats every cell as an object exceeds 64 objects on any grid larger than 64 cells, the full-bank search stopped after at most 23 of its 1,871 candidate programs on 764 of the 1,000 tasks (the first view to exceed the limit, in the listed order, was the single-cell view on 396 tasks, fixed-size tiles on 323 and a single-colour object view on 45), before any move or recolouring was tried, and the three-view search stopped at its first candidate on 45 tasks. All six lost fits are among the stopped tasks; this follows from the saved counts and the deterministic search order. A corrected rule refuses only the oversized view and keeps the executor, the 22 views and every limit unchanged. On invented grids the unchanged search stops where the corrected one fits, and the minimal correction gives identical results wherever the old search did not stop (120 of 120 invented task-and-configuration rows); the

existing corrected wrapper is not a drop-in replacement, because it also changes work accounting, identity scheduling and the handling of tied backgrounds. It is an implementation repair, not a new capability; no corrected natural run has been made, so no coverage or accuracy change is claimed. *What it taught us:* a limit placed on one representation must refuse that representation only; when it ends the whole search, adding views can remove reach instead of adding it, and the small authored grids (at most 49 cells) could not reveal it. *What it opens:* a corrected rerun on the same 1,000 tasks with the same limits that reports fits, clean no-fits and budget stops separately; the added views increase natural coverage only if the corrected full bank fits more tasks than the corrected three views. On an invented 30 by 30 case the corrected full-bank search used its whole 4-second allowance before reaching the fitting program, so budget stops may dominate large grids. Such a rerun would be exploratory, because the correction was designed after these outcomes.

The real ARC-AGI-2 evidence audit. An audit of every completed public-evaluation study looked for one complete causal case: a diagnosis made before the answer, an automatic revision, a program fitting the demonstrations, a correct committed answer, and a comparison with the strongest full-bank and shuffled-failure controls. None was found. The audit recovered 172 incomplete records for 120 tasks, with zero complete causal certificates and zero saved verified neural programs; the neural model's first attempt alone was right on 33 of 172. Our own audit of the same records adds one partial case: the held-back-half test's 2 of 172 outputs are the two queries of one task, and the run's log shows that the fit came from the construction branch, which runs only after the base search has failed. It is a descriptive, previously exposed construction example: construction answered, while the static comparator and the original language abstained. It is not evidence that the operation was necessary, and not a selection result, since the kind of revision was fixed in advance and the task had been examined during design. It adds no answer beyond the scored entry: its first attempts are the CORA-only entry's own first attempts on that task, and across every completed CORA study on the 120 public evaluation tasks, CORA's programs are right on the same three outputs from two tasks; before the guarded-rewriting module existed, no CORA-only configuration solved any of the 120. Of those three, the neural comparison entry was also right on two in both of its rehearsals; the third, the first query output of 62593bfd, is absent from its whole candidate pool. The other positives are fixed-policy effects: inserting the answer of a CORA-constructed program as the second attempt raised an older, weaker neural candidate pool from 17 to 19 correct outputs of 172 (22 to 24 under likelihood ranking); both gains are the two outputs of that same task, whose answer was absent from the whole neural pool, and the mechanism was not confirmed on 207 sealed training-set tasks (8 against 6, $p = 0.5$). On the neural comparison entry's own local offline rehearsal the count stayed at 43 of 172 with and without CORA's step, which proposed no second attempt on the 119 tasks it finished and reached its 15 CPU-second limit on that task; putting single-action candidates into that pool displaced one correct neural answer (43 to 42). *What it taught us:* the components exist (representation change, binding repair, composition and proposed reusable operations), but evidence that choosing the right revision after a failure causes a correct real answer does not. *What it opens:* the prospective causal comparison of Section 8.1, whose design has passed review as a draft only.

What Question 5 established. *Observation:* hand-built revisions of representation and binding (background-adaptive views, guarded rewriting) produced the only new exact public-evaluation program prediction and raised development solves (10 against 3 of 176), but the sealed confirmation was inconclusive (8 against 6 of 207); no grown representation passed its gates, and the one diagnosed extension fitted 18 training tasks against 17 for the base and 17 for a random extension of the same size; a supplied bank of 22 views made 9 of 9 authored cases solvable against 4 of 9 (its 1,000-task demonstration sweep measured a search-stopping defect rather than the views), while a fixed hand-written view score solved 3 of 9, and no real ARC case shows a revision choice causing a correct answer. *Explanation:* a representation change helps only where it adds reach, and one supplied structure did not close a gap that natural tasks show to be wide. *Research implication:* measure the newly reachable tasks of each candidate representation before any search, against random changes of the same size; Conjecture 2 is falsified if representation changes chosen from failure records do no better than random ones.

5.8 Question 6: are acquired capabilities reused?

A capability is worth acquiring only if it is used again. Within the constructed families it was: each of the four promoted operations of the construction test solved 6 of 6 evaluator-only tasks of its family, the merged

parametric upscale solved 6 of 6, and no promoted operation was rolled back (Section 5.5). On held related tasks it was, within one benchmark's family structure; across families and on two-dimensional concept tasks it was not.

Learned operations are not reached or selected on natural tasks (an audited diagnosis, not a transfer or score claim). The companion study tested whether a library learned by CORA's grammar-growth loop helps on 924 previously examined public ARC-style tasks (1,244 outputs from ConceptARC, Mini-ARC and 1D-ARC). A frozen learned library was compared with no library, three random libraries and a fixed library: every condition gave identical predictions, 97 correct of 1,244 outputs (116 answered), so each pre-declared contrast had effect 0, the scorer labelled the test underpowered, and no learned operation was ever selected. On the 120 public evaluation tasks the same six conditions also gave grid-identical predictions, 3 of 172 correct in each, with support reachability 4 of 120 in every condition: zero paired gain, labelled underpowered, and descriptive, since the set was inspected before; it is not learned transfer (the companion study's records; available from the authors on request). On the 1,000 training tasks the same six conditions, as development reachability and unscored, found bounded support fits for 88 to 90 tasks each, tied at 16 of the 176 tasks of the selection partition. A follow-up that interleaved the library into the search order, on 162 source tasks, gave 11 of 168 queries in every condition with no learned program selected; it is descriptive, because its design followed earlier observations. A fit probe found no promoted operation that fitted the demonstrations of any of the 162 source tasks (0 of 162). The grammar-growth mechanism works on its constructed test (8 of 11 tasks against 0 with another task's specification, Section 5.5.2), but its learned operations are not reached or selected on natural tasks: that program coverage and selection gap is the next problem (the companion study's records; available from the authors on request). These results are kept apart from the official Kaggle scores and from the construction test. Through the loop of the introduction they are a diagnosis, not a dead end: growth within the fixed space recombines structures it already has, and natural tasks need the language's expressibility itself to grow; the first hand-supplied one-structure extension did not close the measured reachability either (Section 5.7). It opens growth of expressibility from natural failures, which the object-level step now tests.

Learned library on the public evaluation. The six arms differed only in why the bounded search stopped: without a library, 107 tasks ended in bounded depth-two no-fits and 9 at the execution budget; with the learned library, 73 and 43; with random or full libraries, about 70 to 71 and 45 to 46. These are bounded-search diagnostics, not proofs of inexpressibility: the library spends search budget without adding reach, the utility problem of learned control knowledge [112]. A development-only companion run on training tasks found 88 to 90 fits across arms. In a separate study, libraries learned from solved programs compressed 21 to 26 of 63 held-out programs, but only from a median of 28 nodes to 27, and the number of programs within the search's reach stayed at 2.

The held-back-half test: reuse and construction on held related tasks (independently audited). A pre-registered study grew operations on the source halves of three related benchmarks (ConceptARC, Mini-ARC and 1D-ARC) and ran once on the held halves, 464 tasks with 624 query outputs, with a check on the 120 public ARC-AGI-2 evaluation tasks (172 outputs). *Held* means held out from this study's development only: every held task had been run and scored once on 2 November 2026 by the submitted entries, and once by the companion study's earlier evaluation, so none is unseen. The primary contrast, reuse of the learned object operations against matched static search with the same object cost allowance, gave 227 against 118 correct outputs, +109 with a family-clustered 95% interval of 7 to 204; one task position that the aggregate does not record moves the bounds by at most one, to 6 or 7 and 204 or 205, and cannot change the decision. The gain is a 1D-ARC result. On 1D-ARC (309 outputs) reuse gave 216 against 88, +128 (interval 28 to 222); on ConceptARC (240 outputs) static search won, 9 against 27, -18 (interval -30 to -6), in eight concepts; on Mini-ARC (75 outputs) the reuse registry was empty, and the count was 2 against 3. Task-local construction from typed holes computed from each task's own demonstrations once its search failed got 343 of 624 right (1D-ARC 286, ConceptARC 46, Mini-ARC 11), against 118 for its matched static search in the original language, +225 (interval 150 to 298), and +333 over construction from donor demonstrations. On ConceptARC and Mini-ARC together, reuse got 11 of 315 right against 30 for its static comparator, while construction got 57 right against 30, descriptive arithmetic rather than a declared endpoint. The learned program library got 37 right against 6 for the fixed language but lost to its own matched static search, 37 against 56 (-19; interval -38 to -3; Holm-adjusted $p = 0.018$); static

search reproduced all but 3 of the library's tasks and solved 17 more, and 22 of the library's 37 came from one family. On the exposed ARC-AGI-2 check, construction got 2 of 172 outputs right, both query outputs of task 62593bfd (interval 0 to 6, inconclusive), and every other arm none, including its matched static search, which ran 1,339,667 program executions on that task (63.1 CPU-seconds, against 8.3 for construction) without a fit. The run's log shows that the fit came from the construction step, which runs only after the base search has failed on the demonstrations. The overlap with the scored entry is complete: the first attempts equal, cell for cell, the CORA-only entry's first attempts on that task, which its 1 November score record marks correct, and the same task gave the older gain from inserting a CORA-constructed answer as the second attempt of a neural candidate pool and the learned-search-order study's only ARC-AGI-2 answers. A design review analysed this task, and the entry's guarded-rewriting module implements that review's recommendation of moves by destination. Construction therefore reached automatically, after a failure, answers the entry already had: the union over CORA's studies stays 3 of 172, and this is neither superiority nor a new score. *Scope:* the static comparators searched the original language with extra CPU rather than the same expanded bank, and there was no shuffled-failure arm, so the study supports bounded construction and reuse comparisons; the same-capability test of diagnostic selection is still missing. *Deviations, disclosed:* the run was a local execution takeover on our CPUs instead of the planned private Kaggle kernels, its go-ahead was written under the companion's one-use authorisation, and the frozen amendment kept two result placeholders. Execution took 8.71 wall hours and 13.63 CPU hours. The companion's terminal review passed 182 of 182 metadata checks, and our own post-score audit passed, with 33 custody checks and every statistic reproduced; nine secondary intervals cannot be recomputed without rescoring and are reported as the frozen report gives them. *What it taught us:* learned object operations transfer within 1D-ARC's family structure but not to two-dimensional concept tasks, where static search wins; and our library of whole programs found by search stores successes, so here it shortened searches that the same CPU already completes and bought speed, not reach. *What it opens:* richer two-dimensional views combined with construction, which is the open question of choosing views (Section 8.2); and an admission rule for libraries: admit an entry only if, on source tasks it was not built from, it solves tasks that static search given the entry's discovery cost does not.

Library rivals: operations kept from failures against compression and random libraries (pre-registered; scored once; audited). A pre-registered comparison repeated the object-reuse arm above against libraries of the same size built by compressing solved programs and at random. Its protocol and freeze passed a pre-run review, a re-check and a delta check before the go-ahead; it ran once, detached, in one CPU slot; and its audit passed with notes on the result record's wording only: custody is intact and every number is reproduced from the per-task table (`experiments/library_rivals_20261108/reviews/INDEPENDENT_POST_SCORE_AUDIT.md`). Its exact sentences report it.

1. On 464 held tasks (624 test outputs) from ConceptARC, Mini-ARC and 1D-ARC, held out from the libraries' construction but scored in earlier studies, we added three kinds of library of the same size per benchmark (2, 0 and 19 entries) to the same base program search: operations kept from failures on the source halves, a single-step compression of solved programs, and random whole-grid operations (drawn twice). Each entry entered as one operation of equal cost, under the same budget of 150,000 primitive executions and two attempts.
2. The failure-kept library produced 227 correct outputs against 28 for the compression library. 218 tasks were right only under failure-kept and 22 only under compression (exact McNemar $p < 0.000001$; family-clustered 95% interval for the difference of 199 outputs: 89 to 294).
3. Both random libraries scored 6, the same as the base search alone on every task, and none of their entries appeared in any program the search found.
4. By benchmark, failure-kept scored 9 of 240 outputs on ConceptARC, 2 of 75 on Mini-ARC and 216 of 309 on 1D-ARC, against 4, 2 and 22 for compression and 4, 2 and 0 for the base search alone.
5. Failure-kept led compression by 194 outputs on 1D-ARC (interval 83 to 289), and the ConceptARC difference of 5 outputs was inconclusive (Holm-adjusted p 1.0). Mini-ARC, where no library has entries, is a tie by construction.
6. Compression's only gain over the base search was 22 tasks of the 1D-ARC flip family, where the failure-kept library got nothing right, so in this setting the two libraries are complementary.

7. Read descriptively, this is a low-success regime. The source halves offered few solved programs to compress (on 1D-ARC, 23 of 306 tasks, only 2 distinct programs, all found with four times the base budget), while on the same 1D-ARC source half the failures still supplied specifications from which the builder admitted 19 operations, 12 of which fired on held tasks.
8. These results have firm limits: the failure-kept arm repeats an earlier study's object-reuse arm, whose predictions it matches on all 464 tasks; the design was written knowing that study's held results; construction work was not matched (about 9,300 CPU-seconds for the 1D-ARC failure library against seconds for the compression); the compression is a single step with no holes; the failure material cannot be separated from the object-level operation space; and this is not a general head-to-head of library-learning methods.

In short, in the audit's words: on 464 previously used held tasks, a same-size failure-kept library gave 227 correct outputs, a single-step compression of solved programs 28 and random libraries 6, under the same matched search; compression's only gain was one family the failure-kept library missed. Not a general method comparison.

What Question 6 established. *Observation:* promoted operations were reused within their own family and nowhere else; on held related tasks, reuse beat matched static search within 1D-ARC (216 against 88) and lost on ConceptARC (9 against 27), and the learned program library lost to its static comparator (37 against 56); a frozen learned library changed no prediction on 924 previously examined public ARC-style tasks or on the 120 public evaluation tasks; compression shortened 21 to 26 of 63 held-out programs only from a median of 28 nodes to 27. *Explanation:* in these tests our compression and recombination shortened programs that the search already reached and added no operation the language lacked; this is one test of our own compression, not a result about library learning in general, which can raise what a bounded search reaches (Section 2.2). *Research implication:* measure reuse on held natural tasks of operations grown from natural failures, and merge task-local operations into parameterised ones; reuse is falsified as a source of progress if grown operations are not selected on held tasks of the families that produced them.

5.9 Question 7: does reading the failure choose the change? A pre-registered test

In terms of the core idea: if a failure is evidence about what to change, a system that reads it should choose the right change more often than one that does not, at the same cost.

Questions 3 to 6 measured availability and discovery. This test isolates selection: every arm had the same two changes available and the same budget, and the arms differed only in the order in which they tried the changes. So the decisive test varies only the order of the same changes at equal cost, where order can matter; there, failure-blind or shuffled orders doing as well would refute reading the failure. The study was frozen and reviewed before its held run, scored once, and audited after scoring by the reviewer who wrote its pre-run review; the audit passed, every number of its analysis reproduced, and the pre-registered outcome "none" stands (`experiments/failure_choice_20261108/reviews/POST_SCORE_AUDIT.md`). Its seven allowed sentences report the result.

1. In a pre-registered test on 50 held controlled generated tasks (transformation tasks built from ARC training input grids, not ARC puzzles), a router that reads CORA's failure record gave the same answers as a static bank of the same two changes in a fixed order on every task: at the binding budget of 3.5 million counted primitive executions both solved 30 of 50, as did an oracle told each task's cause (exact McNemar $p = 1$); the pre-registered outcome was none.
2. Failure evidence beat a random order of the same changes (30 against 20 tasks; 10 gained, none lost; Holm-adjusted $p = 0.006$) but not the best fixed order.
3. The reason is in the bank: typed-hole construction rejects an inapplicable hole after about 900 calls (897 to 913 on all 18 held depth tasks), so trying it first costs almost nothing, and the best fixed order already matches the oracle at the binding budget and above.
4. The bank's advantage over spending the same budget on more search pointed the same way as on the source set (10 against 2 tasks) but did not meet the pre-registered rule ($p = 0.039$; family-clustered interval -2.2 to $+34$ points).

5. At a quarter and a half of the binding budget, extra search alone solved more tasks than the bank and the oracle (9 against 7, and 19 against 15), because construction applied but could not finish within the budget; no router in this test used the budget.
6. A learned router trained on the source families did worse than the fixed order (27 against 30; three tasks of one held family sent to search first and lost; Holm-adjusted $p = 0.5$).
7. Choosing what to change can pay only when a change cannot cheaply check whether it applies, or cannot finish within the budget when it does; a proposed second revision would add an object-view search that runs to its cap when it does not apply.

Sentences 1 to 6 give the measured result, and sentence 7 the lesson and the direction it opens. The learned router of sentence 6 is a logistic regression. The tasks are controlled generated tasks, not ARC puzzles, so nothing here is ARC accuracy. The test's full-bank control contract and its checker, a way to test any system's claim to learn from failure against a fixed controller with the same changes, are public in the release at commit 8ec11940 (tag `cora-arc2026-public-2026-11-08c`): `experiments/failure_choice_20261108/fgc/contract.py`, with its checks in `experiments/failure_choice_20261108/checks/CONTRACT_CHECKS.json`.

Revision 2: the decisive selection test (pre-registered; scored once; audited). Revision 1's lesson named the change to add, and revision 2 added it: a search over alternative object views, which cannot cheaply check whether it applies. Its protocol and freeze passed a pre-run review and a delta check before the go-ahead; its 84 held tasks were generated once after the go-ahead in a private Kaggle kernel, measured once in four private kernels, sealed and scored once; and the audit after scoring, by the reviewer who wrote the pre-run review, passed: custody is clean and every number of the result record recomputes exactly (`experiments/failure_choice_r2_20261108/reviews/POST_SCORE_AUDIT.md`). Its eight allowed sentences report the result.

1. In a pre-registered test on 84 controlled generated tasks (transformation tasks built from ARC training input grids, not ARC puzzles), each with one injected cause of failure, a fixed router that reads one fact from CORA's failure record, whether every demonstration keeps its grid size, solved 55 tasks at the pre-declared binding budget of 3,000,000 primitive executions, against 29 for the static full bank in its pre-registered order (26 tasks against 0; exact McNemar $p = 3.0 \times 10^{-8}$; 95% interval over resampled task families, $+11.8$ to $+55.2$ points).
2. The whole gain came from the 28 wrong-object-view tasks, whose fix, a search over alternative object views, cannot cheaply check whether it applies; at every budget the router solved exactly the tasks solved by an oracle router told each task's injected cause.
3. The router also beat its controls: the same rule applied to another task's failure record (18 tasks against 0) and a random order of the same bank (22 against 4), both holding after Holm adjustment; a router learned from the source families only solved 54 of 84 at the binding budget.
4. The advantage shrank as the budget grew, from 26 tasks against 0 at the binding budget to 3 against 0 at twice it, as predicted before the held tasks existed.
5. Against the best fixed order chosen after seeing the held results (construction, then the view search, then deeper search: 44 of 84), the router's advantage is not established: 11 tasks against 0, with an interval from 0.0 to $+26.1$ points.
6. The size fact separates the causes because of how the tasks were generated, and it comes from the demonstrations rather than from the failed search, so the result shows that choosing the order pays when the evidence separates the causes, not that CORA's failure record does so on natural tasks.
7. The pre-registered static order was chosen on a source set with fewer view tasks and scored below a random order on the held mix (29 against 37), and two of the three held causes reuse the previous revision's held families with fresh input grids.
8. Where one change can check cheaply whether it applies, the best fixed order matched the oracle and choosing added nothing (revision 1); where a change cannot, no fixed order of the bank reached the oracle (best 44 against 55 of 84), and the pre-registered router did (revision 2).

Sentence 1 is the pre-registered primary result, sentences 2 to 4 its mechanism, controls and budget, sentences 5 to 7 its limits and sentence 8 the condition across both revisions. The router read only whether the demonstrations

keep their size, so nothing here is evidence that the failed search guided the choice, and nothing is ARC accuracy, an efficiency finding or a learning result. *Deviation, disclosed:* the Kaggle login expired while the generation kernel ran; the kernel kept running and was never pushed again, so generation ran once.

5.10 What the evidence says, in order

The results, read through the three requirements. Discovery moved inside supplied spaces, and a larger gene-rule language raised coverage. Typed holes computed from the demonstrations once search failed found missing operations reliably on controlled tasks (64 against 38 of 95), and post hoc, 13 of the 31 tasks only construction solved had no equivalent program in the starting language at any depth searched (not proof that none exists deeper); reused operations got held related tasks right within their family (227 against 118 of 624, all of the gain on 1D-ARC); task-local construction got 343 of 624 right against the same 118; and a learned order converted a binding budget into correct answers (320 against 234). Selection changed what was solved only on generated tasks where a change cannot cheaply check whether it applies, and only against the pre-registered static order and the shuffled and random controls. Wherever a fixed alternative had the same capabilities and evidence and every change could check cheaply whether it applies, it matched the diagnosis-driven controller (219 against 219 authored Boolean functions), though the fixed controller used 12.9% less counted work (descriptive, not evidence of learned or causal selection), all of it the schedule's one failed build attempt per built function; in the pre-registered order test, failure evidence beat a random order but not the best fixed order (Section 5.9), and in its second revision no fixed order of the bank reached the oracle (best 44 against 55 of 84) while the pre-registered router did, an advantage not established against that best order, chosen after the results; a fixed hand-written view score that searched only its two best-scoring views solved 3 of 9, against 9 of 9 for the full bank; and no real ARC case shows a revision choice causing a correct answer. By the rule of Table 6, the ARC evidence reaches at most "new capabilities help; selection unresolved", on exposed or constructed data.

Guidance changed which answers a budget found, inside what the language and its search reached: a bounded empirical pattern. In every test so far, evidence from failures and from inferred structure changed which answers a fixed budget found, inside what the language and its search could reach. On the 11 constructed tasks, growth from the system's own holes recovered 8 at the base budget, matched search 5, and blind enumeration 10 in some seed at five times the compute (Section 5.5.2); in the companion study's learned action selection on 128 invented tasks, the learned order equalled every control (111 of 128) and a whole-vector shuffled order found more (118 of 128); on 151 held Boolean-network tasks the grown grammar equalled the fixed template, with the right function among the candidates for 76 and absent from the bank for the other 75, so that failure population is not ranking alone (Appendix B.2); in the inferred-wiring test the gain came where the function space was reachable and the inferred structure guided the search inside it, with wrong accepts equal, so the gain is coverage (Section 6.2); and in the feasibility phase of our pre-registered natural-task growth test, a deeper search at four times the budget fitted 34 source tasks against 7 at the base budget, while growth from the failures' typed holes produced no candidate at all (Section 5.5.3). Every attempted change of language, the companion study's representation-growth versions 3 to 15 and the learned-library test, passed no full sequence of gates and gave no usable candidate on the public evaluation queries (Sections 5.7 and 5.8). This is an empirical pattern across the tests run, not a proof: a capped search that fails, or the absence of a known reference program, does not show that the language cannot express the answer. Where the question was settled by analysis rather than by search, it settled the order of the two limits: for the companion study's one-step constructor language, a reachability analysis of 842 ARC training tasks (a diagnosis of the language, not accuracy) found the grammar the limit with certainty for 754 tasks, perfect search inside it worth at most 22 to 88, and a grammar covering the reference solvers able to express a correct program for 219 to 333, while the existing bottom-up search reaches programs of 4 to 6 nodes against a reference median of 29 (Section 5.3). So the language must be broadened first, and the moment it is broadened the binding constraint becomes search: more budget in the present language moves little, and a reference-sized grammar needs a search that reaches programs of 10 to 30 nodes. The Boolean case is settled by proof as well: a target that depends on a regulator outside the variable pool is unreachable by construction, because every candidate is invariant to the omitted variable (Proposition 8; Section 6.2).

The tests above put the limits in an order. *Reach comes first*: no ranking or reordering helps without a fitting program (Questions 2 and 3). *Order comes second*: at a binding budget the search order decides how much of the reach is used (Question 4). *Acceptance comes third*: it trades coverage for precision, and on noisy data a uniform bar is either too strict or not selective (Question 1). Five findings bear on how anyone might reach

85% on ARC-AGI-2. *Representation, construction and selection are different bottlenecks* (Proposition 2). For CORA reachability is the limit: in all 113 audited public evaluation puzzles no demonstration-fitting program was generated within the measured budget, a limit of candidate generation, representation or search rather than proven inexpressibility, so no ranker or one-step revision over that memory can help, and the failure anatomy puts expressivity first in 29 of 46 sampled failures (Section 5.3). The same CORA-only entry that solves 48% of 1D-ARC's outputs within its input limit solves 2% of the public evaluation (Section 5.2). For the neural pool selection matters, but little is left: likelihood selection keeps 22 of the pool's 27 reachable outputs (Table 10), and over the neural comparison entry's own pool no selector can add more than 3 of 172 outputs. On the 41 outputs the neural comparison entry gets right and CORA does not answer, ten times CORA-CAST's budget finds no program, and the first gap is a missing action in 35 of the 36 tasks (Section 5.4; post hoc). It opens broader grammars with deeper search, before any work on selection.

Programs that fit the demonstrations generalize wrongly when the rule is outside the language. Then every fit is spurious, and leave-one-out removes only unstable fits: the counterexamples of Section 5.2 (3 of 11 unfiltered sealed answers, 7 of 7 of the companion study's symbolic candidates, 22 of 33 nuclei deletions) all fit and were wrong. Verification limits harm; it cannot add coverage, and CORA-CAST's public-evaluation answers, all right, are on puzzles we had inspected, so its precision on unseen puzzles is not yet measured. It opens construction where CORA abstains, measured on unseen puzzles.

Our compression of solved programs did not extend search reach. Library learning from solved programs compressed 21 to 26 of 63 held-out programs, but only from a median of 28 nodes to 27; programs within reach stayed at 2 and no condition gained a puzzle (Appendix B.5). This is one test of our own compression, not a general limit: compression leaves what a language can express unchanged and can raise what a bounded search reaches, as DreamCoder's held-out ablations show [57], and construction from typed holes acts on the same quantity. In the capability tests, matched search in the starting language reached a deep macro the inventor could not write (the self-extension follow-up), and a failure specification used descriptively less CPU on 11 constructed tasks, without more coverage (the construction test). It opens a comparison of construction from failure, compression of solved programs and static search on held tasks at matched total compute.

Guarded contracts cut execution cost only where they prune expensive work. Sound contracts (Proposition 11) let a depth-2 search finish within a fixed budget on every task of a development set (budget stops 32 to 0; 44,485 instead of 1,269,443 executions) without losing a fitting program, but the CPU saved is large only for expensive operators such as symmetry completion; guards lifted development solves, while the one sealed confirmation was inconclusive (Section 5.7.1). It opens a sealed test sized to the smaller effect seen there.

Acquired operations are reused within their family. In a controlled, pre-generated ablation, the construction test's four promoted operations, each a reference-equivalent structure from a fixed 211-structure language, solved 6 of 6 evaluator-only tasks of their own families, tasks generated from the removed reference operations and kept only where the reference succeeded, whose reference outcomes had been examined in earlier experiments, but no promoted operation solved a task of another family (the self-extension test), so what was acquired is reused where its family recurs; reuse beyond the family it was built for is open.

What carries forward is a method: grow the language where the failure analysis says, keep a strong static search alongside, prune only with sound checks, and answer only when verified. CORA alone now holds a non-zero leaderboard score (0.83%), but no gain beside a neural entry is attributable to it; the findings narrow where effort should go. In short, CORA is a working, original typed-program reasoner with an official CORA-only entry; it exposes and measures a particular failure mode in the systems measured here (CORA's original engine, CORA-CAST, the second solver, the companion study's constructors and the neural comparison entry's neural candidate pool): the failure to construct, or generate, the right candidate. Its failure-triggered hole constructor is a testable mechanism for growing a grammar, but a score or universal-transfer gain has not yet been established.

6 Beyond visual grids

In terms of the core idea: nothing in it is specific to grids, so this section runs the same pipeline, with a different adapter, on gene networks and on noisy biological data.

If learning what to change is a general problem, the same reasoning pipeline should apply wherever hypotheses are executable and checkable. *Transferability* here means that the pipeline carries over to another domain through an adapter: build an executable rule, check it against the examples, diagnose failures, revise or ask for more evidence, and abstain when unsure. It does not require a system that learns rules on puzzles

and applies them to genes. That narrower question, whether knowledge learned on ARC helps elsewhere, is secondary, and we report it as tested and not supported (Section 6.6). Biology sharpens the question, because there the rules are not given, the evidence is noisy and thin, and a predictive rule is not yet an explanation.

The biology, in plain words. A published gene-network model treats each gene as on or off and gives each gene a rule that sets its next state from the current states of its regulators, for example "Y turns on if A is on and B is off, or if C is on". *Recovering a rule exactly* means that from a few simulated before-and-after states the system writes a rule that agrees with the published one on every possible input state, not only on the examples it saw. These are simulations of published computer models, not laboratory measurements, and reconstructing a published model is not discovering biology.

Transferability, in plain words. What moved between domains is the procedure's design: build an executable rule, accept it only if it fits, and ask for informative evidence, implemented separately for gene networks; the one revision there, a completion rule, was made by researchers. A small two-input truth-table executor, identical byte for byte wherever it is used, runs on ARC grid panels and in later gene-state experiments; it was not used in the results below. The scored CORA-only code ran unchanged on 44 published gene-network models but answered no whole-network query and recovered no rule (Section 6.2); the gene results below come from separately written biology code, which reuses CORA's verify-or-abstain acceptance design and its step of asking for informative evidence, but none of the entries' code. The procedure transfers through adapters, and each domain grows its own grammar from its own failures where the evidence shows it: on generated grid tasks, in logic and in signalling the loop's own failure evidence beat the same loop reading another item's evidence or a shuffled failure description; in spreadsheets, chemistry and the ARC-AGI-3 probe it is not established; the bioimage grammar built from failures was empty, and on measured data nothing admitted helped. Learned content transfers within a family (227 against 118) but not yet between distant domains (177 against 168, 182 and 194). Within a family, operations kept from one half of three related grid benchmarks got 227 of 624 query outputs right on the other half, against 118 for static search in the original language, all of the gain on 1D-ARC (Section 5.8); between distant domains, an operator order counted from ARC recovered 177 of 595 published gene rules exactly, against 168 for plain index order, 182 for the same counts randomly reassigned and 194 for a hand-set simplicity order (Section 6.6). The same loop design was then carried, through separately written adapters, to held tests in logic, enzyme chemistry, spreadsheets, signalling and world models, and whether the grown grammar helped differed by domain (Section 6.11). Noisy continuous measurements are the next adaptation, a hypothesis with strict negative first results (Section 6.9).

Adapters, in plain words. An adapter is a translator between a domain and one shared reasoning core. It says what an object is (a coloured region, a gene), what an operation does (move a shape, evaluate an update rule) and what counts as a check (an exact grid match, agreement with an observed state). It supplies meaning, never the answer: withheld answers never enter an adapter, and sharing syntax alone does not carry meaning.

Evidence class: retrospective framework applicability. Every claim in this section names its mechanism and says whether the submitted entries run it. An audit of the submitted system (`records/NOV1_UNIVERSALITY_AUDIT_FOR_SUBMITTED_SYSTEM.md`) compared the entries' code (CORA-CAST: its two versions, the side code and the neural sources, 35 Python files) with the 236 files behind the biological studies, by file hashes and syntax trees: they share no file, function or class. CORA-CAST's own transfer check records biology as not run. What the two sides share is a design, implemented twice: deleting objects chosen by a guard on their own features, and a leave-one-out stability check (a hard check on each answer in CORA-CAST, a soft support signal in biology).

Where the pipeline carried over, first. On three related ARC-style benchmarks the CORA-only entry's verified answers were right on 81.5% to 95.8% of the outputs it answered, and at most 2.4% with another task's demonstrations (Section 5.2). On BIG-bench list functions the unchanged construction rule recovered 4 of 9 tasks against 1 for blind enumeration at matched CPU, an inconclusive edge, and its three promoted structures each agreed exactly with a deleted operation on the probes and queries (Section 5.5.2). On published Boolean gene-network models, a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code, reconstructed 101 against 66 of 595 published rules with inferred regulators (every exact recovery a one-input rule), in a pre-registered replication; 320 against 291 with informatively chosen observations; and, after a prediction loss, 2,200 against 2,153 of 2,380 held inputs

with a completion vote frozen before it ran, post hoc on a reused cohort (Section 6.2). Conditional construction recovered rules with three or more inputs, 433 against 321 of 595, which a classical learner equals by design. On held real benchmark logic functions the loop recovered 45 against 12 of 217 for a shuffled failure description, reconstructions of known functions (Section 6.11).

Positive signals first, then the open questions. The measured positive signals are: the submitted solvers ran unchanged on three ARC-style benchmarks, right on 81% to 100% of their answers, with precision collapsing under shuffled demonstrations, and 2 of 9 system and benchmark pairs passed the pre-registered transfer rule (the CORA-only entry on Mini-ARC, 19 of 20; the neural comparison entry's CORA component on ConceptARC, 14 of 15); the CORA-only entry's 295 correct 1D-ARC answers come 244 from its second solver and 51 from CORA-CAST, and did not pass the rule, because shuffled demonstrations still drew 250 answers; the same solvers ran on fresh nuclei images; frozen grid programs ran on attributed graphs; a retrospective single-cell application of a related design held its pre-set direction in 41 donors; the frozen bioimage procedure ran unchanged on three held-out image datasets (below); and a tolerant agreement rule admitted one bioimage operator in calibration (Section 5.2). Each negative is an open question with a measured cause and a next test: the strict rule suppresses noisy-image corrections because near-optimal fits disagree, and in the pre-registered bioimage transfer test it made no edit on any of three held-out image datasets (next: an admission bar tied to the target domain; a calibrated bar set on one dataset failed on a new source, Section 6.7); the acceptance check accepted an erase-everything program when demonstration windows held only debris (next: demonstrations that separate debris from nuclei); and no learned policy exists to transfer (next: a trained policy under the same pre-declared success criteria). Three claims are kept separate throughout. *Portability*: the submitted component executes in another domain. *Cross-domain usefulness*: it improves a result there. *Learned transfer*: a policy learned in one domain helps in another. Of the three, portability is established for the submitted component: it ran unchanged on one fresh public nuclei set (below), accepted no edit under the neural comparison entry's rule and improved nothing, and nothing in it is learned. A separately implemented CORA repair procedure with the same design ran on three public biological domains. Table 15 places every universality claim of this report on one scale.

Table 15: The claim hierarchy. Established: executable components, portability, and usefulness on similar problems (limited). Partly established: selection of new operations from a supplied space, in controlled settings (not open invention). Open: harm avoidance by the strict rule, cross-domain usefulness in imaging (partly established within Boolean networks, replicated prospectively), learned transfer and universal reasoning. Portability, cross-domain usefulness and learned transfer are separate claims. The open rows taught us that portability and abstention are not usefulness; they open the admission and coverage tests of Section 8.

claim	status	evidence, and its largest weakness
executable neural-symbolic components	established	the neural comparison entry runs NVARC's neural method with CORA-CAST's executed programs on attempt 2, and every CORA answer is a program executed on the puzzle (Appendix A); on the public evaluation the combination changed no output
portability: the submitted component executes in another domain	established	both solvers ran byte-identical, through the entries' own code, on 455 tasks from 23 nuclei images and on three public ARC-style benchmarks; separately written code ran frozen grid programs on attributed graphs (8 task clusters from 4 engineered Boolean families: 14 of 16 queries and 7 of 8 tasks, against 16 of 16 and 8 of 8); the frozen bioimage procedure ran unchanged on three held-out image datasets; the CORA systems ran on all 44 published Boolean-network models with no errors
the strict rule avoids harmful edits	open (observed only through abstention)	on one nuclei set the neural comparison entry's rule accepted no edit (F1 0.858 unchanged), which abstaining on every task would match; on the fresh fluorescence image set it accepted 11 deletions and 10 were harmful; in the bioimage transfer test it made 0 held-out edits, again non-degradation through abstention
usefulness on similar problems	established, limited	on three public ARC-style benchmarks the entries' components answered 3 to 50% of outputs at precision 0.815 to 1.000, and 2 of 9 system and benchmark pairs met the pre-registered rule; with another task's demonstrations precision fell to 0.03 or less, but answers persisted; a controlled follow-up on the same tasks found higher observed precision for the strict selective policy than for accepting every fitting program acceptance (+0.105), at lower coverage (Table 26)

claim	status	evidence, and its largest weakness
cross-domain usefulness: improves a held biological benchmark	open for imaging; partly established within Boolean networks (exact rule reconstruction with a demonstration-inferred prior, replicated prospectively)	neither solver improved F1 (a tuned intensity filter reached 0.867); the separately written repair procedure's gains (0.692 to 0.705; 0.883 to 0.923) were matched by failure-blind controls; in the pre-registered bioimage transfer test strict CORA made 0 held-out edits (pooled F1 equal to the calibration-tuned baseline on all three targets), while a compute-matched static search over the same operators gave a descriptive, unadjusted +0.0116 on MoNuSAC; on Boolean networks, no whole-network answer, and single-gene answers no better than the majority rule; a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code, with the rule language supplied, inferred each gene's regulators from its demonstrations and exactly recovered 99 of 333 published single-gene rules, against 85 with no wiring and 87 shuffled (148 with the correct wiring supplied, a ceiling), and the inferred-support step replicated prospectively on 75 models new to scoring experiments, from 50 disjoint lineages (101 against 66 of 595, 35 gains and 0 losses, every exact recovery a one-input rule; 222 with correct wiring), which is not the submitted solver's performance and not ARC-to-biology transfer; informative observations gave 320 against 291 of 595 exact rules and, after a completion vote frozen before it ran, 2,200 against 2,153 of 2,380 held inputs (post hoc, reused cohort); conditional construction gave 433 against 321, which a classical learner equals by design
held tests in new domains, through separately written adapters	partly established for logic functions and enzyme reactions; a null for spreadsheets; narrow for signalling; not passed for world models	45 against 12 of 217 held real benchmark logic functions for a shuffled failure description (reconstructions of known functions; 153 recovered by no method); 76 of 200 held substrates against 33 for the best fixed published rule, with a classical nearest-neighbour method at 75 (reconstruction within families, not discovery); 22 against 17 of 53 spreadsheet tasks, not significant; on synthetic signalling tasks a result limited to reconstructing one removed prior edge; on 12 held public ARC-AGI-3 games the pre-registered test did not pass (probe results on public games, not a competition score); Section 6.11
learned transfer: a learned revision policy improves cross-domain adaptation	open	no revision policy was trained, and the learned components tested (a ridge ranking prior, a neural regulator shortlist, learned rule-family weights, an operator order learned on ARC) did not beat their controls; domain-calibrated controllers did not beat their bookkeeping-only ablation; the bioimage grammar induced from failures was empty, so learned transfer is false; a frozen library learned by grammar growth changed no prediction on 924 previously examined public ARC-style tasks; Boolean grammar growth was uninformative, and a supplied three-input OR form selected from biological source models agreed with the fixed form on all 151 tasks by design (Appendix B.2); a later adapter found 22 witnesses in ARC training tasks, and the order counted from them recovered 177 against 194 exact rules for a hand-set simplicity order
autonomous construction of new capabilities from a supplied space	partly established, in controlled settings; selection within a known space, not open invention	the construction test promoted, for each of the four removed operations, a reference-equivalent structure from a fixed, pre-registered 211-structure language (8 of 11 tasks, against 5 for the primary comparison, matched search, 4 from the demonstrations alone and 10 for blind enumeration in any seed), and the neural inventor two fixed-parameter instances; at scale, construction from typed holes recovered 64 against 38 of 95 generated tasks; nothing was invented for an unseen ARC puzzle
universal reasoning	open	no study here tests it

6.1 Graph transformations

Grid programs on graphs (separately written code). In the companion study's series, frozen grid-learned programs ran on attributed graphs through an engineered port interface. The paired unit is 8 task clusters of two queries each, built from 4 engineered Boolean context families that share templates, so the 16 queries are not 16 independent units: the transferred programs answered 14 of 16 queries and fully solved 7 of 8 tasks, against 16 of 16 and 8 of 8 for learning directly on graphs and for static search, with no new uncertainty estimate. This is executable program reuse without an accuracy advantage, not learned perception or transfer (the companion study's records, available from the authors on request; `records/NOV1_INDEPENDENT_RESEARCH_REVIEW.md`).

6.2 Biological regulatory networks

Published Boolean-network models are the setting in which Propositions 8 and 9 bite: each target gene's rule depends on a few regulators among many candidates, and ten demonstrations carry at most ten bits. They are also the setting with the strongest positive result outside ARC. The CORA systems ran without errors on 44 published biological Boolean-network models but answered no whole-network query; with a Boolean rule language supplied and regulators inferred from each task's demonstrations, a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code, exactly recovered 99 of 333 published single-gene rules, and with the correct wiring supplied it recovered 148, a reference ceiling.

Published biological Boolean networks (pre-registered; audited for bounded claims). We used 44 published Boolean-network models of the Biodivine repository [133], in five families, with a whole-network task (predict every gene's next state) and a single-gene task (predict one gene's next value) (`records/NOV4_BOOLEAN_BBM_RESULT.md`). *Portability:* the submitted CORA-only entry and two other CORA systems ran byte-identical on 3,092 real and 2,532 shuffled next-state queries; every started task returned a result and none ended in error. *No whole-network transfer:* none of the three systems answered any of 1,760 whole-network queries, so the pre-declared primary comparison is not testable; as predicted before any data, no demonstration-consistent program was generated, a case of no fit within the declared representation, grammar and bounded search. *A bounded evidence-sensitivity signal on single-gene tasks, attributable to the second solver:* the entry answered 12.5% of single-gene queries (167 of 1,332), with precision 0.898 among its answers, all from its second solver; precision was higher with the task's own demonstrations than with another model's (+0.28, 95% interval +0.10 to +0.43, significant after Holm correction), so the answers depend on the task's evidence. It is not verified transfer, because shuffled answers did not fall to half, and not usefulness beyond the majority rule: in 40 of its 42 answered tasks the entry repeated the value that all ten demonstrations shared, and none of its 42 answered tasks was probe-consistent, so it recovered no rule. Both results taught us that the grid languages did not reach these update rules within their declared search. They opened the adaptation below, with a Boolean rule language supplied. The pre-declared chance level for shuffled single-gene precision was not computed by the frozen scorer, a disclosed limitation.

Inferred wiring, the headline biology result (a within-biology adaptation, separate from the Kaggle entries; pre-registered, with its postscore audit passed). On published Boolean gene-regulation models, a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code, inferred each target gene's likely regulators from its ten simulated transitions, with a Boolean rule language of up to three literals supplied, and recovered the exact published update rule for 99 of 333 targets (29.7%), against 85 (25.5%) without inferred regulators and 87 (26.1%) with them shuffled (+4.20 points; that study's bootstrap resampled models rather than the 30 publication lineages, so we rely on the replication for inference) (`experiments/bbm_wiring_grammar_20261104/results_D/SUMMARY_D.md`). At least 97 of the 99 are one-input rules, in which the gene copies or inverts one other gene. A *wrong accept* is a task where the procedure accepted a rule that is not exactly the published one: 58 with inferred wiring and 58 with none. It answered more query bits (627 against 563 of 1,332) at lower precision on them (95.4% against 96.4%), and the inferred regulator sets were imperfect (precision 0.59 and recall 0.36 against the published regulators). *The supplied-wiring reference ceiling:* with each gene's published regulator set supplied instead, the same verify-or-abstain procedure recovered 148 of 333 (44.4%), against 87 (26.1%) with shuffled documented wiring and 85 (25.5%) with none, with 54 wrong accepts against 59 and 58 (no condition separated the acceptance bar from the prior's preference). Neither the rule language nor any ARC grammar was learned: this is a within-biology prior, not ARC-to-biology transfer, and recovering the rules of published computational models is not wet-lab discovery.

The prospective replication of the inferred support. A frozen, single-run replication on 595 targets from 75 further models in 50 publication lineages (all parsed earlier for an inventory; 33 previously excluded models readmitted) recovered 101 against 66 for each of three controls (shuffled inferred support, the full no-wiring prior and the empty no-wiring prior), which succeeded on the same 66 targets, so they are one comparison, not three confirmations (35 gains, 0 losses; one-sided lineage sign-permutation $p = 0.00002$, Holm-adjusted 0.00006, assuming independent lineages; descriptive whole-lineage bootstrap +3.4 to +8.3 points), and accepted more wrong rules (124 against 112); supplied correct wiring reached 222 of 595, the ceiling. Every exact recovery in the replication was a one-input rule, and only supplied wiring recovered two-input rules (60 of 222); 34 of the 35 gains came from networks with more than 30 genes, larger than any in the original study (within that original range, 15 against 14 of 147; a split made after the fact). Neither result used the shared two-input truth-table executor or anything learned on ARC. A separate automated review of the saved aggregates passed 20,913 checks. Its limits: every model of the collection had been parsed for an inventory before the run, so the cohort is not pristine unseen data; the four largest lineages, all readmitted, supply 191 of the 595 targets (all nine readmitted lineages supply 259), so the targets are not independent units and the inference is at lineage level under declared independence and sign-symmetry assumptions; the outline review arrived after launch, so no end-to-end reproduction of the 333-task result preceded admission (the original engine modules were copied unchanged and hash-checked); a compatibility check completed after the run then reproduced the original 333-task predictions exactly in all five conditions (1,665 condition records, zero mismatches), which is retrospective implementation assurance, not a pre-admission test; the runtime's cached modules were not all pinned; and the result is exact reconstruction of published rules from simulated state transitions, within biology, not discovery,

not learned ARC grammar transfer and not an entry gain (the companion study's evidence packet and its records; available from the authors on request). Inferring each gene's input set from state transitions follows REVEAL [102], preferring consistent functions over fewer inputs is the MIN-FEATURES bias [4], synthesising executable Boolean gene-regulatory networks from single-cell expression data is established [61], and so is learning a whole network's update rules as a logic program from pairs of consecutive states [75], whereas this procedure reconstructs one gene's rule at a time from ten transitions inside a supplied rule language: the inferred wiring is a result measured against controls inside verify-or-abstain, not a new mechanism. The grammar-growth, template-selection and ARC-to-Boolean follow-ups were null or negative (Appendix B.2). Figure 9 draws both studies on one scale.

Table 16: Exact reconstruction of published Boolean update rules from simulated transitions on 333 single-gene tasks (10 demonstrations each). Supplied correct wiring is the labelled ceiling, knowledge the system did not discover; in its own study it answered 808 of 1,332 bits at precision 0.976.

wiring	exact rules	answered bits of 1,332	precision
inferred from the demonstrations	99	627	95.37%
none	85	563	96.45%
shuffled inferred wiring	87		
supplied correct wiring (ceiling)	148		
Prospective replication, 595 targets from 75 models new to scoring experiments, in 50 disjoint lineages			
inferred from the demonstrations	101		
shuffled inferred wiring	66		
no wiring, full prior; no wiring, empty prior	66; 66		
supplied correct wiring (ceiling)	222		

Two learned components did not help in this setting (Appendix C). A ridge ranking prior trained on other lineages' published update functions reconstructed 98 functions against 95 for a fixed minimum-description-length rule on 182 source tasks, within noise, and did not reproduce on 356 tasks from independent lineages (121 against 124 to 126). At scale, a neural-network shortlist of candidate regulators reconstructed 24 published rules exactly on 656 targets, as did a logistic shortlist and the fixed flip-evidence rule, so its pre-declared criterion failed. Most failures in these cohorts were failures of support identification, the obstruction of Proposition 8, and a review of the tie, awaiting the companion study's confirmation, located the loss at the readout and the shortlist rather than in the width of the rule language. An exploratory readout comparison that reused the same 656 targets recovered 97 exact rules with learned rule-family weights against 104 with equal weights. The prior over which variables matter, inferred from the same demonstrations, was the first mechanism with an audited positive result in this setting, as a support prior built from the task's own demonstrations; the reused-cohort studies below add informative observations, a post hoc completion vote and conditional construction, which a classical learner equals by design. The invented-rule design simulation of Section 4.3, which is not biology evidence, suggests where better evidence would help these tasks: above the counting floor and once the variables are found, but not at the present ten transitions for rules with three inputs.

Choosing informative observations: better rule recovery, lower prediction coverage. A frozen exploratory test reused all 595 targets from 75 published models in 50 lineages. Each method received twenty new simulated training labels per target, a *training-label query* being one requested observation of the model's next state; earlier observations were discarded, and the supplied two-input truth-table language and reconstruction method were held fixed. After a first state shared with uniform selection, active selection labels, from a random pool of up to 64 unseen non-withheld states, the state that splits the still-consistent one- and two-input rules most evenly; random states fill in when fewer than two rules remain or a fixed computation cap is reached. The control permutes the variable positions of the actively chosen states and labels them anew (the permuted-variable control). On 595 previously studied target rules from 75 published Boolean models (50 publication lineages), with 20 simulated training labels per target for each method, active example selection recovered 320 rules exactly against 291 for uniform selection and 297 for a control that permutes the variables of the actively chosen states, but it abstained more often on rules with three or more inputs, so it predicted 1,586 of 2,380 withheld inputs correctly against 1,747 and 1,643 (unanswered inputs counted as incorrect). By rule size, all 165 one-input targets were exact under every method, two-input targets 155, 126 and 132 of 156, and no rule with three or more inputs

was recovered (0 of 274); exact recovery against uniform selection was 30 gains and 1 loss, with 21 lineages positive and none negative. Query learning and selection by committee disagreement are classical [6, 147], as is choosing perturbations to discriminate consistent networks [74], and CORA does not claim them. Figure 10 splits the counts by the number of inputs in the published rule: every gain is on two-input rules, and no method recovers a rule outside the two-input language. *What it taught us:* identifying a rule and predicting with it are different endpoints, and the prediction loss came from more abstention on rules the two-input language cannot express. *What it opens:* the completion rule below, and acquisition that tracks a changing language.

A completion rule for the prediction loss (post hoc, reused cohort). The overall outcome (more no-fit abstentions under active selection) motivated a completion rule whose specification and code were frozen before the per-target diagnosis was decoded. A completion rule fixed before it was run, which answers each abstained target by majority vote of the distinct one- and two-input rules that best fit the same 20 labels, gave 2,200 against 2,153 and 2,156 correct, a prespecified active-versus-uniform difference of 47 inputs (1.97 percentage points; 28 of 50 lineages favoured active selection and 14 favoured uniform), of which 30 come from two-input rules that active selection recovered exactly more often. This is exploratory evidence on a reused cohort, with no new training labels and no new significance test: completion recovers no additional exact rule, a fill of zero leaves active selection below uniform (2,002 against 2,028), and because the first scoring attempt stopped at its 120-second limit after evaluating 235 of the 595 targets, the identical sealed predictions were scored once more under a separately frozen and reviewed protocol. By target the paired result was 94 better, 62 worse and 439 tied, and by lineage 28, 14 and 8. In detail, that first attempt evaluated true outputs for 940 of the 2,380 withheld inputs and wrote 3,515 of 8,925 scored rows before it stopped, and these rows were not used; the second scoring had a 600-second limit and local storage, with no new inference and no new labels, under a protocol checked by a separate automated review process within the project. On the 100 targets where both methods abstained, completion scored 313 against 315 of 400; 168 vote positions were exact ties, set to zero by the fixed rule. A fill of zero alone reaches 2,002 and the training-label majority 2,178, so the vote adds 22 over the majority in the active arm and 41 in the uniform arm (Table 17). It is a researcher-made revision, not an autonomously learned repair, not language growth and not discovery. Figure 11 shows where the answers went: active selection was right on 98% of the inputs it answered, and every input it left unanswered belonged to a rule with three or more inputs, which the two-input language cannot express.

Table 17: All fifteen fixed acquisition and readout variants on the reused cohort; each keeps 595 targets and 2,380 held inputs. Completion answers every input, wrong predictions included; a prediction gain is not a gain in exact rule identification. The third column is the permuted-variable control.

readout	active	uniform	permuted	answered
original: abstain on no fit	1,586	1,747	1,643	1,616; 1,876; 1,740
zero on no fit	2,002	2,028	2,004	all 2,380
training-label majority	2,178	2,112	2,122	all 2,380
one least-error function	2,194	2,149	2,146	all 2,380
vote of least-error functions (primary)	2,200	2,153	2,156	all 2,380
exact rules of 595, every readout	320	291	297	

A researcher-made query-policy revision (authored functions). A grown inference language can still ask the wrong questions. In a small authored experiment the starting language describes any rule over two inputs; when exact observations show three different inputs that can each change the answer, the controller knows the language is insufficient and builds a conditional rule by fixing one input and solving the two simpler problems, a *cofactor* split (classical Shannon decomposition). On authored test functions (all 256 Boolean functions of three inputs, each hidden among five input coordinates with four inputs withheld from querying), a researcher-made revision of the query policy, prompted by the first version losing to random querying, raised exact recovery (the whole function correct on all 32 inputs) after 20 answered queries from 148, 138 and 94 to 254, 248 and 188 functions in three fixed embeddings (310 gains and no losses on the same functions; the first embedding is the one that exposed the failure), but a fixed policy that compared the full one-split conditional rule language from the first query recovered all 256 in every embedding, so the result shows that query selection must keep pace

with a growing rule language, not that adaptive growth beats a strong fixed learner. Across all 1,572 units there were 310 gains and 1 loss (six hand-picked four-input functions at 40 labels, 2 of 6 against 3 of 6), and the 310 gains come from 158 distinct functions, so no significance is claimed. The fixed policy used about 16 times the revised policy's work. A separate automated review ran 895 checks on saved aggregates, source hashes and custody records, 840 of which repeat ten bookkeeping checks, with no failed check. *What it taught us:* acquisition must be revised when the hypothesis language changes. *What it opens:* the same revision, tested where no fixed expressive policy is affordable.

A sparse learner at the size of real models (authored functions). In an authored test, a sparse classical query learner (binary search between two differently answered inputs to find each relevant input, then a complete answer table written as nested if-then rules) recovered hidden rules that use three of 60 binary inputs without enumerating all 2^{60} inputs: with 20, 40 and 60 exact answers to inputs it chose, it recovered 36, 248 and 256 of all 256 three-input rules in one random placement and 38, 249 and 256 in a second, that is 1, 210 and 218 and 1, 211 and 218 of the 218 rules that truly use all three inputs, with 80 and 65 wrong fitted rules at 20 answers; a two-input learner given the same answers, which cannot express those 218 rules, recovered 35, 38 and 38 and 37, 38 and 38, and the plain answer table built from the same answers made identical predictions by construction. This shows higher-arity coverage from a known method, not a new learning advantage, a learned policy or a biological result. The method assumes at most four relevant inputs and exact, noise-free answers to any input it chooses, and the two placements are the only replications.

Conditional construction on published models (published-model reconstruction on a reused cohort; independently audited). *Conditional construction* builds a rule over more inputs from two simpler rules, one for each value of a certified input (a cofactor split), and the *two-input rule language* contains every rule over at most two inputs. In published-model reconstruction on a reused cohort of 595 gene update rules from 75 published Boolean network models, conditional construction recovered 433 rules exactly at the predeclared budget of 40 simulated observations, against 321 for the complete two-input rule language on the same observations, all 112 additional rules having three or more inputs (81 of 90 with three inputs and 31 of 184 with four or more); a classical support-table learner given the same observations tied it at 433, so this shows executable coverage, not an advantage over classical reconstruction, learned transfer from ARC or a biological discovery. At 40 observations, 91 programs fitted every observation but were wrong functions and 71 targets had no program, and at least 65 of the wrong functions agree with the truth on all four held inputs, so four held inputs cannot expose them; with 60 observations exact recovery rose to 465 while correctly predicted held inputs fell from 2,070 to 2,054 of 2,380, because more targets ended without a program (80 against 71, as observation gathering stopped early for 68 targets against 30), which outweighed fewer wrong held predictions (6 against 26). At 40 observations the two-input language predicted 1,400 held inputs correctly. The 112 gains are the measured part; that no rule was lost holds by design. The companion's saved-outcome audit passed 618 checks, and our own audit passed 214 (`records/SPARSE_COFACTOR_NATURAL_INDEPENDENT_AUDIT_20261108.md`). The run does not test whether a failure, rather than always using the larger language, triggered the gain.

Two studies that are never mixed. The 656-target readout study (82 models, 68 lineages) contains 222 one-input, 147 two-input and 92 three-input rules, 190 that need more than three inputs and 5 of unknown arity; its language allowed up to three inputs, so the 190 are the rules outside it, not all rules with three or more inputs. The 595-target studies above used a two-input language. The two cohorts have different denominators, languages and readouts, and no number of one is compared with the other. Figure 12 breaks the 656-target study down by the number of inputs in the published rule: the best of 16 arms recovered 196 of 222 one-input rules, 4 of 147 two-input rules and none of the 92 three-input rules its language allowed, so three-input rules need better evidence or better input discovery, and larger rules need a larger language.

6.3 From visual transformations to biological mechanisms

An ARC task gives a few demonstrations, each an input grid and the grid it becomes, and asks for the output of a new input. A biological study gives a few observations, each a measured state of some genes, proteins or cells and what followed, or a perturbation and the response it caused, and asks what will happen under a new condition. Both pose one problem: given observations, infer an executable rule that explains what changes and

why, recognise when that rule is inadequate, and decide what must change (Table 18). The correspondence is not a renaming, and biology is not another collection of grids. Its entities are genes, proteins and cells rather than coloured regions; its relations are regulation and dependency rather than adjacency; its measurements are noisy, often continuous and not always paired in time; and a rule that explains a change does not yet establish what causes it. The correspondence therefore holds only through a domain adapter that declares what each entity, operation and check means, and every biological result in this paper carries an evidence label of Section 1.1 (synthetic demonstration, published-model reconstruction or prediction on real measurements); proposals are marked as conjectures, and there is no laboratory confirmation.

reasoning component	ARC visual reasoning	biological reasoning
demonstration	an input grid paired with its output grid	an initial biological state paired with a measured later state, or a perturbation condition paired with its observed response
objects	coloured regions, shapes, connected components	genes, proteins, cells, molecular complexes
attributes	colour, size, location, orientation	expression, activity, abundance, localisation
relationships	adjacency, containment, alignment	regulation, binding, interaction, dependency
executable operations	move, recolour, rotate, reflect, copy	update an activity, apply a conditional regulatory rule, simulate a state transition
program	a sequence of grid transformations	a Boolean, probabilistic or mechanistic biological model
failure	a predicted grid disagrees with a demonstration	a predicted response disagrees with an observed biological response
missing capability	cannot express an enclosure or an object relationship	cannot express a necessary regulatory dependency or conditional interaction
grammar growth	a reusable visual transformation is introduced	a reusable construction for regulatory rules is introduced
verification	execute a program against the examples	simulate a rule and compare its predictions with measurements
generalization	correctly transform a new grid	correctly predict an independent biological condition or perturbation
external confirmation	the hidden puzzle answer	independent measurements and, ultimately, laboratory experiments

Table 18: The correspondence between grid reasoning and biological reasoning. It holds only through an adapter that declares each domain's meaning: spatial adjacency does not become molecular regulation by renaming. A perturbation-response pair is not necessarily a time transition: a single-cell snapshot taken after a perturbation, as in Perturb-seq, does not show one cell moving from one state to the next. In the published-model studies above the observations are transitions only because they are simulated from each model's update rule.

Where the analogy ends. The correspondence is useful because it is partial. Six differences decide how far it goes.

1. *Exactness.* An ARC output is exact and deterministic, so one disagreement refutes a program. Biological measurements are noisy, continuous and heterogeneous, so one disagreement may be noise. Exact unanimity, which serves ARC, admitted no correction on three microscopy datasets and no rule for 64 Perturb-seq genes (prediction on real measurements).
2. *Time.* An ARC pair is an input and its output. A biological pair may be two snapshots that were never one system at two times (the qualification in the caption of Table 18).
3. *Independence.* In ARC the task is the unit. In biology, targets from one model or lineage, and cells from one donor or well, are dependent: the 595 targets of the main published-model cohort come from 75 models in 50 lineages, and a measured study's unit is the donor, replicate or perturbation, not the cell.
4. *Evidence acquisition.* ARC gives fixed demonstrations and hides its answers. A published model can be queried, at a counted cost, for the next state of any chosen input. A living system can be perturbed, but only

by a laboratory, at real cost and with noise. A laboratory experiment is an additional validation step available in some biological settings, not part of the passive ARC benchmark.

5. *Operations.* Rotating a grid has no biological meaning. Biological operations (evaluating an update rule, composing a conditional rule, fitting a response model) must be defined by the adapter, not borrowed from grids.
6. *What counts as right.* An ARC answer is right when it equals the hidden grid. A biological rule can fit and predict and still be causally wrong, for example when the gene it names only varies with the gene that acts. Explaining a change is not establishing its mechanism.

A worked example of diagnosing a missing regulator (an illustration, not a discovered rule). Take three genes A, B and C and a gene Y whose next state is to be predicted, each on (1) or off (0). A candidate rule is $Y_{t+1} = (A_t \wedge \neg B_t) \vee C_t$: Y turns on when A is on and B is off, or when C is on. Suppose CORA first assumes that only A and B matter and fits $Y = A \wedge \neg B$, then meets two observations: $(A, B, C) = (0, 1, 0)$ gives $Y = 0$ and $(0, 1, 1)$ gives $Y = 1$. A and B are the same in both, yet Y differs, so under a deterministic update no rule over A and B alone can fit both. That is stronger evidence than a search that ran out of budget. A failure record of the kind we want CORA to produce would read: target, the next state of Y; current rule, $A \wedge \neg B$; failure, two observations with equal A and B and different Y; diagnosis, no deterministic function of A and B fits; candidate explanations, a missing input, a wrong time step or updating scheme, or noise and unmodelled context; proposed revision, test whether C explains the disagreement; candidate rule, $(A \wedge \neg B) \vee C$; validation required, fit the known observations, predict independent states, and check biological plausibility. The revision is a cofactor split, $f(A, B, C) = (\neg C \wedge f_{C=0}(A, B)) \vee (C \wedge f_{C=1}(A, B))$, here with $f_{C=0} = A \wedge \neg B$ and $f_{C=1} = 1$. What is acquired is not only Y's rule but the ability to construct conditional rules over one more variable. The record and the revision are an illustration of the mechanism we want, not a claim that the system produced this diagnosis autonomously; with noisy data, or cells measured at incompatible times, adding C could merely overfit, so competing diagnoses must be kept and tested.

The case also runs. The companion study executed it as a machine-generated teaching case: from five training examples, exhaustive search over all 16 two-input functions confirmed the contradiction, a fixed scheduler enabled a supplied conditional template, and the sparse engine constructed IF C THEN 1 ELSE $(A \wedge \neg B)$, sealed before checking. On three held input states the initial rule was right on 1 of 3 and the constructed rule on 3 of 3, as were a fixed expressive learner and an independent minimum-DNF solver. Eight complete three-input functions fit the five examples, so the right completion depends on the supplied simplicity prior; and a full-state example with contradictory labels routes to refusal. It is an executable demonstration, not a benchmark, a learned revision policy or an invented primitive (Figure 13). We checked the saved case without rerunning it. The trace, predictions and source freeze match their recorded SHA-256 hashes. The trace's conflicting pair, program, count of fitting rules and refusal check are computed by the code; its list of assumptions is fixed text, and the choice to split on C was hand-written. The three held outcomes were written by the same person as the observations, so 3 of 3 shows that the simplicity prior agrees with that person, not that the method generalizes. Nor do the observations single out C. Besides the eight fitting rules (the next simplest are $(A \wedge \neg B) \vee (B \wedge C)$ and $(A \wedge \neg B) \vee (\neg A \wedge C)$), an unmeasured gene that varied with C would give the same data, and only an intervention that sets C separates the two. Figure 14 runs the same steps on a real published rule: in the myeloid progenitor network of Krumsiek et al. [93], two observed patterns that agree on PU.1 and cJun but give EgrNab different next values rule out every rule over those two inputs, and adding the input in which they differ, Gfi-1, leaves one of 256 three-input rules, the published one; this is a reconstruction, not a discovery.

Three kinds of capability growth in biology. A new biological rule is a hypothesis about one gene. A new executable abstraction, such as the cofactor split, is a reusable way to construct conditional rules and can improve inference for other genes. A new revision policy decides when evidence justifies expanding a rule, changing variables or obtaining new observations, and is the level that connects most directly to learning what to change. The experiments above give evidence on the first two levels in controlled settings; the third is open.

Three levels of validity, and the experiment that separates two rules. Suppose CORA proposes $Y = (A \wedge \neg B) \vee C$. Three claims must be kept apart. *Computational validity*: the rule executes and matches the specified examples. *Predictive validity*: it predicts biological measurements that were not used to infer it. *Mechanistic validity*: experiments support the proposed relationship, its conditions and its causal reading. They are not

equivalent: changing C may alter Y in the model because C is correlated with another regulator, without C regulating Y directly. The hypotheses $Y = A \wedge \neg B$ and $Y = (A \wedge \neg B) \vee C$ disagree wherever C is on and $A \wedge \neg B$ is off. In the worked example the state $A = 0$, $B = 1$, $C = 1$ was observed with $Y = 1$, but observation alone cannot show that C drives Y rather than something that varied with C. Setting C by intervention at $A = 0$, $B = 1$, where the first rule predicts 0 and the second 1, is the *discriminating intervention*; setting it at $A = B = 0$ and at $A = B = 1$ also separates the revised rule from the next simplest fitting rules. It names a useful experiment: perturb C in a suitable model system, for example by CRISPR interference or activation read out by single-cell expression [54, 65, 140], with non-targeting controls, independent reagents, biological replicates and checks for indirect effects. A consistent change of Y in the predicted direction would support a causal influence of C under the tested conditions, not yet direct transcriptional regulation; and a contradicting result is not a failure to hide but the next evidence for what to change. Figure 15 carries this out on the five-gene cortex model of Giacomantonio and Goodhill [64]: normal development never shows Sp8 on together with Emx2 or Coup-tfi, and 83 rules fit what it does show; the most informative perturbation, turning Sp8 on in posterior tissue, splits them 41 to 42, and seven designed readouts single out the published rule, against 11.3 on average in random order (simulation on a published model; the laboratory experiment is proposed, not done).

6.4 A biological validation ladder

The ladder refines the three levels above: computational validity is rung 1, predictive validity rung 2, and mechanistic validity spans rungs 3 and 4. The ladder from the authors' research discussions (Section 1.1) maps onto it: reproducing what is known is rung 1, generating plausible ideas that can be verified is the step from rung 2 to rung 3, and verifying them is rungs 3 and 4.

A biological rule can be right in four senses, and each needs evidence that the one below cannot supply. It can be *executably consistent*: it runs and reproduces the observations used to build it. It can have *independent predictive validity*: predictions frozen in advance match measurements that played no part in building or choosing it. It can be supported by *intervention-based evidence*: when competing rules predict different outcomes of a feasible intervention, a controlled experiment agrees with one and contradicts the others. And it can reach *mechanistic confirmation*: independent molecular measurements support the route by which the effect arises, under the conditions tested. Each rung rules out an explanation the rung below leaves open. A consistent rule may be overfitted; a predictive rule may name a variable that only travels with the true cause; an intervention can show that changing a gene changes its target without showing that the effect is direct. A laboratory result that supports a predicted response therefore does not by itself establish a direct regulatory interaction (Table 19). Reconstructing a published model is rung 1 with a reference check: stronger than fitting, because the whole rule is compared with an independent reference on every input, but the observations come from the model itself, so it says nothing about living cells; predictions of a published model's withheld states stay on rung 1 for the same reason. Appendix C places every biological result of this paper on the ladder.

Rules that keep each rung honest.

- *Units.* Thousands of cells from one donor, well or perturbation are not thousands of independent cases. Hold out whole perturbations, donors, replicates or projects, and cluster intervals by them; in published models, cluster by lineage.
- *Leakage.* A final test set is never reused to choose thresholds, gene panels, candidate rules or the winning method.
- *Time.* A snapshot after a perturbation is not a trajectory; a transition model needs genuinely time-paired data.
- *Thresholds.* A Boolean threshold on continuous data must be justified by measurement variability and chosen on training data only; otherwise use a continuous or probabilistic response model.
- *Freezing.* Predictions, competing hypotheses and the decision rule are frozen and hashed before the outcome is read, and every outcome is reported, including those that contradict the favoured rule.

6.5 From executable biological hypotheses to experimental tests

The idea. The most useful prospective application of CORA in biology would not be one more prediction but a pair of competing executable hypotheses together with the feasible intervention on which they disagree.

rung	the question	what it does not establish	CORA so far
1. executable consistency	does the rule run and reproduce the observations used to build it; for a published model, does it equal the reference rule on every input?	that it predicts new measurements; any mechanism	reached for authored functions and published models
2. independent predictive validity	do predictions frozen before scoring match measurements on held-out biological units, against strong native baselines?	causation: a variable that travels with the true cause can predict well	attempted on measured data; no robust success
3. intervention-based evidence	do competing rules predict different outcomes of a feasible intervention, and which does a controlled, replicated experiment support?	a direct link: the effect may pass through intermediates	not reached
4. mechanistic confirmation	do independent molecular measurements support the route, its conditions and its limits?	generality across tissues, species or conditions	not reached

Table 19: The biological validation ladder. Each rung needs evidence the rung below cannot supply. Reconstructing a published model is rung 1 with a reference check, because the observations come from the model itself; no CORA rule has reached rung 3.

Because every hypothesis is a program, the disagreement is not a matter of interpretation: the system can list the conditions under which the hypotheses predict different outcomes and rank them by how many surviving hypotheses each condition separates. A laboratory collaborator can then judge whether an appropriate perturbation and measurement exist for those conditions, what they would cost, and which controls they need. Any such design must be reviewed and run by an appropriate biological collaborator; no experiment is initiated by the system.

A worked design (synthetic demonstration). Take the teaching case: five hypothetical observations of genes A, B, C and a target Y, (0,0,0) giving 0, (0,1,0) giving 0, (1,0,0) giving 1, (1,1,0) giving 0 and (0,1,1) giving 1. Eight complete three-input rules fit them, and so does a ninth explanation that observation cannot exclude: C has no causal effect, and an unmeasured gene that varied with C switched Y on. Four of these hypotheses predict, when C is switched on by intervention in each background state of A and B, the outcomes of Table 20.

background (A, B), C switched on	C has no effect	$(A \wedge \neg B) \vee C$	$(A \wedge \neg B) \vee (B \wedge C)$	$(A \wedge \neg B) \vee (\neg A \wedge C)$
$(0, 1)$	Y off	Y on	Y on	Y on
$(0, 0)$	Y off	Y on	Y off	Y on
$(1, 1)$	Y off	Y on	Y on	Y off
$(1, 0)$	Y on	Y on	Y on	Y on

Table 20: Predicted outcomes of switching C on, in each background state of A and B, for four explanations of the five teaching-case observations (hypothetical genes; a synthetic demonstration, not a planned experiment). "C has no effect" means Y follows $A \wedge \neg B$.

The state named above, $(A, B, C) = (0, 1, 1)$, is the background $(0, 1)$: it tests whether C acts at all, separating the no-effect explanation from every rule in which C acts, but it cannot tell those rules apart. Adding the backgrounds $(0, 0)$ and $(1, 1)$ gives the four hypotheses four different patterns of outcome (off, off, off; on, on, on; on, off, on; on, on, off), so three intervention conditions separate them, while background $(1, 0)$ checks only the $A \wedge \neg B$ part. The full four-background design separates all nine explanations, the eight fitting rules and the no-effect explanation (`paper/final_push_20261108/BIOLOGICAL_HYPOTHESIS_FOR_FUTURE_LAB_REVIEW.md`); the enumeration is printed by `paper/final_push_20261108/check_abcy_toy_enumeration.py`, which reads no data. The genes are hypothetical: nothing here is a discovered rule or a planned experiment.

From a computational design to a laboratory test. A collaborator would turn such a design into an experiment in an appropriate model system, for example by switching C on with CRISPR activation or off with CRISPR

interference and reading out Y, alone or by single-cell sequencing in a Perturb-seq design [54, 65, 126]. The controls a qualified laboratory would consider include non-targeting guides; at least two independent reagents per target; measurements confirming that C itself changed and that each background state of A and B held; biological replicates (separate cultures or donors) as the units, rather than cells; more than one time point; and checks for indirect effects through intermediate genes. Real responses are continuous, so "Y on" must be defined in advance from the distribution of the controls, with an effect size and an interval. The predictions of every competing hypothesis are frozen before any measurement is read.

What a result would and would not establish. If switching C on in background $(0, 1)$ raises Y as predicted, that is evidence for a causal influence of C on Y under the tested conditions (rung 3), not for direct transcriptional regulation, which would need binding or intermediate-pathway evidence (rung 4). If Y does not change, the no-effect explanation is favoured and every deterministic rule that fitted the observations is rejected under the toy's assumptions. Either way the result is not a failure to hide. It is new evidence that returns to the diagnosis step, where it may point to a missing variable, a different rule from the fitting set, or a context, noise or timing explanation. This return of external evidence is where the biological loop goes beyond ARC-AGI-2, whose test answers are hidden and which offers no interactive source of new evidence.

What CORA has done toward this, and what it has not. The computational counterpart of choosing a discriminating experiment is choosing a discriminating observation from a simulator, and CORA has measured it. In the reused 595-target cohort, active selection asked, at each step, for the state that split the surviving candidate rules most evenly, from a pool of at most 64 states, and recovered 320 exact rules against 291 for uniform selection; but it abstained more, answering fewer held inputs correctly (1,586 against 1,747 of 2,380) until a post hoc completion vote reversed the loss (2,200 against 2,153) (published-model reconstruction). An invented-rule simulation, which is not biology evidence, showed the same logic at its best: with ten variables and fifteen observations, choosing states where the surviving rules disagree most identified 97% of three-input rules against 8% for random states (synthetic demonstration). A laboratory is harder in four ways: each query is an experiment with a real cost; outcomes are noisy and continuous; not every state can be set; and controls are required. No laboratory experiment has been run on any CORA output, and no data-derived hypothesis currently qualifies for laboratory review: the companion study's dossier is marked unqualified, and our own review reaches the same conclusion.

What a candidate must contain before laboratory review. A candidate must name the biological system; give the executable rule with its inputs and conditions; give the measured observations that support it, with units; say what the earlier rule could not explain and what CORA changed, and whether CORA chose the change; give frozen predictions for independent conditions; list rival rules and non-causal accounts with their predictions, and the measurement that separates them; list the controls a qualified laboratory would consider; state its uncertainty and the alternative causal readings; and cite supporting and conflicting prior work, with a novelty check. Today no candidate exists: the measured studies admitted no rule, the teaching rule is hypothetical, and the revisions were made by fixed rules in authored settings or by researchers.

One loop, three different claims. The shared loop is observe, propose, execute, compare, diagnose, revise and verify; each domain supplies its own meanings through an adapter. A grid adapter defines objects, colours, spatial relations and transformations; a biological adapter defines genes, regulatory states, executable update functions, biological constraints and permissible evidence. Three claims follow from this design, and they need different evidence. *Software portability*: a shared interface or reused implementation runs in both domains; here a small two-input truth-table executor, identical byte for byte wherever it is used, runs on ARC grid panels and in later gene-state experiments, it was not used in the inferred-wiring results, and the full engines are domain-specific. *Application within biology*: the pipeline improves a biological result; the inferred-regulator, informative-observation and completion results above are of this kind. *Transfer of a learned revision policy*: a controller trained to choose revisions on ARC chooses better revisions in biology; that has not been shown.

What an adapter supplies, and what it must never supply. An adapter supplies a domain's meanings: its entities and their attributes; the relations a rule may state; the executable operations and their semantics; what counts as a check (an exact match, or agreement within a declared uncertainty); the model of evidence (exact,

stage	current CORA evidence	what remains
construct executable biological rules	demonstrated for published Boolean models (published-model reconstruction)	more complex and noisy relationships
identify relevant regulatory variables	inferred regulators: 101 against 66 of 595, prospective, all one-input rules; informative observations: 320 against 291 (reused cohort)	stronger comparisons and broader independent data
grow the executable rule language	authored functions: 219 against 38 of 256; a fixed build-then-ask schedule also 219; published models: 433 against 321 of 595, equalled by a classical learner by design	beneficial growth chosen by the system on natural models
predict independent biological outcomes	simulated held inputs only (2,070 against 1,400 of 2,380; post hoc, 2,200 against 2,153); on measured data no rule was admitted for Perturb-seq, and the spatial gain is exploratory, on one donor	robust prediction on independent experimental measurements, with frozen predictions
transfer a learned revision strategy from ARC	not established: an operator order learned on ARC gave 177 against 194 for a hand-set simplicity order	beat strong native and static methods with a frozen transferred policy
confirm a new biological hypothesis in the laboratory	not demonstrated; no hypothesis qualifies	prospective experiments and mechanistic follow-up

Table 21: The evidence boundary in biology, stage by stage. Published-model results reconstruct computer models from simulated observations; none is a discovery, and no CORA rule has been tested in a laboratory.

or noisy with a stated error model); which new evidence may be acquired and at what cost; and the unit that counts as an independent case. It must never supply withheld outcomes, the true regulator set or the reference rule, nor give one comparison arm information that the others lack. A biological prior, such as known wiring, is allowed only when its source and exposure are explicit and every control receives the same prior (Table 22).

6.6 What moved between domains, and what did not

What moved was the procedure's design: the pipeline's acceptance design and its evidence-request step, implemented separately for gene networks, recovered more published rules than its controls; diagnosis and revision there were made by researchers. Learned knowledge did not move. A fixed foreground-mask adapter found exact Boolean relations in ARC training demonstrations, 22 admitted witnesses among 1,000 training tasks (NOR 8, OR 8, XOR 4, AND 1 and TRUE 1), and the order counted from their frequencies was frozen before the biology run. On 595 previously scored targets from 75 published models in 50 lineages, the order counted from 22 of 1,000 ARC training tasks recovered 177 rules exactly, against 168 for plain index order, 182 for one random reassignment of the same counts and 194 for a hand-set simplicity order; its gain over index order did not survive Holm correction (adjusted $p = 0.14$) and was not specific to the ARC counts, and it lost 5 targets and gained none against the permuted order, and lost 20 and gained 3 against the simplicity order. The 274 targets needing three or more inputs were out of reach of the supplied two-input language under every order. An earlier direct grid-to-biology operator transfer found no operator witness at all (Appendix B.2). *What it taught us:* knowledge learned in one domain need not carry over, even with a shared binary alphabet in place, while the procedure's design does. *What it opens:* transferring the revision policy rather than the operators, under the admissible test of Section 6.12.

6.7 Scientific images

The submitted component, byte-identical, ran as a verified object filter on two fresh public nuclei image sets (Appendix C). On S-BSST265, under the neural comparison entry's strict rule, it accepted no edit, so object F1 stayed at the tuned baseline's 0.858, which abstaining on every task would match, while a single intensity filter tuned on the training images reached 0.867. On HeLaCytoNuc the same rule accepted 11 deletions and 10 of them removed real nuclei, because in both tasks it acted on every demonstration window held only debris, so an erase-everything program fitted and survived leave-one-out.

layer	shared between ARC and biology?	measured evidence
the reasoning procedure (observe, propose, execute, compare, diagnose, revise, verify)	shared as a design, implemented separately; in biology, diagnosis and revision were made by researchers	the entries' 35 Python files and the 236 files behind the biological studies share no file, function or class
execution of two-input Boolean rules	shared code, not used in the inferred-wiring results	a small two-input truth-table executor, identical byte for byte wherever it is used, runs on ARC grid panels and in later gene-state experiments; its gene-state adapter has run only in authored tests
conditional (cofactor) construction	shared across the Boolean studies (authored functions, the teaching case, published models); not used on ARC grids	the same sparse engine ran the authored loop, the teaching case and the 595-target run
failure records	shared idea, domain-specific types	ARC: budget stop or exhausted space, typed holes; Boolean: no fit in the declared language, same-input contradiction; Perturb-seq: no stateable relation, insufficient support, binding mismatch
verification	shared design, different strictness	a hard check on each answer in the scored ARC solver; a soft support signal in biology; exact unanimity failed on noisy images
entities, attributes, relations, operations	domain-specific	grids: objects, colours, adjacency; Boolean models: genes, on and off states, update rules; measured data: perturbations, responses, context, cells and donors
evidence model and acquisition	domain-specific	ARC: exact, fixed, no oracle; published models: exact simulator, counted queries; measured data: noisy, continuous, existing data or new experiments
independent unit	domain-specific	ARC: task; published models: lineage; measured data: donor, replicate, perturbation
learned parameters	one test, no benefit	an operator order learned on ARC: 177 against 194 exact rules for a hand-set simplicity order
learned revision policy	none exists	every revision in the biology studies was chosen by a researcher or by a fixed rule

Table 22: What is shared between the ARC and biology studies, and what is domain-specific.

A literature-grounded bioimage grammar, tested for transfer (pre-registered; primary result). The frozen procedure (failure records from the frozen ARC component, an admission check, verify-or-abstain over typed object programs, exact conflict-safe certification, and leak-free splits by parent, the image group that held-out data must not share) ran unchanged on three public biology datasets: phase-contrast cells (LIVECell [55]), H&E nuclei from 31 organs (NuInsSeg [108]) and H&E nuclei from 67 of its 71 TCGA patients (MoNuSAC [162]), with every decision frozen by hash before any evaluation reference was decoded (`records/NOV5_BIO_GRAMMAR_TRANSFER_RESULT.md`). *Portability* is shown; it is an engineering result, not usefulness. The failure records specified ten literature operators, and admission on separate calibration parents admitted none (five had no harmless fit, four failed leave-one-parent-out, and closing failed unanimity), so on the held-out parents of all three datasets strict CORA made 0 edits: its pooled F1 equals the strong, calibration-tuned baseline (LIVECell 0.5628, NuInsSeg 0.5823, MoNuSAC 0.6745), observed non-degradation through zero-edit abstention, not a verified gain. *Cross-domain usefulness* is therefore not shown. The grammar induced on LIVECell is empty, so *learned transfer* is false and grammar growth is not supported; specification ran but was not selective (10 of the 11 operators were specified). Under the same frozen protocol, a compute-matched static search over the same literature operators, fitted on calibration parents, gave descriptive, unadjusted changes in mean per-parent F1 against the strong baseline: +0.0116 [+0.0066, +0.0169] on MoNuSAC (33 of 40 patients better), +0.0094 [−0.0063, +0.0225] on NuInsSeg and +0.0002 [−0.0024, +0.0029] on LIVECell; switching every operator on harmed LIVECell (−0.0248 [−0.0354, −0.0161]). These contrasts are not CORA's and carry no correction for multiple comparisons. The candidate operators thus support a positive matched-search contrast on MoNuSAC that CORA's admission from LIVECell's failures did not find: the bottleneck is admission, whose three requirements (no harm to any demonstration parent, a stable leave-one-parent-out top, and unanimity of the near-optimal programs) were designed for ARC's exact tasks. It opens an admission bar tied to the target domain, calibrated for noisy images. The secondary conditions changed no primary answer (their only edits were 4,000, almost all neutral, on NuInsSeg), and a calibration-informed tolerant rule admitted one operator on LIVECell calibration, an admission count and not a biological success (`records/NOV5_BIO_GRAMMAR_TRANSFER_RESULT.md`). On held data the tolerant rule made one harmful edit and no beneficial one on LIVECell, which the failure-blind and shuffled control seeds that admitted the same operator also made; on NuInsSeg it made 32 accepted edits, 10 beneficial, 16 harmful and 6 neutral; on MoNuSAC it made none; and no target passed its control gates, while unmatched control CPU rules out any claim that failure evidence helped (Figure 2).

The bioimage study's follow-up and its private maize target. A frozen follow-up crossed the original object features with an expanded bank (adding each object's area relative to its nearest neighbour) and crossed

unanimity with a declared parent-level evidence bar, beside no-edit, unguarded, failure-blind and shuffled controls, on one image from each of 78 held parents. Its 624 predictions were sealed and replay-checked; its single run then failed with an out-of-memory error during an attempted MoNuSAC reference decode, after the LIVECell and NuInsSeg references had been processed and scored in memory, because the inherited decoder kept one full-frame mask per annotated object; how many MoNuSAC references were reached is unknown. No scored-rows file was written, so there is no qualified endpoint, and the failure is not a null. It taught us that a pre-run review must bound the memory of the scoring path on the largest real input. A reviewed bounded-memory decoder exists as source only; any score-only recovery of the sealed predictions would need a newly reviewed protocol, and would keep the earlier reference exposure on record. The run cost 141 CPU-seconds and no GPU. The same study's private target was the first author's own maize leaves (Figure 1): on 16 evaluation leaves CORA abstained on every leaf, the cell-trained baseline as shipped scored a pooled F1 of 0.0000, and plain search over the same operations made 120 edits, 103 helpful, at a pooled F1 of 0.0113 (Section 1).

A calibrated acceptance rule on fresh melanoma images (pre-registered; independently audited). After the strict rule abstained and the tolerant rule was not selective, a calibrated rule was tested once on a fresh source, PUMA, a public melanoma dataset that no CORA study had used (Zenodo record 13859989, CC BY 4.0; audit `experiments/calibrated_acceptance_20261108/reviews/POST_SCORE_AUDIT.md`, section 6). This experiment tested a calibrated acceptance rule, not learning from failure: the order of CORA's candidate operators came from per-class oracle gains (the F1 an oracle fix of each error class would recover) on 6 of 12 NuInsSeg calibration organs, CORA's failure probes found no fitting program for any probed class, and acceptance was a statistical certificate computed on the 12 calibration organs (236 images). An operator was accepted only if, across leave-one-organ-out folds on those 12 organs, the one-sided 95% lower bound of the mean per-image F1 change was above 0 and the one-sided 95% upper bound of the harmful-edit rate was at most 0.60; operators were tested in order, stopping at the first one not accepted. The bound of 0.60 is a weak safety promise: it allows up to 6 harmful edits in every 10 accepted, and it was relaxed from 0.20 during development, on the same calibration organs, because 0.20 accepted nothing. The rule was frozen and pre-registered, then tested once on 100 held-out PUMA melanoma image regions (50 primary, 50 metastatic; 48,013 expert-corrected nucleus outlines) chosen after the go-ahead by a hash of the go-ahead file; the other 105 regions remain unopened for one replication. Every rule started from a strong baseline, StarDist with detection thresholds tuned on NuInsSeg plus a deletion rule chosen there, whose mean per-image F1 over the 100 regions was 0.889. (Confirmatory) The rule accepted one operator, which adds low-threshold StarDist detections, and made 605 edits on the 100 regions: 430 of the 605 were harmful (0.711, 95% interval 0.662 to 0.757), mean per-image F1 fell by 0.0020 (95% interval 0.0014 to 0.0027), and 69 of the 100 regions lost F1 while 31 gained. (Confirmatory) Under the pre-registered decision rule the label is "harmful", because the whole 95% interval of the F1 change lies below 0; the gain hypothesis is not supported, and the harm bound was exceeded, with even the lower end of the observed rate's interval, 0.662, above 0.60. (Confirmatory) The oracle-gain order did not beat random operator orders under stop-at-first-failure testing: the mean per-image F1 difference was -0.0008 (95% interval -0.0011 to -0.0006) over the 100 regions. Because 3 of the 5 random orders also drew the adding operator first and reproduced the calibrated rule's edits exactly, while the other 2 made no edit, this contrast is exactly $2/5$ of the primary one and is not separate evidence about CORA's failure analysis. (Descriptive) The strict verify-or-abstain rule made no edit on the 100 regions and lost nothing, while every rule that edited lost F1 against the baseline: the calibrated rule (605 edits) by 0.0020, a majority-vote rule (318 edits) by 0.0023, and two unverified searches (3,271 and 17,694 edits) by 0.0143 and 0.0194. Lesson: a harm bound calibrated across organs of one dataset did not hold on a new source; the adding operator's harmful-edit rate was 147 of 337 (0.436) on left-out calibration organs (development, optimistic) and 430 of 605 (0.711) on the new source. (Exploratory) An added object raises F1 on average only if the share of false additions is below about $1 - F1/2$, so a fixed bound of 0.60 left room for gain at the calibration organs' pooled F1 of 0.503 (break-even about 0.75) but not at the new source's baseline pooled F1 of 0.893 (break-even about 0.55). Direction: tie any future harm bound to the target's break-even accuracy, estimated on a small labelled pilot set kept apart from the test images and fixed before looking; abstain by default on a new source; and test a revised rule once, under a new pre-registration, on the 105 unopened PUMA regions or on another fresh source. The result concerns nuclei in melanoma image regions from one source, PUMA, with StarDist as the baseline segmenter.

No biology gain from the submitted component, because its demonstrations did not separate debris from nuclei. On one nuclei set the neural comparison entry's rule accepted no edit (F1 0.858 unchanged); on the fresh fluorescence image set the neural comparison entry's rule accepted 11 deletions, 10 of them harmful, and the plain CORA-only configuration's weaker rule 66, 43 of them harmful, lowering F1 by 0.0014 per image, while a calibration-selected simple-rule search gained 0.0123 mean per-image F1, not a matched comparison; and in the pre-registered bioimage transfer test strict CORA made no edit on any of three held-out image datasets, while a compute-matched static search over the same operators showed a descriptive, unadjusted gain on MoNuSAC; and on 44 published Boolean-network models it answered no whole-network query, since no demonstration-consistent program was generated, while its single-gene answers equalled the majority rule (above, and Appendix C). It opens demonstrations chosen to separate the rule from its rivals, and rule languages that fit the domain, as the supplied Boolean language did.

A repair procedure with the same design. A separately implemented repair procedure, which shares a design with CORA-CAST but no code and which the entries do not run, improved two image results: pooled object F1 from 0.8827 to 0.9230 on the 50 official BBBC039 test images from a watershed start, and from 0.692 to 0.705 on top of StarDist's pretrained segmentations on 50 fresh images. In both, failure-blind controls matched it, so the gains belong to the repair procedure, not to a learned diagnosis (Appendix C).

6.8 Other biological data

In drug response, typed rules revised on development compounds and applied frozen to 416 held-out compounds of the PRISM screen gave mean balanced accuracy 0.5249 with CORA's diagnosis against 0.5295 for try-all at equal compute; grammar induction from failures accepted no production in any of three domains; and domain-calibrated controllers did not beat their own bookkeeping-only ablation (Appendix C). A retrospective single-cell application of a related design predicted a direction that held in an independent cohort of 41 donors, as every control condition's did, so the question could not credit the design (Appendix C). A source-only spatial transcriptomics study on breast tumour sections was negative under stronger controls (Appendix D).

6.9 Noisy continuous data: the next adaptation of the pipeline, as a hypothesis

The original strict results are a first-step diagnosis. Three measured domains gave strict negatives. On real Perturb-seq perturbation-response profiles no rule other than an alias of the fixed reference was admitted for any of 64 genes, and the typed failure record said why: most failed attempts found no relation the language can state (Section 3.1). On three held-out microscopy datasets the strict rule made 0 edits (Section 6.7). On a held donor of human prefrontal-cortex spatial transcriptomics, a marker-gene correction made 58 corrections against 65 harms and its pre-registered gate failed (Appendix B.2). Each suggests what may need to change: the relations a rule can state, and the acceptance bar; in microscopy a tolerant bar was not selective either (on one dataset 32 edits: 10 beneficial, 16 harmful and 6 neutral), so acceptance alone is not shown to be the missing piece. In the imaging and Perturb-seq studies a uniform bar was either too strict or not selective; whether noise is the cause is an untested hypothesis.

The hypothesis, and its limits. We propose, as a research hypothesis and not a law, that the pipeline carries over where a domain offers discrete or relational states, compositional and contextual rules, and executable verification or active probing, and that noisy continuous domains need two adapter changes: relational guards that survive rescaling, such as comparing two genes instead of thresholding one, and a prior-weighted acceptance bar with a least-error vote instead of exact unanimity. Source review bounds this. A pairwise comparison is invariant only under one common increasing transformation, not under independent rescaling of each gene; the saved Perturb-seq profiles are responses to perturbation, so the tested relations compare imputed response coordinates, not measured expression of a new query; the original microscopy adapter already applied operators to objects with scale-normalised and neighbour features; the four spatial sections come from one previously exposed donor; and the acceptance equation of Section 4.4 is a proposed bar, not a calibrated biological posterior. Discrete tasks can also fail from language, ambiguity or search, and continuous tasks for reasons other than noise.

Three frozen follow-ups (independently audited). Each ran once under a reviewed freeze, with all predictions sealed before scoring; our own audit passed the spatial and Perturb-seq results and found the microscopy run inconclusive. *Spatial:* the pre-declared combined relational gate raised pooled macro-F1 from 0.931555 to 0.933612, with 23 corrections against 9 harms, but failed its declared criterion because one section worsened; the pre-declared neighbour-only ablation reached 0.934997, with 36 corrections against 12 harms and a gain in all four sections, a modest exploratory signal from a predeclared secondary ablation on one previously used donor, without a matched neighbour-only shuffle; its five-of-six rule was set by hand without a check on the source donors. A rank-only gate made results worse, and the graph-shuffled gate changed nothing, which by construction was expected to make almost no change, so it is equivalent to the native readout and does not test whether neighbour information beats a random filter of the same size. On its source validation donor the original rule had corrected 101 of 122 changed spots, against 58 of 123 on the held donor, so a correction's value moved with how often the base model was wrong where the rule fired. In a post hoc analysis of the saved counts, the neighbour guard kept 36 of the rule's 58 corrections but only 12 of its 65 harms, where a random filter of the same size would keep about 23 corrections; the rank guard (38 corrections, 48 harms) showed no better selectivity than that random-filter model. With one donor there is no donor-level uncertainty; even if the four sections were independent, four of four improving would give a one-sided sign-test p of 0.0625; and no standard spatial-smoothing baseline was run. *Perturb-seq:* every one of the 16 repair and static rule banks admitted 0 of 64 rules, and directional macro-F1 was zero in all 20 arms; overall accuracy of 16,286 of 16,320 calls reflects class imbalance, since the truth held only 32 directional responses; even on training data no candidate gained more than one net correction over the baseline, against the four required, so this run could not distinguish scalar from pairwise representations. Under its fixed constants the fold vote's bar amounts to four net corrections per gene (Proposition 10); the number of correctable events per gene among the 243 admission perturbations has not been counted, and it can be counted on training data alone. Whether too few correctable events existed for any per-gene rule to reach the declared bar is a testable hypothesis, not an established cause, so the null neither supports nor refutes relational representations. *Microscopy:* the run failed technically, with a memory error while rasterising reference masks, and was not retried, so it gives no result; it has no qualified endpoint and is not a null; a reviewed bounded-memory decoder exists as source only (Table 36). *What they taught us:* on one previously used donor, a neighbour-agreement guard turned a correction rule that harmed more than it helped (58 corrections, 65 harms) into one that helped (36 and 12), as a predeclared ablation while the primary failed its gate; the tested rank predicate failed here, and why (for example, that both of its genes are raised in the missed tissue) remains a hypothesis; and in Perturb-seq the amount of correctable evidence, not only the representation, may set the floor. *What they open:* donor replication with a matched neighbour-only shuffle, an acceptance bar with declared likelihood and prior, and a separately reviewed, score-only recovery of the sealed microscopy predictions that discloses the earlier reference exposure.

6.10 Thirteen domain families where the loop design ran

Table 23 counts the domain families in which CORA's loop design ran, with its adapter, what ran and the result, positive results first. One rule decides what is counted: a family is counted when the loop design, build an executable rule or edit, check it against the examples, keep it only if it holds up and otherwise abstain, ran on it through a declared adapter with a recorded result. On grids, sequences and lists the code is the entries' own; for every other family it was written separately, so what carried over is the design, not the code. By that rule the attributed-graph test (Section 6.1) is not counted, because it ran frozen grid programs rather than the loop.

6.11 Held tests in new domains, through separately written adapters

After the biology results, the loop design was carried to five further domains, each through an adapter written for that domain and described in an adapter card that says what was reused and what was rebuilt (`experiments/transfer_logic_20261108/ADAPTER_CARD.md` and the same file in each study's folder). Each test was frozen and reviewed before its held run, scored once and audited after scoring by the reviewer who wrote its pre-run review, not by its designer; each audit lists the sentences a paper may use, and this section uses only those, with their labels. By the first author's decision these were quick checks of the loop's portability, run with small budgets, and most of the work stayed on CORA's own learning from failure and on the biology that already worked.

domain family	adapter	what ran	result	label
two-dimensional grids: ARC-AGI-2, ConceptARC, Mini-ARC	coloured grids as objects, views and relations	the scored solver; construction from typed holes; the held-back-half test	0.83% official; precision 81.5% (ConceptARC) and 95.0% (Mini-ARC); 64 against 38 of 95 generated tasks; ConceptARC reuse 9 against 27; ARC-AGI-2 construction 2 of 172	official score; related benchmarks; generated tasks; held evaluation, exposed
one-dimensional sequences: 1D-ARC	a row of cells as an object sequence	the scored solver; held-back-half reuse and construction	precision 95.8% (295 of 308); reuse 216 against 88 of 309; construction 286 of 309	related benchmark; held evaluation, exposed
list functions: BIG-bench	integer lists with invertible prefix and suffix operations	construction from typed holes at matched CPU	4 of 9 against 1, inconclusive; three promoted structures equal the deleted operations	controlled authored tasks
Boolean gene networks: published models	gene states, update rules and simulated transitions	inferred regulators; informative observations; a completion vote; conditional construction	101 against 66 of 595; 320 against 291; 2,200 against 2,153 of 2,380; 433 against 321, a classical learner also 433	published-model reconstruction, reused cohort
logic functions and access rules	truth tables; constants, single inputs and two-input gates; a failure description naming an input	the loop splitting on the input the failure names	45 against 12 of 217 held real benchmark functions for a shuffled failure description; 153 of 217 recovered by no method	held items; reconstruction of known functions
enzyme chemistry: EnzymeMap reactions	reaction rules cut from the observed reaction centre	the loop, keeping a cut rule only if it beat the best published rule on leave-one-out checks	76 against 33 of 200 held substrates; a classical nearest-neighbour method 75	held items; reconstruction of known transformations within families
spreadsheet text programs: SyGuS string tasks	a hand-written space of spreadsheet text operations	the loop's six steps, re-implemented	22 against 17 and 17 of 53, neither comparison significant; mainly faster search	held items; a null
signalling logic models: the DREAM hepatocyte network	pieces added to failing readouts from nodes of the learner's network	200 held synthetic tasks, one prior edge removed in each	a result limited to reconstructing one removed prior edge on this network and design (Section 6.11)	held synthetic tasks on a real network
world models: public ARC-AGI-3 games	frames as grids; an actor that moves and a colour map for clicks; 23 kinds of pieces	the build-and-check loop on 12 held games	the pre-registered test did not pass; gains in 3 of the 12 games	probe results on public games, not a competition score
single-cell perturbations: Perturb-seq	perturbation-response profiles as typed relations	strict construction, then a relational follow-up	0 of 64 rules admitted, strictly and after the follow-up	prediction on real measurements; diagnosis
microscopy: LIVECell, NuInsSeg, MoNuSAC; PUMA	cells and nuclei as objects	a strict literature-operator grammar; a follow-up; a calibrated acceptance rule	0 edits on three held-out image datasets; the follow-up failed technically; the calibrated rule harmed (430 of 605 edits harmful)	prediction on real measurements; diagnosis
spatial transcriptomics: prefrontal cortex	spots with markers and neighbours	a marker correction, then relational gates	58 corrections against 65 harms; the gated primary failed its criterion; neighbour-only 36 against 12	prediction on real measurements; diagnosis, one exposed donor
drug response: the PRISM screen	typed rules from a pathway model	the repair procedure on development compounds, applied frozen to held-out compounds	0.5249 balanced accuracy with diagnosis against 0.5295 for try-all at equal compute (416 held-out compounds)	prediction on real measurements; a negative

Table 23: Thirteen domain families where CORA's verify-or-abstain loop design ran through a declared adapter: the entries' own code on grids, sequences and lists, and separately written code for every other family. The first four families and the logic and chemistry tests give positive results (list functions inconclusive); spreadsheets are a null, signalling is narrow, the world-model probe did not pass, and the four measured-data domains are diagnoses that suggest what may need to change next. Every held ARC-style task had been scored once on 2 November 2026 by the submitted entries.

Logic functions and access rules (held items; independently audited). To test whether CORA's learn-from-failure loop transfers to a new domain, we connected it through an adapter (building blocks, a checker and a failure description) to exact recovery of Boolean functions from sparse probes. The real test items were published two-level benchmark functions with 6 to 12 inputs (the Espresso example files, which include the MCNC benchmarks); the methods in the primary comparison saw the same random 25% of each function's truth table, and a recovery counted only if the returned program matched the function on every input, including the 75% never shown. When no block of the base language (constants, single inputs and two-input gates) fit the probes, the loop named the input implicated by the closest conflicting probe pairs and split on it; a control ran the same loop with the named input replaced at random. The protocol, code and data were frozen before any held item was chosen; the held items were selected with a salt taken from a file committed before the run after an independent pre-run review, the single scored run was audited after scoring, and all of its rows were reproduced exactly. On 217 held real benchmark functions in 35 files, the loop recovered 45 exactly (20.7%), against 7 (3.2%) for the base language of two-input gates and 12 (5.5%) for the same loop with a shuffled failure description. Against the shuffled description, the loop recovered 34 functions the control missed and missed 1 that the control recovered (exact McNemar test, Holm-adjusted $p = 2.1 \times 10^{-9}$); the difference in exact-recovery rate was 0.152, with a 95% bootstrap interval clustered by file of 0.075 to 0.241. Both pre-registered primary comparisons, against the base language and against the shuffled description, met the support rule: a Holm-adjusted exact McNemar test and a file-clustered 95% interval above zero. The gain was not concentrated in a few files: the loop beat the shuffled control in 13 of the 35 held files and lost in none. Because the shuffled control also keeps every program the base language finds, the comparison with the shuffled control is the test that the failure description itself carries the gain; the comparison with the base language shows only that growth adds recoveries. The advantage over the shuffled control held at 12.5% probes (29 against 9 of 217) and at 50% probes (67 against 15 of 217), and on 120 synthetic access-control policies generated from real ClassBench seed statistics (44 against 28 of 120); all eight pre-registered secondary comparisons rejected after Holm adjustment. At the same probe budget, choosing three quarters of the probes where the surviving candidate programs disagreed most raised the loop's exact recoveries from 45 to 63 of 217 held circuits and from 44 to 56 of 120 synthetic policies. As descriptive references with no pre-registered test, a CART decision tree recovered 39 and an Espresso-style two-level minimiser 38 of the same 217 held circuits; on the synthetic policies, an unrestricted greedy decision-list learner recovered 49 of 120, more than the loop's 44. The result has clear limits: 153 of the 217 held circuits, including every function in 16 of the 35 files, were recovered by none of the six methods from a quarter of their truth table. A four-fold rebuild check passed 54 of the loop's circuit programs, but only 33 of them were exact, so agreement between rebuilds does not certify an exact recovery. On policies, building rule templates from the failure description made the shuffled control lose but also kept the loop below an unrestricted learner (44 against 49 of 120), which points to using the description as a search order rather than a restriction. In development, 8 of the loop's 12 circuit recoveries came from a single file; on held items no file from that family was present, and the loop's advantage over the shuffled control spread over 13 files. All numbers are from held items; circuit results are real benchmark functions, policy results are synthetic from real seeds, and every recovery is a reconstruction of a known function, not a discovery (`experiments/transfer_logic_20261108/reviews/POST_SCORE_AUDIT.md`).

Enzyme chemistry (held items; independently audited). We applied CORA's learn-from-failure loop to enzyme chemistry through an adapter and ran a pre-registered held test on real enzymatic reactions from EnzymeMap (derived from BRENDA): for each of 40 held enzyme families, the loop saw 3 example reactions and predicted the exact main product of 5 new substrates of the same family [69]. In each family the loop kept a rule cut from the observed reaction centre only if it beat the best published rule on leave-one-out checks over the 3 examples, and otherwise kept that published rule (in 10 of the 40 families no published rule produced any example's product). The loop gave the exact product, and nothing else, for 76 of 200 held substrates, against 33 of 200 for the best of 1,224 published generalized enzymatic reaction rules (Ni et al. 2021), chosen on the same 3 examples and applied one substrate at a time (difference +0.215; 95% family-bootstrap interval 0.115 to 0.330; exact McNemar $p = 2.3 \times 10^{-13}$, with 43 targets recovered only by the loop and none only by the published rule) [125]. All 43 extra recoveries came from the 13 of 40 families in which the loop kept a cut rule (44 of 65 targets, against 1 of 65 for the published rule). The best published rule produced the true product among its outcomes for 132 of the 200 substrates but produced it alone for only 33; the kept cut rules recovered 29 of the 99 targets where it also produced other products, and 14 targets whose product it never produced. These results are reconstruction of known enzymatic transformations on held substrates of the same families, not discovery of new chemistry and not transfer across families; an exact match means the same canonical

SMILES with stereochemistry removed. A classical nearest-neighbour method, which applies the rule cut from the most similar example, recovered 75 of 200, about as many as the loop (difference +0.005; 95% interval −0.110 to 0.115); the gain over the published rules comes from cutting rules from the observed change, which the classical method also does. A control that built its rule from another family's change never passed the family's own checks and fell back to the published rule on all 200 targets; it shows that the gain needs the family's own change, but it is not an independent test. The gain over the published rules was smaller on held families than in development (+0.215 against +0.374). The first held run stopped after 3 of 40 families on an unhandled RDKit error while cutting a rule; a dated amendment, made before any held outcome was opened, removed stereochemistry before cutting rules and made any remaining failure mean 'no rule', and the rerun on the same held set finished and reproduced the first run's 3 completed families exactly. In a post hoc split by the loop's own status, the loop recovered 44 of 65 targets in the 13 families where it kept a cut rule (nearest neighbour 36 of 65) but 7 of 85 in the 17 families where no rule level fitted all 3 examples and it fell back to the published rule (nearest neighbour 19 of 85); using the nearest example's rule as the fallback is a hypothesis for a new pre-registered test, not a result (`experiments/transfer_chemistry_20261108/reviews/POST_SCORE_AUDIT.md`).

Spreadsheet text programs (held items; a null; independently audited). CORA's learn-from-failure loop was applied to a new domain, spreadsheet string transformations. Its six steps were re-implemented in a self-contained adapter: propose, check, describe the failure, build the missing piece, check again, and keep the piece only if it holds up. The loop was tested on 53 held-out tasks from the public SyGuS-Comp programming-by-example string benchmark: 46 real spreadsheet-forum problems and 7 tasks from families written by the benchmark authors. Before any held-out task was run, the protocol, code, data split and two primary comparisons were frozen and independently reviewed. The held-out tasks were scored in a single run, and an independent audit recomputed every reported number from the run records. A task counts as recovered when the program reproduces every held-out example of the task exactly. At the pre-registered budget, the loop recovered 22 of the 53 tasks. It was compared with two arms, which recovered 17 each: the same search without new pieces, given four times the loop's base budget (the loop's maximum search spending); and pieces built from another task's failure. Neither pre-registered comparison was statistically significant. The discordant tasks split 5 to 0 against the deeper search and 6 to 1 against pieces from another task's failure; both Holm-adjusted exact McNemar p-values were 0.125. Given 16 times its primary budget (2,560,000 operation executions), the search without new pieces also recovered 22 of the 53 tasks. The loop's gain at the pre-registered budget is therefore mainly faster search. Four tasks were recovered by the loop but not by the search without new pieces at any budget up to 2,560,000 executions. These are candidates for new ability, not evidence of it: each answer is a single operation from the hand-written space of spreadsheet text operations; blind enumeration of that space also found all four; and each rests on a single held-out example. The pre-registered acceptance rule rebuilds each new piece without one training example and requires the program to reproduce that example. It rejected 6 correct answers and blocked 1 wrong one, and 5 of the 6 rejections were on tasks with only two training examples. Without the acceptance rule, the loop recovered 28 of the 53 tasks. Comparing that arm with the others (28 against 17) was not pre-registered, so that comparison is exploratory. Blind enumeration of every piece in the same space recovered about as many tasks as building pieces from the failure: 21 against 22 at the pre-registered budget, and within 2 tasks at every budget. It used more CPU time: 9.8 against 6.7 CPU-seconds. Matched by total CPU time to the nearest run of the search without new pieces, the loop recovered 4 to 7 more of the 53 tasks. This comparison is descriptive, and CPU time depends on machine load. What failed, and what was learned: with a single held-out example for most tasks, a gain of about five tasks in 53 was not detectable; rebuilding a piece from one remaining example was too weak an acceptance test; and a confirmatory test needs more held-out examples per task and an acceptance rule suited to two or three training examples. These results reconstruct known benchmark functions. The space of spreadsheet text operations was written with the development tasks in view. The results are not discoveries, and they show no transfer beyond this benchmark (`experiments/transfer_spreadsheet_20261108/reviews/POST_SCORE_AUDIT.md`).

Signalling logic models (held synthetic tasks on a real network; independently audited). We tested whether the loop transfers, through an adapter, to signalling logic models. The test used 200 held synthetic tasks on the real DREAM hepatocyte prior network and experimental design. In each task, one prior edge used by a random true program was removed from the learner's network (one of 6 removable edges). On these tasks, the grown program predicted 863 of 1,000 held perturbation conditions exactly. The other arms predicted 731 (the static prior-network program), 805 (the same loop fed a shuffled failure description) and 728 (a CellNOpt-style

comparator). Both pre-registered hypotheses were supported. The grown program beat the shuffled-failure control in 50 tasks and lost 11 (Holm-adjusted sign test $p = 4.6 \times 10^{-7}$). It beat the static program in 62 tasks and lost 12 ($p = 5.7 \times 10^{-9}$). Most of the gain came from one of the 6 edges, mek12 to erk12, which the loop rebuilt exactly in 51 of its 58 tasks. That edge accounted for 118 of the 132 net held conditions gained over the static program and 44 of the 58 gained over the shuffled control. In the other 142 tasks, the grown program led both controls by 2.0 percentage points (617 against 603 of 710 held conditions), which was not significant (sign tests $p = 0.15$ and 0.17). The result is limited to reconstructing one removed prior edge on this network and design. Three conditions apply: by construction, the static program cannot represent the truth; the primary replicates a development finding on new draws; and the shuffled control moves the failing conditions across time points while keeping the readout and direction, which makes it weaker than a shuffle within each time point. On the 25 real DREAM conditions, every arm predicted 14 exactly (grown against static: 1 condition won, 1 lost; McNemar $p = 1.0$). Growth lowered training errors in 4 of 5 folds but did not change held recovery. These data were seen once during development and give low power. On 100 held synthetic tasks on the 12-node CellNOpt toy network, the shuffled control matched the grown program (121 against 118 of 200 held conditions; the shuffled control won 13 tasks and lost 11, $p = 0.84$), as development had predicted. A Python re-implementation of CellNOpt's objective and genetic algorithm, not checked against R, matched the static program (728 against 731 of 1,000). This is consistent with the gain coming from growing the model space rather than from a stronger search of the prior space [144]. The test was frozen and reviewed before any held task existed. The held tasks were drawn from a salt fixed by a GO file committed after the review. The single scored run was audited independently (`experiments/transfer_signalling_20261108/reviews/POST_SCORE_AUDIT.md`).

World models on public ARC-AGI-3 games (a probe for a future direction; independently audited). We applied CORA's build-and-check loop, through an adapter, to real public ARC-AGI-3 games [82]: from 30 transitions observed under random actions, a model must predict the exact next 64 by 64 frame. This is world-model induction, not game play, and it reconstructs the games' own rules rather than discovering new ones. The loop starts from a fixed rule language (an actor that moves and a colour map for clicks). It describes the cells it predicts wrongly, builds candidate pieces of 23 kinds, and keeps a piece only if the piece fixes cells, breaks no transition that was right, and passes a leave-one-out check. Twelve of the 25 public games were held out by a hash rule fixed before any frame was collected. No transition from them was seen before a single pre-registered run in a private Kaggle kernel. On the 12 held games, the grown language predicted 1,204 of 3,168 test frames exactly, against 902 for the fixed language (0.380 against 0.285, a difference of +0.095). It was right on 302 transitions where the fixed language was wrong, and never the reverse. The pre-registered test did not pass: the 95% bootstrap interval over games was [0.0, 0.232], and its lower bound is not above zero. The gains came from 3 of the 12 games. In lf52 (+202 transitions) and tu93 (+21), they came from a piece that turns the first cell of a colour, in scan order, into another colour, as a step-counter display does. In dc22 (+79), they came from a piece that recolours the cells the actor leaves, as a trail does. Because the gains sit in 3 of 12 games, a resample of 12 games misses all three with probability $(9/12)^{12}$, about 3.2%. That is more than the 2.5% tail, so a bootstrap over games cannot exclude zero. The same loop guided by another game's failure ranking predicted 1,125 of 3,168 frames exactly. The loop's advantage over it, +0.025 (95% interval [0.0, 0.076]), came entirely from one game, dc22, so the contribution of an item's own failure description is not established. Copying the previous frame predicted 930 of 3,168 frames exactly, and so did memorising the observed (frame, action) pairs. Among pixels that changed, the grown language predicted 14.7% exactly (11,567 of 78,521), the fixed language 13.5% (10,587) and copying none. The fixed language was less exact than copying in two games (cn04: 51 against 54 frames; ft09: 236 against 261), because its fitted rule changed frames that did not change. The loop kept no piece in those games and inherited the errors. In development, on 13 other games, every gain also came from the step-counter piece, in 4 of the 13 games. Gains from growing the language appeared only where a game showed a regularity that the piece kinds could express. A test at the level of games needs more games or a wider set of piece kinds. These are probe results on public games, not a competition score (`experiments/transfer_worldmodel_20261108/reviews/POST_SCORE_AUDIT.md`).

What the new domains add, and what they do not. Read together, and only as their audits allow, these tests show the loop design running through five more adapters on held items, with one result in which the failure description's own content carried a gain over a shuffled control (logic functions), one gain over fixed published rules that a classical method matched (enzyme chemistry), one null (spreadsheets), one result limited to one removed edge (signalling) and one probe that did not pass (world models). Each domain's code was written

separately; nothing learned in one domain was carried to another; and every result is a reconstruction of known functions, reactions, models or game rules, not a discovery.

6.12 Reconstruction is not discovery, and what a transfer test would need

For the repair procedure, which the entries do not run, portability is shown: the same sketch, completion, revision and controller code ran in three biology domains, with traces. Utility is not: in no domain did the procedure beat every matched control, and where it improved a result (nuclei repair, repair on top of StarDist) failure-blind controls matched it, so the gains belong to the repair procedure, not to a learned diagnosis policy. **Learned cross-domain transfer is open**: nothing was trained.

The object worth transferring next is not ARC's grid operations but the mechanism that worked in the construction test: a specification of the missing operation deduced after a failure, completed by symbolic construction and admitted through the same checks, frozen with its controller. An admissible test would (i) freeze and hash the controller, checks and proposer prompts before any outcome; (ii) give each domain an adapter that supplies typed primitives with roles, an executor on development units, a validation pool and a regression set, checked on already-inspected development fixtures only; (iii) compare the frozen procedure with failure-blind, shuffled-report and compute-matched search conditions on units that are donors, images or compounds, never cells or pixels, with intervals clustered by those units; and (iv) open each sealed set once, under a rule fixed in advance like the follow-up's (Appendix B.3). Microscopy needs a new fresh image set, since the one above is spent; the 811 sealed drug-response compounds can serve a second, semantically different domain, but only for a controller that first passes the development criterion. Because the ARC confirmation of guarded rewriting was inconclusive, the neural inventor did not beat failure-blind invention or matched search, and the construction test's edge over matched search is inconclusive and awaits independent replication, no such test has been run, and none is reported.

A matched comparison of conditional construction against a learning-from-failures inductive logic programming (ILP) learner with predicate invention, Popper [47], on the 595 sparse targets, at equal observations and resources, is the next experiment, to be pre-registered. It requires a validated whole-program four-input constraint inside Popper's search rather than a filter on its output. Two further tests are planned, and neither is a result. The first is a held replication on a fresh cohort of curated gene-network models outside the Biodivine collection [78]. It is not feasible today: a citation-only review has so far certified no case as fresh, and the published models that meet the replication's lineage, size and licence criteria give only 102 targets from 13 lineages, against a planned size of about 150 (`records/FRESH_BOOLEAN_COHORT_CANDIDATES_20261108.md`). The second is a cellular-automaton benchmark that separates query cells the demonstrations identify from those they leave ambiguous [105, 106]. Published enzyme reaction rules, the chemistry domain in which one atom-mapped example fixes a whole bond edit [69, 125, 149], have since been tested within families (Section 6.11); transfer across families is untested. Drug-combination effects and table repair remain candidate domains. Section 8.5 gives the designs and what would refute each.

7 The proposed paradigm: failure-grounded co-induction

In terms of the core idea: its next step is to learn which change to make, and this section states that step as a research paradigm.

7.1 The paradigm stated

We propose to treat a reasoning system's representation and its executable language as hypotheses, induced together from failures under the same evidential discipline as the programs they make expressible. *Co-induction* means that a change of representation and a change of language are chosen jointly, because each changes what the other can say: a new view shortens many programs at once, and a new operation can be constructed only over objects that some view provides (Figure 3 needed both). *Failure-grounded* means that the change is chosen from the typed record of what failed, and adopted only on evidence it was not built on. The object of learning is then the revision policy of Section 4.2, together with the representations and languages it produces, and the measure of success is *acquisition efficiency*: verified correct answers gained on held tasks per unit of counted work spent diagnosing, constructing and admitting, at matched precision, against a static system that spends

the same work on search. This is skill-acquisition efficiency [26] applied to the reasoner's own capabilities. Figure 16 draws the cycle and marks, stage by stage, what CORA has implemented, what it has tested only narrowly and what remains a proposal.

7.2 What CORA currently establishes, and what decisive evidence is missing

The organising frame: know the gap, then grow expressibility. The central idea is to know the gap and to grow the language's expressibility to mitigate it. The work is a loop of three steps, each with its own evidence status. (1) *Detect the gap.* Typed failure records report where a search failed: no fit within the declared representation, grammar and bounded search, as distinct from a search stopped by its budget. That is a diagnosis, not a proof that no program exists. Evidence: the Perturb-seq record, whose failed attempts are mostly typed `no_relation`, meaning that no relation the language can state explains the change (Section 3.1), and the finding that the fixed construction language has no structure for "keep the shape, then apply a shared palette map", since it has no identity geometry (the companion study's records; available from the authors on request). (2) *Grow within a fixed space.* The construction test builds the missing operation from the fixed 211-structure language and works on constructed tasks, 8 of 11 against 0 with another task's specification, though not better than matched search (Section 5.5.2). It is not enough on natural tasks: a frozen learned library changed no prediction there, and no promoted operation fitted the demonstrations of the source tasks (Section 5.8), which motivates testing other representations, operators and binding rules. (3) *Grow the language's expressibility itself* from the diagnosed gap. The first such extension, identity geometry followed by a palette map, has been tested against a random extension of the same size (Section 5.7): on the 120 public evaluation tasks all 13 language variants, the extension included, fitted 0 tasks, and on the 1,000 training tasks (development only, reachability only) the extension fitted 18 against 17 for the base language and 17 for the random extension, so its pre-declared criterion failed. That hand-supplied one-structure extension did not close the measured reachability. Read through this frame, the null results are a diagnosis: growth must extend expressibility, not only recombine a fixed set of structures; they motivate further representation, operator and binding experiments, and they do not establish a minimum number of new structures.

The central hypothesis and the architecture. A reasoning system can improve generalization by using execution failures to determine which representation, program structure or executable capability to revise. CORA's architecture follows it: observations, candidate representations, program construction, execution, failure evidence, candidate revisions, verification, a capability library, and reuse. The full self-extension loop exists across research modules (Appendix I), but not as one deployed system: the failure-to-specification construction and the library run only as companion research, and the submitted Kaggle notebooks do not run the loop end to end: the CORA-only entry contains no grammar promotion, the growth prototype runs in a separate public research notebook (`https://www.kaggle.com/code/chimdiwalter/cora-grammar-growth-research-replay?scriptVersionId=365118951`), and its integration into a scored entry is future work. The entries run only the last step, verify or abstain (Appendix A); construction from failure is a separate controlled test (Section 5.5); Table 15 grades what the experiments establish.

A target hypothesis, not a result. We treat a failed explanation as a decision among revisable levels: the representation, the executable rule language, the search, and the evidence needed to tell hypotheses apart. The query output is not available to that decision. What might transfer between grids and biology is the discipline of proposing a revision, checking it against observations, and rejecting it when equally equipped alternatives do as well. The construction test supports bounded construction and reuse of operations, not learned selection among all four levels.

What is missing. Five pieces of evidence would turn the paradigm from a proposal into a demonstrated approach, and none exists yet. (i) A learned revision policy that beats fixed intervention orders and matched search at equal cost on held tasks (Conjecture 1); no such policy has been trained, the learned revision ranker lost to streaming static search, and in the first pre-registered test of failure-guided choice a router that read the failure record gave the same answers as a fixed order of the same changes (Section 5.9). (ii) Representation changes chosen from failure records that add reach on natural tasks (Conjecture 2); the one diagnosed extension was hand-supplied and its reachability criterion failed. (iii) Construction beyond a declared structure space; the constructed-task results, 8 of 11 and 64 of 95, are inside a 211-structure space that contained the answers, and on held related tasks construction and reuse gained within 1D-ARC while reuse lost to static search on

ConceptARC. (iv) An admission bar keyed to prior knowledge, tested against uniform bars at matched precision; a calibrated bar set on one dataset has already failed on a new source (Section 6.7). (v) Learned cross-domain transfer of a revision procedure (Conjecture 3); what crossed domains so far is software and design, not learning.

Learning what to change, as a research paradigm. We propose learning what to change as a research paradigm opened by this evidence, not as a finished result. Its object is the third requirement, capability selection: a system that, after a failure, chooses among more search, a changed representation, a new operation, more evidence and abstention, and is judged by verified answers gained per unit of work against strong static access to the same capabilities. The evidence above shows that availability and discovery can be engineered and measured; it shows no learned selection yet. The paradigm is falsifiable through the predictions of Section 4.5 and the experiments of Section 8, and it is refuted for a system if failure-blind or shuffled selection does as well.

What the frontier gap is, in a biological case. Modern neural models learn representations, predict outcomes, generate programs and improve their answers with more computation; program synthesis searches for executable explanations; grammar induction discovers reusable abstractions; and verification tests consistency against declared specifications. Each of these is strong, and this paper does not claim otherwise. The harder problem is deciding which component should change when they fail. A failed regulatory prediction makes the decision concrete, because it may need any of six different interventions: more search, a better choice of regulatory variables, a richer logical rule, a different representation of biological context, more observations, or a revised assumption about noise or dynamics. CORA's biology studies measured each of these separately (Table 24). Inferred regulators reconstructed 101 against 66 of 595 published rules; informative observations gave 320 against 291; a richer conditional language gave 433 against 321, equalled by a classical learner by design; more observations raised exact rules from 433 to 465 while correct held predictions fell from 2,070 to 2,054; a relational representation of Perturb-seq responses still admitted no rule; and a neighbourhood guard in spatial transcriptomics gave a modest exploratory gain on one donor. In every case a person, or a fixed hand-written rule, chose the intervention. CORA's long-term objective is a system that reads the failure and makes that choice itself. That decision process has not been demonstrated on natural biological tasks: the one loop with a fixed hand-written router ran on authored functions, took the same path in every built case, and a fixed build-then-ask schedule recovered the same 219 of 256.

a failed prediction may need	what points to it	what CORA measured	who chose it
more search	the search stopped at its budget, not at the end of its space	a fixed build-then-ask schedule matched the diagnosis loop (219 of 256 authored functions); in a language too small, exhaustive search over all 16 two-input rules shows search cannot help (authored)	fixed rule
better regulatory variables	the true regulator is outside the candidate set	inferred regulators: 101 against 66 of 595; an essential regulator lay outside the shortlist for 176 of 356 tasks (published models)	researcher
a richer logical rule	observations agree on every input the rule reads and differ in outcome	conditional construction: 433 against 321 of 595, all gains with three or more inputs, equalled by a classical learner by design (published models)	researcher; fixed rule in the authored loop
a different context representation	the record says no stateable relation explains the change	Perturb-seq: 1,842 of 2,508 failures typed no relation, and a relational follow-up still admitted 0 of 64; spatial: a neighbourhood guard, 36 corrections against 12 harms on one donor (measured data)	researcher
more observations	fitting rules disagree at the query, or the data carry too little information	informative observations: 320 against 291 of 595; 60 instead of 40 observations: exact rules 433 to 465, held predictions 2,070 to 2,054 (published models)	researcher
a revised noise or dynamics assumption	identical full states with different outcomes	the teaching case refuses a deterministic rule on an identical full state with opposite labels (authored); no measured study yet	fixed rule; open

Table 24: Six interventions a failed regulatory prediction may need, what points to each, and what CORA measured. In every row the intervention existed and was chosen by a researcher or a fixed rule; choosing among them from the failure evidence is the open problem.

A step toward open-ended growth, bounded today. Learning what to change is a step toward open-ended capability growth: a reasoner that keeps turning its own failures into new, checked capabilities, and keeps choosing what to learn next, without a fixed list of skills. Open-endedness has a long research line: search for novelty without a fixed objective [99], systems that generate their own problems and solutions [166], algorithms that generate the learning process itself [44], agents that grow a library of executable skills [164], and the

argument that open-endedness is essential for superhuman intelligence [73]. CORA differs in where each new capability comes from and how it is kept: it grows a capability from its own failure evidence, checks it before keeping it, and measures it against matched static controls. Today's growth is bounded: it selects from a supplied 211-structure space, the operations it can build are supplied, and the reuse evidence is within families. The open-ended version is a direction with testable predictions, not a result.

7.3 What would distinguish it experimentally from earlier approaches

Each neighbouring line of work makes a different prediction from the paradigm, and each difference can be measured. *Library learning from solved programs* [16, 57, 66] leaves what the language can express unchanged and raises what a bounded search reaches, as DreamCoder's held-out ablations show; construction from typed holes, which selects from a supplied space that contains the answers, acts on the same quantity, so the two make no different prediction until they are compared on held tasks at matched total compute (construction from failure, compression of solved programs and static search). Our own compression test is one instance, not that comparison (Section 5.8). *Neural guidance of synthesis and test-time training* [2, 11, 138, 150] changes the search order inside a fixed language, or a transductive model's weights for each task; guidance changes what a binding budget finds while leaving what a complete search could reach unchanged, which holds for complete enumeration in any approach, and the held search-order test fits that reading at the budgets tried (Table 13), though 20,000 counted operations is not an asymptote; a language change alters what can be reached. *Neuro-symbolic filtering of proposals* [50, 76] keeps proposals consistent across demonstrations from a fixed language; the paradigm adds what is changed after the filter rejects everything. *Rational metareasoning and learned meta-control* [143, 173] choose computations of a fixed object-level system; the paradigm's action set changes the object-level system itself, and so needs held validation of every persistent change. *Refinement from counterexamples* [21, 43] changes a representation when a counterexample proves spurious; in the inductive setting a counterexample can itself be ambiguous (Proposition 4), so a refinement must be validated on tasks it was not built on. *Growth from failure in grammar induction, bias shift and predicate invention* [14, 18, 46–48, 59, 116, 160] adds terms, predicates or rules, or prunes them, where analysis fails; the paradigm adds typed holes computed from the demonstrations, selection from a declared space instead of pruning, charged admission, and controls that replace only the content of the failure.

The decisive experiment therefore has the same shape whichever neighbour it is aimed at. At matched counted work on held tasks, and on tasks where the order of trying changes can matter, it compares strong static search; the same work spent on more search; failure-blind revision; revision from another task's failure record; revision from the task's own failure record; and the same with verified reuse of admitted capabilities. It records executable programs constructed, exact predictions, cost, capabilities accepted and regressions, and it reports failures as outcomes. The paradigm is supported only if the authentic failure-conditioned arms beat every other arm on newly reachable held tasks; it is refuted, for that system and task family, if failure-blind or shuffled revision does as well.

7.4 The working hypotheses and their status

The working hypotheses, mapped to the evidence. (hypothesis 1) A revision policy, given a failed explanation and its structured residual and choosing what to revise, can be learned in one family of domains and help in another: *open*; no policy is trained yet, and domain-calibrated controllers did not beat their bookkeeping-only ablation (Section 6.8). That taught us that their domain calibration added nothing beyond their bookkeeping. It opens a trained revision policy, tested against the same ablation. (hypothesis 2) Structured failure records improve revision choices over failure-blind controls at matched compute: *open, with one held test in support*; on 217 held real benchmark logic functions the loop recovered 45 against 12 for the same loop with a shuffled failure description, the test that the failure description itself carries the gain (reconstructions of known functions; Section 6.11), while on 50 held controlled generated tasks a router that read CORA's failure record gave the same answers as a fixed order of the same changes (Section 5.9); the language-model inventor's failure reports tied failure-blind invention (2 against 2 of 11 tasks), the learned revision ranker's failure-blind and shuffled conditions matched it (Appendix B.5), and its dedicated pre-registered test, on BIG-bench list functions at matched CPU, recovered 4 of 9 tasks from the failure's typed hole against 1 for blind enumeration of the same space: all 3 discordant tasks favour the hole, but with 3 of them the smallest attainable p is 0.25, so the pre-declared result is not confirmed (`records/NOV2_U3B_RESULT.md`). This taught us that 9 tasks cannot decide the

question. It opens a larger list-function assay. The two ties taught us that neither the inventor nor the ranker used the failure record's content, while the symbolic constructor did. They open proposers given the typed specification itself. (hypothesis 3) On real ARC tasks the binding constraint is mostly representation, not search depth: *consistent with the evidence for CORA's language on the public evaluation*, as a measurement rather than a pre-registered test: within the measured budget no demonstration-fitting program was generated for any of 113 audited puzzles, a reachability limit that does not by itself separate representation from search, and the failure anatomy puts expressivity first in 29 of 46 sampled failures (Section 5.3). It opens broader grammars with deeper search, varied one at a time to separate the two. (hypothesis 4) The value of such a policy is measurable by counterfactual ablation at matched compute, with planted-cause tasks: *the design is in place* (the construction test plants a deleted operation and runs shuffled, demonstrations-only, blind and matched controls), and the learned policy it would measure does not exist yet.

8 Open problems and testable predictions

In terms of the core idea: each open problem below asks how a failure can be used better as evidence.

Each problem below is stated as a hypothesis, an experiment that could support it, and the result that would disprove it. All are open; none is a result of this paper.

8.1 Real ARC-AGI-2: the missing causal evidence

The missing evidence is a verified improvement on real ARC-AGI-2 answers caused by choosing the right revision after a failure. The components exist: representation change, binding repair, composition and proposed reusable operations, each implemented and each with its own measured cases (Sections 5.5 to 5.8). What does not yet exist is the chain on a real task: a diagnosis made before the answer, an automatic revision, a program that fits the demonstrations, a correct committed answer, and a win over strong full-bank, static and shuffled-failure controls (Section 5.7). On the 36 exposed tasks of Section 5.4 the first link exists in a limited form: a diagnosis from the demonstrations alone names the missing kind of change for each task, though its rules were refined on those tasks; the revision, the fit and the controls are still missing. Two results bound the present position. The 1,000-task demonstration sweep exposed an early-stop defect in the expanded view search; a minimal correction that refuses only the oversized view passes invented-grid regression tests, no corrected natural run has been made, and no new ARC accuracy result follows from it. The held-back-half test's check on the public evaluation gave 2 of 172 outputs for task-local construction and none for every other arm, from one task, an inconclusive result on exposed tasks; both outputs are the CORA-only entry's own answers. A companion experiment on choosing the right change, with authentic against shuffled failure descriptors over identical capabilities, a fixed full-bank controller and an extra-search baseline, has passed review as a draft and has not run. Two versions on generated tasks with planted causes have each run once, pre-registered and audited: in the first, failure evidence beat a random order of the same changes but not the best fixed order, the pre-registered outcome none; in the second, where a change cannot cheaply check whether it applies, no fixed order of the bank reached the oracle (best 44 against 55 of 84) and the pre-registered router did, an advantage not established against that best order, chosen after the results (Section 5.9). Real ARC-AGI-2 cannot decide the question yet: every CORA mechanism together solves 2 of the 120 public tasks, so even a complete split of those tasks gives a two-sided exact p of at least 0.5. The decisive comparison must use a budget that binds, because fixed alternatives tied whenever the budget was generous (219 against 219 authored Boolean functions; 327 against 329 correct outputs of 624 at 20,000 counted operations per hole), it must use changes whose order can matter, and it must report two-dimensional tasks separately. *Disproved*, for the tested system, if, where the order of trying changes can matter, authentic failure-conditioned routing does no better on held real answers than static access to the same capabilities at matched complete cost.

8.2 Six open questions, each with how it could be disproved

1. *Inducing the operations that the neural gap names.* A task-by-task post-mortem of the outputs the neural method solved has now been run (Section 5.4; post hoc, exposed public evaluation, not an official score). None of the 41 query outputs that the neural comparison entry gets right and CORA does not answer gains a program at ten times the budget; the first gap is a missing action in 35 of the 36 tasks (in 14 of them the object views also

fail), a missing guard in 1, and never depth or budget. The hypothesis that the gap is mostly representation and language therefore survived the test that could have disproved it, and neural guidance would help most by proposing new actions and views. The open question is whether the named operations (drawing on the background, editing parts of objects, assembling new output grids, panel and pattern views, guards that relate one object to another) can be built from each task's few demonstrations, with or without the neural attempt as one more observation, and then reused. *Disproved* if operations built from these failures add no right answers on held tasks over static access to a library of the same size at matched counted work, or are admitted from shuffled failures as often as from the real ones.

2. *Scoring a representation without running actions.* Views might be chosen before any program search by cheap scores on the demonstrations' object graph: whether the object count is invariant, whether the multiset of shapes is conserved, and the description length of the correspondence between input and output objects. This would gain reach without the search tax and the spurious fits that a larger language brings. The one action-free score tried so far, a fixed hand-written score that searched only its two best-scoring views, solved 3 of 9 authored cases against 9 of 9 for the full bank, so the test must compare with executed actions. *Disproved* if such scores choose views that add reach no more often than a random order of the bank.

3. *Minimum-residual projection on grids.* This is the ARC analogue of the biology completion vote. When no program fits every cell, keep the programs that explain at least 90% of objects, checked by leaving out the objects each governs, and use them to repair local errors in neural guesses or to vote on what happens to each object. A first observation, on one design-exposed task: on 0934a4d8 a symmetry completion program fixes 19 of the 27 cells of the query output, and both neural attempts are wrong only in the 8 it leaves open (Section 5.4), so a partial program can locate a neural error that it cannot repair. *Disproved* if, at matched work on held tasks, such projections add no correct output over the unrepaired neural guesses, or lower precision.

4. *Why selectors ignore failure evidence.* Learned selectors gave byte-identical outputs across authentic, failure-blind, fixed and random arms (111 of 128 each), and a frequency table (318) and a linear model (323) matched the learned guide (320). Two hypotheses compete: a feature bottleneck, in which the hole features cannot separate operator families, so learning collapses to base rates; and candidate dominance, in which a few general schemas win early under the cap. A post hoc analysis of development data (the source halves of ConceptARC, Mini-ARC and 1D-ARC; 85 of its numbers re-derived by separate code) separates the two. The failure features turned out to be informative: they carried at least 2.82 of the 2.93 bits needed to say which kind of edit would succeed and named it in 98.1% of the 633 holes that had an exact program, but a plain table of past success rates already ran a working search first in 95.6% of those holes, so the learned guide, a linear model and the table fitted 284, 286 and 285 of the 425 development tasks at 500 operations per hole, against at most 289 for any possible order (297 if the eight tasks whose programs were never retried also fitted) (post hoc, development data). This taught us that the tie came from candidate dominance, not from a feature bottleneck: a few general edits almost always work when the comparison of demonstration pairs proposes them (gravitate in 109 of 109 holes, connect in 149 of 161), so once the language holds the right edit, choosing what to try first is easy and a learner has almost nothing left to win (post hoc, development data). It opens the harder half of the problem: 642 of the 1,275 holes had no exact program at all, so progress needs building the missing edit, and a fair test of learned guidance needs tasks where the right edit is rare and depends on the task (post hoc, development data). The scorer of that analysis read each benchmark's single solutions file, which also holds the held outputs, and kept only source tasks; nothing held was used or scored. *Disproved*, for candidate dominance, if on tasks where the right edit is rare and depends on the task a table of success rates still matches a guide that reads the failure features.

5. *The false-discovery curve of promoted operations.* In biology, dropping the held-out check raised exact rules from 84 to 200 but wrong accepts from 55 to 451; in microscopy a tolerant gate's 32 accepted edits on held NuInsSeg images were 10 beneficial, 16 harmful and 6 neutral. The false-discovery rate should be measured against the number of demonstrations and against an operator's degrees of freedom, since a colour table can overfit far more than a parameter-free move, and then an acceptance bar weighted by description length tested. A post hoc analysis of the same development data gives the first curve. Of the 4,832 constructed rules that fit every demonstration and were suggested independently by at least two of them, 562 (11.6%) were wrong on the task's query output, from 8.5% on 1D-ARC to 57.9% on ConceptARC, and rebuilding the same 277 1D-ARC tasks from fewer demonstrations raised the wrong share among fitting rules from 9.7% with all three to 30.3% with one (post hoc, development data); the 11.6% pools rules, most of them from 1D-ARC, and is not a rate per task. This taught us that fitting is weak evidence and that ranking is not where the

errors are: in 25 of the 41 tasks whose first answer was wrong no right rule had been built, 14 of the other 16 were one rule read under two ways of cutting the grid into objects that agree on every demonstration, and preferring shorter descriptions changed one task each way (post hoc, development data). This is why selection is the frontier: what separated right rules from wrong ones was more evidence, in the form of more demonstrations, and the second attempt that ARC scoring allows already covered all 281 tasks that had a right rule, so the open problem is deciding when the evidence singles out one rule, which observation to ask for when it does not, and when to hold back (post hoc, development data). For the bar weighted by description length, keeping every right rule, the shortest description-length cut lowers the wrong share only from 57.9% to 51.4% on ConceptARC (107 of the 208 rules it keeps) and from 47.5% to 45.5% on Mini-ARC (35 of the 77 it keeps), and not at all on 1D-ARC (8.5%), so such a bar fails on 1D-ARC and helps little elsewhere (post hoc, development data). Whether more free constants make a rule wrong inside a task cannot be read from these data: only 5 of the 31 tasks that accepted both kinds of rule could show a difference. *Disproved*, as a direction, if a bar weighted by description length, frozen before a held run, gives no lower false-discovery rate than the uniform bar at matched recall.

6. *Breadth and depth of biological queries at three or more inputs.* Active selection recovered all 165 one-input and 155 of 156 two-input rules but none with three or more inputs, and produced more no-fit abstentions than uniform choice (191 against 126). Simulating adaptive arity expansion at 20, 40 and 60 labels, with a cofactor split triggered once a third essential input is certified, asks how many labels separate a true three-input rule from noise; the first natural run reached 433 against 321 rules at 40 labels, which a classical learner equals by design. *Disproved*, as a claim for adaptive expansion, if it recovers no more three-input rules than a fixed expressive learner at the same number of labels.

8.3 Representation revision on natural tasks

Hypothesis: failure records can choose representation changes (views, relations, output canvases, object correspondences) that add reach on natural tasks. *Experiment:* choose each change from source-half failure records only; measure its newly reachable held tasks at matched counted work against random changes of the same size and against library growth alone; then score held-query correctness once on the newly reachable tasks. The held-half test of the object-level step and the construction test at scale have run: the first gained within 1D-ARC and lost on ConceptARC, and the second found operations reliably inside a supplied space (Sections 5.8 and 5.5.3); neither chose representation changes from failure records, so the hypothesis is untested. *Disproved* if failure-chosen changes add no more newly reachable held tasks than random changes of the same size (Conjecture 2).

8.4 Useful grammar growth

Hypothesis: operations grown from natural failures are selected again on held tasks of the families that produced them. *Experiment:* under agreement-based acceptance, report the set of newly reachable tasks, not total fits (Proposition 7); merge task-local operations that differ only in a literal by anti-unification [136], whose typed parameters are bound from each task's demonstrations, and test the merge against expanding every value at the same compute and against random merges; and pair a reference-sized grammar with a search that reaches programs of 10 to 30 nodes, since perfect search inside the present language reaches at most 22 to 88 of 842 training tasks. *Disproved* if grown or merged operations are not selected on held tasks of their own families, or if merging does no better than expansion or random merges at matched compute.

8.5 Evidence calibration and the choice of data

Hypothesis: an admission bar keyed to consistency with prior knowledge (3), and evidence chosen for the hypothesis at hand, admit more correct revisions than a uniform bar and random evidence at matched precision. *Experiment:* vary the bar with the prior fixed, and the prior with the bar fixed, which the inferred-wiring design could not separate; compare random, positive-only and actively chosen demonstrations at equal numbers. The invented-rule simulation of Section 4.3 predicts the shape of the result: designed states should win only above the counting floor and once the variables are found, and should then expose the shortlist, the acceptance rule and the language as the next bottlenecks. *Disproved* if the keyed bar admits no more correct revisions than the

best uniform bar at matched precision, or if chosen demonstrations do not raise exact recovery over random ones at equal numbers.

The Boolean studies are limited by the evidence in each task as well as by the rule language (Proposition 9), and these limits suggest four further tests. First, fresh curated networks: of the 61 expert-curated models in the meta-analysis of Kadelka et al. [78] that do not come from the Cell Collective database, 42 have no publication match in the Biodivine Boolean Models collection [133]. A citation-only review of these 42 has certified none as fresh: six carry earlier exposure alerts and stay quarantined, and of the other 36, 20 are independent on bibliographic evidence, 10 unresolved and 6 likely derived from models used before (`records/NOV8_KADELKA_PARENT_EXPOSURE_LEDGER.md`). After an overlap check at the level of rule content, the frozen reconstruction can be rerun on the independent ones under a plan registered before any rule file is opened. Second, choosing the data: whatever states are shown, ten binary demonstrations leave a single consistent rule for at most 1,024 of the 388,656 rules with up to three inputs in a 23-gene model, yet eight states that cover every combination of a rule's three inputs, with all other genes at 0, leave only that rule whenever it depends on all three, and a binary search between two states with different next values finds each relevant input in about $\log_2 n$ chosen states for n genes [146]. Third, structure: 4,827 of the 5,110 rules analysed by Kadelka et al. are nested canalising, meaning that the inputs are checked in a fixed order and the first input found at its canalising value fixes the output, and restricting the language to such rules lowers the worst-case number of demonstrations for a six-input rule in a 23-gene model from 81 to 35. Fourth, chemistry: EnzymeMap atom-mapped recorded reactions from the BRENDA enzyme database by applying the published enzyme reaction rules of Ni et al. [69, 125], and the Pickaxe software ships the 1,224 generalized rules together with less generalized variants [149]. A held test within families has since run: on 200 held enzymatic reactions, given 3 examples per family, the loop gave the exact product for 76, against 33 for the best fixed published rule and 75 for a classical nearest-neighbour template method (Section 6.11); these results are reconstruction of known enzymatic transformations on held substrates of the same families, not discovery of new chemistry and not transfer across families. What remains is one typed molecular-graph language tested across families, on real reactions and on simulated ones that give exact negatives and allow chosen substrates, with one round of grammar growth from the failures on half of the rules before a single frozen run on the other half. Finally, CellARC [105, 106] separates query cells that the demonstrations identify from those they leave ambiguous, which tests whether abstentions fall where they should; its learned rule proposer with an exact verifier is the closest published design to ours. The measured five-gene yeast network IRMA [24] offers one case beyond simulation, although its designers specify wiring rather than rules. In every case, ambiguity is a property of the data relative to the rule language.

A larger rule language therefore needs a sharper support, a prior over rule forms, such as the short read-once and canalizing rules that are over-represented in published models [159], or more informative data. We will test a learner whose rules may read any number of inputs, as in learning from interpretation transition [75], at ten and at thirty recorded transitions, and growth triggered when the fitting rules disagree rather than when none fits. At scale the acceptance policy hid some exact two-input rules, so the readout and the acceptance policy, not the rule language, are the first things to diagnose.

8.6 Exact and noisy evidence

Hypothesis: one executable revision loop can serve exact and noisy domains, but its acceptance step should differ: consistency plus ambiguity checks when observations are exact; a likelihood comparison under a calibrated, class-conditional noise model, applied only above the event floor of Proposition 10 and judged by its precision on the population where it is used, when observations are noisy. *Experiment:* in two noisy domains and one exact domain, cross scalar against relational representations with consistency, a fixed-constant comparison and a calibrated comparison, after counting informative events on training data, with failure-blind, shuffled, matched random-filter and standard-method controls and donors or perturbations as units. *Disproved* if calibrated comparison does not beat consistency above the floor in the noisy domains, if it beats it equally under shuffled evidence, or if the same tolerance helps as much in the exact domain. The present biological follow-ups are consistent with parts of the hypothesis but were not designed to test it: whether Perturb-seq sat below the event floor has not been counted, the spatial signal is one donor, and the microscopy test has no qualified endpoint. The calibrated acceptance test of Section 6.7 bears on the last clause: a harm bound calibrated on one dataset did not hold on a new source. The noisy-data test of the reconstruction procedure is a planned experiment and has not run: its written plan fixes a declared noise model, calibration only on permitted development data or authored fixtures, a frozen split and full cost accounting before any held run, and comparisons with the best

fixed schedule, blind growth, shuffled failures and a noise-aware classical method (the future-experiment plan in the companion study's microscopy erratum record); it reports no result.

8.7 Capability reuse

Hypothesis: capabilities admitted from failures are reused on later tasks that need them, at a lower cost than rediscovering them. *Experiment:* freeze the grown registry after the source halves, run it once on the held halves and on unseen families, and count tasks solved through a promoted operation at the base budget against the same search without the registry. *Disproved* if promoted operations solve no more held tasks at the base budget than the registry-free search, or are never selected outside the tasks that produced them (Prediction 5).

8.8 Transfer of learned revision policies

Hypothesis: a revision policy learned in one family of domains helps in another through its procedure and second-order structure, not its operations (Conjecture 3). *Experiment:* the admissible test of Section 6.12, with the policy frozen and hashed before any outcome, domain adapters checked on inspected fixtures only, failure-blind, shuffled-record and compute-matched search controls, units that are donors, images, compounds or models rather than cells or pixels, and each sealed set opened once. Transfer between unrelated domains should be sought in shared second-order templates, such as a threshold on a count over a neighbourhood, rather than in operations [51], and an inductive logic programming learner run on CORA's object-level facts, as relational decomposition did with pixel facts [70], is the natural external baseline for richer relational selection rules. *Disproved* if the transferred policy does no better than the same procedure with failure-blind or shuffled records, or than a policy learned in the target domain alone at matched data.

8.9 The decisive test of the central conjecture

Conjecture 1 is tested by the experiment of Section 7.3: strong static search, matched extra computation, failure-blind revision, shuffled-failure revision, authentic failure-conditioned revision, and authentic revision with verified capability reuse, on held tasks, at matched counted work, with a computationally accounted route that preserves the static baseline's answers. Its outcomes are executable programs constructed, exact predictions, search cost, capabilities accepted and regressions. The conjecture is disproved for the tested system if the authentic arms do not beat matched extra computation and the failure-blind and shuffled arms. The studies still in progress, each held behind a pre-declared success criterion, are in Appendix D.

8.10 The next experiments, and why they matter

Seven experiments follow from the evidence, in order. (1) *Failure-conditioned routing against strong static access*: identical capabilities and demonstrations in every arm, with authentic and shuffled failure descriptors changing only the routing, on real ARC-AGI-2 answers; its first version, on generated tasks, is reported in Section 5.9, and its audit's lesson sets the condition for the next one: changes that cannot cheaply check whether they apply. (2) *A learned router on held tasks*: the same comparison with a trained router, reported apart from any hand-written one, since a fixed rule is not learned metareasoning. (3) *Active evidence acquisition in interactive settings*, in the spirit of ARC-AGI-3 [82], where the system chooses its own observations, as the biology studies already do in simulation; a first probe of world-model induction on 12 held public ARC-AGI-3 games did not pass its pre-registered test (probe results on public games, not a competition score; Section 6.11). (4) *A fresh biology cohort*: a citation-only parent and exposure ledger for the expert-curated models outside the collection used so far is complete and certifies no case as fresh (`records/NOV8_KADELKA_PARENT_EXPOSURE_LEDGER.md`), so a roster frozen before any model body or label is opened comes first. (5) *Laboratory-testable hypotheses*: rules frozen before scoring, a discriminating intervention state for each pair of rival rules (Section 6.5), and a qualified collaborator before any laboratory claim. (6) *Changing the representation when it collapses*: on the first author's maize leaves the starting representation scored near zero and CORA abstained everywhere (Section 1), so the next step detects a collapsed baseline and changes the representation itself before editing. (7) *Building the operations the neural gap names* (Section 5.4): actions first, then views, guards and depth, judged on held tasks against static access to the same operations and against shuffled failures.

Why it matters: skill-acquisition efficiency is about how much a system learns per unit of experience [26], and a system that can tell which part of its reasoning to change spends that experience on the right revision. In science the same choice decides whether a failed prediction becomes a new variable, a new measurement or a rejected hypothesis. Both depend on the selection requirement that this paper leaves open.

8.11 Limitations of the present evidence

- *Fitting but wrong programs, learned from few examples.* This is the central scientific limitation of executable reasoning from a few demonstrations. A program can reproduce every demonstration exactly and still be wrong on the test: from rival fits under true ambiguity, and from spurious fits when the rule is inexpressible (Section 3.1). CORA's 98 and 162 fits on one evaluation puzzle, the original engine's 33 fitted and wrong evaluation puzzles, and the 3 fitting but wrong sealed predictions of the frozen CORA-CAST core are instances (that core ran without its acceptance filter, so whether the filter would remove them is not measured). Exact verification on the demonstrations, and rebuilding without each of them, test consistency and stability, not truth; no measurement here shows that CORA knows when its program will generalize. It opens acceptance that also weighs consistency with prior knowledge.
- *Accuracy and eligibility.* CORA alone solves few unseen ARC-AGI-2 puzzles (the CORA-only entry scored 0.83%); the neural comparison entry's attempt 1 is NVARC's method, and CORA can contribute to the neural comparison entry's score only through attempt 2, which changed no public-evaluation output; the CORA-only entry is CORA alone, keeps its public-evaluation answers on Kaggle's CPUs only because its caps were raised to 45 CPU seconds, and scored 0.83%; prize eligibility is not confirmed for the neural comparison entry. Local tests on exposed tasks suggest that coverage, not precision, is CORA's main engineering limit; they do not explain the official score. It opens construction where CORA abstains.
- *Inspected data.* The 120 public evaluation puzzles were used to diagnose CORA, and the neural checkpoint is contaminated for them; the one new exact evaluation prediction is on a puzzle that shaped the design. Source puzzles are public training puzzles, and prior knowledge of public ARC tasks cannot be excluded for any pretrained model.
- *Extensions.* The companion study's frozen one-pass assay on 24 source-derived ARC-style tasks found the diagnosis-specific condition's gain over its deliberately weak base (20 against 9 tasks; 11 paired wins, no losses) fully matched by the static and alternative conditions (20 each, the same on all 24 tasks): a reason to rank proposals and improve representation rather than add more search in the same language. It is not ARC-AGI-2 accuracy, a Kaggle gain, biological transfer or a new primitive (the companion study's records; available from the authors on request). The extensions that reached real puzzles were hand-built; the self-extension tests recovered removed operations on constructed families, with one 4B proposer whose coding ability is the binding limit, and at matched compute the starting language's search recovered more motivating tasks than invention did.
- *Power and statistics.* The sealed confirmation had little power at its observed effect size; the capability tests have 11 motivating tasks in 4 families, and their seeds replicate within a task. They were first analysed with task-by-seed units, which counted tasks several times; all inference here is at task level (`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`).
- *The positive causal result is narrow.* The construction test shows that a specification deduced after a failed search selects the right structure in a known, pre-registered 211-structure language, in a controlled ablation whose evaluator tasks and reference outcomes had been examined in earlier experiments. At task level only the comparison with a shuffled specification is significant (8 to 0, $p = 0.0078$); against the demonstrations alone (4 to 0, $p = 0.125$) and matched search (4 to 1, $p = 0.375$) the edges are not established; blind enumeration of that language covered at least as many tasks with five times the compute; and the result awaits independent replication. It taught us that inside a known space the specification saves compute rather than adding coverage. It opens independent replication, and the same test in spaces too large for blind enumeration.
- *Learned controllers.* In every controlled pilot a strong static baseline matched or beat them; the pilots are small, generated or authored, and not always budget-matched. They taught us that inside a fixed language strong static search already covers what learned control adds. They open learned control in spaces static search cannot cover, at matched budgets.

- *Representation growth and learned selection in the companion study (audited negatives).* The companion study's representation-growth series, versions 3 to 15 (object relations, typed crops, canvas filters, object maps, role and palette maps, object correspondence, crop-and-palette roles and multi-object layouts), none passed its full sequence of pre-declared gates (on the 1,000 exposed training tasks, then on the 120 public evaluation inputs), and a sixteenth, an output-canvas constructor, is paused without a result; its learned action selection on 128 invented tasks (111 of 128) equalled every control, and a whole-vector shuffled order found more (118 of 128); and its frozen constructor on ARC training tasks did not beat full-budget static search at equal total work (Appendix B.2). None is an entry result, an accuracy figure or a transfer result. Our own pre-registered natural-task growth test, in its feasibility phase on source halves, found that 96% of base-search failures localise but that none of 425 construction units has a hole constructible from the 211 pre-registered whole-grid structures (12 near misses, 0 promotions), so the construction loop of Section 5.5.2 does not reach natural tasks as it stands (Section 5.5.3); its object-level step is development evidence on the three source halves and an 80-task training sample (promotions on ConceptARC and 1D-ARC, none on Mini-ARC); its held test has since shown gains within 1D-ARC and a loss on ConceptARC (Section 5.8). These negatives taught us that coverage comes before ranking: no grown representation reached what its gates required. They open object-level construction from natural failures with richer two-dimensional views.
- *Generated puzzles.* Every synthetic family was written by the researcher who designed the systems, and every generator drew on background 0.
- *Construction at scale.* The 95 generated tasks lie inside a 211-structure language that contained the removed operations; the positive contrast depends on eight clusters, and leaving out either of two worlds gives intervals that include zero; reuse on excluded tasks failed its pre-registered independence condition; and two memory errors escaped the rerun rule. Compute favoured the comparator: static search used 8.47 measured CPU-hours, and construction was charged 2.61 CPU-hours (measured growth about 0.31 CPU-hours plus 902 diagnosis CPU-seconds). The reachability analysis is post hoc, its static programs are the shortest known rather than proven shortest, and no equivalent at any depth searched is not proof that none exists deeper.
- *Selection.* Two pre-registered revisions of the failure-guided choice test used controlled generated tasks, not ARC puzzles. The first (two changes) gave its pre-registered outcome, none; the second (three changes) beat its pre-registered static order (55 against 29 of 84), but that order scored below a random order (29 against 37), the fact the router reads comes from the demonstrations rather than from the failed search, and against the best fixed order chosen after the results the advantage is not established (11 tasks against 0; interval 0.0 to +26.1 points) (Section 5.9). No learned or failure-conditioned choice of what to change has beaten a strong fixed alternative in any completed study.
- *New domains.* Each adapter was written separately, the tests were quick checks with small budgets, and each result is a reconstruction of known functions, reactions, models or game rules: the logic result leaves 153 of 217 functions recovered by no method; in chemistry a classical nearest-neighbour method matched the loop; the spreadsheet primaries were a null; the signalling gain is limited to one removed edge; and the world-model probe did not pass its pre-registered test (Section 6.11).
- *The neural gap analysis.* It is post hoc, on the exposed public evaluation set, and not an official score; its diagnostic rules were refined on the same tasks; and the neural checkpoint is contaminated for that set (Section 5.4).
- *Exposure of the held halves.* Every held task of the held-back-half test had been run and scored once on 2 November 2026 by the submitted entries and once by the companion study's earlier evaluation; the run was a local takeover rather than the planned Kaggle kernels; its static comparators searched the original language rather than the expanded bank; and it had no shuffled-failure arm.
- *Biology.* The studies are retrospective, held-out data are held out from CORA only, and the diagnostician's ARC thresholds were never recalibrated within a pre-registered study. In biological data, held prediction alone does not establish a mechanism: heterogeneity and measurement error can imitate a failed rule. We therefore compare revision policies at matched work, keep abstention, and require robustness across biological units, predeclared ablations, agreement with known markers or related atlases, and independent cohorts or orthogonal measurements before a predictive rule is raised to a biological explanation, the

standard that comments on the discussion draft also set out. The prospective replication of the inferred support (Section 6.2) had prior inventory parsing of its models, readmitted models, 191 of 595 targets in four lineages, and pre-admission checks narrower than those requested (the 333-task compatibility check, zero mismatches, was completed only after the run); it is exact reconstruction of published rules within biology, not discovery. The informative-observation, completion and conditional-construction studies reused the replication cohort, and a classical learner equals the last by design; on measured data no rule has yet shown independent predictive validity; a calibrated acceptance rule harmed on a fresh melanoma source; the maize trial used the first author's unpublished images; and no CORA rule has been tested by an intervention or in a laboratory.

8.12 Conclusion

The question of this paper is what a reasoning system should change when its reasoning fails, and how it could learn to answer that question itself. The evidence does not yet show a system that learns it. It shows something more basic, which any such system must respect: the same failure can have different causes, and a failure record alone does not say which; verification establishes consistency, not intended correctness; reach comes before ranking, and search order only decides how much of the reach a budget uses; agreement-based acceptance turns language growth into abstention wherever the old language already fitted; and a few demonstrations carry little information, so chosen evidence and priors matter as much as search. Within those limits CORA has tested capabilities: verify-or-abstain answers that are precise on related benchmarks, and construction of a missing operation from typed holes computed from the demonstrations once search failed, inside a declared space. It has measured improvements: construction that beat static search in the starting language on controlled generated tasks while using less compute; reuse and task-local construction on held related tasks, within one benchmark's family structure; a learned search order at a binding budget; through a separately written biology procedure, regulator support, informative observations and a completion vote in published gene-network models, and, through separately written adapters, held tests in logic functions and enzyme chemistry. Its official CORA-only score is 0.83% (submission 57902131). In the terms of the three requirements, discovery moved inside supplied spaces, and selection did not on real tasks. Where every change could check cheaply whether it applies, a fixed alternative with the same capabilities and evidence matched the diagnosis-driven choice, and a router that read the failure record gave the same answers as a fixed order of the same changes; on controlled generated tasks where a change cannot, no fixed order of the bank reached the oracle (best 44 against 55 of 84) and the pre-registered router did, an advantage not established against that best order, chosen after the results. Its negative results, each stated with what it established and the experiment it motivates, locate the next problem: choosing which change to make from the evidence of failure, and showing on real tasks that the choice, not the extra capability, produced the correct answer. Beside a strong neural system, CORA explains the few answers its language reaches and, for the rest, names from the demonstrations alone what its language lacks; that list is where learning what to change must start on real tasks. That argument stands even if the particular learning mechanisms tried here fail, because it says what must be measured, and in which order, for any system that revises itself.

Acknowledgements

Toni Kazic led high-level discussions on the big problems in science: what AI can and cannot do, grammar induction and program synthesis across domains, and what the next big leap could be. We thank Chris Fluharty, who started this project with us last year in a hackathon (the LLM route), and Pierce Taylor for high-level discussions on single-cell omics in biology and its bottlenecks (dispersion versus displacement). The single-cell case study of Appendix C builds on the DISPERSION project, a single-cell study of gene-expression dispersion that we wrote with Pierce Taylor as a course project at the University of Missouri.

A The submitted system and its methods

The paper must describe the submitted notebooks. The neural comparison entry (submission 57867631, submitted 1 November 2026), whose neural core is NVARC's published method, scored 24.03% on the public leaderboard; the CORA-only entry (submission 57902131), submitted on 2 November 2026, scored 0.83%, the

first non-zero official score of a CORA-only entry (`records/OFFICIAL_SCORE_AND_SELECTION_STATUS.md`); the paper links the CORA-only entry, CORA only (the first author's decision of 2 November), and reports the neural comparison entry (submission 57867631) as the comparison. The Paper Track does not say that the linked entry must be one of the two final selections; we infer that it should be, and the CORA-only entry is one of the two. Earlier candidates (the neural entry alone, its CORA-CAST variant, and a hybrid whose separately built constructor may fill only a duplicated attempt 2, which made 0 insertions on a full rehearsal) are recorded in `records/NOV1_COMPETITION_COMMAND_DECISION.md` and are not entries.

The neural comparison entry (submission 57867631, 1 November 2026): the hardened neural entry with CORA-CAST version 2 on attempt 2. Attempt 1 is the neural method below, in a hardened process supervisor (recovery after an ordinary termination signal; a private child process writes the submission file; its 120-puzzle offline rehearsal completed). Attempt 2 is the answer of CORA-CAST version 2 (`cast_selective_v2`) when the frozen selective acceptance rule accepts it (the strict selective policy: genuine top-1 leave-one-out rounds, every round predicting the same query outputs as the full run, agreement across the union of modules, and restart stability; chosen by the first author, who accepted its replacement risk), otherwise the neural second attempt. On the 120 public evaluation puzzles CORA-CAST version 2 timed out at its 15 CPU-second cap on 62593bfd, the one puzzle that only CORA-CAST answers and where the neural model has one of its two outputs, both locally and in the Kaggle rehearsal; The strict policy accepted only 981571dc, already the neural answer, so there the neural comparison entry equals the neural method alone (43 of 172, 0 gained, 0 lost; `records/NOV2_FINAL_SUBMISSION_STATUS.md`).

The CORA-only entry (submission 57902131, 2 November 2026; public score 0.83%): CORA only, on CPU. Every non-fallback answer is a program prediction that a CORA solver executed on the puzzle, and every other output is the input-copy fallback; there is no neural model, no GPU and no external data (Kaggle notebook `chimdiwalter/cora-cast-r29-v2-arc-agi-2`, notebook version 1, the scored version, `https://www.kaggle.com/code/chimdiwalter/cora-cast-r29-v2-arc-agi-2?scriptVersionId=364571398`; notebook hash `f683136c`). The driver (`kaggle/score_first_2026/configs/sfo/driver_plus.py`) runs two solvers one after the other, each in forked children over up to four shards: CORA-CAST version 1 (`cast_v1`) with caps of 45 CPU and 90 wall seconds per puzzle, then a second reviewed CORA solver from the companion study (`r29_cpu_package_20261101`): forward, diagnosed-inverse and conditional-beam constructors, each of which must fit every demonstration, replay it independently and pass every genuine leave-one-out round, with each fit capped at 1 CPU second . Attempt 1 is the answer of the earlier solver in that order, else the other's, else a copy of the query (`sfo/driver_plus.py` line 60; on the public evaluation 169 of 172 outputs got the copy, and 3 came from a solver); attempt 2 is the next distinct answer from the other, otherwise a copy of the query. The caps of CORA-CAST version 1 were raised from 15 to 45 CPU seconds by the first author's decision of 1 November, after a Kaggle CPU rehearsal at 15 seconds (Python 3.13, 4 CPUs) reached the cap on 5 puzzles, 62593bfd among them, and answered 1 of 172 (981571dc; 0.58% per output, task mean 0.83%; `kaggle/score_first_2026/configs/results/op_rehearsal_v1b/score.json`). At 45 seconds the Kaggle CPU rehearsal had no timeout and was byte-identical to the local run: 3 of 172 outputs (2 puzzles, 62593bfd and 981571dc), all right, in 219 s (`records/NOV2_FINAL_SUBMISSION_STATUS.md`). These puzzles were inspected while building, and 62593bfd shaped the move constructor, so this shows that the entry works, not what it will score or how precise it is on unseen puzzles; the second solver answers none of the 172 outputs and 1 of 32 inspected source puzzles (`records/SUBMITTED_METHOD_TO_CODE_MAP.md`). On the sealed source puzzles (public training puzzles in a mechanism study, Section 5.7.1) CORA-CAST version 1's core, without its acceptance filter, answered 11 tasks, 8 right; as training puzzles they check the mechanism and are not a performance estimate. The plain CORA-only configuration (CORA-CAST version 1 alone at 15 seconds, with the test-input copy as attempt 2) is kept as a checked fallback.

The neural method in the neural comparison entry: NVARC's published method, in our own code. The notebook re-implements the method of NVARC, winner of ARC Prize 2025 [87, 155], from its public description (the repository also holds a copy of NVARC's notebook, pulled to build the reference rerun 57160429): NVARC's public ARC-specialized 4-billion-parameter checkpoint (`sorokin/qwen3_4b_grids15_sft139`, Apache 2.0 weights), a fresh LoRA adapter trained on each puzzle's demonstrations (rank 64, alpha 64, learning rate 10^{-4}, 48 steps, at most 8,192 tokens per sequence; colour permutations appear only in these training sequences), decoding

and scoring under the 8 dihedral symmetries with identity colours, and selection by mean teacher-forced log-likelihood, two attempts always written (Table 25). Every adapter starts from the frozen base, so nothing carries between puzzles. The method is NVARC's, and so is any score it earns. An earlier build of this code passed an independent code review with a go verdict; the neural comparison entry's hardened version differs from it on deadline, exception and restart paths only and was reviewed only by this project's own reviewers (`records/NOV2_FINAL_SUBMISSION_STATUS.md`).

CORA-CAST's acceptance rule, entry by entry (`experiments/cora_cast_20261026/kaggle_pkg/`). Both versions run the frozen CORA-CAST core in a fixed order: general symmetry completion, guarded relational rewriting (rule lists, per-feature table rewrites and guarded crop-to-object frames, `cast/rewrite.py` lines 482 to 505), and a contract-checked typed search (depth 2, a budget of about 1,000 executions, checked between candidate programs). In rewriting, each demonstration object's fate (kept, deleted, recoloured, holes filled, outlined, moved) proposes actions; the eight actions that explain the most changed objects are kept; guards are atoms over the objects' features and relations with values seen in the demonstrations; and an ordered list of at most two guarded rules and a default (each object takes the earliest rule whose guard holds) is a candidate when every object a guard matches is explained by its action. Every candidate is verified by executing the whole list on every demonstration (`experiments/cora_cast_20261026/cast/rewrite.py`). Fitting programs are grouped by their predictions and ranked by completeness, then group size, program cost and enumeration order. The sort key also holds a leave-one-out field, but it is always 0 (`cast_v1/candidates.py` lines 48 and 51; symmetry completion sets it to 0, `cast_v1/api.py` line 74, and the other modules never set it), so leave-one-out acts only in acceptance, below. An answer must be a valid grid of at most 30 rows and 30 columns (`cast_v1/grids.py` lines 192 to 196), and it may not copy the query when every demonstration changes its input (`cast_v1/candidates.py` lines 28 to 39; `cast_v1/api.py` lines 162 and 173). In CORA-CAST version 1 completeness is a sort key, not a filter: symmetry completion keeps the fits complete on every query when any exist, else those complete on any query (`cast_v1/api.py` line 97), and the answer is the top attempt with no per-query completeness check (lines 169 to 177). On a multi-query puzzle it can therefore return a partial completion: a synthetic two-query probe returned one with 2 undetermined cells (`experiments/systems_review_20261102/probe_m4_partial.json`). This never happened on the public evaluation, where CORA-CAST version 1's only symmetry answer, 981571dc, has one query output; CORA-CAST version 2 requires a complete top group (`cast_selective_v2/api.py` lines 173 and 174). Puzzles with fewer than two demonstrations are never accepted. *CORA-CAST version 1, in the CORA-only entry* (`cast_v1`): the answer is accepted if, with each demonstration held out in turn, the same module rebuilt from the rest ranks the held-out output among its top two attempts. An audit of CORA-CAST version 1 (`records/CAST_LEAVE_ONE_OUT_CORRECTNESS_AUDIT.md`) found the rounds to be genuine re-syntheses: each re-runs the whole module, representation, guard space and construction included, on the remaining demonstrations, and the held-out output is used only in the final comparison. It also found three defects: a round passed when the held-out output was among its two attempts rather than its selected one; disagreement between the rounds' hypotheses and the full hypothesis was not recorded; and too little evidence was not reported as such (with exactly two demonstrations, every round is a one-demonstration synthesis). *CORA-CAST version 2, in the neural comparison entry* (`cast_selective_v2`) fixes them and, under the strict policy, accepts an answer only if every round's top answer is the held-out output, every round predicts the same query outputs as the full run, every fitting program of every module predicts the same output for the query, and a restart in reversed enumeration order selects the same behaviour. An accepted answer replaces attempt 2; if it equals attempt 1, the neural pair is kept; if CORA-CAST version 2 finds nothing acceptable, times out, fails or returns a malformed grid, the neural attempts stand. Both rules are stricter than the procedure of the sealed confirmation (Section 5.7.1), which ran CORA-CAST version 1's core without its acceptance filter. CORA-CAST version 2's measured results: on the 120 public evaluation puzzles it accepted only 981571dc and timed out on 62593bfd (above); on held-out source puzzles its validated policies accepted 13 answers, all correct (Section 5.7.1); on fresh nuclei images it accepted none (Section 6.7). The rule is the same for every task: no task identifiers, no memorized answers. Calls are capped at 15 CPU and 30 wall seconds per puzzle in the neural comparison entry and at 45 and 90 in the CORA-only entry, are deterministic, and touch no file, network or subprocess (Algorithm 2).

Where each completeness item is. The architecture is Figure 17, the pseudocode Algorithm 2, and the hyperparameters and resource limits Table 25; every statement of this section is mapped to a file, function and line range in `records/SUBMITTED_METHOD_TO_CODE_MAP.md`. The failure analysis is Section 5.3; the ablations and

Algorithm 2 CORA-CAST on one puzzle, as both versions run it (`experiments/cora_cast_20261026/kaggle_pkg/`).

Require: demonstrations $D = \{(x_j, y_j)\}_{j=1}^{m}$, queries X^*, per-puzzle CPU and wall caps

if $m < 2$ **then**
  return abstain on every query
end if
for module M in symmetry completion, guarded rewriting, typed search (depth 2, about 1,000 executions) **do**
  $P \leftarrow$ the programs of M that reproduce every (x_j, y_j) exactly
  (CORA-CAST version 1, symmetry completion) keep the programs complete on every query, else those complete on any
  group P by their predictions on X^*; if every demonstration changes its input, drop a prediction equal to its query input
  rank the groups: complete first, then group size, program cost, enumeration order (the leave-one-out key is always 0)
  if ACCEPT(M, D) **then**
    for each query $x^* \in X^*$ still without an answer **do**
      answer the top-ranked valid prediction for x^* (CORA-CAST version 2: only from a complete top group)
    end for
  end if
end for
return the answers; every other query abstains and the entry's fallback applies

function ACCEPT(M, D)
  for $j = 1, \ldots, m$ **do**
    rebuild M from scratch on $D \setminus \{(x_j, y_j)\}$
    CORA-CAST version 1 (CORA-only entry): require y_j among the rebuilt module's two attempts for x_j
    CORA-CAST version 2 (neural comparison entry, strict policy): require y_j as its top attempt, and the full run's test predictions
  end for
  CORA-CAST version 2 also requires no disagreement on the query across modules, and the same result on a restart
  return true if every requirement holds within the caps
end function

controls are Tables 10, 12, 28 and 29 and Section 5.7.1; the official scores and submission identifiers are Table 9; licensing and provenance follow, and the exact commands are in Appendix H.

Licensing and provenance. The neural comparison entry attaches NVARC's public checkpoint, a third-party model, unchanged (labelled Apache 2.0 on its Kaggle page, whose training-data and base-model fields are empty; the base model of NVARC's published recipe is an open 4-billion-parameter model released under Apache 2.0) and a peft 0.21.2 wheel; its neural code is our re-implementation, and NVARC's own notebook is used only for the reference rerun (its licence: reported Apache 2.0 on its Kaggle page, not independently verified; NVARC's GitHub repository is unlicensed). The CORA-only entry uses no pretrained model and no external data. Both notebooks' stored sources equal the local builds cell for cell, and every CORA solver is fixed by its manifest hash: notebooks `0e09afd4` (neural comparison entry) and `f683136c` (CORA-only entry); CORA-CAST version 2 `aa8551ad`, CORA-CAST version 1 `acd57e12`, the second solver `11eb2ee9`. The puzzles are the ARC Prize Foundation's. The licence for our code is open (below), and the release plan is in `records/NOV1_OPEN_SOURCE_RELEASE_PLAN.md`.

Prize eligibility awaits the host's answer. Two material questions need the host's written answer (`records/COMPETITION_ELIGIBILITY_OPEN_ISSUES.md`). Rule 5.a.1.a, read literally, asks for an open-source model as the

	neural comparison entry (submission 57867631)	CORA-only entry (submission 57902131)
machine	Kaggle, 4 NVIDIA L4 GPUs (22 GB each), 48 CPUs, internet off	Kaggle CPU notebook, 4 CPUs, internet off
inputs	the competition data; NVARC's checkpoint; a peft 0.21.2 wheel	the competition data only
neural attempt	a fresh LoRA adapter per puzzle (rank 64, alpha 64, dropout 0, attention and feed-forward (MLP) projections); AdamW, learning rate 10^{-4}, 10% warm-up then linear decay, gradient clip 1.0, bf16; 48 steps, at most 8,192 tokens per sequence; leave-one-demonstration-out training sequences, each under one random dihedral symmetry and one colour permutation that keeps 0; 70% of the puzzle's budget, a soft cap of 900 s; decoding and scoring under the 8 dihedral symmetries only, with identity colours; selection by mean teacher-forced log-probability	none
CORA caps per puzzle	CORA-CAST version 2: 15 CPU and 30 wall seconds (its own timers); the side process kills at 26 CPU or 40 wall seconds	CORA-CAST version 1: 45 CPU and 90 wall seconds (kills at 56 and 100); the second solver: each fit 1 CPU second
total budget	one CORA side process at nice 10 for at most 10,800 s; neural deadline 41,400 s; guard 42,600 s	driver deadline 39,600 s; worst case 8.93 h serially, 2.23 h on 4 shards
measured on the 120	8,630 s for the notebook; CORA 332 CPU seconds in 341 s	219 s

Table 25: Hyperparameters and resource limits of the linked entries (`records/SUBMITTED_METHOD_TO_CODE_MAP.md`, sections 3 and 5).

Open Source Initiative defines it, and neither the base model nor the checkpoint documents its training data (the published recipe lists six training subsets and an evaluation validation subset, but the exact hosted checkpoint lineage, full data rights and prize eligibility remain unverified; the data and its generator carry no licence, and part of the data was generated by closed commercial models). And the licence for our own code differs between Kaggle's rules (CC BY 4.0) and arcprize.org (CC0 or MIT-0). Neither question blocks a leaderboard submission. Entry the CORA-only entry uses no pretrained model, so the open-model question does not arise for it; the code licence question does. The paper track requires the linked code submission to demonstrate the approach the paper describes: the CORA-only entry runs CORA alone and the neural comparison entry runs CORA-CAST on attempt 2; neither runs the capability-growth cycle, which this paper reports as research.

Engineering. A valid fallback `submission.json` is written before any GPU work; four replicas, one per L4 GPU, pull puzzles from a shared queue; each puzzle's budget is the time left divided by the puzzles left, with a soft cap of 900 s (in the neural comparison entry's Kaggle rehearsal 2 puzzles ran past it, the longest 1,099.5 s); the file is rebuilt atomically every 20 s, with a global deadline at 11 h 30 min, and checked against the competition schema (every task, one entry per query, two rectangular grids of 1 to 30 cells a side with values 0 to 9). Measured on 15 size-stratified public evaluation puzzles, the deployable path took 238 s per puzzle on average (666 s at most) with a peak of 17.4 GB of 22 GB, which projects to 4.0 h for 240 puzzles on four L4 GPUs. The neural comparison entry's own Kaggle rehearsal of the 120 took about 8,630 s (2.40 h) for the whole notebook on four L4 GPUs, which projects to about 4.8 h for the 240 hidden puzzles inside an 11.5 h watchdog, and both output files were schema-clean (`records/NOV2_FINAL_SUBMISSION_STATUS.md`).

Research experiments no candidate runs. Everything else in this report: the self-extension cycle and its tests (Appendix B.3); the five architecture families, the program backing of neural answers, the neural program writers with diagnosis-conditioned revision and the proposer comparison (Appendix B.5); CORA-CAST's bindings and bounded dynamics; every generated-puzzle study; and the biology studies (Section 6). They are reported as research, never as properties of the entry, and no growth result contributes to the CORA-only entry's score.

A.1 CORA-CAST: contracts, symmetry completion and guarded relational rewriting

CORA-CAST (task-conditioned structural contracts for adaptive program synthesis, `experiments/cora_cast_20261026/cast/`) is a hand-written extension, and nothing in the scored entry is learned (a small binding ranker in CORA-CAST's development code is not in the scored package). Every grid's background is its majority colour (ties to the smallest colour); no module assumes colour 0. Every operator is a deterministic, bounded function from grid to grid, or undefined; an undefined result on any demonstration rejects the program, and no model-written code is executed.

- *Contracts.* Each operator carries forward and backward contracts over a reduced product of shape, per-cell colour sets, palette and counts; an exhaustive finite-scope checker validates them, and the search prunes a partial program only when a contract proves that no completion can fit. The checker is also the interface through which an invented operator would be validated.
- *General symmetry completion.* A masked colour is filled from every exact symmetry of the grid (any row or column axis, any 180-degree centre, any diagonal), optionally cropped to the mask; conflicting orbits are undefined.
- *Guarded relational rewriting.* A scene holds the grid's entities (single-colour 4- or 8-connected, or multicolour 8-connected objects on the majority background) with masks, boxes and feature and relation tables. A rule pairs a *guard*, a typed test of an entity's features or relations such as `size_unique` (no other entity has the same size) or `has_hole` (the entity has a hole), with an action: recolour, delete, crop to an entity, look up a small table keyed by one feature, or move. Writes are synchronous with an explicit overlap rule; moves are sequential, nearest to the target wall first, with destination modes *until blocked* and *to the border*. A program is a first-match list of at most two rules and a default, constructed from the demonstrations and verified by exact execution. It is an ordinary guarded graph rewrite, not a double-pushout implementation.
- *Also built, not in the core*: executable bindings that re-read a legend from every input, and bounded local dynamics with declared neighbourhoods and schedules.

A.2 The self-extension cycle

The cycle (Figure 18) starts from a starting language and its bottom-up search (150,000 primitive executions and 20 CPU seconds per task). When the search finds no fit, a search four times deeper is tried; if that also fails, the controller decides to invent an operation, otherwise to keep searching. The failure report states the search's completeness, the deeper search's outcome, the closest program and its residual, the output-size relation, sketches that would fit if one hole were filled together with each hole's input and output specification, and competing diagnoses. A pretrained 4-billion-parameter instruction-tuned language model [139] (inference only), writes $K = 4$ candidate operations per episode in a sandbox (a bounded grid-function language, with the library refused), where existing operations carry neutral codes re-permuted per seed and no removed name appears. A candidate is kept only if it runs and returns grids, is not reproduced by any starting-language program of cost at most 7 (a rename), lets the motivating task fit, changes no fit or first prediction on 14 previously solved tasks, and passes validation on separate tasks; it is then promoted into a versioned registry with its provenance, and failures are kept with their stage and reason. Promoted operations are available to the search on excluded tasks.

A.3 Earlier routes, in brief

Earlier work, reported in Appendix B.5, tried learning inside a fixed language: a language model writing complete CORA programs and revising them from CORA's typed diagnosis, language-model writers of grid functions, a learned search guide, and five further architecture families. Their methods are in the cited records; the residual masks and the diagnosis levels are those defined above.

B ARC experiment records in detail

The records below are the full versions of the studies that Section 5 summarises, in the order of its questions.

B.1 Verification: the controls follow-up and the source-puzzle policies

The controls the first similar-problem test lacked (a pre-registered follow-up on the same tasks). The follow-up reran the same 924 tasks and 1,244 query outputs (ConceptARC 480, Mini-ARC 149, 1D-ARC 615) with controls declared, after an independent pre-run review, before their runs: accepting the top program that fits every demonstration, the leave-one-out variants, and blind enumeration in CORA-CAST's own language within the per-task CPU that the neural comparison entry's CORA-CAST component actually used, the only compute-matched comparator (`records/NOV2_U2B_CONTROLS_RESULT.md`). Intervals are 95% bootstraps clustered by 1D-ARC type and ConceptARC concept (Mini-ARC by task). It is a controlled follow-up on the same tasks, not a fresh holdout; the neural comparison entry's neural part was not rerun; and it is not a hidden-set, accuracy or learned-universality result. An independent audit reproduced the arithmetic from the aggregates.

condition (pooled over 1,244 outputs)	answered	correct	wrong	coverage	precision	accuracy
CORA-only entry (frozen)	382	358	24	0.307	0.937	0.288
the neural comparison entry's CORA component, strict selective policy (frozen)	82	71	11	0.066	0.866	0.057
plain CORA-only configuration, CORA-CAST version 1 (frozen)	116	97	19	0.093	0.836	0.078
accept every fitting program	134	102	32	0.108	0.761	0.082
top-2 leave-one-out	115	96	19	0.092	0.835	0.077
genuine top-1 leave-one-out	92	78	14	0.074	0.848	0.063
blind enumeration in CORA-CAST's language, at the CPU the neural comparison entry's component used	18	18	0	0.014	1.000	0.014
by benchmark, precision (coverage)	ConceptARC, 480		Mini-ARC, 149		1D-ARC, 615	
every fitting program	0.673 (0.115)		1.000 (0.087)		0.788 (0.107)	
strict policy	0.933 (0.031)		1.000 (0.060)		0.828 (0.094)	
blind enumeration	1.000 (0.013)		1.000 (0.074)		1.000 (0.002)	

Table 26: The pre-registered follow-up on the similar-problem tasks: every condition, pooled, and the three benchmarks with their output counts (`records/NOV2_U2B_CONTROLS_RESULT.md`).

The selective acceptance rule, measured. A protocol frozen before scoring (`records/CAST_SELECTIVE_ACCEPTANCE_PROTOCOL.md`) compared acceptance policies of CORA-CAST version 2 on the two development sets of 176 unselected source puzzles (`records/CAST_CANDIDATE_RISK_COVERAGE.md`). CORA-CAST constructed 15 candidate answers; the three validated policies accepted the same 13, all correct (precision 1.0, Wilson 95% lower bound 0.77), and abstained on 93% of the 182 queries; the 2 candidates they rejected were exactly the 2 wrong ones. Both wrong candidates were unique, restart-stable programs (a table rewrite keyed by shape, and a recolour then crop), the signature of a missing capability, and they are recorded as cases for capability growth. On the 120 public evaluation puzzles every policy accepted the same 3 answers, all correct, within the cap on that run; in the neural comparison entry's rehearsal CORA-CAST version 2 timed out on 62593bfd, so acceptance near the cap depends on machine speed. Against the neural model's fixed attempts the validated policies replace attempt 2 on 62593bfd only: $+2/-0$ under either neural selection rule (`records/CAST_NEURAL_PAIRWISE_GAIN_LOSS.md`); a second CORA-only call on top of CORA-CAST added nothing (0 gains, 0 losses; `experiments/cora_cast_20261026/results/cast_only_v2/CAST_ONLY_V2_MEASUREMENT.md`). On 32 further training puzzles drawn from CORA-CAST's own design partition (so not fresh for CORA-CAST), the first version answered 1 correctly and 1 wrongly; the deployed rule accepted the correct one and rejected the fitting but wrong one; an independent conditional constructor answered 3 others, so the union is 4 of 32 (`experiments/cora_cast_20261026/results/fresh32_cast_selective_v2/SUMMARY.md`). The rule was not run on the sealed set, which is used up; whether it would have removed the 3 sealed errors is not measured, and we do not claim it. Mean cost per puzzle: 1.5 CPU seconds, at most 15.

Programs that fit and are wrong, because acceptance certifies stability, not truth. On ARC, 3 of the 11 sealed answers of CORA-CAST's core without its acceptance filter, and all 7 of the companion study's verified symbolic candidates on the public evaluation, fit and were wrong; on list functions, for 2 concepts the full language fits the demonstrations and mispredicts the queries; on microscopy, 22 of the 33 nuclei deletions proposed without the leave-one-out check removed real nuclei, and on a fresh fluorescence image set the check itself passed an erase-everything program because every demonstration window held only debris (Section 5.2). When the

demonstrations cannot separate the rule from a rival, a stable fit is still a wrong one. It opens acceptance that also weighs consistency with prior knowledge.

B.2 Generation, search and selection

Coverage, because of reachability. CORA's frozen solvers answered only 3% to 50% of the query outputs of three public ARC-style benchmarks, and the CORA-only entry solves 2% of the public evaluation against 48% of 1D-ARC's outputs within its input limit (Section 5.2). In all 113 audited public evaluation puzzles no demonstration-fitting program was generated within the measured budget (a reachability limit of candidate generation, representation or search, not proven inexpressibility), so no ranker or one-step revision over that memory can help, and the failure anatomy puts expressivity first in 29 of 46 sampled failures (Section 5.3).

Table 27 is organised by mechanism and dataset and is meant to receive audited results as they arrive. On the ARC-AGI-2 public evaluation set, which we inspected while building, every number is descriptive. On the 1,000 ARC-AGI-2 training tasks only reachability and gap type are reported, never accuracy, as the Paper Track asks. A cell is filled only where an audited number exists; the work still in progress is described in Appendix D, not here.

The neural model's own development scores are contaminated and are not this work's: 17 of 172 outputs with vote selection and 22 with likelihood selection (task-averaged 11.1% and 13.9%); with per-puzzle adaptation on a 15-puzzle pilot, 8 of 23 outputs against 1 without it (+7, 0 regressions).

The submitted hybrid on the neural entry's own predictions. On the hardened neural entry's offline rehearsal of the 120 (43 of 172 outputs, 25.0%, task mean 27.1%; contaminated, and NVARC's method), the neural comparison entry's attempt 2 changed nothing: 43 of 172, 0 gained, 0 lost; CORA-CAST accepted one answer, 981571dc, which the neural model already had (`records/NOV2_FINAL_SUBMISSION_STATUS.md`). CORA-CAST version 2 timed out at its 15 CPU-second cap on 62593bfd, where the neural model has one of its two outputs, so the zero is partly a time-cap loss, not only redundancy; the neural comparison entry's Kaggle rehearsal gave the same (43 of 172, task mean 26.7%). The +2 above came from an earlier neural pool without per-puzzle training, where 62593bfd was unsolved, and from a CORA-CAST run that answered 62593bfd within its cap. This taught us that CORA's step adds an answer only where the neural pool lacks it and the cap lets the search finish. It opens construction aimed at the pool's missing outputs, with a budget that reaches them.

Learned repair corrected no neural errors, because its correct candidates duplicated the model's. NVARC's model without per-puzzle training answered 162 of 172 outputs, 8 of them right; the companion study's repairs of its residuals corrected none and broke none, and their correct candidates duplicated correct neural answers (Section 5.3, Table 34).

The finished companion tests: the measured coverage and representation limit (audited; none is in either Kaggle entry). Each test below ran once under a frozen protocol and passed a postscore or postrun audit; for our neural-guesses test and our two Boolean-network conditions, the scoring go was issued by the companion study's custodian after an independent output-chain review (the companion study's records; available from the authors on request). *Neural guesses turned into verified programs* (the 120 previously examined public evaluation tasks): CORA's earlier construction grammar turned 0 of 172 frozen neural first guesses into a program that passes every demonstration, as with no target (0 of 172), while the neural first attempt alone was right on 33 of 172; every primary run stopped at the 150,000-execution and 400-closure bounds, so the effect is not shown and inexpressibility is not proven (`experiments/neural_to_program_b_20261103/runs/score/SCORE_REPORT.md`). The same neural file (SHA-256 `558b6ab1`) was later searched with CORA-CAST's five modules, which explain 2 of its 344 attempts, both right (Section 5.4). A separate eight-task typed-object probe on exposed training tasks found 0 of 8 fits; its denominator is kept apart. *Boolean grammar growth:* the first Boolean grammar-growth stage was uninformative: only 1 of 182 source tasks failed its base fit, so every learned library was empty. A second stage with a smaller base language found 14 failures in 8 source lineages and chose a supplied three-input OR form, which recovered 60 of 151 held rules from 19 other, previously scored models; because the chosen arm reuses that fixed form, the two agree on all 151 tasks, and the base language (59), the full eight-form language (60) and the source-tuned and shuffled choices (60 and 59) did as well or within one task. Both stages learned from biological source models, not from ARC; the right function was among the candidates on 76 of the 151, so 16 were reached

but not chosen. *ARC-to-Boolean transfer:* negative at first; no operator witness was found in 1,000 ARC training tasks, so the biology stage never ran; a later foreground-mask adapter found 22 witnesses among 1,000 training tasks (NOR 8, OR 8, XOR 4, AND 1, TRUE 1); the order counted from them recovered 177 of 595 exact rules, against 168 for plain index order, 182 for the same counts randomly reassigned and 194 for a hand-set simplicity order; its gain over index order did not survive Holm correction (adjusted $p = 0.14$), and it lost 5 targets and gained none against the reassigned counts and lost 20 and gained 3 against the simplicity order (Section 6.6). *A learned Boolean ranking prior:* a development observation on source models (98 against 95 of 182, within noise) that did not reproduce on 46 models of independent lineages (121 against 124 to 126 of 356), most misses being failures of support identification (Section 6.2). *The companion study's frozen constructor on ARC training tasks* (a development diagnosis on exposed tasks, each task's first two demonstrations predicting its third; under the Paper Track's rule against training-set performance no counts are reported): at equal total work it did not beat full-budget static search; its losses were time-outs rather than wrong answers, since it emitted nothing where static search reached the program later in the same schedule; its probes could only detect solutions that crop a selected object; and its adaptation produced verified outputs only on tasks that static search already solved. A reachability analysis of that language on the same 842 tasks (`records/NOV5_REAL_ARC_FAILURE_STRUCTURE.md`; a diagnosis of the language, not accuracy and not training-set performance) found that for 754 of 842 tasks no program in the language reproduces even the two demonstrations the solver was given, so there the grammar is the limit with certainty; perfect search inside the language would reach at most 22 to 88 tasks; a grammar covering the published reference solvers would contain a correct program for at least 219 and probably 333 of the 842, but the only search that exists for such a grammar, bottom-up enumeration at the same execution cap, found 15 of 194 and 5 of 63 known programs, reaching programs of 4 to 6 nodes where the reference programs have a median of 29 nodes. *A held donor in brain tissue* (spatial transcriptomics of the dorsolateral prefrontal cortex): the pre-registered gate failed; a CORA marker-gene correction changed pooled macro-F1 from 0.931555 to 0.932028 (+0.000473), with 58 corrections against 65 harms, and 2 of 4 tissue sections were worse. *The companion study's drug-response test:* no result; its single authorized launch exited with an error before any outcome was read, and its 811 sealed outcomes stay unopened. *The companion study's constructor on the external benchmarks and the ARC-AGI-2 public evaluation* (one frozen scoring pass of exact two-attempt query counts; independent postrun review passed; every cohort previously exposed to the project): ConceptARC 15 of 480 queries against 13 for full-budget static search (five queries gained and three lost, the gains in two tasks and the losses in one), Mini-ARC 5 against 7, 1D-ARC 0 against 100, and the ARC-AGI-2 public evaluation 0 in every arm on all 172 queries; pooled over the external benchmarks, 20 against 120 of 1,244. That is no broad superiority, a narrow component signal on ConceptARC, and a zero on ARC-AGI-2 reported as a zero. *The companion study's ARC representation-growth series* (object relations, typed crops, canvas filters, object maps, role and palette maps, object correspondence, crop-and-palette roles and multi-object layouts): no grown representation passed its sequence of gates. Its gates on the 1,000 exposed training tasks were null or failed, except a crop-and-palette role grammar that gained reaches there and then produced 0 executable candidates for the 172 public evaluation queries it processed (input only, stopped before any scoring); the latest, a multi-object layout grammar fitted from the first two demonstrations, reached the third in 0 of 842 eligible tasks in every arm. *Repairing NVARC's own answers with object masks* (32 training tasks): no rescue; raw NVARC and every repair condition were right on 27 of 32 in two attempts, and all five neural errors lay outside the repair language. Read together, these results measure CORA's coverage and representation limit: the missing programs were not reached within the present representation, grammar and bounded search, and none of these tests shows an entry gain or learned cross-domain transfer. Their common lesson is that coverage comes before ranking or transfer: no selection, repair or transfer helps where the language does not reach. They open broader grammars with deeper search, guided by the holes and missing relations these tests recorded.

B.3 The self-extension test with a neural inventor

Design. The starting language is the arc-dsl language [71] as translated for the architecture campaign, with four operations removed: compress, downscale, trim and upscale. Within cost 12 no program of the starting language expresses compress, downscale or upscale; trim is a macro of cost 10. For each removed operation, 11 tasks were built from templates over real source input grids: 3 motivating tasks, 2 in the system's validation pool and 6 excluded tasks seen only by the evaluator; 4 decoys need only deeper search, and 14 previously solved tasks guard against regressions. The proposer (Appendix A.2) ran with identical templates, token caps (640) and sampling seeds in every condition, two seeds and 22 episodes per condition. Conditions: the task's *own*

failure report; *failure-blind* invention (the language card and the puzzle only); a *shuffled* report (another family's motivating task); *whole-program writing* by the same model (samples plus one repair with execution feedback); and *20 times more search* (a 60 CPU-second cap). The label is capability recovery, not invention of an unforeseen capability (`experiments/capability_growth_20261027/PROTOCOL.md`).

Results, stage by stage. *Diagnosis:* the controller decided to invent on 19 of 20 family tasks and to keep searching on the 4 decoys and on 1 trim task whose macro the deeper probe found; all match the evaluator's truth. Its levels are coarse, though: the proposer stated "operation" in 243 of 243 parsed proposals, so representation, relation and composition levels were never exercised. *Proposal:* 73 of 88 own-report proposals reached execution (71 failure-blind, 80 shuffled); renames of an existing program, rejected by the novelty check, were 5 against 15 and 15; none of 241 distinct proposals is near-verbatim arc-dsl, and none uses a removed name. *Recovery* (Table 28): downscale by 2 and upscale by 2 or 3, each as a fixed-parameter operation that agrees exactly with the removed reference at its own factor and that no starting-language program reproduces within a million executions (an expansion of the starting language, not a macro); compress and trim were never recovered. *Promotion and reuse:* 4 promoted operations and 3 quarantined (held back after failing validation) in the own-report condition (2 promoted failure-blind, 0 shuffled), with 0 regressions on the 14 solved tasks and no rollback; promoted downscale by 2 solved 4 of 6 excluded downscale tasks and upscale by 2 solved 2 of 6, while the quarantined factor-3 instances would have solved the other 4 (a false negative of a validation pool that held only factor-2 tasks); no cross-family transfer. *Positive control:* hand-written implementations of the four operations pass every check and solve 6 of 6 excluded tasks each, so the pipeline is not the bottleneck. *Ceiling check:* given each removed operation's exact input and output behaviour on the task's own grids, the proposer recovered 1 of 11 tasks, and its compress, downscale and trim proposals agreed with the references on 0 to 10% of probe grids: the binding limit is the 4B proposer's ability to implement a grid operation, not CORA's evidence or checks. The test cost 2.34 CPU hours and 1.64 GPU hours (`experiments/capability_growth_20261027/reports/TEST_I_REPORT.md`).

Scope of this test. Every failure hit the search's closure cap, so no inexpressibility is shown, only no fit in the enumerated fragment; the 20× search used 8 to 29 CPU seconds per task against about 70 GPU seconds plus decoding per invention episode, so it is not a compute-matched control (the follow-up's matched search reversed its result); three protocol amendments followed observation of the first generations (the stopped runs were kept and nothing from them was scored); the gains sit in two families; the inventions are fixed-parameter (the follow-up's merge lifted one pair of them to a parametric operation); one 4B model, two seeds. Because the proposer's coding is the binding limit, the next hypothesis, proposed and not run, separates invention into a neural stage that states a typed behavioural specification of the missing capability and a symbolic stage that constructs an implementation with CORA-CAST's typed rewriting, sketches and contracts, verified on permitted development cases before it enters the registry.

The pre-registered follow-up. The follow-up (`experiments/capability_growth_20261027/PROTOCOL_TEST_IB.md`, frozen before generation) reran the own-report, failure-blind and shuffled conditions on new seeds 2 to 4 (33 runs each, 11 tasks by 3 seeds), added the own report without its sketch block, and a search control matched at 150 CPU seconds per task. Its rule: own reports beat failure-blind invention only if the exact McNemar $p < 0.05$ and a bootstrap 95% interval, clustered by task family, lies above 0; otherwise the result is inconclusive.

The rule was not met (`experiments/capability_growth_20261027/reports/TEST_IB_REPORT.md`; Table 29). At task level (seeds rarely changed an outcome; `records/NOV1_S1_TASK_LEVEL_CORRECTION.md`), own reports recovered 2 of the 11 motivating tasks and failure-blind invention 2 (discordant 1 to 1, exact McNemar $p = 1.0$): inconclusive; the unit-level test first reported (5 against 3 of 33 runs) counted tasks several times and is withdrawn. Removing the sketch block did not lower recovery (6 runs against 5), so the hole specifications were not what helped. Another family's report recovered none. Search with the starting language at matched compute, 150 CPU seconds per task, recovered 5 tasks against 2 for own-report invention (3 to 0, $p = 0.25$): with equal compute, the old language's search found programs that fit each motivating task and answer its test, where the proposer mostly could not write the operation, although no task-level comparison reaches $p < 0.05$. The pooled unit-level comparisons across both tests are withdrawn for the same reason. The proposer's failures taught us that writing the operation, not the evidence, is the binding step. That opened the symbolic construction test of Section 5.5.2.

Two positive engineering results came with it. A symbolic merge, run on each condition's registry, replaced the constant of promoted instances that differ only in one literal by a parameter and re-validated the result: the factor-2 and factor-3 upscale instances became one parametric operation (domain 2 to 3) that agrees exactly with the removed reference at both factors and solved 6 of 6 excluded tasks of its family, an expansion of the starting language. No promoted operation was rolled back, and the leak scan of 474 distinct proposals found none near-verbatim and none using a removed name. An exploratory pilot outside the causal test, a neural specification completed by symbolic search over the starting language, recovered no motivating task (only macros were possible there). So prediction 2 is not supported, and in this test prediction 3 is not supported at matched compute (5 against 2 tasks, not significant); what survives is that verified, promoted operations are reused within their family (predictions 4 and 5). It opened the natural-task growth test, which asks whether such operations reach tasks outside the constructed families.

Construction needs an invertible operation. On list functions the hole specification recovered 5 of 9 tasks against 0 for another task's ($p = 0.0625$) and equalled the demonstrations alone, because the deletion removed the only invertible unary operation and left the identity as the only decomposition (Section 5.5.2). It opens tests in which each deleted operation leaves an invertible decomposition.

B.4 Strong static baselines: the controlled pilots

A parallel series of controlled pilots, run in separate worktrees and verified by a read-only review (`records/NOV1_INDEPENDENT_RESEARCH_REVIEW.md`), tested learned controllers against static baselines. All of it is development evidence on generated, authored or public training tasks (its 256 source tasks lie in the training split, outside every sealed partition), and each task set is kept separate (Table 30).

The synthesis that survives the review is a negative result with a mechanism: in every pilot a strong static search or constraint baseline matched or beat each learned controller, and failure-blind controls tied failure-conditioned ones; learned components helped only when they re-ranked inside a fixed program bank, never when they pruned search. The construction test differs in kind: there, failure evidence did not route or prune a search but stated what a construction must satisfy (Section 5.5.2).

B.5 Learning inside a fixed language: what each control rules out

Each result below is kept because it rules something out, and together they motivate self-extension: none of these routes changes what the language can reach. The five families were screened on source puzzles; their public-evaluation column is the only ARC number here, and it is 0 throughout.

Two earlier tests found that failure evidence did not help choose. On 288 synthetic groups, a demonstration summary chose the correct repair 59.1% of the time; adding failure evidence changed this by -0.6 points (interval -2.6 to $+1.4$), and the task's own evidence did no better than a matched donor's ($+0.5$, interval -1.9 to $+2.8$) (`records/ITEM2_V15_SELECTION_RESULT_20261001.md`). In a 27-task synthetic repair study, the configurations using failure evidence, no failure evidence and a donor task's evidence each certified 3 correct repairs, that is, accepted them under the study's verification, in development on synthetic tasks, and the entry built on it (submission 57354496) scored 0.00% (`records/CORA_FHR_DEV_RESULTS_20261014.md`). This taught us that a failure descriptor helps only if it carries information the demonstrations lack. It opens features taken from the failed search itself.

Blind repair matched evidence-directed repair. On 40 fresh source puzzles with identical saved pools and 400 executions per condition, repair ordered by CORA's evidence was test-correct on 0 puzzles, blind ordering on 1 and shuffled evidence on 0 (exact McNemar $p = 1.0$; `records/CORA_MECHANISM_TEST_FRESH40_RESULTS_20261026.md`). The pre-registered verdict is *not supported*.

Generated puzzles: large engineering gains, confined to CORA's own language. On held-out puzzles generated from families written in CORA's own rule language, recovering the program classes of CORA's own language (rule language version 2) raised the number solved from 43 to 162 of 200; construction from residual masks (rule language version 3) from 166 to 271 of 296 (106 gains, 1 loss), mostly through construction

(construction alone 81.8%, new selection over the old candidates 61.1%); rules over parts of objects and lines (rule language version 4) raised its target groups from 15.4% to 97.3% (104 gains, 0 losses), with 9 of the 11 target families written in its own language; rule language version 5 was right on 40 of 40 target tasks against 20 for rule language version 4. Every rule language version solved 0 of the 120 public evaluation puzzles. All generators drew on background 0, which explains part of the gap (Appendix B.7). A guide trained on frozen features of a 4-billion-parameter open language model solved 198 of 395 held-out generated compositions, against 30 untrained and 51 with shuffled labels, but the hand-designed order solved 358, and at equal budgets the guide trailed (198 against 306 at 500 candidates, 292 against 393 at 3,000). These results taught us that gains confined to CORA's own language do not reach real puzzles, and that the learned guide did not beat a hand-designed order. They open generators that draw on real grids, and guides tested against the hand-designed order.

Failure-derived pruning: sound in practice, not faster. Skipping candidates that a failed single-rule candidate rules out (Popper's idea [46], adapted) lost nothing in practice: none of 10,129 pruned candidates fits. It reached the first correct candidate earlier on 53 of 127 generated tasks and later on none, but solved barely more (121 against 119 at a budget of 300; equal at 1,000 and 3,000) and saved no CPU (ratio 0.98). This is an empirical soundness check, not a proof (Appendix B.7). It taught us that pruning from failed candidates was sound but removed too little work to save time. It opens such pruning aimed at expensive operators, where the contracts did save CPU.

Learned revision ranking with baseline preservation (first test negative). The companion study's signed-delta proposal ranker, with 261 parameters, first runs a protected static trajectory of 12 candidates, so a static success in that prefix is kept. Its pre-registered primary comparison was negative: at 64 validations it solved 8 of 16 held grid tasks against 12 for streaming static search, and 5 of 16 graph tasks against 13, with no paired gains; failure-blind and shuffled conditions had zero gains and zero losses against it (the companion study's records; available from the authors on request). *Diagnosis:* the learned conditions pay for the protected prefix and for probes of parent candidates that the streaming comparator skips; at 32 validations they solved no held task against 4 grid and 5 graph tasks for streaming static, and at 128 every condition, untrained, failure-blind and shuffled included, reached the same totals. This taught us that the ranker's totals did not depend on the failure evidence. *Pre-declared success criterion:* a new pre-registered test must beat streaming static search at matched validations before any learned-ranking claim.

B.6 Program writers on the public evaluation

Program writers on the first 20 public evaluation puzzles. No proposer produced a program that reproduces the demonstrations of any of the first 20 public evaluation puzzles (32 query outputs): an adapted SOAR-7B baseline [138] (936 candidates with refinement, 589 executable, 0 fitting), a 30-billion-parameter open code model (4 of 20 puzzles finished within its cap) and a 4-billion-parameter open model, all executed by CORA's grid-function backend (`kaggle/hybrid_2026/llm_writer/PROPOSER_COMPARISON.md`). Our earlier writer of CORA programs, revised from CORA's typed diagnosis, also fit nothing on those puzzles in any condition (Table 32); CORA's diagnosis made more revisions change the program (60% against 38% with generic feedback) and more reduce the wrong cells (37 against 19), but most revisions were unchanged or worse. This taught us that the diagnosis steers revisions, but free-form program writing rarely reaches a fit. It opens revisions made through typed construction under the same diagnosis.

B.7 Corrections forced by an independent code review

A read-only review of the synthesis code (`records/CORA_SYNTHESIS_REVIEW_20261027.md`, sections 1.2 and 4) reproduced one representation defect and required five claims to be corrected before they enter the paper. All five are applied here.

1. *The background defect, and its fix.* Six of CORA's seven views, and its original symmetry completion, assumed background colour 0, while 96 of the 120 public evaluation puzzles have a non-zero majority colour in some grid (67 in every input grid; 59 of the 81 same-size puzzles are affected). On such grids, multicolour, 8-connected and colour-layer objects swallow the background. Every synthetic generator drew on zeros, so no synthetic check could see it. The fix, in CORA-CAST, takes each grid's majority

colour as background. Alone it gave no new fit (0 of the 59 affected same-size puzzles, rule language version 5 search with adaptive backgrounds); it is the representation that the 62593bfd program needs. The correction taught us that a representation fix enables programs without creating fits by itself. It opens constructors that use the corrected background. It is reported as a correction, not as a capability.

2. *"Proved unsatisfiable" becomes "no fit in the enumerated fragment".* Relational completion had been reported to prove 1,825 of 1,826 sketches unsatisfiable. Its selector domains were capped without the cap being recorded, and 84% of the sketches lay outside the capped fragment, so the correct statement is that no fit was found in the enumerated fragment; that gap is itself evidence of an expressivity limit.
3. *Spurious fits, not ambiguity*, on 97d7923e (Section 3.1).
4. *Not "exact, proven".* The collapse of equivalent views is not exact and proven, and the second failure-derived pruning rule is not provably sound; the measured statement is that none of 10,129 pruned candidates fits.
5. *Fit counts and compute.* The rule language version 5 fit count on 97d7923e may depend on the hash seed (a tied sort key at a binding cap of 64); and matched-compute statements for rule language versions 4 and 5 exclude their head and leave-one-out phases, which run outside the soft CPU cap (a 0.5 s cap used 4.4 s).

The review also found that the near-miss attempt rewards doing nothing: ranked by raw wrong cells, 26 of 98 near-miss attempts in CORA's language, and 110 of 153 in grid functions, equal the query. CORA-CAST and its deployed acceptance rule reject the unchanged query.

B.8 Representation growth and neural-to-program verification

Representation growth. Natural runs used 1,000 exposed training tasks, demonstrations only. Object relations: structured 30, relation 31, random 33; relation-only 0, random-only 2, $p = 1.0$. Typed crop: crop-only 5, random-only 1, $p = 0.109$; 69 of 999 tasks shape-feasible. Canvas filter: zero fits in every arm. Object maps on authored tasks: growth 78 of 80, fixed same-language search 80 of 80, random 30 of 80, earlier renderers 0 of 80; on natural tasks every arm fit one task. Three further authored structures reached 80 of 80, 48 of 48 and 32 of 32 tasks, each tied by fixed same-language search, and growth needed 1.44 and 1.19 times the fixed work in two of them. Role and palette bindings: 24 of 842 against 23 with roles shuffled; object correspondence: 5 of 841 against 4 for static search. An audited scan of all 1,000 training tasks counted 30 conditional palette fits, 320 shape changes and 297 new output colours.

Neural-to-program verification. The output chain was reviewed before scoring (1,540 of 1,540 arms, every arm ending without a fit in its bounded proposal) and scored once, with a postscore audit passed. A control search without the guess as a target reached none of the 172 outputs. That test searched CORA's earlier construction grammar, first attempts only, and every search stopped at its execution budget; Appendix B.9 records the later search of the same file with CORA-CAST's modules.

B.9 Neural attempts and the CORA gap: definitions and records

Post hoc, exposed public evaluation, not an official score. Records: `paper/final_push_20261108/neural_vs_cora/` (the two analyses, their check, the gap table and the data folder).

Terms. An *object view* is how CORA-CAST cuts a grid into objects on its majority-colour background: one-colour cells joined through edges, one-colour cells joined through edges or corners, or cells of any non-background colour joined through edges or corners. A guarded-rewriting rule pairs a *guard*, which picks the objects it acts on, with an *action*: keep, erase, recolour, recolour to the colour of a touching or enclosed object, fill holes, draw a one-cell outline on the background around the object, or move it in one direction until blocked or to the border. A program has at most two guarded rules and a default and gives each object at most one action; guarded rewriting also has recolour tables keyed by one feature and, when the size changes, a crop to the box of one object that a guard picks. Typed search chains at most two whole-grid operations (turns and flips, colour swaps, crops, scaling, tiling, joining with a transformed copy, filling enclosed areas, outlines, gravity), and symmetry completion fills the cells under a mask colour from the grid's symmetries. *Depth* is how much a program may chain. A program *fits* when it reproduces every demonstration exactly.

How each cause was diagnosed. The diagnostics read only the demonstrations and queries, never an answer, and their code names no task. *Scene parsing:* under each view, every demonstration input and output is cut into objects and matched; a view is consistent when every input object is kept, changed as a whole or found again with the same shape, and every changed output object is a recoloured or moved input object or new cells on the background. A task passes when one view is consistent in every demonstration. *Language:* every program that CORA's modules build from one demonstration is run on all of them. On same-size tasks, CORA's own evidence step lists the single actions that fit each object's change, and changed cells that no action of any object can write mean a missing action. If every changed cell can be written, a program that gives each object the action it needs is replayed with CORA's own rules, and a rule list over CORA's guards is built one rule at a time; no list, or a list that does not carry over between demonstrations, means a missing guard, while an object that needs two actions at once, or a list longer than two rules, means depth. On size-changing tasks, an output that is not a window of its input is built, not cut out, which is a missing action. Each task gets one verdict, from the view that comes closest. *Budget:* the shipped three modules were rerun on each task, in a fresh process, at the CORA-only entry's budget and at four and ten times it (Table 33). The largest of the 36 tasks took 9.4 CPU-seconds at the entry's budget and 8.7 in the wider check at ten times (cap 450); typed search stopped at its 1,000-execution cap on 4 of the tasks at the entry's budget and, given four or ten times its executions, finished its whole space of two-step programs on all 36 (at most 1,480 executions). Of the 36 tasks, 23 keep the grid size and 13 change it; in none does a program that CORA builds from one demonstration reproduce even that demonstration; and in 3 (aa4ec2a5, d8e07eb2 and de809cff) an object needs two actions at once even on the part of the change that CORA can already write.

The main missing piece, by task. By the diagnostics' labels:

- drawing lines or filling regions on the background, 11 tasks: 1ae2feb7, 36a08778, 6e453dd6, 7666fa5d, 7b80bb43, 8b9c3697, 8f3a5a89, a395ee82, b6f77b65, d8e07eb2 and db0c5428;
- a tie between that and erasing part of an object, 1 task: 31f7f899, 16 cells each;
- recolouring part of an object, 4: 1818057f, 4c3d4a41, 71e489b6 and 7c66cb00; erasing part of an object, 2: 135a2760 and de809cff;
- copying a shape to a new place, 2: 7ed72f31 and 9aaea919; new cells bordering an object, 1: aa4ec2a5; moves by distances that CORA's stopping rules do not produce, 1: b5ca7ac4;
- size changes, the output built from pieces of the input that are not objects, 9: 20270e3b, 2d0172a1, 38007db0, 3a25b0d8, 45a5af55, 4c7dc4dd, 58f5dbd5, 78332cb0 and a251c730; copies of input objects arranged in a new grid, 2: 2ba387bc and e8686506; input objects arranged with new content, 2: 65b59efc and bf45cf4b;
- a missing guard, 1: d35bdbdc.

Some tasks are close. The changed demonstration cells that no CORA action can write number 2 of 66 in 4c3d4a41, 3 of 31 in 7b80bb43, 36 of 174 in b6f77b65, 42 of 178 in 6e453dd6, 44 of 162 in 9aaea919 and 128 of 449 in aa4ec2a5; they are half or more in 12 of the 21 same-size tasks that have such cells. In none of the 13 size-changing tasks is every demonstration output a window of its input, even after a turn, a flip or a recolouring, so the crop rule cannot apply. At the entry's budget guarded rewriting skipped some views of 3 tasks (4c7dc4dd, a251c730 and d8e07eb2) for having more than 120 objects summed over the demonstration inputs; the wider check searched them, with no fit.

The explanation search. Each module first searches the demonstrations with no target; every fitting program is kept with its outputs on the queries, and a neural attempt is explained when one of those outputs equals it. Programs that reproduce every demonstration exist for 4 of the 120 tasks: 62593bfd and 981571dc, e3721c99 (whose program returns the queries unchanged) and 0934a4d8 (whose program leaves 8 cells of the query output open). A second route adds the attempt as an extra worked example and searches again, as the earlier test did; it explained nothing more. Every counted program was run again outside the module that proposed it, on every demonstration and on the query; none failed, and no module raised an error. At the starting budget (1,000 typed-search executions, 15 CPU-seconds per module call), 40 searches aimed at a neural attempt were cut short, 39 in typed search and 1 in local growth; at ten times the budget all of them finished. As a filter, keeping a first attempt only when it is explained keeps 1 of the 172 outputs, which is right, against 33 of 172 right first attempts without the filter. Controls: aimed at CORA's own answer, the search recovers it on all

3 query outputs where CORA has one and rejects it with one cell changed; on 3 tasks whose demonstration outputs were replaced by a planted two-step program (flip upside down, then let cells fall), it finds that program and rejects the same target turned a quarter. Without a target, typed search stopped at its 1,000-execution cap on 11 tasks, 4 of them in the gap; rerun with 10,000 executions, all 11 finished within 1,480 executions, and none fitted. The search code never opens the answer file; the answers were joined afterwards.

Checks on the diagnostics. The rerun at the entry's budget reproduces the CORA-only entry's 3 answers exactly. On 62593bfd (Figure 3), which CORA solves, the diagnostics' rule-list learner recovers CORA's own two-rule program from the demonstrations. 981571dc fails the object diagnostics yet is solved by symmetry completion, so those diagnostics speak only for guarded rewriting. The two analyses and our check used about 2.1 CPU-hours of local CPU, with no GPU and no neural inference.

B.10 Where construction stalls

Where construction stalls. The companion study's construction-bottleneck table (Table 34) diagnoses seven ways in which construction failed in its closed studies. Its counts are in different units (records, candidate programs, task banks and atom occurrences, tasks, queries) and are not pooled; none of the seven studies produced an accuracy gain. Each row's supported conclusion is what it taught us; together they point to construction that keeps distinct bindings and validates beyond the ranked prefix, and to re-ranking rather than routing.

C Cross-domain studies in detail

The biological procedure is CORA's ARC diagnostician (sha256 `ba9b65c2`, with its feedback record `e99890fd`) behind one interface (`cora_cross_domain_transfer/interface.py`), with domain adapters and revision policies; the nuclei decisions came from the revision policies in `cross_domain/scrna_dispersion_20261025/revision_policies.py` (sha256 `9fa3d97a`) over the BBBC039 adapter `cora_cross_domain_transfer/adapters/bbbc039.py`. A replay of the recorded feedback reproduced the diagnostician's scores on 217 of 218 nuclei-repair steps and 58 of 58 single-cell steps, and its action orders on 21 of 21 and 7 of 7. In every study the procedure started from a fixed hypothesis, revised it on development units only, was validated by an evaluator that alone read held-out labels, and was compared at equal budget with failure-blind, shuffled-feedback, generic-order and try-all alternatives.

C.1 Biological regulatory networks: learned priors and the readout

A learned ranking prior: a development observation on source models that did not reproduce on independent lineages (the companion study's study, independently reviewed; a supplied Boolean rule language and a ranking prior learned within computational biology, not learned on ARC tasks and not biological discovery: the targets are published model equations and the demonstrations are simulated state transitions of those models, not measurements). On 182 single-gene tasks from 25 published Boolean-network models in 11 publication lineages, a ridge ranker trained on the other lineages' published update functions reconstructed 98 functions exactly, against 95 for a fixed minimum-description-length rule, 95 for the fixed rule tuned on the source models and 92 for a gradient-boosting classifier trained on the same examples (5 gains and 2 losses against the tuned rule; exact paired test $p = 0.23$ one-sided, 0.45 two-sided: a development observation, within noise). On 356 tasks from 46 further models in 43 lineages sharing no publication lineage with the training models, evaluated once under a frozen protocol, the same ranker reconstructed 121 functions against 124, 125 and 126 for the same comparators (2 gains and 5 losses against the tuned rule; lineage-bootstrap difference -0.84 points, 2.5th to 97.5th percentiles -2.33 to $+0.56$). The prospective cohort could detect a gain of about 3 points with 80 percent power; the source gain of 1.6 points lies outside the prospective interval, so the source gain did not reproduce on independent lineages. Most failures in both cohorts were failures of support identification: for 59 of 182 source tasks and 176 of 356 prospective tasks a regulator proven essential lay outside the eight-variable shortlist chosen from the ten demonstrations, so no candidate in the shared bank could be correct and no ranking rule could repair it (of the prospective misses, 16 more had their correct function absent from the bank and 43 were reachable but unselected). It opens better support identification, before any ranking. Ten demonstrations carry at most ten bits, while naming three regulators among twenty variables needs 10.2 bits, and at scale a learned support ranking did not get past that bound: on 656 targets in 82 models and 68 lineages a neural-network

shortlist reconstructed 24 published rules exactly, as did a logistic shortlist and the fixed flip-evidence rule (24 each; difference 0, Holm-corrected $p = 1$), against 3 with its shortlists shuffled and 21 for random shortlists, so its pre-declared criterion failed (the companion study's audited record, 1,859,206 checks); a prior over which variables matter, inferred from the same demonstrations, was the first mechanism with an audited positive result in this setting (the 99 against 85 of 333 above), as a support prior built from the task's own demonstrations, and it replicated prospectively (101 against 66 of 595, every exact recovery a one-input rule). The tie taught us where the information was lost. A review of the saved outputs, awaiting the companion study's confirmation, found that the three shortlists gave identical answers on all 656 tasks, that the readout never accepted a rule with two or three inputs, and that 191 targets with at most three inputs lost a regulator at the shortlist. That 190 of the 656 targets need more than three inputs remains true, but every arm loses those targets alike, so they do not explain the tie. It opens readout and acceptance design, and more informative examples. Disclosure: all 285 models of the collection, the 46 prospective models included, had been parsed on 2 November 2026 for an inventory that evaluated each update function to classify genes as constant, identity or regulated and recorded each gene's number of documented regulators; no outcome of the 46 models was evaluated before the prospective run and no per-gene statistic of those models was summarised before it, so the prospective cohort is new relative to every ranking experiment but is not unseen data (the companion study's records; available from the authors on request).

An exploratory readout comparison (target reuse). This comparison reused 656 previously tested targets from 82 models in 68 lineages, inside the supplied three-input language; it has no fresh confirmation and no success gate. With the neural guide, learned rule-family weights (a learned readout) recovered 97 exact rules against 104 with equal weights (7 gains, 14 losses), below the fixed guide with the same learned weights (133) and the fixed single choice with the neural guide (200). That fixed choice recovered 200 exact rules (196 one-input, 4 two-input and no three-input) with 451 wrong and 5 unknown accepts; with the held-out check it recovered 84, with 55 wrong and 1 unknown. The targets comprise 222 one-input, 147 two-input and 92 three-input rules, 190 that need more than three inputs and 5 of unknown arity. This taught us that the acceptance policy hides some exact two-input rules and that success is still mostly one-input. It opens comparisons of informative examples against random ones at a matched budget. An earlier learned-support test on these targets used different guide and policy factors, so its 24 exact rules are not a baseline for these counts, and none of these counts is a learned or neural gain. The neural guide plays the role of DeepCoder's network, which predicts which functions a program contains in order to order exact enumeration [11]: it proposes which regulators a rule reads and leaves the rule to exact search in a language of at most three inputs, whereas a network trained to output logic-program rules from transitions could not recover a five-node network containing a four-input rule [135]. Published regulatory functions are over-represented among short read-once and nested canalizing forms [159], whereas this language is capped by number of inputs, not by formula length.

Boolean models, further detail. Table 16 lists the arms. In the supplied-wiring study, 236 of the 333 genes are expressible in the three-literal language, and correct wiring recovered 148 of them; an oracle-structure reference with the documented regulators recovers the published rule in 207 of 333 tasks. A separate grammar-growth stage on 151 held tasks in 19 models chose a supplied three-input OR form after 14 base failures in 8 lineages, selected from biological source models, not from ARC, and recovered 60 of 151 exact functions in two attempts; the right function was among its candidates on 76 of 151, so 16 were reached but not chosen; because the selected arm reuses the fixed form, the two agree on all 151 tasks, and three other forms and the full language also reach 60 (base 59, shuffled 59, random mean 58.85; no contrast survives Holm correction). Grid-to-biology operator transfer first found zero operator witnesses in 1,000 ARC training tasks, so its biology stage never ran; a later foreground-mask adapter found 22 witnesses among 1,000 training tasks (NOR 8, OR 8, XOR 4, AND 1, TRUE 1); the order counted from them recovered 177 of 595 exact rules, against 168 for plain index order, 182 for the same counts randomly reassigned and 194 for a hand-set simplicity order; its gain over index order did not survive Holm correction (adjusted $p = 0.14$), and it lost 5 targets and gained none against the reassigned counts and lost 20 and gained 3 against the simplicity order (Section 6.6). A late review of the replication's protocol outline arrived after launch, so the compatibility check against the 333-task source study was completed only retrospectively, after the run; it then matched that study's predictions exactly on all 333 tasks and 1,665 condition records. Not every runtime cache was individually pinned.

Where each biological result sits on the validation ladder. Table 35 places every biological result of this paper on the ladder of Section 6.4. CORA's biology is strong at rung 1, has been tried at rung 2 on real measurements without a robust success, and has not been taken to rungs 3 or 4.

C.2 The submitted component on fresh nuclei images

The submitted solvers on fresh nuclei images. The pre-declared follow-up (`records/NOV1_UNIVERSALITY_EXPERIMENT_PROPOSAL.md`; protocol frozen before any mask was decoded) ran both entries' solvers, byte-identical (31 files verified 13 times) and through the entries' own code, as a verified object filter on StarDist's output for S-BSST265 [92], a CC0 fluorescence nuclei set none of our studies had scored: 3,639 StarDist objects of 23 test images, posed as 455 ARC-format tasks (`records/NOV1_UNIVERSALITY_RESULT.md`). The component ran on every task. Under the neural comparison entry's strict rule it accepted no edit, so object F1 stayed at the tuned baseline's 0.858; the 33 deletions its fitting programs proposed without the leave-one-out check were two-thirds wrong (22 were real nuclei). Abstaining on every task gives the same F1, so this shows non-degradation through complete abstention, not a safety advantage. Under the plain CORA-only configuration's weaker rule it accepted 7 deletions, 4 of them real nuclei, about as often as with shuffled demonstrations (10), and lowered F1 by 0.0006. A single intensity filter tuned on the training images did better (0.867). Neither predeclared endpoint passed (selectivity; utility). The component is portable; it did not improve biology, and no learned transfer is involved; one dataset, 23 evaluation images. It taught us that stability under rebuilding did not pick out useful corrections in these images, while a tuned simple filter did. It opens an admission bar keyed to prior knowledge about nuclei.

The submitted component on a fresh fluorescence image set: the acceptance check passed a fitting but wrong program (companion experiment, pre-registered). The frozen CORA component of both entries, byte-identical, verified object deletions learned from four labelled calibration windows per task on HeLaCytoNuc, a publicly licensed image set none of our studies had used: 161 held-out images, 11,549 StarDist objects, 1,444 tasks, analysed by parent image (`records/NOV3_FRESH_IMAGE_TRANSFER_RESULT.md`). A calibration-only check confirmed room for improvement: deleting every false positive would have added 0.09 F1. Under the neural comparison entry's rule the component accepted 11 deletions, and 10 of them removed real nuclei: in both tasks it acted on, every demonstration window held only debris, so an erase-everything program fitted and survived leave-one-out. Under the plain CORA-only configuration's weaker rule it accepted 66 deletions, 43 of them harmful, and lowered F1 by 0.0014 per image (interval -0.0025 to -0.0006). A calibration-selected simple-rule search on the same calibration data (the best conjunction, small and touching objects), with much less CPU and not a matched comparison, gained $+0.0123$ mean F1 per image (interval $+0.0082$ to $+0.0164$). The component is portable and usually abstains; here it did not pick out beneficial corrections, and a calibration-selected simple-rule search did. It taught us that when every demonstration window holds only debris, a stable fit can be a harmful one. It opens demonstration windows chosen to separate the rule from its rivals. The shuffled-evidence control proved weak, and the image grouping may be optimistic if images share wells.

C.3 What the repair procedure achieved (a mechanism the entries do not run)

Nuclei images: repairing a segmentation. The public BBBC039 collection [23, 104] (200 fields of U2OS nuclei, CC0; official split 100 training, 50 validation, 50 test) tests object repair on public images. Starting segmentations are a thresholded watershed and plain connected components; repairs are split, cut, bridge and remove; feedback classes merged nuclei as whole-object errors, splits and boundary errors as part errors, missed nuclei as additions and false positives as broken (`cross_domain/bbbc039_repair_20261026/REPORT.md`).

From the watershed start the diagnostic condition improved 44 of the 50 test images (5 regressed, 1 unchanged) and beat the generic order, the shuffled diagnosis and equal-budget try-all, but not failure-blind revision: one seed reached 0.9234, and the paired mean difference against the failure-blind seeds was $+0.003$ (interval -0.001 to $+0.008$). From the components start five controls beat it. Development and test F1 tracked closely (0.920 against 0.923). The auditor's status is blocked, so its comparisons cannot be credited: the diagnostic condition used 54 executions against 60 for every comparison, an 11.0% gap over its 10% tolerance. It taught us that the diagnosis added nothing measurable over failure-blind revision here, and that a small compute mismatch can block a comparison. It opens a rerun at matched executions against failure-blind revision.

Repair on top of a pretrained neural segmenter, on fresh images. Mirroring the ARC setting, where CORA works beside a strong neural model, the study's frozen repair procedure started from StarDist's pretrained `2D_versatile_fluo` segmentations [145] (BSD-3-Clause code and weights) on 150 BBBC039 development images, and each condition's frozen program was then applied to 50 fresh images of a different screen and instrument (a public collection [10], CC BY 4.0, not in StarDist's training data; masks read by the evaluator only, under a protocol frozen before any segmentation). Every condition, diagnosed or not (all 13, try-all included), made the same single revision (delete objects dimmer than 0.05 of full scale, one of the adapter's own hand-written candidate rules): pooled object F1 rose from 0.692 to 0.705, per image by $+0.014$ (95% interval $+0.010$ to $+0.019$; 38 images better, 0 worse, 12 unchanged), mainly by removing 93 of StarDist's 995 false positives at the cost of 7 more missed nuclei. The gain over the neural start is the repair procedure's; the diagnostician tied the failure-blind, shuffled, generic and equal-compute conditions exactly (the auditor's verdict: complete, no confirmed gain; `cross_domain/universality_v2/neural_repair/summary.json`). It taught us that the gain came from the shared candidate rule, not from the diagnosis. It opens diagnoses tested on candidate banks where the conditions can choose differently.

Single-cell lung atlas: displacement or dispersion (a retrospective application of a related design). This study applies the separately written repair procedure, which shares a design with CORA-CAST but no code, to a published analysis; it is not evidence that the Kaggle model learned a transferable policy, and nothing in it was learned. The original analysis is joint work with Pierce Taylor. Do alveolar epithelial cells in lethal COVID-19 shift together towards one damaged state (*displacement*) or spread apart across several (*dispersion*)? Three kinds of result are kept apart. (a) *Findings of the original research, not of CORA.* The data are 22,128 alveolar epithelial nuclei of a public lung atlas [19, 111]. An April 2026 analysis by Pierce Taylor, Chimdi Walter Ndubuisi and Toni Kazic reported that coherent displacement was not supported (one-sided Mann-Whitney $p = 1.0$) and that dispersion was (variance ratio 2.3); a later donor-level audit by the first author found 26 patients behind the 27 sample labels and reversed the headline: the 2.28 ratio is not significant at the donor level ($p = 0.18$), and coherent displacement is robust in every representation (frozen-rule $p = 0.0024$). (b) *What CORA's framework newly produced.* Under a protocol frozen before its single run, with patients as units (16 development, 10 held-out, and a second cohort over a shared panel of 83 genes), the procedure started from the April claims, removed the spread change and added a displacement fitting all 16 development patients; fixed in advance, the predicted direction held in an independent cohort of 41 donors from the CELLxGENE Census [49] ($p = 0.0016$), but its fitted size overshoots: in that cohort the reported final predicts all 19 COVID-19 donors worse than no effect, and every condition's final p lay between 0.0006 and 0.0016, the reported value being the weakest. The pre-registered held-out test was invalid (its items were constant in three controls), and a recalibrated transitional state did not transfer ($p = 0.22$). (c) *Against the controls: no measured gain*; every condition found the displacement (`records/CROSS_DOMAIN_SCRNA_CASE_STUDY_20261025.md`). It taught us that the displacement was strong enough for every condition to find, so it cannot credit the design. It opens single-cell questions on which the conditions disagree.

C.4 Where controls equal or beat the repair procedure

Drug response. Typed rules from the first author's ontology-guided pathway model, revised on development compounds and applied frozen to 416 held-out compounds of the PRISM screen [45], gave mean balanced accuracy 0.5249 with CORA's diagnosis, 0.5248 with the generic order, 0.5246 failure-blind, 0.5259 shuffled and 0.5295 for try-all at equal compute, which was better than the diagnosed condition (-0.0046, interval -0.0092 to -0.0002); the diagnostician returned the generic order on all 968 diagnosed steps (the drug-response study's report).

Grammar induction from failures. New typed operations were proposed from recurring failures by eight proposers (symbolic and a pretrained language model, with or without failure information), and a production was kept only if it lowered the error on units not used for proposing. No production was accepted in any of the three domains, so every held-out unit ties with the starting language (`records/GRAMMAR_INDUCTION_DOMAIN1_SCRNA_20261026.md`, the drug-response domain's record and `records/GRAMMAR_INDUCTION_DOMAIN3_BBBC039_20261026.md`).

Domain-calibrated controllers. Controllers recalibrated per domain made evidence-dependent decisions in every domain but did not beat their own bookkeeping-only ablation on held-out development units (drug response balanced accuracy 0.5174 against 0.5252; nuclei pooled F1 0.8467 against 0.8840), so no controller passed the development criterion and the sealed drug-response compounds stayed sealed (`cross_domain/universality_v2/UNIVERSALITY_EVIDENCE_AUDIT.md`). The ARC diagnostician's hand-set thresholds explain part of this: in drug response its conditions never fired, so its order was constant. It opens thresholds calibrated within each domain before any controller comparison.

D Studies completed after the reviewed package, and studies in progress

Completed after the reviewed package, with their audit status. Each ran once under a frozen protocol, and none is in either Kaggle entry. Independently verified by our own recomputation: construction at scale (Section 5.5.3). Independently audited by our own post-score audit: the held-back-half test (Section 5.8). Independently audited by us, claim by claim, from saved aggregates, rows and source: the bounded diagnosis loop and its saved-work efficiency analysis (Section 5.5.3; `records/audit_20261108/AUTONOMOUS_REASONING_LOOP_RESULTS.md`, section 2), the cofactor query-policy revision and the sparse engine (Section 6.2; same file, sections 4 and 5), active selection and the completion vote (Section 6.2; `records/audit_20261108/BIOLOGICAL_TRANSFER_AUDIT.md`, section 4), conditional construction on published models (Section 6.2; `records/SPARSE_COFACTOR_NATURAL_INDEPENDENT_AUDIT_20261108.md`, 214 checks), the view bank, the coverage sweep and the early-stop defect (Section 5.7; `records/audit_20261108/NATURAL_ARC_CONSTRUCTION_REVIEW.md`, sections 2 and 3; `paper/final_push_20261108/SEARCH_TERMINATION_BUG_REVIEW.md`), the real ARC evidence (Section 5.7; same file, section 4), the saved trace of the worked teaching case (Section 6.3; `paper/final_push_20261108/BIOLOGICAL_RULE_GROWTH_CASE_STUDY.md`) and the three continuous-data follow-ups (Section 6.9: spatial and Perturb-seq pass, microscopy inconclusive). Where a claim failed as worded, this version uses the audit's corrected wording. Completed and independently audited after scoring by the reviewer who wrote each pre-run review: the five held tests in new domains (Section 6.11; each study's `reviews/POST_SCORE_AUDIT.md`), the test of failure-guided choice (Section 5.9) and the calibrated acceptance rule (Section 6.7). Post hoc and checked by separate code, not audited: the neural gap analysis (Section 5.4; `paper/final_push_20261108/neural_vs_cora/CHECK.md`), the reachability analysis (Section 5.5.3) and the selection-frontier diagnostics (Section 8.2; 85 numbers re-derived).

In progress. The companion study's experiment on choosing the right change, comparing authentic and shuffled failure routing over identical capabilities with a fixed full-bank controller and an extra-search baseline, has passed review as a draft and has not run (Section 8.1); the specification-reading guide of Section 5.6.1 has since been scored once and audited; and a citation-only parent and exposure ledger for a fresh biology cohort is complete and certifies no case as fresh (`records/NOV8_KADELKA_PARENT_EXPOSURE_LEDGER.md`). The second revision of our pre-registered order test has since been scored once and audited (Section 5.9). The companion study's learned selection study on authored Boolean functions, a controlled study and not the published gene rules, did not run its held study: its pre-declared gate found no room for a selection effect on the development tasks, where every method tied (`records/boolean_selection_review_20261108/V2_PRECHECK_CHECK.md`).

Earlier in-progress companion research, not results. None of it is in the Kaggle entries, and none of it supports a claim of this report. Each item gives its state, its source and the exact pre-declared success criterion it must pass before any claim. Records of the companion study are available from the authors on request.

A domain-neutral failure-to-specification core (software-validated only). The same core (`kernel.run`) handles synthetic ARC grid-program tasks and fabricated Perturb-seq-style target-by-gene point tasks, with typed-hole ordering, exact replay, finite checks that no admitted rule is an alias, and a work ledger; domain adapters supply the inverse semantics (the companion study's records; available from the authors on request). The core's second version reproduced all 10 domain-by-control run records, but its review revealed three boundary failures; a third version, repairing those three failures, passed independent review with 14 of 14 package hashes, 13 of 13 local dependency pins, 15 of 15 synthetic tests, and exact stable fields and work in all ten two-domain control run records (the companion study's records; available from the authors on request). Its scope is fixed, trusted-code synthetic mechanics: no ARC accuracy, real biology or transfer follows. *Pre-declared success criterion:* the frozen ARC binding-preserving inverse-goal assay (48 source grid identifiers, 144 constructed demonstration outputs and 96 unlabelled query inputs; nine synthetic five-condition custody tests

(checks that no held-out answer is read) passed; no query truth opened) must pass independent review before its once-only source confirmation run.

The bioimage grammar test: its design and remaining conditions (pre-registered). The protocol, frozen before any download (`experiments/universality_bio_grammar_20261103/PROTOCOL.md`), admits cited bioimage operators only where the frozen CORA component's own failure records on calibration images show no fit within its declared representation, grammar and bounded search for the needed correction, uses them under verify-or-abstain, induces the grammar on LIVECell (phase contrast) and applies it frozen to NuInsSeg (H&E nuclei) and MoNuSAC; a third dataset was ineligible, and MoNuSAC replaced it under an amendment. Its primary result is in Section 6.7: procedural portability, 0 held-out edits and no learned transfer; its secondary conditions changed no primary answer, and its tolerant rule admitted one operator on calibration data, made 32 accepted edits on held NuInsSeg images, 10 beneficial, 16 harmful and 6 neutral, and passed no pre-declared gate. *Pre-declared success criteria, fixed in advance:* CORA is labelled cross-domain useful on a dataset only if the lower bounds of CORA minus each of abstain-all, all-on, matched search, failure-blind and shuffled are above 0; grammar growth is supported only if the static comparators had every candidate operator, matched search had CORA's measured compute and the failure-blind and shuffled conditions ran the same procedure at the same work; admission is reported apart from useful construction; every admitted operator passed the check that it is not an alias of an existing operator; admitted operators changed held-out outputs; and CORA beats matched search and all-on.

Spatial transcriptomics and Perturb-seq pilots, held behind pre-declared success criteria. The HER2ST [5] source-only study was negative under stronger controls: CORA's macro-F1 on the source patient, 0.183546, equalled its failure-blind and shuffled conditions and fell below balanced 24-rule static search (0.192603), so the pre-specified deployment criterion rejected it and the held patients stay sealed (the companion study's records; available from the authors on request). *Pre-declared success criterion:* a source-qualified hypothesis must pass that admission, and two manifest gaps found by the integrity review must be repaired and independently reviewed, before the held patients are scored once. For the Perturb-seq pilot [140], the engine is cleared only for metadata and synthetic mechanics (14 tests and a six-condition replay without expression values), a metadata-only phase pin (a recorded hash that fixes each phase's inputs) passed 16 tamper and symlink checks and reproduced 1,213, 243 and 255 admission counts, and the 255 held source-validation perturbations of this reused cohort (one human leukaemia cell line, K562; earlier exposure elsewhere cannot be ruled out) have since been scored once by a frozen two-by-two follow-up, which admitted no rule (Section 6.9), and the 1,993 target-held perturbations remain unopened (the companion study's records; available from the authors on request). *Pre-declared success criterion:* an exact source-row-to-phase-bank digest pin, runner and extraction attestation, and independent review of the production assembly come before one source-value run; a transfer claim needs a held outcome under a pre-registered rule. The construction on real perturbation-response profiles has since run and admitted no rule for any of 64 genes, and its frozen follow-ups are in Section 6.9.

Completed companion tests. The neural-guesses test and the two follow-up Boolean-network conditions, regulators inferred from each task's demonstrations and the growth of supplied rule templates, are complete and audited: the inferred-wiring result is in Section 6.2, and the others are with the finished companion tests in Appendix B.2. The audited six-condition learned-library test on the 120 public evaluation tasks is a descriptive null (Section 5.8); it is neither a leaderboard result nor evidence that a learned grammar improved accuracy. It taught us that compression cannot add an operation the language lacks. It opens constructing missing operations from failures.

Roadmap. (1) The domain-neutral core: independent review of the inverse-goal assay, then its once-only source confirmation. (2) The bioimage grammar: its frozen two-by-two follow-up sealed its predictions and failed during reference decoding (out of memory), so it has no qualified endpoint; a first complete score-only recovery of the sealed predictions, keeping the earlier reference exposure on record, would need a newly reviewed protocol. (3) HER2ST: sealed until it meets its criterion. Perturb-seq: the source-validation follow-up is complete and admitted no rule; the 1,993 target-held perturbations stay sealed. (4) Learned ranking: a new protocol, frozen before any outcome. (5) Failure-guided choice: its second revision has been scored once and

audited (Section 5.9); next, the same test on natural tasks. A claim of cross-domain reasoning needs a held, pre-registered win over strong static, failure-blind and shuffled controls in a second domain; none exists yet.

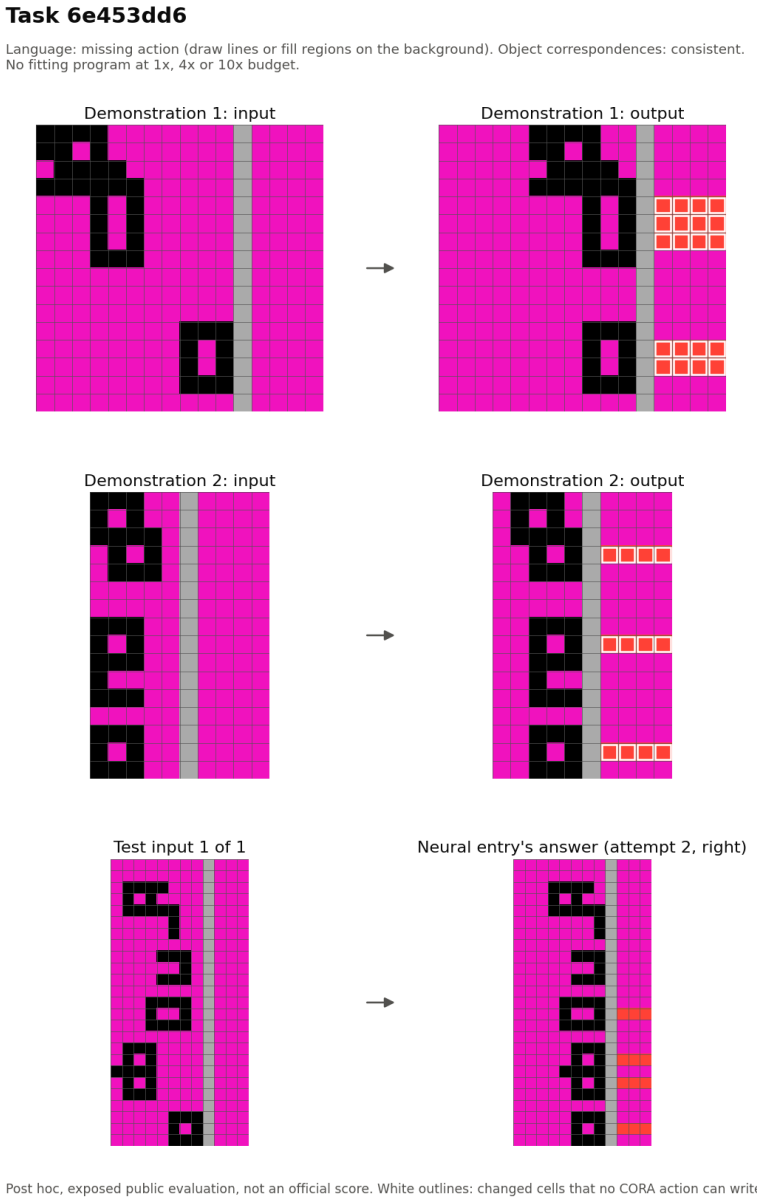


Figure 8: A right neural attempt that CORA cannot explain: a missing action. Task 6e453dd6 of the public evaluation set: two demonstrations, the query and the neural comparison entry's right second attempt. White outlines mark changed demonstration cells that no CORA action can write (42 of 178); the diagnostics' label is drawing lines or filling regions on the background. On inspection, shapes slide right until they touch the grey wall, which CORA can write as a move until blocked, and shapes with an enclosed gap also get red cells on the far side of the wall, in the rows of the gap, which no CORA action draws; the move and the drawing would also have to combine in one program. No CORA-CAST program reproduces the demonstrations at the entry's budget or in the wider check at four or ten times it (labelled 1x, 4x and 10x in the figure). Post hoc, exposed public evaluation, not an official score. Puzzle data: ARC Prize Foundation.

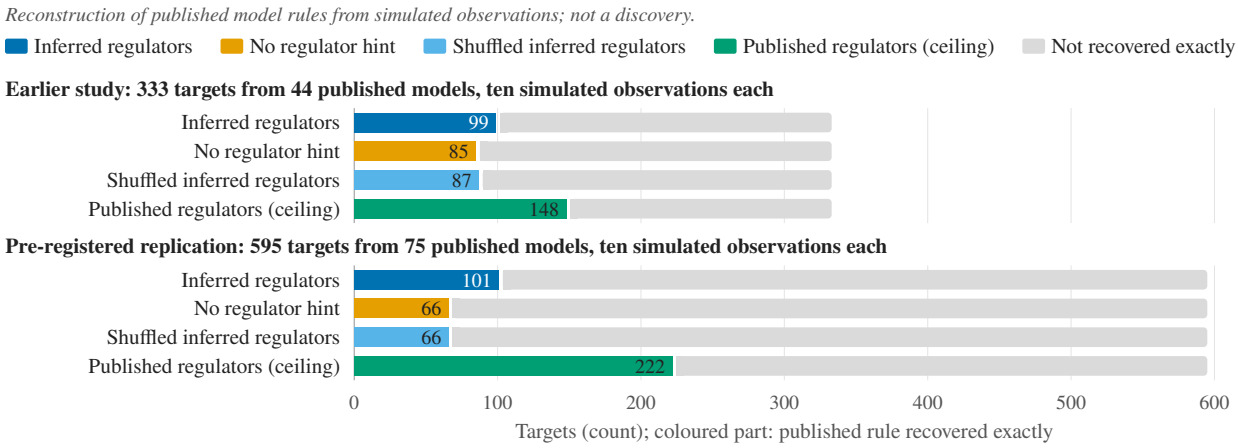


Figure 9: Inferred regulators raise exact recovery of published gene rules (reconstruction of published model rules from simulated observations; not a discovery). Each target is one gene or protein in a published Boolean network model [133]: from ten simulated network states and its next value in each, the procedure must recover its published update rule exactly. Each bar spans all targets of a study; the coloured part counts exact recoveries. Inferred regulators (the inferred wiring of the text) are the genes ranked most likely to matter from the same ten observations. In the pre-registered replication they gave 101 of 595 exact rules, against 66 for each control, every exact recovery a one-input rule; being given the published regulators, a reference ceiling and not a competitor, gave 222.

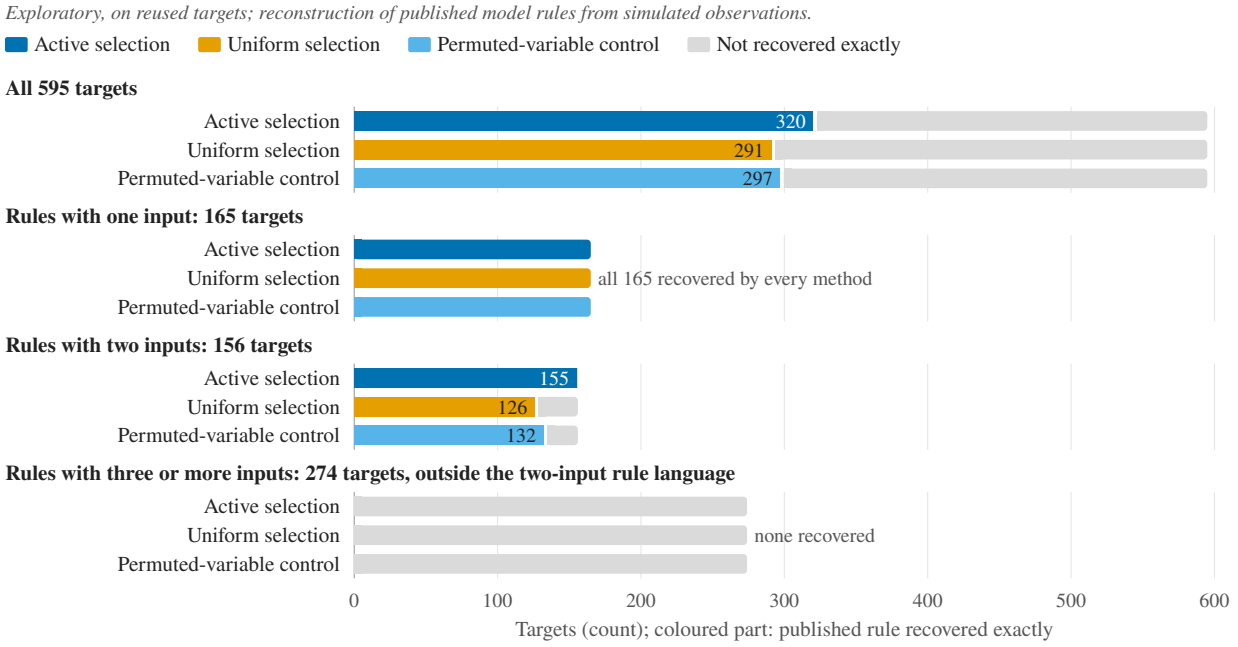


Figure 10: Choosing informative observations recovers more two-input rules (exploratory, on the same 595 reused targets; reconstruction of published model rules from simulated observations). Each method requested 20 simulated observations per target, then used the same rule language (every rule with at most two inputs) and the same rule choice. Uniform selection picks network states at random; active selection picks each next state, from a pool of at most 64, to split the remaining candidate rules as evenly as possible; the permuted-variable control randomly permutes the variable positions of the actively chosen states and labels them anew. Every method recovered all 165 one-input rules; active selection recovered 155 of 156 two-input rules, against 126 (uniform) and 132 (permuted-variable control). The 274 rules with three or more inputs lie outside the language, and none was recovered.

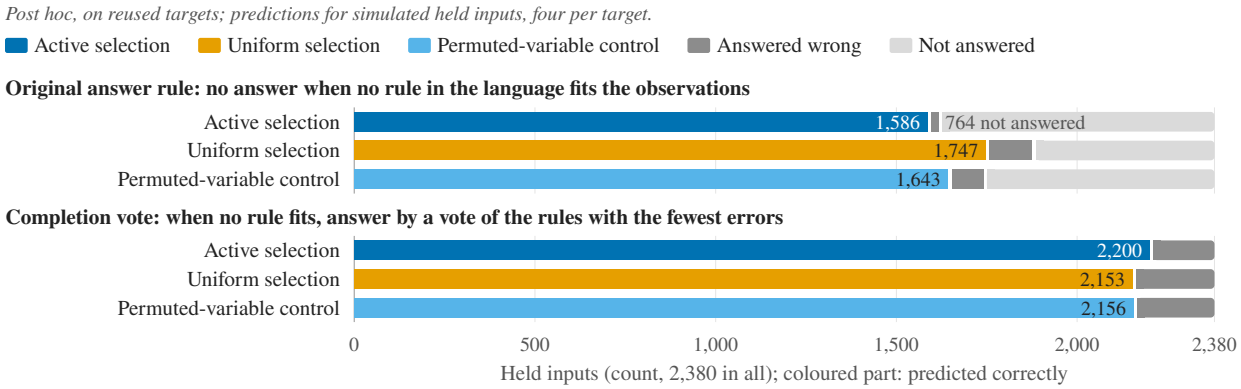


Figure 11: Prediction trade-off on held inputs, and a completion rule fixed before it ran (post hoc, on the same 595 reused targets; simulated observations). A held input is a network state kept out of training whose next value must be predicted; there are four per target (2,380 in all). The original answer rule gives no answer when no rule in the language fits the observations. Active selection was right on 98% of the inputs it answered but left 764 unanswered, all from rules with three or more inputs, so it predicted fewer correctly than uniform selection (1,586 against 1,747). The completion vote, motivated by this overall outcome and frozen before the per-target diagnosis was decoded, answers those inputs by a vote of the rules with the fewest errors; active selection then predicts 2,200 correctly against 2,153 (+2.0 percentage points).

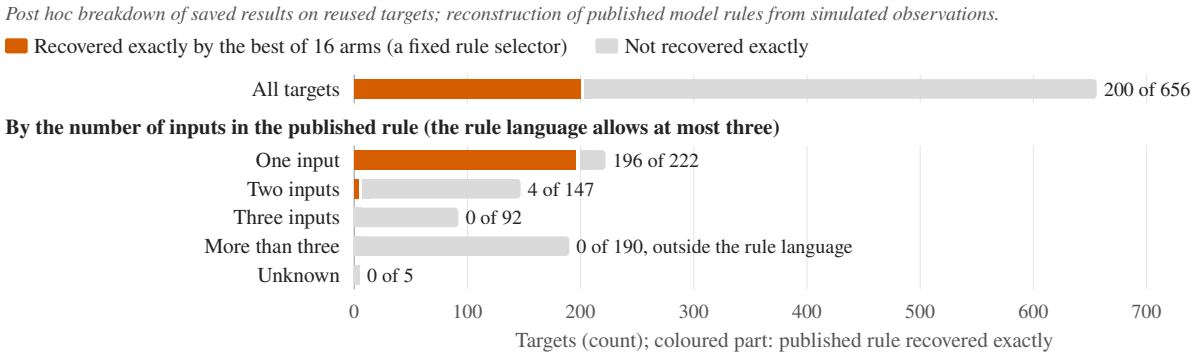


Figure 12: Where exact recovery stops: the number of inputs in the published rule (post hoc breakdown of saved results on 656 reused targets from 82 published models; reconstruction of published model rules from simulated observations). Bars group the targets by how many inputs their published rule has. The best of 16 arms, a fixed rule selector that answers every target (the fixed single choice with the neural guide of Appendix C), recovered 200 rules exactly: 196 of 222 one-input rules and 4 of 147 two-input rules. No arm recovered any of the 92 three-input rules, although the rule language allowed three inputs; the 190 rules with more than three inputs (up to 16) lie outside the language. The selector's other 456 answers were wrong rules (451) or could not be checked (5).

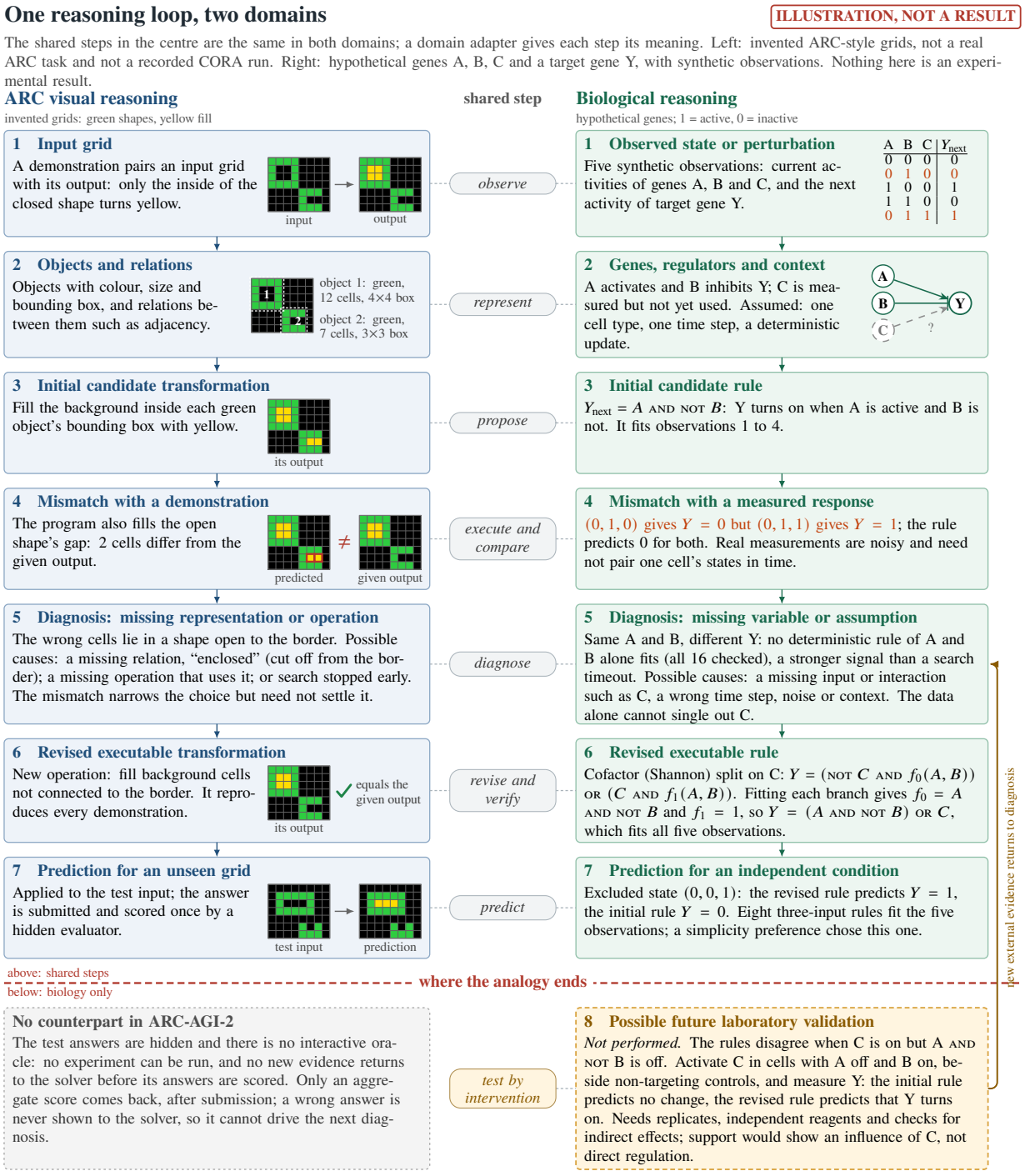
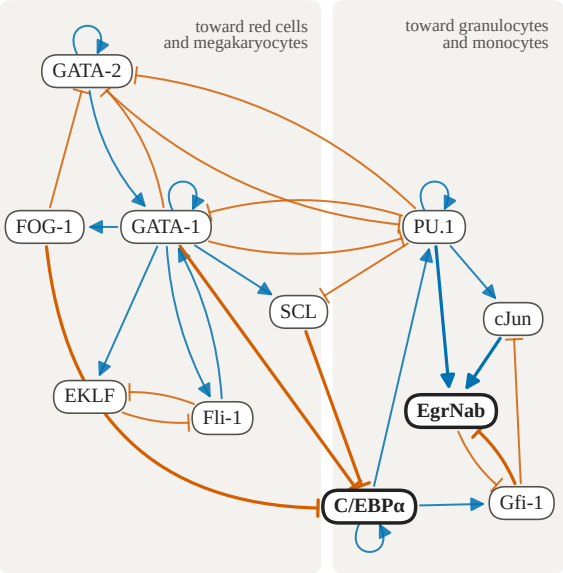


Figure 13: One reasoning loop, two domains (an illustration, not a result). Left: invented ARC-style grids, not a real ARC task and not a recorded CORA run. Right: hypothetical genes A, B and C and a target gene Y, with five synthetic observations. The shared steps are the same in both domains, and a domain adapter gives each step its meaning. The revised rule comes from a cofactor split on C, the decomposition of Boole [15] and Shannon [148], which is established mathematics, not a CORA invention; eight complete three-input functions fit the five observations, and a simplicity preference chose this one. Step 8 has not been performed, and ARC-AGI-2 has no counterpart to it: no new evidence returns to the solver before its answers are scored. The figure shows no ARC accuracy, biological discovery or laboratory result.

Published model; illustration of the reasoning steps on real model rules; not a CORA discovery.

a A published gene network for blood cell choice

Krumsiek et al. 2011: 11 transcription factors, one link per rule input



switches on switches off bold: rules in b and c

What the rules produce

Start: GATA-2, C/EBPα and PU.1 on, the authors' early progenitor state. Genes switch one at a time, each by its rule: 160 states can be reached, and every path ends in one of 4 stable states, which the authors named as cell types:

Erythrocyte	GATA-1	FOG-1	SCL	EKLF
Megakaryocyte	GATA-1	FOG-1	SCL	Fli-1
Monocyte	PU.1	C/EBPα	cJun	EgrNab
Granulocyte	PU.1	C/EBPα	Gfi-1	

Genes on in each stable state. Simulation on the published model.

b Reasoning steps on one published rule: EgrNab

published rule, as code: `egrnab_next = pu1 and cjun and not gfi1`

1 Observe › 2 Propose › 3 Verify › 4 Diagnose › 5 Revise

	1 inputs now PU.1	cJun	Gfi-1	EgrNab next	states seen	2 pu1 and cjun proposed		5 pu1 and cjun and not gfi1	
	0	0	0	0	50	0	✓	0	✓
	0	0	1	0	25	0	✓	0	✓
	0	1	0	0	50	0	✓	0	✓
	0	1	1	0	25	0	✓	0	✓
	1	0	0	0	2	0	✓	0	✓
	1	0	1	0	2	0	✓	0	✓
4	1	1	0	1	4	1	✓	1	✓
4	1	1	1	0	2	1	✗	0	✓
					160	3 7 of 8 fit		8 of 8 fit	

4 **Diagnose:** the two marked patterns share PU.1 and cJun but differ in EgrNab next, so no rule of those two inputs fits both; only Gfi-1 differs.

5 **Revise:** add Gfi-1. Of the 256 rules on three inputs, exactly one fits all 8 patterns, and it is the published rule.

Observations: the 160 reachable states of a; no two-input rule fits all 8 patterns. Reconstruction of a published rule, not a discovery.

c The same steps on the published record

Consistency check by simulation; not a biological discovery

	states	transitions	
1 reported in the article	232	789	
2 3 rules as listed, simulated	160	477	✗
5 one edit, simulated	232	789	✓

4 **Diagnose:** of 93 distinct single edits to one rule, only one gives both counts:

as listed `cebpa_next = cebpa and not (gata1 and fog1 and scl)`

one edit `cebpa_next = cebpa and not (gata1 or fog1 or scl)`

The article's rule tables need GATA-1, FOG-1 and SCL all on to switch C/EBPα off; its text also allows any one of them, the reading the reported counts fit.

Proposed experiment (not done): switch on GATA-1 in progenitor cells lacking FOG-1, then measure C/EBPα. As listed, C/EBPα stays on; with the edit, it switches off.

Figure 14: A published gene network and the reasoning steps on its real rules (published model; an illustration, not a CORA run and not a discovery). (a) The myeloid progenitor model of Krumsiek et al. [93]: 11 transcription factors, each with a published update rule (blue: switches a gene on; vermillion: switches it off). From the authors' early progenitor state, changing one gene at a time, the rules reach 160 states and four stable states, which the authors named as blood cell types (simulation on the published model). (b) For EgrNab, the best two-input rule fits 7 of 8 observed input patterns; two patterns that agree on PU.1 and cJun but differ in the output name the missing input, Gfi-1, and with it exactly one of 256 three-input rules fits, the published one (a reconstruction). (c) The rules as listed give 160 states and 477 transitions, not the 232 and 789 the article reports; of 93 distinct single edits, only one, to the C/EBPα rule, gives both (a consistency check by simulation; evidence, not proof), and a proposed experiment (not done) would separate the two readings.

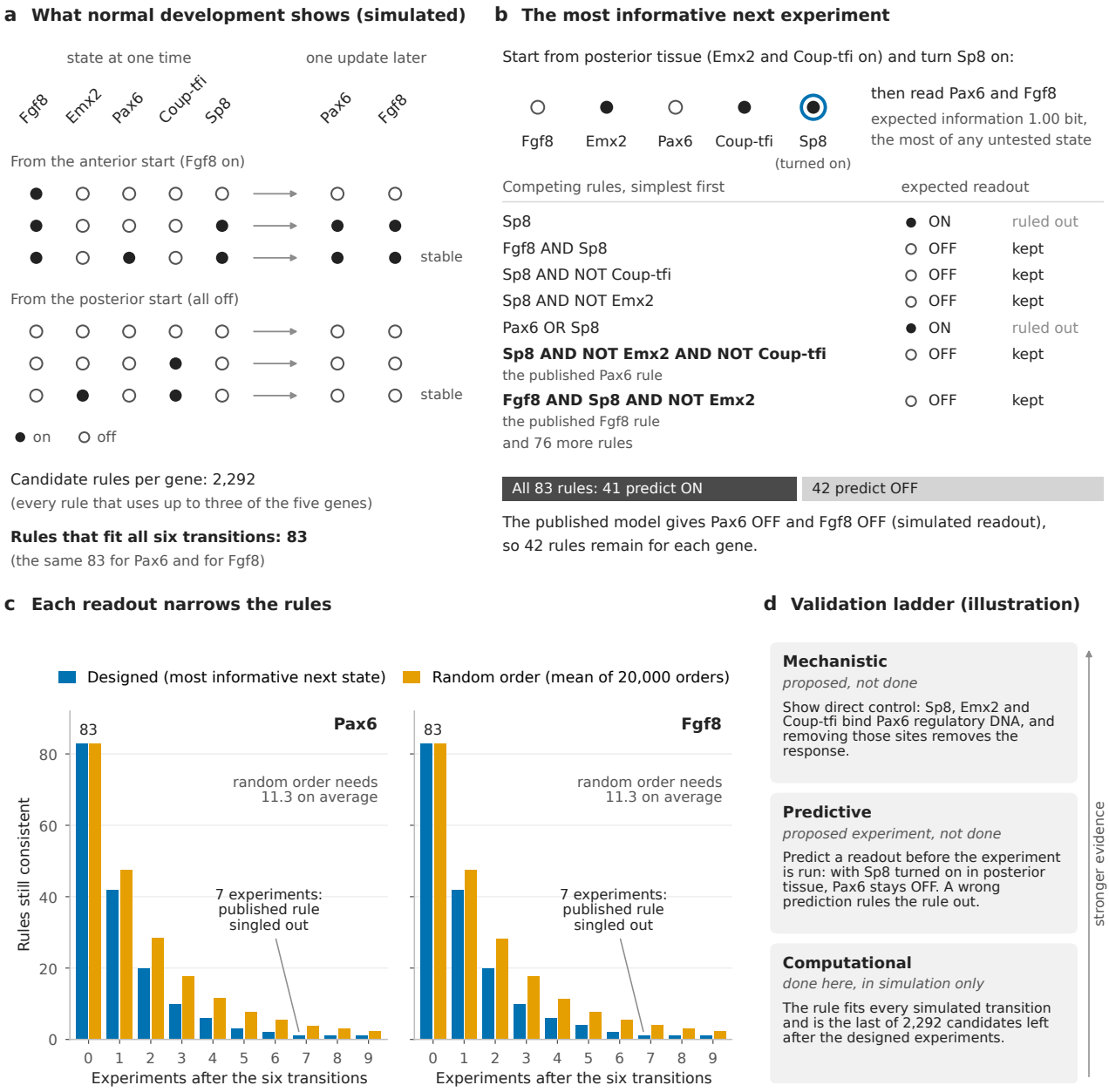


Figure 15: Designing the experiment that best separates competing rules, on a published gene network (simulation on a published model; not a laboratory result and not a discovery). (a) The six transitions that the five-gene cortex model of Giacomantonio and Goodhill [64] shows from its front (anterior) and back (posterior) starting states; filled circles are genes that are on. Of 2,292 rules that use up to three of the five genes, 83 fit them, the same 83 for Pax6 and for Fgf8. (b) The most informative next experiment, turning Sp8 on in posterior tissue, splits the 83 rules 41 to 42; the model gives OFF for both genes, leaving 42. (c) Rules still consistent after each experiment: choosing the most informative state each time singles out the published rule after 7 experiments, the fewest possible in the worst case, while random order needs 11.3 on average (20,000 orders). (d) What would turn such a rule into biological evidence (illustration; the predictive and mechanistic steps are proposed, not done).

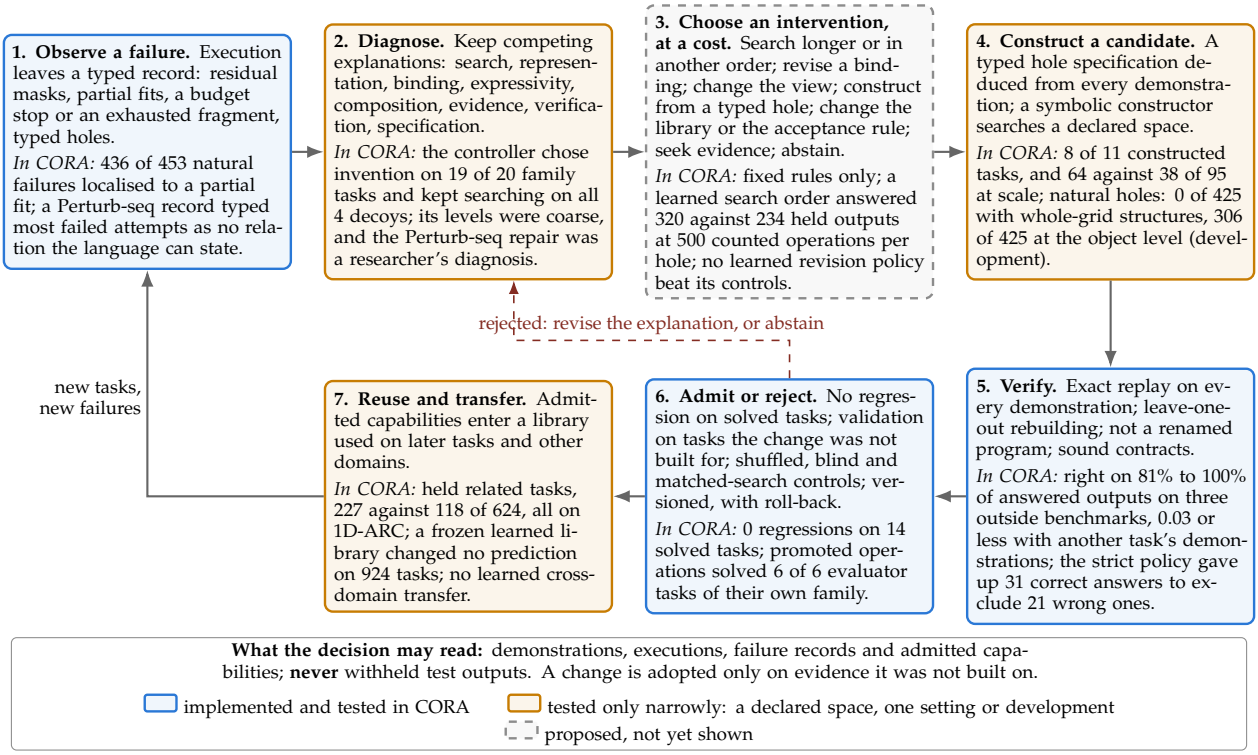


Figure 16: The evidence-grounded capability-acquisition cycle proposed in this paper, with each stage's status in CORA. A failure is observed and recorded, explained by competing hypotheses, answered by a costed intervention, turned into a candidate capability, verified on the demonstrations, admitted only on evidence the change was not built on, and reused. The counts are from the records of Sections 3 to 6: natural-task localisation and constructibility (Section 5.5.3), the self-extension controller (Section B.3), the learned search order (Section 5.6.1), the construction test (Section 5.5.2), verification on outside benchmarks and its controls (Section 5.2), promotion and reuse (Section 5.8). Stages 3 and 7 are where the paradigm's central conjecture lies (Section 4.6).

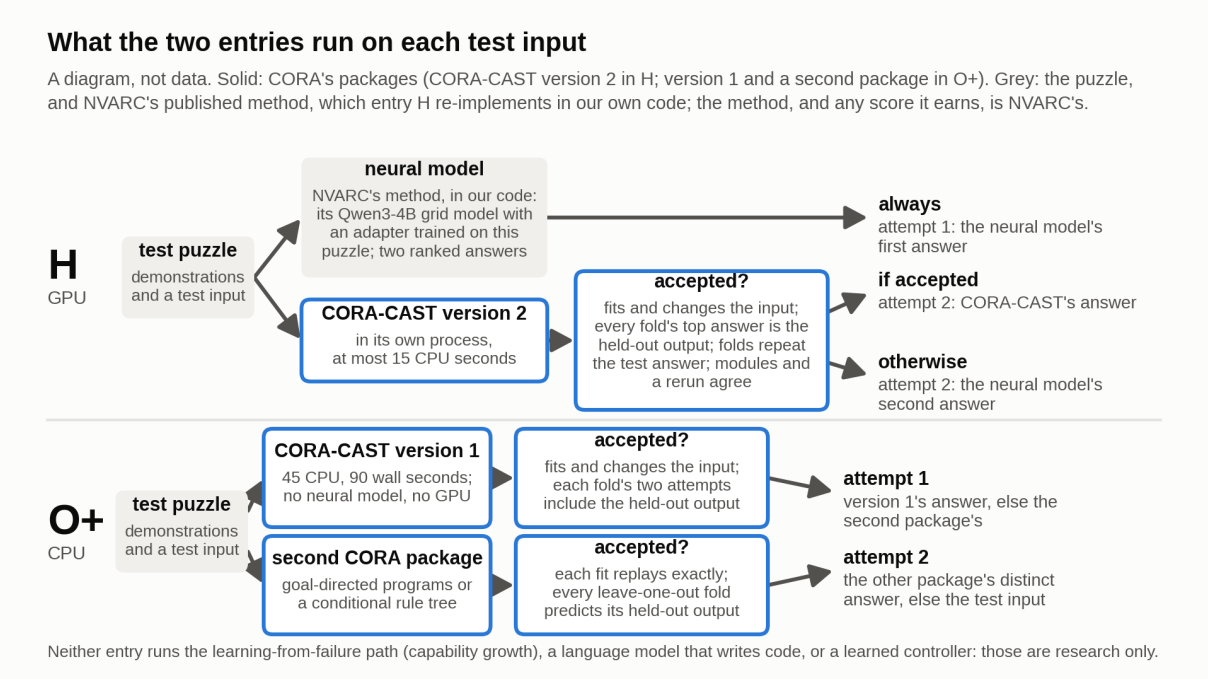


Figure 17: The architecture of the two linked entries on each query (a diagram, not data), drawn from the method-to-code map (`records/SUBMITTED_METHOD_TO_CODE_MAP.md`); its script is `paper/writeup/media/make_entries_diagram.py`. CORA-CAST's steps are Algorithm 2.

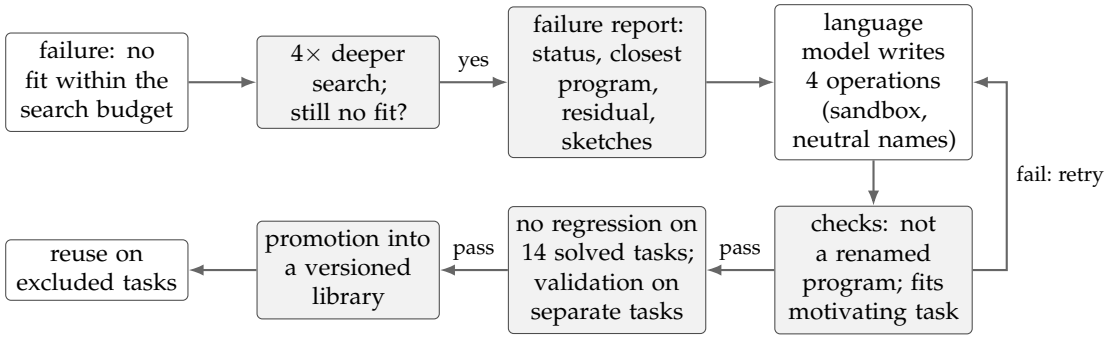


Figure 18: The self-extension cycle as implemented (`experiments/capability_growth_20261027/cgrow/`). Shaded boxes are CORA's own code; the language model only proposes. Every step ran as a real computation in the capability-recovery test; the cycle is a research experiment and is in no candidate notebook.

mechanism	public evaluation, 120 tasks: reachability and gap type	public evaluation: answered, correct, precision; CPU ceiling	training tasks: reachability and gap type, a diagnosis only, never accuracy
fixed CORA (CORA-CAST, then the second solver: the CORA-only entry)	reachability, for CORA's original engine: no demonstration-fitting program generated within its budget for any of 113 audited puzzles; gap type, reviewers' labels for 46 sampled failures: expressivity first in 29, the output canvas in 13, the object representation in 4	3 of 172 outputs answered, all 3 correct; 45 CPU seconds per puzzle for CORA-CAST and 12 for the second solver	one-step reachability in the architecture campaign: 1 construction-reachable and 482 unreachable of 572 training states
plus the learned library (the six-condition test on public evaluation; the separate support-fit probe on training tasks)	support reachability 4 of 120 tasks in every condition, 116 not fitted within the cap; gap types differ (no library: 107 bounded depth-two no-fits and 9 execution-budget limits; learned library: 73 and 43)	3 of 172 answered, all 3 correct, with grid-identical predictions in all six conditions (no library, learned, three random, full) at one shared per-task CPU ceiling: zero paired gain, labelled underpowered	the separate support-fit probe, not the six-condition test: no promoted operation fitted the demonstrations of any of 162 source tasks (0 of 162), as for the random and fixed libraries; a one-step program of the full 211-structure language fitted 1
plus the expressibility extension (identity geometry, then palette map)	no demonstration-fitting program for any of 120 tasks (0 of 358 support-only folds) in any of 13 language variants		18 tasks fitted against 17 for the base language and 17 for a random extension of the same size (999 eligible; development only); the pre-declared criterion failed
neural guesses turned into verified programs	the neural-guesses test, in CORA's earlier construction grammar: 0 of 172 frozen neural first guesses turned into a program that passes every demonstration, as with no target (0 of 172), every primary run stopped at the 150,000-execution and 400-closure bounds; earlier route: 0 of 1,002 backing searches admissible, the 21 correct top-3 candidates included	the neural first attempt alone right on 33 of 172; verified programs added none (effect not shown)	the separate eight-task typed-object probe: 0 of 8 fits, a development diagnosis with its own denominator
neural attempts explained by CORA-CAST (post hoc; Section 5.4)	2 of the neural comparison entry's 344 attempts explained, both right, none of the 301 wrong; on the 41 query outputs it gets right and CORA does not answer, no program at ten times the budget; first gap a missing action in 35 of 36 tasks (14 also a view), a missing guard in 1	kept only when explained: 1 of 172 first attempts kept, and it is right (unfiltered: 33 of 172 right); 15 CPU-seconds per module call, 150 at ten times	

Table 27: Results by dataset and mechanism. Each number is linked to a source, with outcome audits, preflight reviews and development analyses labelled separately; an empty cell has none yet. The public evaluation numbers are descriptive (inspected puzzles), and training tasks carry no accuracy. Sources: the reachability audit and failure anatomy (Section 5.3), the official rehearsal (Appendix A), the learned-library fit probe and the neural-guesses test (Section 5.8 and Appendix B.2) and the five architecture families (Table 31). Every row teaches the same lesson, that the missing programs lie outside what the language and its search reach, and opens the same direction, broader grammars with deeper search.

removed operation (starting language)	own report	failure-blind	shuffled	whole program	20× search
compress (no program)	0/6	0/6	0/6	0/6	0/6
downscale (no program)	2/6	0/6	0/6	0/6	0/6
trim (macro at cost 10)	0/4	0/4	0/4	0/4	0/4
upscale (none within cost 12)	4/6	2/6	0/6	2/6	0/6
total runs, of 22 (task by seed)	6	2	0	2	0
tasks recovered in any seed, of 11	4	1	0	1	0

Table 28: Motivating runs recovered (task by seed): a proposal lets the task fit and its test prediction is right. Seeds rarely changed an outcome, so the task is the unit of inference. Against the own report (4 tasks): failure-blind 1 and whole-program writing 1 (discordant 3 to 0, exact McNemar $p = 0.25$ each), shuffled reports and 20× search 0 (4 to 0, $p = 0.125$ each); no comparison reaches $p < 0.05$. The unit-level tests first reported are withdrawn (`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`).

condition (seeds 2 to 4)	own report	failure-blind	no sketches	shuffled	matched search
motivating runs recovered, of 33	5	3	6	0	15
motivating tasks recovered, of 11	2	2	n/r	0	5
runs with a promoted operation	2	2	4	0	n/a

Table 29: The pre-registered follow-up of the self-extension test. Primary comparison at task level: own report against failure-blind, 2 against 2 tasks (1 to 1, $p = 1.0$), inconclusive; n/r, not recomputed at task level. The matched search is deterministic, so its 5 recovered tasks count once per seed. Budget 2.03 GPU hours and 2.81 CPU hours.

pilot	task set	result	caveat (from the review)
learned failure controller (one typed revision chosen from failure features)	generated: 64 held tasks, 128 queries; a 64-task graph domain	33 of 128 against static search's 87 (failure-blind 86; half-ceiling static 49); graph 14 against 72	it could only prune relative to the static condition, so the loss shows that hard pruning hurt, not that failure evidence hurt
learned hierarchy ranking a fixed 24-program bank	generated: 64 held tasks	32 of 64 executable against a flat ranker's 10 and static 64	one seed; shorter pooled sequences are a confound
frozen grid programs run on attributed graphs	authored: 8 task clusters of 2 queries, from 4 engineered Boolean families	14 of 16 queries and 7 of 8 tasks, against 16 of 16 and 8 of 8 (direct graph learning and static)	reuse, no benefit; not 16 independent units
typed programs with anti-unification and repair coverage primitives; conditional programs	authored: 4 tasks per stage public training, 32 pseudo-held queries each	8 of 8 in every condition (validation), 0 of 8 (confirmation); repair tied by failure-blind 3 against 1 of 32 (+2/−0); 3 against 1 at about twice the compute	checks of mechanism, not of benefit training-set diagnostics only

Table 30: The controlled pilots of the parallel series (development evidence). Our own campaigns agree: a learned version-space controller helped only on corrupted states, a calibrated revision filter harmed no answer but gained none, a learned view order (5 against 2 of 176 source puzzles) helped by re-ranking, and matched search beat the neural inventor (Appendix B.3).

architecture	strongest baseline	result	evaluation
learned search control over a persistent version space (a set Transformer)	plain enumerator	replay gains only on corrupted variants (natural states 21 of 21 unchanged); pooled features match it	0 of 172 outputs
failure-conditioned grammar induction, with a library-compression condition	plain language, no library	0 gains in every condition, task set and version; learned definitions compress 21 to 26 of 63 held-out programs, but only from a median of 28 nodes to 27, and programs within reach stay at 2	0 of 120
joint representation and program learning	learned view order (5 of 176)	4 of 176 (+4/−2 against fixed views, $p = 0.69$); its one representation-change solve saves search cost and adds no expressivity	0 of 120
calibrated revision (a value network deciding whether to revise)	linear model	replay 90 to 91 against the linear model's 91, with 0 harmed answers and 64 to 98% fewer executions end to end	0 of 172
bidirectional neural and program inference (programs backing the grid model's answers)	neural voting, 17 of 172	0 of 1,002 backing and 0 of 344 construction searches admissible, including the 21 correct top-3 candidates	17 for every condition

Table 31: Five architecture families, first screens (`experiments/architecture_sweep_20261026/ALL_FIVE_RESULTS.md`, `records/A5_AND_LOOP_STEP1_RESULTS_20261027.md`). None advanced, so the sealed partition stayed closed until the CORA-CAST confirmation. The common cause is reachability (Section 5.3).

condition (CORA's language)	candidates	type-check	fits	correct outputs	tokens, GPU s
no feedback	160	62%	0	0 of 32	39k, 974
diagnosis	1,836	91%	0	0 of 32	231k, 9,507
generic feedback	2,068	95%	0	0 of 32	231k, 9,265
equal compute	1,854	64%	0	0 of 32	357k, 9,357

Table 32: The diagnosis-conditioned writer's smoke test on the first 20 public evaluation puzzles (development evidence; `records/NEURAL_CORA_NEURAL_PROGRAM_WRITER_RESULTS_20261025.md`). A bounded grid-function language gave the same result.

bound	entry	four times	ten times
CPU-seconds per task	45	180	450
typed-search executions	1,000	4,000	10,000
typed-search fitting programs kept	32	128	320
rewriting rule lists kept (move rule lists)	64 (16)	256 (64)	640 (160)
raised only in the wider check (bounds the code treats as part of the language):			
actions considered per view	8	32	80
objects, summed over the demonstration inputs	120	480	1,200
size values offered as guards	12	48	120

Table 33: Bounds of the gap reruns of the shipped three modules. The explanation search used CORA-CAST's five modules at 15 CPU-seconds per module call, then ten times every execution budget, kept-list budget and CPU cap, with the three language bounds fixed. The neural entry's CORA second attempt had 15 CPU-seconds per task. Post hoc, exposed public evaluation, not an official score.

bottleneck	counts, with their unit	supported conclusion	closed study
binding collisions	68 of 88 historical positive records alias negative outcomes under 53 features (45 of 65 base, 23 of 23 continuation; 9,490 rows); records are not tasks	distinct bindings were lost in the neural features; no accuracy gain	a source provenance audit
validation starvation	260 candidate programs, a validation allowance of 24; the intended forms ranked 47 to 260; all 11 conditions 0 of 8 synthetic cases	the right structure lay beyond the validated prefix; no inexpressibility conclusion	a causal pilot
finite atom truncation	78 of 520 task banks lost 1,204 legal atom occurrences (679 recolour, 213 pack, 78 fill, 234 symmetry); full banks of at most 121 against a cap of 96	the omission is real; a later full-bank intervention on the public 120 still gave 0 of 172 candidates	an audit of 520 retained source banks
no retained relational witnesses	both paired conditions 0 of 120 retained fits and 0 of 172 query candidates (137,281 edges, 7,540 contexts, 13,103 validations; 0 new-operator replays)	more construction produced no verified program; a resource-bounded unknown is not proof of absent semantics	a 14-family relational screen
hard routing	grid 33 of 128 queries against static 87 and blind 86; graph 14 of 128 against static 72 (64 tasks per domain, 2 queries each); rebinding 32 of 64 and 53 of 64; 0 selected learned-macro expansions	lower loss did not improve execution; learned routing suppressed useful branches	a learned-routing ablation
demonstration fit, wrong query	7 retained verified candidates, 0 of 7 correct (0 of 172); the neural comparison entry 43 to 42 (0 gained, 1 lost); the CORA-only entry 3 to 3	a fixed, consistent action prior did not transfer; not inserted into the neural comparison entry	a single-action policy scored once on the public 120, `experiments/independent_scoring_20261102/single_action_v1.score.json`
residual repair copies successes	raw neural 162 of 172 answered, 8 correct; 10 combined candidates, 3 correct, 2 changes, 0 corrections; hybrid 8 of 172	candidate hits duplicate correct neural answers; joint construction is a separate, unscored hypothesis	a neural-to-CORA residual study, `experiments/independent_scoring_20261102/neural_repair_v1.score.json`

Table 34: The companion study's construction-bottleneck table (companion experiments; aggregate rows only; the companion study's records, available from the authors on request). The two paths in the last column are in this report's repository.

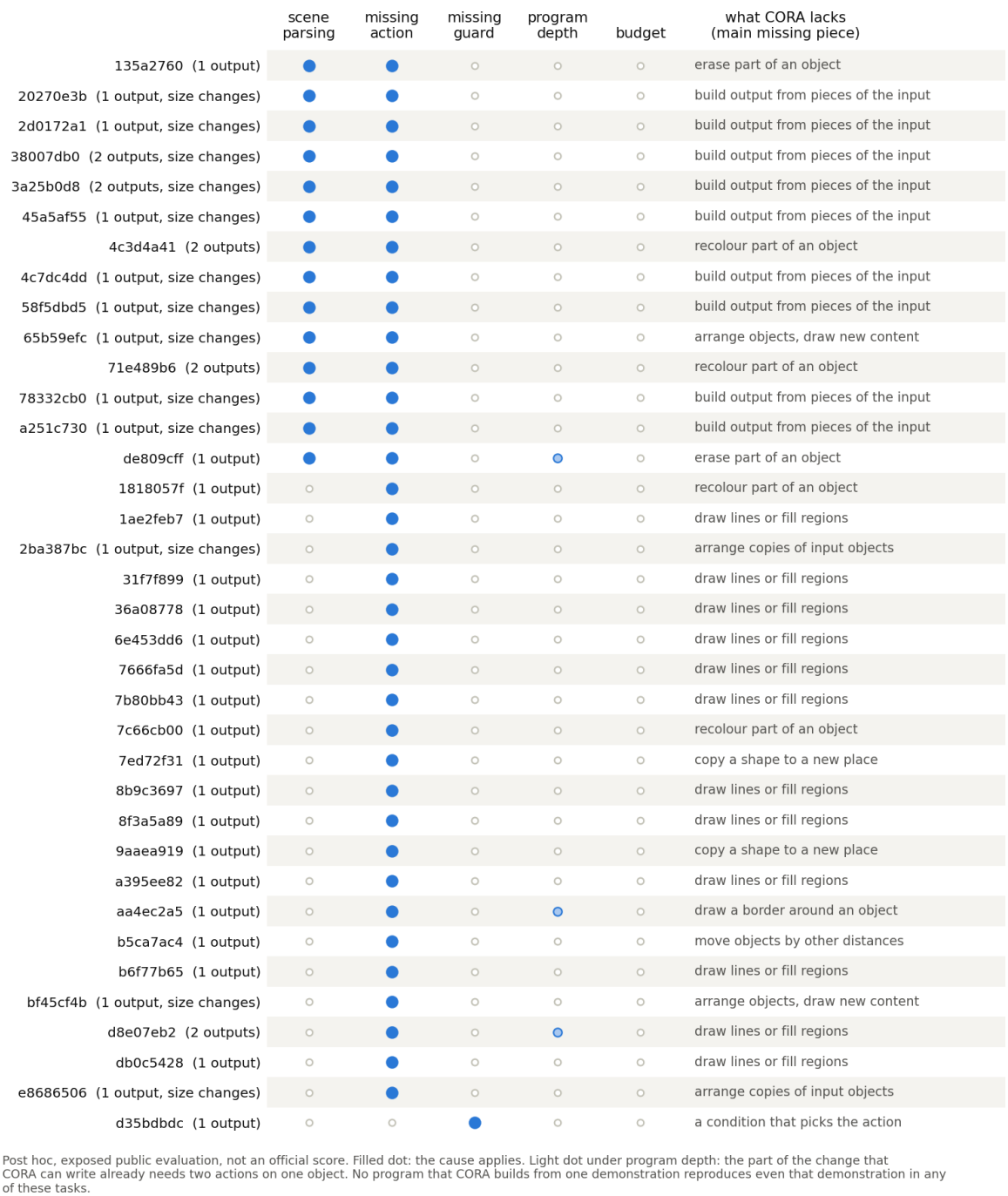


Figure 19: The 36 tasks of the gap, by cause. Each row is a public evaluation task on which the neural comparison entry was right and CORA did not answer, with the number of such query outputs. Filled dot: the cause applies; light dot under program depth: the part of the change that CORA can already write needs two actions on one object. The right-hand column is the diagnostics' main missing piece in plain words; for aa4ec2a5 the diagnostic records new cells that border an object, and on 31f7f899 erased object cells and new line or fill cells tie at 16 cells each, so the label shown is the first of the two in alphabetical order. In none of these tasks does a program that CORA builds from one demonstration reproduce even that demonstration. The diagnostic rules were refined while being run on these tasks. Post hoc, exposed public evaluation, not an official score.

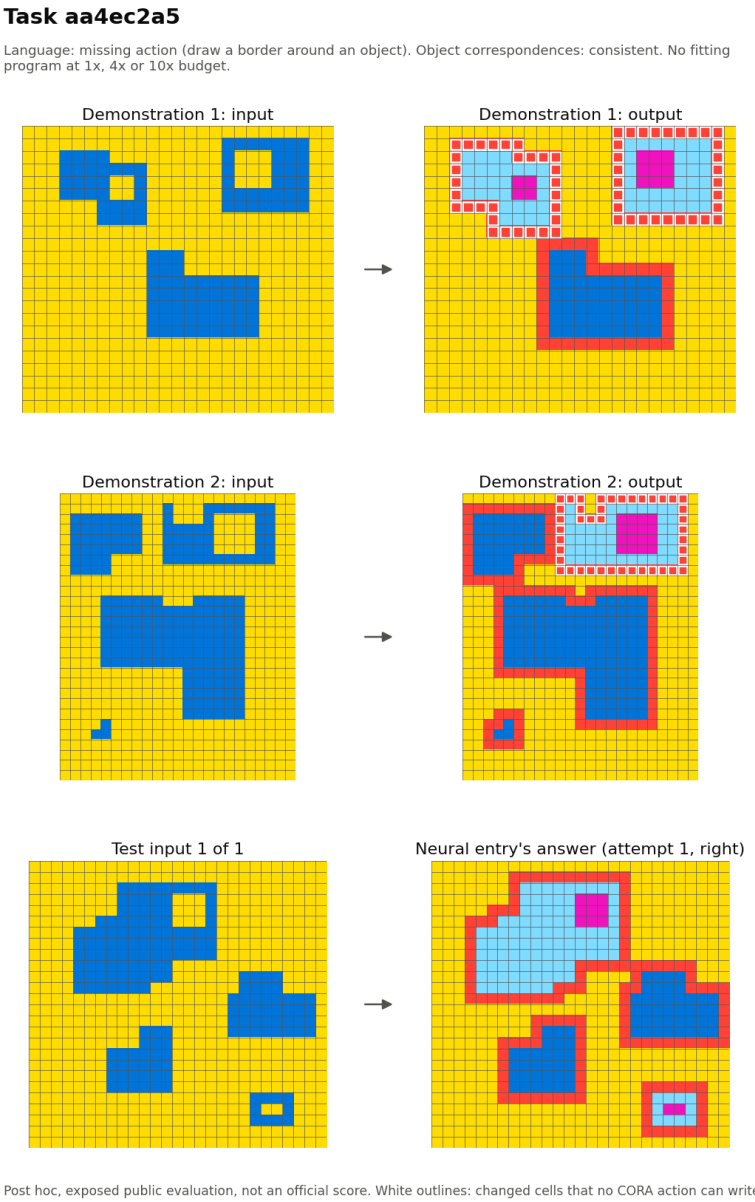


Figure 20: A near miss. Task aa4ec2a5 (the neural first attempt is right). Every blue shape gets a red border; shapes with holes also turn light blue and their holes turn magenta. White outlines: 128 of the 449 changed demonstration cells, all of them new cells bordering an object, which no CORA action writes. On inspection, CORA's outline action writes every background cell next to an object, holes included, so it cannot draw the border of a shape with holes, and such a shape also needs two actions at once (recolour and fill its holes), which one action per object does not allow. No CORA-CAST program reproduces the demonstrations at the entry's budget or in the wider check. Post hoc, exposed public evaluation, not an official score. Puzzle data: ARC Prize Foundation.

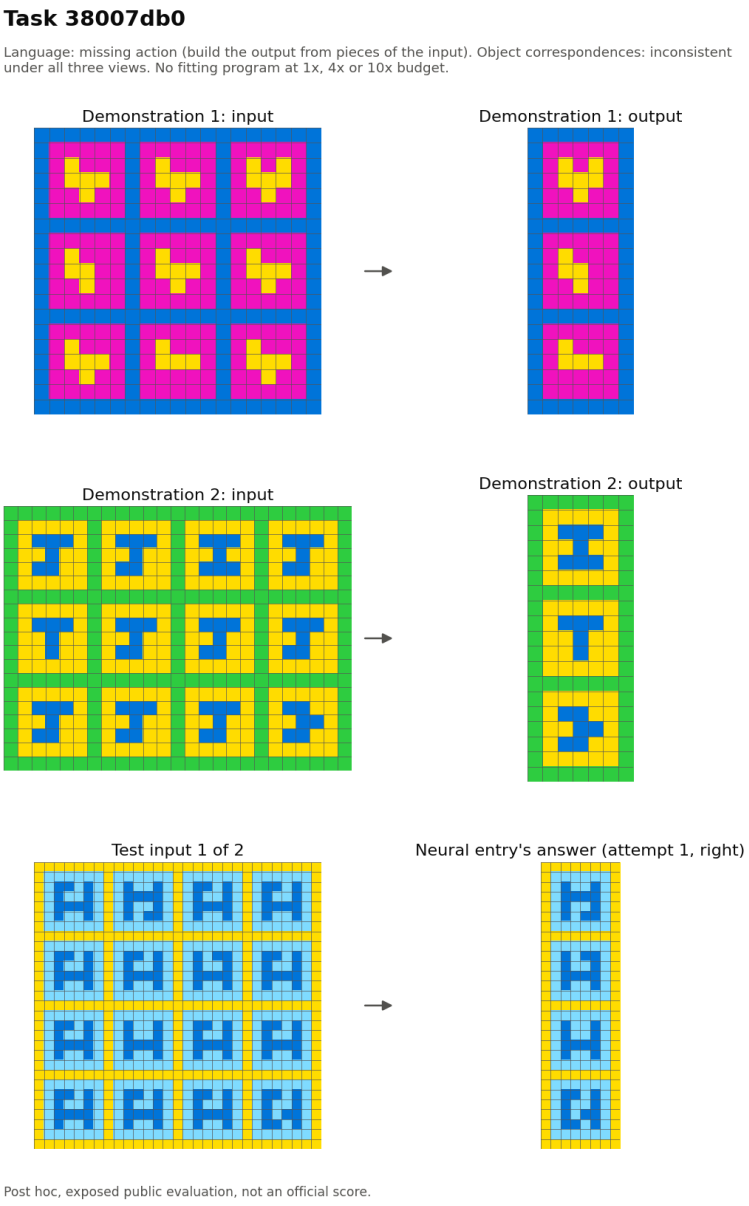


Figure 21: Building a new grid. Task 38007db0 (both of its query outputs right on the neural first attempt). The diagnostics find that the output is built, not cut out of the input, and that no object view gives consistent correspondences: the output's pieces are not objects under any view. On inspection, separator lines cut the grid into panels; in each row of panels one panel differs from the others, and the output is the column of those odd panels, so a panel view, a guard that compares a panel with its row and an action that assembles the chosen panels would all be needed. Post hoc, exposed public evaluation, not an official score. Puzzle data: ARC Prize Foundation.

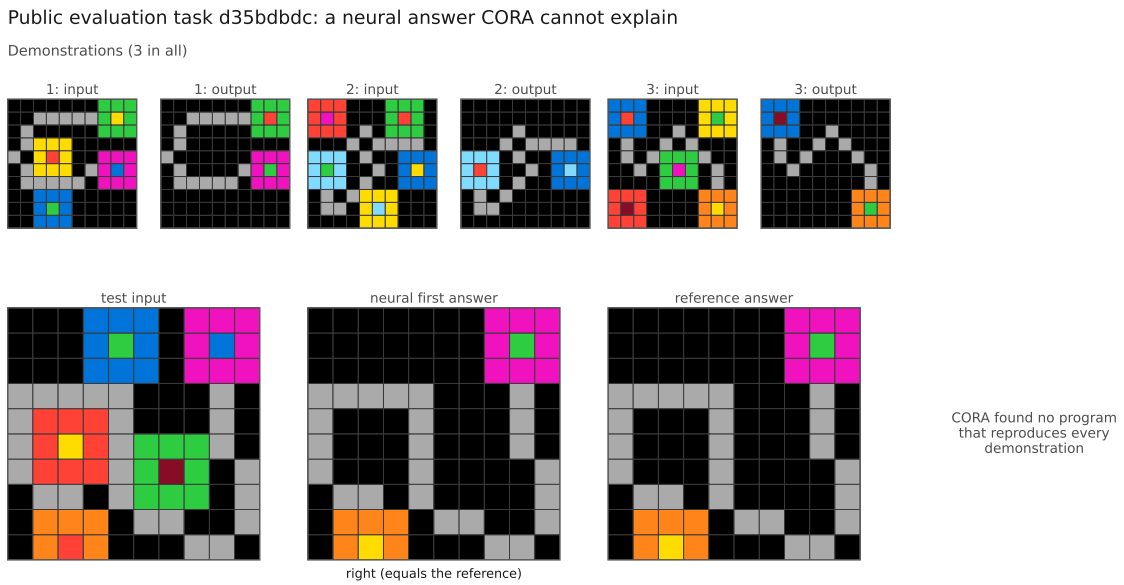


Figure 22: A missing guard. Task d35bdbdc, third query output (the neural first attempt is right). Every change can be written by CORA's actions, but no rule list over CORA's guards assigns them, and no program in CORA-CAST's five modules reproduces the three demonstrations, at the starting budget or at ten times it; the one search that had stopped at its execution cap, typed search without a target, finished without a fit when rerun with 10,000 executions. On inspection, which box's centre colour names which other box's frame colour decides the output, while CORA's guards test an object's own features and the colours touching or enclosing it. Post hoc, exposed public evaluation, not an official score. Puzzle data: ARC Prize Foundation.

result	label	rung	outcome
teaching case; bounded diagnosis loop (219 of 256 against 38); query-policy revision; sparse engine	synthetic demonstration	1	reached, on authored functions; no advantage over equally expressive fixed methods (a fixed build-then-ask schedule also 219; a fixed expressive query policy recovered all 256; the sparse engine equals a classical answer table by design)
inferred regulators (99 against 85 and 87 of 333; 101 against 66 of 595)	published-model reconstruction	1, reference-checked	reached
informative observations (320 against 291); completion vote (2,200 against 2,153 of 2,380, post hoc)	published-model reconstruction, reused cohort	1, reference-checked	reached; held inputs are simulated states
conditional construction (433 against 321; classical learner 433)	published-model reconstruction, reused cohort	1, reference-checked	reached; no advantage over the classical learner
Perturb-seq construction and relational follow-up	prediction on real measurements	2, attempted	no rule admitted for any of 64 genes, twice; directional F1 zero in all 20 compared configurations
microscopy (three held-out datasets; follow-up; calibrated acceptance on PUMA)	prediction on real measurements	2, attempted	0 edits; the follow-up failed technically and gives no result; the calibrated rule harmed (430 of 605 edits)
spatial transcriptomics, one exposed donor	prediction on real measurements	2, attempted	original rule 58 corrections against 65 harms; follow-up primary failed its criterion; neighbour-only gate 36 against 12, exploratory
single-cell lung atlas, 41 independent donors	prediction on real measurements	2, attempted	the predicted direction held, but so did every control's, so the design gets no credit
drug response, 416 held compounds	prediction on real measurements	2, attempted	0.5249 against 0.5295 for try-all
any CORA rule tested by a designed intervention		3	none
any CORA rule with mechanistic support		4	none

Table 35: Where each biological result sits on the validation ladder.

Table 36: The three continuous-data follow-ups (companion study; each frozen, sealed and run once on 8 November 2026; audited by us from saved aggregates and cross-checked by the companion). None ran in either Kaggle entry.

domain (units)	design and controls	outcome	audit
Perturb-seq (255 held source-validation perturbations, one cell line, reused cohort)	scalar or pair relation × exact fit or likelihood fold vote; label and pair shuffles, exhaustive static, ridge, mean, constant; 326,400 sealed predictions	0 of 64 rules admitted in all 16 program configurations; directional macro-F1 0 in all 20; 32 of 16,320 responses directional	audited null; an event floor is a testable hypothesis
microscopy (78 held parents)	original or expanded object features × unanimity or parent-level bar; no edit, unguarded, failure-blind, shuffled; 624 sealed predictions	failed during an attempted MoNuSAC reference decode (out of memory), after the LIVECell and NuInsSeg references were processed; no qualified endpoint	technical failure, not a null
spatial (1 donor, 14,234 spots)	neighbour and marker-rank guards, readout fixed; graph-shuffled combined gate (made no change by construction, so it does not test neighbour information); 85,404 sealed predictions	primary 23 corrections, 9 harms, one section worse (gate failed); neighbour-only 36 and 12, all sections up, macro-F1 0.93500 against 0.93155	exploratory, one donor

condition, at equal budget	watershed start	components start
fixed start, unrevised	0.8827	0.7880
CORA's diagnostic order	**0.9230**	**0.8478**
generic fixed order	0.9120	0.8013
failure-blind, five seeds	0.9077 to 0.9234	0.8081 to 0.9209
shuffled diagnosis, five seeds	0.9092 to 0.9120	0.8013
try-all, equal budget	0.8885	0.8827
try-all, double budget	0.9091	0.8916

Table 37: Pooled object F1 (IoU above 0.5) on the 50 official test images.

E Related work in detail

Table 38 compares the closest lines of work, row by row: what each uses a failure for, whether its language is fixed or grown, how it checks answers, and where CORA differs. CORA's construction step, as audited: once a bounded search fails, typed holes are computed from the demonstrations; an operation is selected for them from a supplied space (211 whole-grid structures in the generated-task test) and admitted only if a bounded search in the starting language does not reproduce it, it fits its task, it breaks no solved task and it fits another unsolved task; admitted operations are reused. CORA-CAST answers only when its program fits every demonstration and, rebuilt without each one, still ranks that demonstration's output among its top two (the CORA-only entry's rule).

Table 38: How earlier systems use a failure, whether their language grows, how they check, and where CORA differs.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
Learning from failed programs: Popper [46] and its predicate-invention version [47]; conflict-driven synthesis, Neo [59]	Pruning. Popper: a program that is too specific (misses a positive example) or too general (covers a negative one) becomes constraints that prune its specialisations or generalisations. Neo: the cause of a failed partial program becomes a learned constraint that prunes others failing for the same reason.	Fixed: background predicates and syntactic bounds (Popper), a given domain-specific language (Neo). The predicate-invention version adds predicates while it learns one task.	Popper: entails every positive and no negative example. Neo: satisfies the input and output examples.	Popper and Neo read each failure; on ARC-style tasks CORA does not: a failed search only triggers construction, and the hole comes from the demonstrations. CORA adds an operation instead of pruning, and keeps it across tasks only after checks on other tasks. Our Popper run (5 previously examined training puzzles, unequal budgets) is uninformative and is not a result.
Bias shift and predicate invention: Utgoff's bias shift [160]; inverse resolution [116]; predicate invention as a bias shift [157]	Trigger and growth. Utgoff: when no description in the current language is consistent with the examples, a new term is added to the language. Inverse resolution invents predicates that were not in the vocabulary, and a human names them. Stahl: for which language biases inventing predicates is a useful shift of bias.	Grown: the concept-description language (Utgoff); the predicate vocabulary (inverse resolution)	consistency with the examples	The closest precedents for growing a language when it fails. CORA's addition is the measurement: pre-registered controls that keep the growth machinery and replace only what the task or the failure supplies (another task's holes, 16 against 64 of 95 generated tasks; a shuffled failure description, 12 against 45 of 217 held logic functions, reconstructions of known functions), and admission checks on other tasks whose compute is charged.
Growth where analysis fails: Berwick's grammar acquirer [14]; missing grammar rules from failed parses [48]; chunking in Soar [95, 96]; DeepCoder's sort-and-add search [11]	Trigger, and in one case location. Berwick: when no known rule applies, build one new rule that does, and save it. Cussens and Pulman: infer by abduction the edges a failed parse lacked, build rules that would supply them, then generalise. Soar: an impasse (too little knowledge to select or apply an operator) starts reasoning in a subgoal, and chunking compiles that reasoning into rules that avoid the impasse later. DeepCoder: whenever search fails, add the next most probable existing function to the active set and restart.	Grown: grammar rules (Berwick; Cussens and Pulman) or production rules (Soar). DeepCoder: a wider active subset of a fixed language.	Berwick: the new rule must apply where no known rule did. Cussens and Pulman: candidate rules are evaluated on the data, ordered by coverage, and added only when the user commits. Soar: not assessed. DeepCoder: the program must fit every example.	The closest precedents for a failure that triggers a new, kept piece. Like Berwick's acquirer, CORA builds one piece when its language fails; unlike Cussens and Pulman, it does not use the failed attempt to locate the gap. Its additions are the checks before keeping (new, fits, breaks no solved task, fits another unsolved task) and a control that takes the holes from another task's demonstrations (16 against 64 of 95 generated tasks). No control isolates the trigger itself.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
Sketching with holes: Sketch [153, 154]; type-and-example-directed synthesis, Myth [130]	Sketch: a sketch with no valid completion is reported as buggy; counterexamples refine its inputs (row 6). Myth: a guessed term that disagrees with an example is discarded.	Fixed: the programmer writes the sketch, and holes stand for constants or programmer-defined kinds of expression (Sketch); a typed functional language (Myth).	Sketch: equal to a reference program on every input, checked with satisfiability solvers over finite programs. Myth: consistent with every example.	The same idea: a hole with a specification, which Myth also derives by pushing examples into holes. CORA's sketches are three fixed shapes (the hole alone, or one invertible operation applied after or before it), not written by a programmer; the hole is filled from a supplied space of 211 structures, and the filler becomes a reusable operation. With a few demonstrations and no reference program, CORA's checks show consistency, not correctness.
Deduced hole specifications: witness functions (given what an operation must output, they work out what its arguments must be) in FlashMeta [137], which re-implements FlashFill [67]; λ^2 [60]; inverse semantics on ARC [3]	Pruning: a sub-specification that no program meets yields an empty set, and that branch is dropped (FlashMeta); deduction refutes hypotheses that cannot be completed (λ^2). FlashFill also asks the user for outputs on inputs that may have several interpretations.	Fixed: a domain-specific language whose designer supplies the witness functions (FlashMeta); a functional language with combinators such as map and fold (λ^2).	Every kept program fits all examples by construction; a ranking picks one (FlashMeta, FlashFill); λ^2 returns the simplest program that fits.	The closest precedent for the hole: undoing one invertible operation is a simple witness function. The one-operation sketches beat the demonstrations alone (64 against 48 of 95 generated tasks; 16 gains, no losses; all 7 of the 8 task groups that differ favour them), which measures this known idea inside CORA. The difference: where FlashMeta drops a branch no program meets, CORA selects an operation for it from a supplied space and, if admitted, keeps it for later tasks.
Library learning: DreamCoder [57]; LILO [66]; Stitch [16]	None: unsolved tasks are retried in later cycles; new routines are compressed from programs found for solved tasks.	Grown: a library of routines (LILO adds names and documentation written by a language model).	DreamCoder: a list program must predict every observed output; routines are chosen by a Bayesian objective. Stitch: abstractions chosen by how much they compress a corpus of programs.	Both grow a language, and both change what a bounded search reaches. DreamCoder: on drawing and tower-building tasks, neither ablation (no library learning, or no recognition network, the neural guide of its search) solved more than 60% of held-out tasks, against nearly 100% for the full system. CORA: trying its 211 structures in random order without holes found 74 of the 95 tasks in at least one of three repeats but 3 in all three, against 64 in all three for the holes. CORA builds from unsolved tasks' holes and admits after checks on other tasks; with no matched comparison against these systems, no advantage over them is claimed, and the same-size comparison of Section 5.8 is not a general method comparison. Our library of whole programs found by search, with their sub-programs, beat the starting language at the base budget (37 against 6 of 624 held query outputs, every held task scored once before by the submitted entries) but lost to static search given its whole cost (37 against 56). That tests our library, not DreamCoder's; a test that also matches construction work, which the same-size comparison of Section 5.8 did not, is still needed.
Counterexample-guided loops: inductive synthesis, CEGIS [153, 154]; abstraction refinement, CEGAR [43]	Refinement. CEGIS: each counterexample input from the verifier joins the synthesizer's inputs. CEGAR: a spurious counterexample (allowed by the abstract model, impossible in the real system) splits the last abstract state of its shortest prefix that no real run matches.	CEGIS: a fixed space of hole values. CEGAR: the abstraction grows finer.	CEGIS: a verifier with a complete specification, on bounded inputs. CEGAR: model checking; each refinement is guaranteed to eliminate the spurious counterexample.	CEGAR is the closest analogue to a failure changing the representation, and it uses the failure more: the counterexample places and justifies the change. CORA's holes come from fixed shapes over the demonstrations, and admission is empirical. ARC supplies only the demonstrations, with no verifier that can return new counterexample inputs. Direction: let the failed search place the hole, and test that against holes from the demonstrations alone.
Program repair from failing tests: GenProg [97]; SemFix [124]; Angelix [110]	Localisation and repair: GenProg weights statements run by the failing test and mutates them with code copied from elsewhere in the program; SemFix and Angelix derive from the tests what a suspicious expression must compute, then synthesize a replacement.	Fixed: GenProg does not invent new code; SemFix searches repair expressions layered by complexity.	Pass the test suite; patches that pass the tests can still overfit them [152].	SemFix is close: a located hole, a derived specification, a filler from a fixed space. Repair locates the fault from the failing run and patches one program; CORA's hole ignores the failing run, and its filler becomes a language operation, checked on other tasks and reused. Whether CORA's admission checks prevent overfitting to one task's demonstrations was not tested separately: no reported control removes them.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
Self-repair by language models: Self-Debugging [25]; a self-repair study [128]	Repair: the model reads execution results, or explains its own code, then rewrites it.	Open-ended code; nothing is kept for later tasks.	Execution on available tests; once repair's cost is counted, gains are often modest.	CORA's change comes from a supplied space, persists and must pass checks on other tasks; no language model runs in the scored entry. Our result: a 4-billion-parameter language model writing the missing operation from CORA's failure report recovered 2 of 11 constructed tasks, as many as without the report, while static search in the starting language at matched compute recovered 5; given the operation's exact input and output behaviour, it recovered 1 of 11. Lesson: the binding limit was the model's ability to write a grid operation. Direction: hand the hole's specification to a symbolic constructor, which on the same 11 tasks recovered 8 (another task's specification 0; matched static search 5, $p = 0.375$; the same space tried in random order 10 in at least one repeat, at about five times the processor time).
ARC program search, test-time training and held-out demonstrations: the 2020 Kaggle winner [169]; test-time training [2]; NVARC, the 2025 Kaggle winner [155], built on the ARChitects' method [62], who were second in 2025 with a masked-diffusion model (one that fills in a masked answer over several passes) that refines its answers [87]; held-out demonstrations on ARC [76]; jackknife+ [12]	None. The 2020 winner keeps the programs that solve the most demonstrations, then the shallowest. Test-time training trains on leave-one-out tasks built from the demonstrations. Joffe and Eliasmith hold out each demonstration in turn to choose which object properties a rule reads.	2020 winner: fixed and hand-written (up to 4 of 142 grid transformations built from 42 functions). Test-time training: no program; a network's weights are adapted to each task (NVARC adds synthetic training puzzles).	Fit to the demonstrations, then ranking (2020 winner); votes across invertible transformations of the task (test-time training) or the model's own output probabilities across augmented views (the ARChitects' method); jackknife+ gives intervals with a coverage guarantee when the data are exchangeable (their order carries no information).	CORA-CAST is in the program-search line (typed search, exact fit) and abstains when a program fails the rebuild check. Holding out one demonstration at a time is shared with test-time training and with Joffe and Eliasmith, but CORA-CAST uses it to decide whether to answer; unlike jackknife+ it gives no coverage or precision guarantee, and its effect apart from the other acceptance checks was not isolated. Our neural comparison entry, which re-implements NVARC's method, scored 24.03% against 0.83% for the CORA-only entry.
ARC Prize 2025 paper awards: Tiny Recursive Model (TRM), first [77]; SOAR (Self-improving Operators for Automated program Refinements), second [138]; CompressARC, third [103]	TRM: no explicit failure step; a 7-million-parameter network refines a latent state (an internal vector) and its answer over recursive steps, supervised at each of up to 16 steps. SOAR: repair and learning; a language model refines programs using their execution on the demonstrations, and its attempts are relabelled as correct programs for the tasks they do solve, then used for fine-tuning. CompressARC: no explicit failure step; a network is trained on the one puzzle to minimise its description length (the bits needed to encode the puzzle).	TRM and CompressARC: no program, a network (TRM with an embedding for each puzzle and augmentation; CompressARC with 76,000 parameters and no pretraining). SOAR: Python, with model weights updated between rounds.	TRM: majority vote over 1,000 augmentations (transformed copies of the puzzle). SOAR: programs run on the demonstrations; each output grid scored by the summed accuracy of the programs that produce it. CompressARC: the most frequently sampled answer.	SOAR is the closest ARC work that learns from failed programs, but what it learns lives in weights; CORA's change is one explicit operation that must pass checks and can be inspected and reused. TRM refines inside a fixed network and always gives an answer; CORA-CAST abstains when its check fails.

F The ARC Prize Paper Track criteria

The condensed ARC paper is organised by the narrative of this paper and maps its evidence to the six criteria of the Paper Track near its end. Table 39 gives, for each criterion, the evidence contributed and what it means, with the next test, rather than a self-assigned grade. Measurements on tasks built from public training puzzles are mechanism evidence, never performance or accuracy.

G Reproducibility map

Table 40 ties every reported result to its code, its frozen protocol, where it ran and its result record. Public now: the CORA-only entry's notebook, the neural comparison entry's notebook, the grammar-growth research notebook, a verification notebook that recomputes the headline numbers from public files (`https://www.kaggle.com/code/chimdiwalter/cora-verify-the-paper-s-headline-numbers`; version 3; 22 of its 26 checks match and none differ; it does not recompute the held-back-half counts 227, 118 and 343, or the 0.83% hidden-test score), and the GitHub release, the public branch arc2026-sprint at commit 8ec11940, tagged `cora-arc2026-public-2026-11-08c`; the submitted version will be tagged `arc2026-paper-20261108`. Since 8 November that release holds, besides both entries' notebooks with their embedded code, the code and records of the construction-at-scale, learned-order, held-back-half and related-benchmark studies, the packages they import, the code, records and audits of the five new-domain studies, the companion study's count-only aggregates (`records/companion_aggregates_20261108/`), the failure-guided choice test (both revisions) with its full-bank control contract, the library-rivals test, the calibrated acceptance test, the specification-reading guide test, two Boolean review records and the records this map cites. In the last column, *release* means the code and the cited record are in that release, *code* that the code is public and the record not yet, *Kaggle* a public notebook, *on request* companion code and records available from the authors, and *private data* the first author's unpublished images. The experiments marked Kaggle in the fourth column ran on private Kaggle CPU kernels; their outputs, receipts and hashes are in the release record; the private Kaggle notebooks stay private. Freezes are SHA-256 prefixes of the original freeze files; where the public copy was sanitised (a private folder name replaced, as for the construction and held-half freezes), `releases/2026-11-08/SANITISED_COPIES.json` maps the original's SHA-256 to the public copy's, and the release manifests list every public file with its SHA-256. A go-ahead (GO) commit released each held run; GO, result and analysis entries are commits of the project's private research history, cited in the public audits but not on the public branch.

Table 40: The reproducibility map. "Local CPU" means the project's own processors; every run's custody, cost and seal records sit beside its result record. The CORA-only entry's code is the scored notebook as released, not the earlier configuration of 21 October.

result	code folder	freeze	where it ran	result record	public
CORA-only entry, 0.83%	`kaggle_entries/0_plus_v2_cora-cast-r29-v2-arc-agi-2_v1` (the scored notebook, byte-identical)	notebook version 364571398	Kaggle, scored	submission 57902131	Kaggle; release
neural comparison entry, 24.03%	`kaggle_entries/H_sf-h-c4-cast-arc-agi-2_v1`	notebook version 364496615	Kaggle, scored	submission 57867631	Kaggle; release
headline-number check	the verification notebook	public notebook	Kaggle	the notebook's outputs	Kaggle
grammar-growth replay	`experiments/capability_growth_20261027`	notebook version 365118951	Kaggle	the notebook's outputs	Kaggle
related benchmarks, 81.5% to 95.8%	`experiments/universality_similar_problems_20261102`	pre-registered protocol in the folder	local CPU	`records/NOV2_UNIVERSALITY_SIMILAR_PROBLEMS_RESULT.md`	code
construction at scale, 64 against 38	`experiments/construction_at_scale_20261105`	b17f28d025a8	three private Kaggle CPU kernels	`runs/analysis/ANALYSIS.json` in the folder	release
reachability, 13 and 17 of 31	a post hoc reading of the study above	post hoc	local CPU	`records/CONSTRUCTION_REACHABILITY_ANALYSIS_20261108.md`	on request
learned search order, 320 against 234	`experiments/guided_construction_20261106`	d876eeded96c	three private Kaggle CPU kernels	`runs/held_run/RESULTS.json` in the folder	release
held-back-half test, 227 against 118	`experiments/natural_growth_b_20261105`	9d9d2248c511	local CPU (rehearsals on private Kaggle kernels)	final aggregate bfccc6832... and `reviews/INDEPENDENT_POST_SCORE_AUDIT.md`	release (audit); aggregate on request
late CPU copy	`experiments/late_submission_cpu_20261106`	reviewed copy	private Kaggle CPU kernel	`records/eval120_summary.json` in the folder	no
inferred wiring, 99 against 85 of 333	`experiments/bbm_wiring_grammar_20261104`	pre-registered protocol in the folder	local CPU	`results_D/SUMMARY_D.md` in the folder	no
replication, 101 against 66 of 595	companion study	9da4707166ae	local CPU	companion result record	on request

result	code folder	freeze	where it ran	result record	public
chosen observations, 320 against 291	companion study	1f10e860a7b2	local CPU	companion result record	on request
completion vote, 2,200 against 2,153	companion study	0c2b7d4db1a3	local CPU	aggregate 5bf69a11a511	on request
conditional construction, 433 against 321	companion study	2487788cc1c8	local CPU	result 38406658eaf6	on request
cofactor query policy, 310 gains	companion study	880bb45cfb6c	local CPU	result 8b25e4586ec5	on request
sparse engine, 60 coordinates	companion study	3c30b4d6cccf	local CPU	companion result record	on request
bounded loop, 219 of 256	companion study	f284caaec787	local CPU	result c462a3cb6844	on request
view bank, 9 of 9	companion study	e59a8b28b950	local CPU	companion result record	on request
coverage sweep and early-stop defect	companion study	18d62260e508	local CPU	companion public summary	on request
continuous follow-ups	companion study	324e85d69e7d, 08d549ca5c94, 40ac5d2bc7f3	local CPU	companion result records	on request
logic functions, 45 against 12 of 217	`experiments/transfer_logic_20261108`	go-ahead bca541e7	local CPU	`reviews/POST_SCORE_AUDIT.md` in the folder	release
enzyme chemistry, 76 against 33 of 200	`experiments/transfer_chemistry_20261108`	go-ahead 190756a6, amendment 849c4b50	local CPU	`reviews/POST_SCORE_AUDIT.md` in the folder	release
spreadsheets, 22 of 53	`experiments/transfer_spreadsheet_20261108`	go-ahead fccbb0ff	local CPU	`reviews/POST_SCORE_AUDIT.md` in the folder	release
signalling, 863 of 1,000	`experiments/transfer_signalling_20261108`	go-ahead 6ae7f578	local CPU	`reviews/POST_SCORE_AUDIT.md` in the folder	release
world models, 1,204 of 3,168	`experiments/transfer_worldmodel_20261108`	go-ahead ed754a3d	private Kaggle kernel	`reviews/POST_SCORE_AUDIT.md` in the folder	release
failure-guided choice, 30 of 50	`experiments/failure_choice_20261108`	go-ahead 87f30e19	three private Kaggle kernels	`reviews/POST_SCORE_AUDIT.md` in the folder	release
failure-guided choice, revision 2, 55 against 29 of 84	`experiments/failure_choice_r2_20261108`	go-ahead 4f88bb15; freeze fae62f8842be	private Kaggle kernels (one to generate, four to measure)	`reviews/POST_SCORE_AUDIT.md` in the folder	release
library rivals, 227 against 28 of 624	`experiments/library_rivals_20261108`	go-ahead 3faf93d6; freeze 0ce263495c9b	local CPU	`reviews/INDEPENDENT_POST_SCORE_AUDIT.md` in the folder	release
calibrated acceptance, 430 of 605	`experiments/calibrated_acceptance_20261108`	protocol 9f283c2c	local CPU	`reviews/POST_SCORE_AUDIT.md` in the folder	release
neural gap, 41 in 36 tasks	`paper/final_push_20261108/neural_vs_cora`	post hoc	local CPU	`CHECK.md` in the folder	on request
maize trial, 16 leaves	`experiments/universality_bio_grammar_20261103`	Amendment 8 freeze 62208c3079862fa1	local CPU	freeze only	private data

H Reproducibility: exact commands

The submitted entries and their reproducer are public in the repository `https://github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI`, release branch `arc2026-sprint`, now at commit `8ec119409becff92c266561488f13e7481cd08f2`, tagged `cora-arc2026-public-2026-11-08c`; the submitted version will be tagged `arc2026-paper-20261108` (`https://github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI/tree/arc2026-sprint`); the entries were first released at commit `636bdfa6`. The release holds both entries' notebooks with their embedded code, the builder, the expected predictions and the scorer, and, since 8 November, the code, records and audits of the studies marked "release" in Table 40. The rest of the companion research code (capability growth beyond the released studies, cross-domain and biology), its frozen protocols and records, and this report are not yet released; they are available from the authors on request. A public verification notebook recomputes the headline numbers from public files (`https://www.kaggle.com/code/chimdiwalter/cora-verify-the-paper-s-headline-numbers`). The release's code licence does not settle the data rights or open-model eligibility of the neural comparison entry's third-party checkpoint.

The claim-to-code-and-results tables (Appendix I) give the code, result file and status of every claim, and the results-to-source table (`paper/writeup/RESULTS_TO_SOURCE.md`) maps every number in this report to its record and line. Main locations: CORA's engine `geocat_arc/`; typed rule languages `cora_neural_v2/` to `cora_neural_v2g5/`; CORA-CAST `experiments/cora_cast_20261026/cast/` and its Kaggle solvers `experiments/cora_cast_20261026/kaggle_pkg/`; the self-extension cycle `experiments/capability_growth_20261027/cgrow/`; the neural entry `kaggle/score_first_2026/sf/`; the shared cross-domain code `cora_cross_domain_transfer/`.

Commands run from the repository root unless a working folder is named. Interpreters: three Python 3.12 virtual environments, one for capability growth, the CORA-CAST studies and the nuclei conditions, one for the CORA-only entry checks, the ARC-style benchmarks and the GPU steps, and one for the nuclei adapter and grammar induction, and the system `python3` 3.12.3; Kaggle ran Python 3.13.15 with numpy 2.1.3. A command marked *reconstructed* is taken from the run's recorded configuration because its launch line was not written down.

Code and notebooks.

- The scored CORA-only notebook, submission 57902131: `https://www.kaggle.com/code/chimdiwalter/cora-cast-r29-v2-arc-agi-2?scriptVersionId=364571398`.
- The neural comparison entry's notebook, submission 57867631, the attributed comparison (NVARC's method, re-implemented): `https://www.kaggle.com/code/chimdiwalter/sf-h-c4-cast-arc-agi-2?scriptVersionId=364496615`.
- The GitHub release, the public branch `arc2026-sprint` at commit `8ec11940`, tagged `cora-arc2026-public-2026-11-08c` (the submitted version will be tagged `arc2026-paper-20261108`): `https://github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI/tree/arc2026-sprint`.
- The verification notebook, which recomputes the headline numbers from public files: `https://www.kaggle.com/code/chimdiwalter/cora-verify-the-paper-s-headline-numbers`.
- The grammar-growth research notebook, version 365118951: `https://www.kaggle.com/code/chimdiwalter/cora-grammar-growth-research-replay?scriptVersionId=365118951`. Its cell sources match the reviewed local notebook (20 of 20 cells), though the notebook container's bytes differ.

The linked entries. Rebuild and verify, then run the local tests (both CPU only):

```
python3 kaggle/score_first_2026/configs/build_configs.py --only H,HRH,OP2,OP2RH
python3 -I kaggle/score_first_2026/configs/h/verify_c4_copy.py
python3 kaggle/score_first_2026/configs/verify_builds.py --only H,HRH
python3 kaggle/score_first_2026/configs/write_manifest.py
sh kaggle/score_first_2026/configs/run_h_tests.sh
setsid nohup sh kaggle/score_first_2026/configs/dev_effect/run_op2_checks.sh >> outputs/score_first/configs_run/op2_checks.log
↪  2>&1 &
```

The neural comparison entry on Kaggle (rehearsal, entry, submission 57867631), with the Kaggle command-line tool:

```
(cd outputs/score_first/kernels/sf-h-rehearsal-eval120 && kaggle kernels push -p .)
sh kaggle/score_first_2026/configs/dev_effect/process_h_rehearsal.sh 1
(cd outputs/score_first/kernels/sf-h-c4-cast-arc-agi-2 && kaggle kernels push -p .)
sh kaggle/score_first_2026/configs/dev_effect/check_h_entry.sh
kaggle competitions submit arc-prize-2026-arc-agi-2 -k chimdiwalter/sf-h-c4-cast-arc-agi-2 -v 1 -f submission.json -m "H: C4
↪  neural TTT attempt 1 (NVARC method re-implemented) + CORA cast_selective_v2 Tier 1 B3 attempt 2"
```

The CORA-only entry on Kaggle: the rehearsal `chimdiwalter/cora-cast-r29-v2-rehearsal-eval120` and the entry `chimdiwalter/cora-cast-r29-v2-arc-agi-2` are pushed the same way and checked with `kaggle/score_first_2026/configs/dev_effect/check_cpu_kernel.sh`; the full sequence and the submit line are in `records/NOV2_FINAL_SUBMISSION_STATUS.md` (the section on the CORA-only entry). Score any submission file on the 120:

```
python3 -I kaggle/score_first_2026/configs/dev_effect/score.py --challenges
↪  outputs/score_first/data/arc-agi_evaluation_challenges.json --pred
↪  O_local=kaggle/score_first_2026/configs/results/o_local_eval120/submission.json --pred
↪  OplusV2_kaggle=outputs/score_first/runs/op2_rehearsal_v1/submission.json --out
↪  kaggle/score_first_2026/configs/results/op2_rehearsal_v1/score.json
```

The CORA calls inside the notebooks, as used outside them (from `experiments/universality_similar_problems_20261102/PROTOCOL.md`, with `C` a challenges file and `D` an output folder):

```
python -m sfo.driver_plus --challenges C --out D --submission S --packages '[{"name": "cast_v1", "entry": "cast_candidates",
↪  "cpu_s": 45, "wall_s": 90, "details": 1}, {"name": "r29_cpu_package_20261101", "entry": "candidates", "cpu_s": 12, "wall_s":
↪  24, "details": 0}]' --deadline-s 39600 --grace-s 10 --shards 1
python -m sfb.cast_side --challenges C --out D/side --package cast_selective_v2 --entry evaluate --mode evidence --deadline T
↪  --cpu-s 15 --wall-s 30 --grace-s 10 --nice 10
```

then the neural comparison entry's merge (`sfb.merge.merge_files_packages` with the strict selective policy, code parameter `tier1:B3`).

The submitted solvers on fresh nuclei images, and the studies in progress.

```
setsid nohup sh experiments/universality_submitted_pkg_20261101/scripts/run_all.sh > /dev/null 2>&1 < /dev/null &
setsid nohup sh experiments/universality_similar_problems_20261102/scripts/chain.sh chain1 "c a" > /dev/null 2>&1 < /dev/null &
setsid nohup sh experiments/universality_similar_problems_20261102/scripts/chain.sh chain2 "b" > /dev/null 2>&1 < /dev/null &
sh experiments/universality_similar_problems_20261102/scripts/finish.sh
setsid nohup sh experiments/universality_fresh_image_20261103/scripts/chain.sh > /dev/null 2>&1 < /dev/null &
```

Capability growth (working folder `experiments/capability_growth_20261027`; the construction test, the self-extension follow-up and the list-functions replication are resumable chains):

```
setsid nohup sh chain_s1.sh >> runs/s1_chain.log 2>&1 < /dev/null &
setsid nohup sh chain_ib.sh >> runs/ib_chain.log 2>&1 < /dev/null &
setsid nohup sh chain_u3.sh >> runs/u3_chain.log 2>&1 < /dev/null &
sh chain_after_gen1c.sh
```

The self-extension test's earlier steps are *reconstructed* from their run configurations; each module ran through the launcher `launch.sh` (arguments: run identifier, kind, an optional GPU flag, then the module):

```
cgrow.tasks
cgrow.experiment prep prep_v4
cgrow.proposer generate runs/prep_v4/jobs.jsonl runs/gen1c/replies.jsonl       (GPU)
cgrow.experiment process process_v1 prep_v4 gen1c
cgrow.positive_control posctl_v1 prep_v4
```

The task-level correction (`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`) has no script; it recomputes from the units files that the record names.

CORA-CAST studies (working folder `experiments/cora_cast_20261026`):

```
PYTHONPATH=. PYTHONSAFEPATH=1 python -m cast.run launch --id confirm_v1 --family CAST-COMBO --kind confirm --force --config
↪  protocols/CONFIRMATION_PROTOCOL.md -- sh confirm/chain.sh confirm_v1
PYTHONPATH=. PYTHONSAFEPATH=1 python -m cast.run launch --id selective_v1 --family CAST-COMBO --kind screen --force -- sh
↪  reliability/chain.sh selective_v1
PYTHONPATH=. PYTHONSAFEPATH=1 python -m cast.run launch --id reliability_post --family CAST-COMBO --kind audit --force --after
↪  selective_v1 -- sh reliability/post.sh
python -m cast.runa export --out results/runA
python -m cast.report
python -m cast.tables > results/tables.md
```

The guarded-rewriting development run and the fresh 32 are *reconstructed* from their recorded run configurations:

```
sh runs/chain_v2/code/cast/stages/chain_v2.sh chain_v2
sh fresh32/chain.sh
sh fresh32/selective_chain.sh
```

The sealed confirmation cannot be rerun as a sealed test: its puzzle set is used up.

Reachability audit (working folder `experiments/architecture_sweep_20261026`, `PYTHONPATH=.`):

```
python -m harness.build_episodes --out runs/episodes_v1/data --stages S_train_syn S_select_syn S_train S_select ARC_eval_dev
python -m harness.reach
python -m harness.scorer --reach runs/episodes_v1/data/ARC_eval_dev/reach_predictions.jsonl --out
↪  runs/episodes_v1/reach_eval_dev.json
```

The neural-symbolic analyses of 1 and 2 November. Documented usages (the exact input paths are recorded by hash in the result files and in the companion study's requests, not as command lines):

```
python3 -I kaggle/score_first_2026/configs/h2/headroom.py POOLS_DIR H_SUBMISSION CHALLENGES OUT_JSON
python3 -I kaggle/score_first_2026/configs/h2/score_h2.py H_SUBMISSION H2_PREDICTIONS CHALLENGES OUT_JSON
python3 -I experiments/independent_scoring_20261102/score_neural_repair.py REPAIR.json NEURAL.json SOLUTIONS.json OPLUS.json
↪  H.json OUT.json
python3 -I experiments/independent_scoring_20261102/score_independent_candidates.py PREDICTIONS.json SOLUTIONS.json OPLUS.json
↪  H.json OUT.json
```

Biology studies (separately written code). BBBC039 repair, from its folder `cross_domain/bbbc039_repair_20261026`:

```
nice -n 19 ionice -c3 python3 -I scripts/s03_run.py --run
setsid nohup nice -n 19 ionice -c3 python3 -I scripts/s06_generic_arm.py --run > runlogs/s06.stdout 2>&1 &
python3 -I scripts/s07_audit_export.py
python3 -I ../universality_audit/universality_audit.py audit/manifest.json --report audit/AUDIT_REPORT.md
```

Drug response, from its scripts folder (STUDY stands for the study's folder), and grammar induction (domain 1; the other two domains' lines were not recorded), from its study folder:

```
cd STUDY/scripts && setsid nohup nice -n 19 ionice -c3 python3 -I r02_run_classtransfer.py > ../runlogs/r02_run.log 2>&1 &
cd cross_domain/grammar_induction_20261025 && sh run_domain.sh domains/scrna.json MODEL_DIR
```

The repair on top of StarDist (`cross_domain/universality_v2/`) has no recorded command; its scripts state their interpreter. The single-cell study:

```
cd cross_domain/scrna_dispersion_20261025/scripts && nice -n 19 ionice -c3 python s02_run_case_study.py --run
```

The neural-versus-CORA analyses (Section 5.4), from the repository root, each through the CPU-slot wrapper at the lowest priority, with `python3` from a Python 3.12 virtual environment:

```
D=paper/final_push_20261108/neural_vs_cora/scripts; P="python3 -B"; S="sh scripts/with_cpu_slot.sh nice -n 19"
$S $P $D/01_score_entries.py; $S $P $D/02_budget_runs.py --k 1 4 10; $S $P $D/03_diagnose.py
$S $P $D/04_build_table.py; $S $P $D/05_figures.py 6e453dd6 aa4ec2a5 38007db0
$S $P $D/explain_01_search.py driver --phases 1x:own,1x:a1,1x:a2,10x:own,10x:a1,10x:a2 --cpu-limit 6200
$S $P $D/explain_02_score.py; $S $P $D/explain_04_partial.py; $S $P $D/explain_03_figures.py; $S $P $D/explain_05_controls.py
$S $P $D/check_01_counts.py; $S $P $D/check_02_cast_own.py; $S $P $D/check_03_join.py
```

The companion study's pilots record no command lines in this repository; their result artifacts are listed in `records/NOV1_INDEPENDENT_RESEARCH_REVIEW.md`.

I Claim-to-code-and-results table

Table 41: The main claims, their code, result files and status. Full matrices: `records/CORA_PAPER_CLAIM_EVIDENCE_MATRIX.md` (rows 1 to 49) and `records/CORA_FINAL_PAPER_EVIDENCE_MATRIX.md` (rows 50 to 72).

claim	code	result file	status
every CORA-only configuration: 0 of 120; official 0.00%	`geocat_arc/`, `cora_neural_v2*/`	`records/RESULTS_SYNV4_REALCHECK_KAGGLE_DEV_20261022.md`; `records/KAGGLE_NATIVE_V1_OFFICIAL_RESULT_20261013.md`	supported (negative)
no demonstration-fitting program generated within the measured budget, none within one action correct, in 113 audited puzzles	`experiments/architecture_sweep_20261026/harness/reach.py`	`experiments/architecture_sweep_20261026/runs/episodes_v1/reach_eval_dev.json`	supported
background-0 defect; majority-colour fix	`experiments/cora_cast_20261026/cast/grids.py`	`records/CORA_SYNTHESIS_REVIEW_20261027.md`	supported
guarded rewriting: 10 against 3 of 176	`experiments/cora_cast_20261026/cast/rewrite.py`	`experiments/cora_cast_20261026/M2_REWRITE_REPORT.md`	provisional
sealed confirmation: 8 against 6 of 207, inconclusive	`experiments/cora_cast_20261026/confirm/`	`experiments/cora_cast_20261026/results/confirm/CONFIRMATION_REPORT.md`	guard effect not confirmed
62593bfd: exact new executed-program prediction	`experiments/cora_cast_20261026/cast/rewrite.py`	`experiments/cora_cast_20261026/REAL_ARC_RESULTS.json`	supported as a fact, on a puzzle that shaped the design
CORA-CAST explains 2 of the neural comparison entry's 344 public-evaluation attempts, both right; 0 of 301 wrong	`paper/final_push_20261108/neural_vs_cora/scripts/explain_01_search.py`	`paper/final_push_20261108/neural_vs_cora/data/explain/explain_rows.csv`; `paper/final_push_20261108/neural_vs_cora/data/check/check_counts.json`	supported; post hoc, exposed
the 41-output gap: no program at ten times the budget; a missing action first in 35 of 36 tasks, a missing guard in 1	`paper/final_push_20261108/neural_vs_cora/scripts/03_diagnose.py`	`paper/final_push_20261108/neural_vs_cora/gap_table.csv`; `paper/final_push_20261108/neural_vs_cora/data/gap_summary.json`	supported; post hoc, exposed; rules refined on these tasks
failure-guided choice: the failure record and a fixed order 30 of 50 each; a random order 20	`experiments/failure_choice_20261108/fgc/`	`experiments/failure_choice_20261108/reviews/POST_SCORE_AUDIT.md`	outcome none, as pre-registered; audited

claim	code	result file	status
failure-guided choice, revision 2: the router 55 of 84 against 29 for the static order; the best fixed order 44	`experiments/failure_choice_r2_20261108/fgc/`	`experiments/failure_choice_r2_20261108/reviews/POST_SCORE_AUDIT.md`	pre-registered outcome met against the static order; not established against the best fixed order; audited
library rivals: failure-kept 227, compression 28 and random 6 of 624	`experiments/library_rivals_20261108/lr/`	`experiments/library_rivals_20261108/reviews/INDEPENDENT_POST_SCORE_AUDIT.md`	as stated, with its limits; audited
attempt 2: 17 to 19 and 22 to 24 of 172, 0 losses	`experiments/cora_cast_20261026/cast/runa.py`	`experiments/cora_cast_20261026/ABLATIONS_AND_COSTS.md`	provisional (inspected puzzles)
the self-extension cycle runs end to end; 2 of 4 operations recovered	`experiments/capability_growth_20261027/cgrow/`	`experiments/capability_growth_20261027/reports/TEST_I_REPORT.md`	supported (capability recovery)
own report beats shuffled reports and 20× search	`experiments/capability_growth_20261027/cgrow/invent.py`	`experiments/capability_growth_20261027/reports/TEST_I_REPORT.md`	not significant at task level (4 to 0, $p = 0.125$)
own report beats failure-blind invention	`experiments/capability_growth_20261027/cgrow/testib.py`	`experiments/capability_growth_20261027/reports/TEST_IB_REPORT.md`	unsupported (2 against 2 tasks)
Construction test: a wrong specification recovers nothing (8 against 0 tasks); against demonstrations alone 8 against 4; against matched search, the primary, 8 against 5; blind enumeration 10 in any seed	`experiments/capability_growth_20261027/`	`records/NOV1_S1_TASK_LEVEL_CORRECTION.md`	supported ($p = 0.0078$); not established ($p = 0.125$; $p = 0.375$)
the proposer's coding is the binding limit (1 of 11)	`experiments/capability_growth_20261027/cgrow/proposer.py`	`experiments/capability_growth_20261027/reports/TEST_I_REPORT.md`	supported
five architecture families, none advanced	`experiments/architecture_sweep_20261026/`	`experiments/architecture_sweep_20261026/ALL_FIVE_RESULTS.md`	supported (negative)
the neural comparison entry's attempt 1 is NVARC's method, re-implemented; CORA-CAST version 2 on attempt 2	`kaggle/score_first_2026/sf/`	`records/KAGGLE_SCORE_FIRST_SUBMISSION_READY_20261026.md`	supported as a description
portability in three biology domains	`cora_cross_domain_transfer/universal/`	`cross_domain/universality_v2/UNIVERSALITY_EVIDENCE_AUDIT.md`	supported
repair gains (0.883 to 0.923; 0.692 to 0.705)	`cross_domain/bbbc039_repair_20261026/`	`cross_domain/bbbc039_repair_20261026/REPORT.md`; `cross_domain/universality_v2/neural_repair/summary.json`	supported, the procedure's
utility beyond matched controls; learned transfer	`cora_cross_domain_transfer/universal/`	`cross_domain/universality_v2/UNIVERSALITY_EVIDENCE_AUDIT.md`	unsupported
held tests in new domains: logic 45 against 12 of 217; chemistry 76 against 33 of 200; spreadsheets 22 against 17 of 53; signalling limited to one removed edge; world models not passed	`experiments/transfer_logic_20261108/`, `experiments/transfer_chemistry_20261108/`, `experiments/transfer_spreadsheet_20261108/`, `experiments/transfer_signalling_20261108/`, `experiments/transfer_world model_20261108/`	each folder's `reviews/POST_SCORE_AUDIT.md`	audited; reconstructions, not discoveries
calibrated acceptance on PUMA melanoma regions: 430 of 605 edits harmful; F1 down 0.0020	`experiments/calibrated_acceptance_20261108/`	`experiments/calibrated_acceptance_20261108/reviews/POST_SCORE_AUDIT.md`	harmful, as audited
the submitted solvers on fresh nuclei: no edit under the neural comparison entry's rule, no gain	`experiments/universality_submitted_pkg_20261101/`	`records/NOV1_UNIVERSALITY_RESULT.md`	supported (portable; no edit, so no degradation through abstention; no gain)
the plain CORA-only configuration on Kaggle's CPUs: 1 of 172, against 3 locally (cap)	`kaggle/score_first_2026/configs/sfo/`	`kaggle/score_first_2026/configs/results/op_rehearsal_v1b/score.json`	supported; caps raised to 45 CPU seconds (Appendix A)

criterion	evidence contributed	interpretation and next test
Accuracy	two executable submitted systems: the CORA-only entry (submission 57902131) scored 0.83% on the hidden test, against 0.00% for every earlier CORA entry; the neural comparison entry (submission 57867631), credited to NVARC's method, scored 24.03%; both are public-leaderboard scores on 120 hidden tasks; on the common exposed rehearsal CORA added no output and lost none (43 of 172 with and without it)	end-to-end entries are demonstrated, and CORA's answers are precise where it answers on the exposed public evaluation (3 of 172, all right), and how the hidden score splits into precision and coverage is not observed; broader coverage is the main accuracy goal, the hybrid's hidden increment is unknown, and later research is separate from the scores
Universality	the loop design ran in thirteen domain families through declared adapters (Table 23): precision 81.5% to 95.8% on three related benchmarks; list functions; published gene networks (101 against 66 of 595 rules; 320 against 291 with chosen observations; 2,200 against 2,153 of 2,380 with a completion vote, post hoc); held real benchmark logic functions (45 against 12 of 217 for a shuffled failure description; reconstructions of known functions); held enzymatic reactions (76 against 33 of 200 for the best fixed published rule, a classical nearest-neighbour method 75; reconstruction within families); a null on spreadsheets; a narrow signalling result; a world-model probe that did not pass	the procedure's design carries over through adapters, with separately written code outside grids, sequences and lists; learned priors did not (177 against 194), and the measured-data domains are diagnoses; next, a fresh biology cohort and noisy-data adapters under matched controls
Progress	typed holes computed from the demonstrations once search failed beat static search in the starting language, 64 against 38 of 95 generated tasks, with less compute (independently verified), and post hoc 13 of the 31 tasks only construction solved have no equivalent static program at any depth searched (not proof that none exists deeper); held-back-half reuse 227 against 118 and construction 343 of 624 (independently audited; gain on 1D-ARC, loss on ConceptARC); learned search order 320 against 234 at 500 counted operations per hole; the exposed ARC-AGI-2 check, 2 of 172 by construction, both the CORA-only entry's own answers	discovery moved inside supplied spaces; at a binding budget a learned guide solved 1,083 of 1,128 held generated tasks against 953 for the hand-set order, 881 for a frequency table and 1,018 for a linear model, search efficiency, not new reach; on selection, where every change could check cheaply whether it applies, fixed alternatives with the same capabilities matched (219 against 219; a router reading the failure record gave the same answers as a fixed order), and on controlled generated tasks where a change cannot, the pre-registered router reached the oracle (55 of 84) and no fixed order did (best 44), an advantage not established against that best order; next, the same test on real answers
Theory	three requirements (availability, discovery, selection) tied to a decision model with an information term; elementary conditional results: solved $\leq$ generated $\leq$ expressible, demonstrations underdetermine programs, verification certifies consistency and stability, not correctness, selective acceptance trades coverage for precision, an omitted essential input cannot be recovered by search, demonstrations carry bounded information, and under noise a likelihood bar is a count of net corrections; each with the test that bears on it (Table 8)	the results account for the measured pattern under their assumptions, and growth under agreement of all fitting programs is untested; they are not impossibility results for ARC, deep learning or library compression; each prediction states what would falsify it
Completeness	scored notebook identities, architecture, an executed program example, frozen protocols, failed attempts, all controls, resource accounting, our own audits, a public verification notebook that recomputes the headline numbers (version 3; 22 of its 26 checks match and none differ) and the reproducibility map (Appendix G)	some private Kaggle kernels and companion code are described, not linked; the platform word count, release rights and deadline must be checked before submission
Novelty	pre-registered controls that replace only the content of what is built (another task's holes, 64 against 16 of 95; a shuffled failure description, 45 against 12 of 217 held logic functions); construction that reaches operations bounded static search did not (13 of 31 tasks, post hoc); the three requirements measured apart, with the measured condition under which choosing can pay; one loop design carried through adapters with held tests and negatives; each compared with the closest prior work (Table 4; Appendix E)	every ingredient has a predecessor, and the claims are the controlled measurements; no claim of priority beyond the work read

Table 39: The six Paper Track criteria: what the evidence contributes and what it means.

References

[1] Tatsuya Akutsu, Satoru Miyano, and Satoru Kuhara. Identification of genetic networks from a small number of gene expression patterns under the Boolean network model. In *Pacific Symposium on Biocomputing*, volume 4, pages 17–28, 1999. doi: 10.1142/9789814447300_0003.

[2] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 942–963, 2025. URL https://proceedings.mlr.press/v267/akyurek25a.html. arXiv:2411.07279; first version titled "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning".

[3] Simon Alford, Anshula Gandhi, Akshay Rangamani, Andrzej Banburski, Tony Wang, Sylee Dandekar, John Chin, Tomaso Poggio, and Peter Chin. Neural-guided, bidirectional program search for abstraction and reasoning. In Rosa Maria Benito, Chantal Cherifi, Hocine Cherifi, Esteban Moro, Luis M. Rocha, and Marta Sales-Pardo, editors, *Complex Networks & Their Applications X, Volume 1: Proceedings of the Tenth International Conference on Complex Networks and Their Applications COMPLEX NETWORKS 2021*, Studies in Computational Intelligence, pages 657–668. Springer, 2022. doi: 10.1007/978-3-030-93409-5_54. arXiv:2110.11536.

[4] Hussein Almuallim and Thomas G. Dietterich. Learning with many irrelevant features. In *Proceedings of the Ninth National Conference on Artificial Intelligence (AAAI-91)*, pages 547–552, 1991. URL https://cdn.aaai.org/AAAI/1991/AAAI91-085.pdf.

[5] Alma Andersson, Ludvig Larsson, Linnea Stenbeck, Fredrik Salmén, Anna Ehinger, Sunny Z. Wu, Ghamdan Al-Eryani, Daniel Roden, Alex Swarbrick, Åke Borg, Jonas Frisén, Camilla Engblom, and Joakim Lundeberg. Spatial deconvolution of HER2-positive breast cancer delineates tumor-associated cell type interactions. *Nature Communications*, 12:6012, 2021. doi: 10.1038/s41467-021-26271-2.

[6] Dana Angluin. Queries and concept learning. *Machine Learning*, 2:319–342, 1988. doi: 10.1023/A:1022821128753. URL https://homepages.math.uic.edu/~lreyzin/papers/angluin88.pdf.

[7] ARC Prize. Post on X, 16 October, 2025. URL https://x.com/arcprize/status/1978872651180577060.

[8] ARC Prize Foundation. ARC-AGI-2 public training and evaluation data, 2025. URL https://github.com/arcprize/ARC-AGI-2.

[9] ARC Prize Foundation. ARC-AGI Community Leaderboard. https://arcprize.org/leaderboard/community, 2025. Semi-private results run and verified by ARC Prize; accessed 2 November 2026.

[10] Malou Arvidsson, Salma Kazemi Rashed, and Sonja Aits. An annotated high-content fluorescence microscopy dataset with Hoechst 33342-stained nuclei and manually labelled outlines. *Data in Brief*, 46:108769, 2023. doi: 10.1016/j.dib.2022.108769.

[11] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01989. arXiv:1611.01989.

[12] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Predictive inference with the jackknife+. *The Annals of Statistics*, 49(1):486–507, 2021. doi: 10.1214/20-AOS1965.

[13] Taylor Berg-Kirkpatrick and Dan Klein. Phylogenetic grammar induction. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, pages 1288–1297, 2010. URL https://aclanthology.org/P10-1131/.

[14] Robert Cregar Berwick. Computational analogues of constraints on grammars: A model of syntactic acquisition. In *18th Annual Meeting of the Association for Computational Linguistics*, pages 49–53, Philadelphia, Pennsylvania, USA, 1980. Association for Computational Linguistics. doi: 10.3115/981436.981453. URL https://aclanthology.org/P80-1014/.

[15] George Boole. *An Investigation of the Laws of Thought, on Which Are Founded the Mathematical Theories of Logic and Probabilities*. Walton and Maberly, London, 1854.

[16] Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 7(POPL):1182–1213, 2023. doi: 10.1145/3571234. Article 41; arXiv:2211.16605.

[17] Eric Brill. Automatic grammar induction and parsing free text: A transformation-based approach. In *31st Annual Meeting of the Association for Computational Linguistics*, pages 259–265, 1993. doi: 10.3115/981574.981609. URL https://aclanthology.org/P93-1035/.

[18] Eric Brill. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565, 1995. URL https://aclanthology.org/J95-4004/.

[19] Broad Institute Single Cell Portal. Study SCP1219: Columbia University/NYP COVID-19 Lung Atlas, 2021. URL https://singlecell.broadinstitute.org/single_cell/study/SCP1219.

[20] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. doi: 10.1109/TC.1986.1676819.

[21] Alan Bundy and Xue Li. Representational change is integral to reasoning. *Philosophical Transactions of the Royal Society A*, 381(2251):20220052, 2023. doi: 10.1098/rsta.2022.0052.

[22] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2305.17126.

[23] Juan C. Caicedo, Jonathan Roth, Allen Goodman, Tim Becker, Kyle W. Karhohs, Matthieu Broisin, Csaba Molnar, Claire McQuin, Shantanu Singh, Fabian J. Theis, and Anne E. Carpenter. Evaluation of deep learning strategies for nucleus segmentation in fluorescence images. *Cytometry Part A*, 95(9):952–965, 2019. doi: 10.1002/cyto.a.23863.

[24] Irene Cantone, Lucia Marucci, Francesco Iorio, Maria Aurelia Ricci, Vincenzo Belcastro, Mukesh Bansal, Stefania Santini, Mario di Bernardo, Diego di Bernardo, and Maria Pia Cosma. A yeast synthetic network for in vivo assessment of reverse-engineering and modeling approaches. *Cell*, 137(1):172–181, 2009. doi: 10.1016/j.cell.2009.01.055.

[25] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html. arXiv:2304.05128.

[26] François Chollet. On the measure of intelligence, 2019. URL https://arxiv.org/abs/1911.01547.

[27] François Chollet. Post on X, 17 June, 2024. URL https://x.com/fchollet/status/1802790666420035646.

[28] François Chollet. Post on X, 15 January, 2025. URL https://x.com/fchollet/status/1879583863368032432.

[29] François Chollet. Post on X, 21 August, 2025. URL https://x.com/fchollet/status/1958410017683681698.

[30] François Chollet. Post on X, 8 October, 2025. URL https://x.com/fchollet/status/1975849178057334871.

[31] François Chollet. Post on X, 11 December, 2025. URL https://x.com/fchollet/status/1999183528492433672.

[32] François Chollet. Post on X, 25 March, 2026. URL https://x.com/fchollet/status/2036861192619384989.

[33] François Chollet. Post on X, 25 March, 2026. URL https://x.com/fchollet/status/2036879665655406944.

[34] François Chollet. Post on X, 25 March, 2026. URL https://x.com/fchollet/status/2036881543973790004.

[35] François Chollet. Post on X, 6 August, 2026. URL https://x.com/fchollet/status/2085324762637574183.

[36] François Chollet. Post on X, 7 August, 2026. URL https://x.com/fchollet/status/2085796727362052241.

[37] François Chollet. Post on X, 14 August, 2026. URL https://x.com/fchollet/status/2088243704603824311.

[38] François Chollet. Post on X, 21 August, 2026. URL https://x.com/fchollet/status/2090838046937645398.

[39] François Chollet. Post on X, 3 September, 2026. URL https://x.com/fchollet/status/2095599835932135919.

[40] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report, 2024. URL https://arxiv.org/abs/2412.04604.

[41] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2025. URL https://arxiv.org/abs/2505.11831.

[42] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report, 2026. URL https://arxiv.org/abs/2601.10904.

[43] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV 2000)*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000. doi: 10.1007/10722167_15.

[44] Jeff Clune. AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. arXiv:1905.10985, 2019. URL https://arxiv.org/abs/1905.10985.

[45] Steven M. Corsello et al. Discovering the anticancer potential of non-oncology drugs by systematic viability profiling. *Nature Cancer*, 1(2):235–248, 2020. doi: 10.1038/s43018-019-0018-6.

[46] Andrew Cropper and Rolf Morel. Learning programs by learning from failures. *Machine Learning*, 110(4): 801–856, 2021. doi: 10.1007/s10994-020-05934-z. arXiv:2005.02259.

[47] Andrew Cropper and Rolf Morel. Predicate invention by learning from failures, 2021. URL https://arxiv.org/abs/2104.14426.

[48] James Cussens and Stephen Pulman. Incorporating linguistics constraints into inductive logic programming. In *Fourth Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, pages 184–193, 2000. URL https://aclanthology.org/W00-0740/.

[49] CZI Cell Science Program et al. CZ CELLxGENE Discover: A single-cell data platform for scalable exploration, analysis and modeling of aggregated data. *Nucleic Acids Research*, 53(D1):D886–D900, 2025. doi: 10.1093/nar/gkae1142.

[50] Anugyan Das, Omkar Ghugarkar, Vishvesh Bhat, and Asad Aali. Compositional neuro-symbolic reasoning. arXiv:2604.02434, 2026. URL https://arxiv.org/abs/2604.02434. Poster, ICML 2026 Workshop on Compositional Learning.

[51] Jesse Davis and Pedro Domingos. Deep transfer via second-order Markov logic. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 217–224, 2009. doi: 10.1145/1553374.1553402.

[52] Alejandro de Miquel Bleier and Roderic Guigó Corominas. ARC Kaggle competition: Second-place solution to the Abstraction and Reasoning Challenge. https://github.com/alejandrodemiquel/ARC_Kaggle, 2020. Also at https://github.com/RodericGuigoCorominas/ARC_Kaggle.

[53] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, 2017. URL https://arxiv.org/abs/1703.07469. arXiv:1703.07469.

[54] Atray Dixit, Oren Parnas, Biyu Li, Jenny Chen, Charles P. Fulco, Livnat Jerby-Arnon, Nemanja D. Marjanovic, Danielle Dionne, Tyler Burks, Raktima Raychowdhury, Britt Adamson, Thomas M. Norman, Eric S. Lander, Jonathan S. Weissman, Nir Friedman, and Aviv Regev. Perturb-Seq: Dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell*, 167(7):1853–1866.e17, 2016. doi: 10.1016/j.cell.2016.11.038.

[55] Christoffer Edlund, Timothy R. Jackson, Nabeel Khalid, Nicola Bevan, Timothy Dale, Andreas Dengel, Sheraz Ahmed, Johan Trygg, and Rickard Sjögren. LIVECell: A large-scale dataset for label-free live cell segmentation. *Nature Methods*, 18(9):1038–1045, 2021. doi: 10.1038/s41592-021-01249-6.

[56] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11(53), 2010.

[57] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.

[58] Yu Feng, Ruben Martins, Jacob Van Geffen, Isil Dillig, and Swarat Chaudhuri. Component-based synthesis of table consolidation and transformation tasks from examples. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 422–436, 2017. doi: 10.1145/3062341.3062351.

[59] Yu Feng, Ruben Martins, Osbert Bastani, and Isil Dillig. Program synthesis using conflict-driven learning. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 420–435. ACM, 2018. doi: 10.1145/3192366.3192382.

[60] John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.

[61] Jasmin Fisher, Ali Sinan Köksal, Nir Piterman, and Steven Woodhouse. Synthesising executable gene regulatory networks from single-cell gene expression data. In *Computer Aided Verification (CAV 2015)*, Lecture Notes in Computer Science, pages 544–560. Springer, 2015. doi: 10.1007/978-3-319-21690-4_38.

[62] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 17657–17671, 2025. URL https://proceedings.mlr.press/v267/franzen25a.html. arXiv:2505.07859.

[63] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[64] Clare E. Giacomantonio and Geoffrey J. Goodhill. A Boolean model of the gene regulatory network underlying mammalian cortical area development. *PLoS Computational Biology*, 6(9):e1000936, 2010. doi: 10.1371/journal.pcbi.1000936.

[65] Luke A. Gilbert, Max A. Horlbeck, Britt Adamson, Jacqueline E. Villalta, Yuwen Chen, Evan H. Whitehead, Carla Guimaraes, Barbara Panning, Hidde L. Ploegh, Michael C. Bassik, Lei S. Qi, Martin Kampmann, and Jonathan S. Weissman. Genome-scale CRISPR-mediated control of gene repression and activation. *Cell*, 159(3):647–661, 2014. doi: 10.1016/j.cell.2014.09.029.

[66] Gabriel Grand, Lionel Wong, Maddy Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.19791.

[67] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.

[68] Sankha Narayan Guria, Jeffrey S. Foster, and David Van Horn. Absynthe: Abstract interpretation-guided synthesis. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1584–1607, 2023. doi: 10.1145/3591285.

[69] Esther Heid, Daniel Probst, William H. Green, and Georg K. H. Madsen. EnzymeMap: curation, validation and data-driven prediction of enzymatic reactions. *Chemical Science*, 14(48):14229–14242, 2023. doi: 10.1039/d3sc02048g.

[70] Céline Hocquette and Andrew Cropper. Relational decomposition for program synthesis. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*, pages 4526–4534, 2025. doi: 10.24963/ijcai.2025/504. arXiv:2408.12212.

[71] Michael Hodel. arc-dsl: a domain-specific language for the Abstraction and Reasoning Corpus, 2023. URL https://github.com/michaelhodel/arc-dsl.

[72] Qinheping Hu, Jason Breck, John Cyphert, Loris D'Antoni, and Thomas Reps. Proving unrealizability for syntax-guided synthesis. In *Computer Aided Verification (CAV)*, Lecture Notes in Computer Science, pages 335–352. Springer, 2019. doi: 10.1007/978-3-030-25540-4_18.

[73] Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktäschel. Position: Open-endedness is essential for artificial superhuman intelligence. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 20597–20616, 2024.

[74] Trey E. Ideker, Vesteinn Thorsson, and Richard M. Karp. Discovery of regulatory interactions through perturbation: Inference and experimental design. In *Biocomputing 2000: Proceedings of the Pacific Symposium*, pages 305–316. World Scientific, 2000. doi: 10.1142/9789814447331_0029.

[75] Katsumi Inoue, Tony Ribeiro, and Chiaki Sakama. Learning from interpretation transition. *Machine Learning*, 94(1):51–79, 2014. doi: 10.1007/s10994-013-5353-8.

[76] Isaac Joffe and Chris Eliasmith. Vector symbolic algebras for the Abstraction and Reasoning Corpus. *npj Unconventional Computing*, 3(1):54, 2026. doi: 10.1038/s44335-026-00100-5. arXiv:2511.08747.

[77] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL https://arxiv.org/abs/2510.04871.

[78] Claus Kadelka, Taras-Michael Butrie, Evan Hilton, Jack Kinseth, Addison Schmidt, and Haris Serdarevic. A meta-analysis of Boolean network models reveals design principles of gene regulatory networks. *Science Advances*, 10(2):eadj0822, 2024. doi: 10.1126/sciadv.adj0822.

[79] Ashwin Kalyan, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. Neural-guided deductive search for real-time program synthesis from examples. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1804.01186. arXiv:1804.01186.

[80] Greg Kamradt. Post on X, 14 November, 2024. URL https://x.com/GregKamradt/status/1857060130975531329.

[81] Greg Kamradt. Post on X, 16 September, 2025. URL https://x.com/GregKamradt/status/1968000354659995674.

[82] Greg Kamradt. Announcing ARC-AGI-3: A new challenge for frontier agentic intelligence. ARC Prize blog, 2026. URL https://arcprize.org/blog/arc-agi-3-launch. Published 25 March 2026.

[83] Greg Kamradt. Post on X, 3 September, 2026. URL https://x.com/GregKamradt/status/2095600045873828013.

[84] Subin Kim, Prin Phunyaphibarn, Donghyun Ahn, and Sundong Kim. Playgrounds for abstraction and reasoning. In *NeurIPS 2022 Workshop on Neuro Causal and Symbolic AI (nCSI)*, 2022. URL https://openreview.net/forum?id=F4RNpByoqP. Mini-ARC data: https://github.com/KSB21ST/MINI-ARC.

[85] Dan Klein and Christopher Manning. Corpus-based induction of syntactic structure: Models of dependency and constituency. In *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics*, pages 478–485, 2004. doi: 10.3115/1218955.1219016. URL https://aclanthology.org/P04-1061/.

[86] Mike Knoop. Post on X, 6 December, 2024. URL https://x.com/mikeknoop/status/1865107183014400304.

[87] Mike Knoop. ARC Prize 2025 results & analysis. ARC Prize blog, 2025. URL https://arcprize.org/blog/arc-prize-2025-results-analysis. Published 5 December 2025.

[88] Mike Knoop. Post on X, 15 January, 2025. URL https://x.com/mikeknoop/status/1879584411420966928.

[89] Mike Knoop. Post on X, 19 May, 2025. URL https://x.com/mikeknoop/status/1924609247725424725.

[90] Mike Knoop. Post on X, 16 July, 2026. URL https://x.com/mikeknoop/status/2077819572452036896.

[91] Mike Knoop. Post on X, 3 September, 2026. URL https://x.com/mikeknoop/status/2095600676919455857.

[92] Florian Kromp, Eva Bozsaky, Fikret Rifatbegovic, Lukas Fischer, Magdalena Ambros, Maria Berneder, Tamara Weiss, Daria Lazic, Wolfgang Dörr, Allan Hanbury, Klaus Beiske, Peter F. Ambros, Inge M. Ambros, and Sabine Taschner-Mandl. An annotated fluorescence image dataset for training nuclear segmentation methods. *Scientific Data*, 7(1):262, 2020. doi: 10.1038/s41597-020-00608-w.

[93] Jan Krumsiek, Carsten Marr, Timm Schroeder, and Fabian J. Theis. Hierarchical differentiation of myeloid progenitors is encoded in the transcription factor network. *PLoS ONE*, 6(8):e22649, 2011. doi: 10.1371/journal.pone.0022649.

[94] Harri Lähdesmäki, Ilya Shmulevich, and Olli Yli-Harja. On learning gene regulatory networks under the Boolean network model. *Machine Learning*, 52(1–2):147–167, 2003. doi: 10.1023/A:1023905711304.

[95] John E. Laird. Introduction to Soar, 2022. URL https://arxiv.org/abs/2205.03854.

[96] John E. Laird, Paul S. Rosenbloom, and Allen Newell. Chunking in Soar: The anatomy of a general learning mechanism. *Machine Learning*, 1(1):11–46, 1986. doi: 10.1007/BF00116249.

[97] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[98] Doyoon Lee, Woosuk Lee, and Kwangkeun Yi. Inductive program synthesis by meta-analysis-guided hole filling. *Proceedings of the ACM on Programming Languages*, 10(POPL):1500–1529, 2026. doi: 10.1145/3776694.

[99] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[100] Lihong Li, Michael L. Littman, and Thomas J. Walsh. Knows what it knows: A framework for self-aware learning. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 568–575, 2008. doi: 10.1145/1390156.1390228.

[101] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/35ee921eb17e9b4c3e73b6e2ae0d55ba-Paper-Conference.pdf. arXiv:2411.02272.

[102] Shoudan Liang, Stefanie Fuhrman, and Roland Somogyi. REVEAL, a general reverse engineering algorithm for inference of genetic network architectures. In *Pacific Symposium on Biocomputing*, volume 3, pages 18–29, 1998. URL https://psb.stanford.edu/psb-online/proceedings/psb98/liang.pdf.

[103] Isaac Liao and Albert Gu. ARC-AGI without pretraining, 2025. URL https://arxiv.org/abs/2512.06104.

[104] Vebjorn Ljosa, Katherine L. Sokolnicki, and Anne E. Carpenter. Annotated high-throughput microscopy image sets for validation. *Nature Methods*, 9(7):637, 2012. doi: 10.1038/nmeth.2083.

[105] Miroslav Lžičař. CellARC: Measuring intelligence with cellular automata, 2025. Version 1, marked as a working draft.

[106] Miroslav Lžičař. CellARC: An oracle-calibrated benchmark for few-shot rule induction. In *ICML 2026 Workshop on Combining Theory and Benchmarks: Towards a Virtuous Cycle to Understand and Guarantee Foundation Model Performance*, 2026. URL https://icml.cc/virtual/2026/68262. Poster, 10 July 2026.

[107] Matthew V. Macfarlane, Clément Bonnet, Herke van Hoof, and Levi H. S. Lelis. Gradient-based program synthesis with neurally interpreted languages. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2604.18907. arXiv:2604.18907.

[108] Amirreza Mahbod, Christine Polak, Katharina Feldmann, Rumsha Khan, Katharina Gelles, Georg Dorffner, Ramona Woitek, Sepideh Hatamikia, and Isabella Ellinger. NuInsSeg: A fully annotated dataset for nuclei instance segmentation in H&E-stained histological images. *Scientific Data*, 11:295, 2024. doi: 10.1038/s41597-024-03117-2.

[109] Ryan McDonald, Slav Petrov, and Keith Hall. Multi-source transfer of delexicalized dependency parsers. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 62–72, 2011. URL https://aclanthology.org/D11-1006/.

[110] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 691–701, 2016. doi: 10.1145/2884781.2884807.

[111] Johannes C. Melms et al. A molecular single-cell lung atlas of lethal COVID-19. *Nature*, 595, 2021. doi: 10.1038/s41586-021-03569-1. Pages 114 to 119.

[112] Steven Minton. Quantitative results concerning the utility of explanation-based learning. *Artificial Intelligence*, 42(2–3):363–391, 1990. doi: 10.1016/0004-3702(90)90059-9.

[113] Tom M. Mitchell. Generalization as search. *Artificial Intelligence*, 18(2):203–226, 1982. doi: 10.1016/0004-3702(82)90040-6.

[114] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997. ISBN 0-07-042807-7. URL http://www.cs.cmu.edu/~tom/mlbook.html.

[115] Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. The ConceptARC benchmark: Evaluating understanding and generalization in the ARC domain. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2305.07141.

[116] Stephen Muggleton and Wray Buntine. Machine invention of first-order predicates by inverting resolution. In John Laird, editor, *Proceedings of the Fifth International Conference on Machine Learning (ICML)*, pages 339–352, San Mateo, CA, 1988. Morgan Kaufmann. doi: 10.1016/B978-0-934613-64-4.50040-2.

[117] Stephen H. Muggleton, Dianhuan Lin, Niels Pahlavi, and Alireza Tamaddoni-Nezhad. Meta-interpretive learning: application to grammatical inference. *Machine Learning*, 94(1):25–49, 2014. doi: 10.1007/s10994-013-5358-3.

[118] Stephen H. Muggleton, Dianhuan Lin, and Alireza Tamaddoni-Nezhad. Meta-interpretive learning of higher-order dyadic datalog: predicate invention revisited. *Machine Learning*, 100(1), 2015. doi: 10.1007/s10994-014-5471-y.

[119] Sach Mukherjee and Terence P. Speed. Network inference using informative priors. *Proceedings of the National Academy of Sciences*, 105(38):14313–14318, 2008. doi: 10.1073/pnas.0802272105.

[120] Kevin Murphy. Model Discovery Agent: LLM-assisted Bayesian experiment design for data-efficient discovery of mechanistic world models, 2026. URL `https://arxiv.org/abs/2608.09696`.

[121] Chimdi Walter Ndubuisi. A phenotype atlas of maize lesion organization: geometry and a locked within-rater comparison. Unpublished working manuscript, 21 September 2026; aggregate results only, 2026.

[122] Chimdi Walter Ndubuisi. An image-derived segmentation pilot on maize leaf lesions. Unpublished exploratory report, November 2026; aggregate results only, 2026.

[123] C. G. Nevill-Manning and I. H. Witten. Identifying hierarchical structure in sequences: A linear-time algorithm. *Journal of Artificial Intelligence Research*, 7:67–82, 1997. doi: 10.1613/jair.374.

[124] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 772–781, 2013. doi: 10.1109/ICSE.2013.6606623.

[125] Zhuofu Ni, Andrew E. Stine, Keith E. J. Tyo, and Linda J. Broadbelt. Curating a comprehensive set of enzymatic reaction rules for efficient novel biosynthetic pathway design. *Metabolic Engineering*, 65:79–87, 2021. doi: 10.1016/j.ymben.2021.02.006.

[126] Thomas M. Norman, Max A. Horlbeck, Joseph M. Replogle, Alex Y. Ge, Albert Xu, Marco Jost, Luke A. Gilbert, and Jonathan S. Weissman. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365(6455):786–793, 2019. doi: 10.1126/science.aax4438.

[127] Augustus Odena, Kensen Shi, David Bieber, Rishabh Singh, Charles Sutton, and Hanjun Dai. BUSTLE: Bottom-up program synthesis through learning-guided exploration. In *International Conference on Learning Representations (ICLR)*, 2021. URL `https://arxiv.org/abs/2007.14381`. arXiv:2007.14381.

[128] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://arxiv.org/abs/2306.09896`. arXiv:2306.09896.

[129] Laurent Orseau, Levi H. S. Lelis, Tor Lattimore, and Théophane Weber. Single-agent policy tree search with guarantees. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL `https://arxiv.org/abs/1811.10928`.

[130] Peter-Michael Osera and Steve Zdancewic. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 619–630, 2015. doi: 10.1145/2737924.2738007.

[131] Saswat Padhi, Todd Millstein, Aditya Nori, and Rahul Sharma. Overfitting in synthesis: Theory and practice. In *Computer Aided Verification (CAV 2019)*, volume 11561 of *Lecture Notes in Computer Science*, pages 315–334. Springer, 2019. doi: 10.1007/978-3-030-25540-4_17.

[132] Eric Pang. [ARC-AGI-2 SoTA] efficient evolutionary program synthesis. Blog post, 16 September 2025, `https://ctpang.substack.com/p/arc-agi-2-sota-efficient-evolutionary`, September 2025. Code and README: `https://github.com/epang080516/arc_agi`; a DreamCoder-inspired, LLM-assisted program synthesis system; 26.0% on ARC-AGI-2, semi-private set.

[133] Samuel Pastva, David Šafránek, Nikola Beneš, Luboš Brim, and Thomas Henzinger. Repository of logically consistent real-world Boolean network models, 2023. bioRxiv preprint; Biodivine Boolean Models, https://github.com/sybila/biodivine-boolean-models.

[134] Fernando Pereira and Yves Schabes. Inside-outside reestimation from partially bracketed corpora. In *30th Annual Meeting of the Association for Computational Linguistics*, pages 128–135, 1992. doi: 10.3115/981967.981984. URL https://aclanthology.org/P92-1017/.

[135] Yin Jun Phua and Katsumi Inoue. Variable assignment invariant neural networks for learning logic programs. In *Neural-Symbolic Learning and Reasoning (NeSy 2024)*, volume 14979 of *Lecture Notes in Computer Science*, pages 47–61. Springer, 2024. doi: 10.1007/978-3-031-71167-1_3. arXiv:2408.10709.

[136] Gordon D. Plotkin. A note on inductive generalization. In B. Meltzer and D. Michie, editors, *Machine Intelligence 5*, pages 153–163. Edinburgh University Press, 1970. URL https://homepages.inf.ed.ac.uk/gdp/publications/MI5_note_ind_gen.pdf.

[137] Oleksandr Polozov and Sumit Gulwani. FlashMeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 107–126, 2015. doi: 10.1145/2814270.2814310.

[138] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 49659–49688, 2025. URL https://proceedings.mlr.press/v267/pourcel25a.html. arXiv:2507.14172.

[139] Qwen Team. Qwen3-4B-Instruct-2507, model card, 2025. URL https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507. Apache 2.0; revision cdbee75 used here.

[140] Joseph M. Replogle, Reuben A. Saunders, Angela N. Pogson, Jeffrey A. Hussmann, Alexander Lenail, Alina Guna, Lauren Mascibroda, Eric J. Wagner, Karen Adelman, Gila Lithwick-Yanai, Nika Iremadze, Florian Oberstrass, Doron Lipson, Jessica L. Bonnar, Marco Jost, Thomas M. Norman, and Jonathan S. Weissman. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185:2559–2575.e28, 2022. doi: 10.1016/j.cell.2022.05.013.

[141] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. doi: 10.1016/0005-1098(78)90005-5.

[142] Ronald L. Rivest. Learning decision lists. *Machine Learning*, 2, 1987. doi: 10.1007/BF00058680.

[143] Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. doi: 10.1016/0004-3702(91)90015-C.

[144] Julio Saez-Rodriguez, Leonidas G. Alexopoulos, Jonathan Epperlein, Regina Samaga, Douglas A. Lauffenburger, Steffen Klamt, and Peter K. Sorger. Discrete logic modelling as a means to link protein signalling networks with functional analysis of mammalian signal transduction. *Molecular Systems Biology*, 5(1):331, 2009. doi: 10.1038/msb.2009.87.

[145] Uwe Schmidt, Martin Weigert, Coleman Broaddus, and Gene Myers. Cell detection with star-convex polygons. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*, volume 11071 of *Lecture Notes in Computer Science*, pages 265–273. Springer, 2018. doi: 10.1007/978-3-030-00934-2_30. arXiv:1806.03535.

[146] Rocco A. Servedio. The probably approximately correct learning model in computational learning theory, 2025.

[147] H. Sebastian Seung, Manfred Opper, and Haim Sompolinsky. Query by committee. In *Proceedings of the Fifth Annual ACM Workshop on Computational Learning Theory*, pages 287–294. ACM, 1992. doi: 10.1145/130385.130417. URL https://collaborate.princeton.edu/en/publications/query-by-committee/.

[148] C. E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi: 10.1002/j.1538-7305.1949.tb03624.x.

[149] Kevin M. Shebek, Jonathan Strutz, Linda J. Broadbelt, and Keith E. J. Tyo. Pickaxe: a Python library for the prediction of novel metabolic reactions. *BMC Bioinformatics*, 24(1):106, 2023. doi: 10.1186/s12859-023-05149-8.

[150] Kensen Shi, Hanjun Dai, Kevin Ellis, and Charles Sutton. CrossBeam: Learning to search in bottom-up program synthesis. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2203.10452.

[151] Kensen Shi, Hanjun Dai, Wen-Ding Li, Kevin Ellis, and Charles Sutton. LambdaBeam: Neural program search with higher-order functions and lambdas. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2306.02049. arXiv:2306.02049.

[152] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? Overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 532–543, 2015. doi: 10.1145/2786805.2786825.

[153] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013. doi: 10.1007/s10009-012-0249-7.

[154] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[155] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025, 2025. URL https://github.com/1ytic/NVARC.

[156] Aarohi Srivastava et al. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. URL https://openreview.net/forum?id=uyTL5Bvosj. arXiv:2206.04615; list functions task: https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/list_functions.

[157] Irene Stahl. The appropriateness of predicate invention as bias shift operation in ILP. *Machine Learning*, 20 (1–2):95–117, 1995. doi: 10.1007/BF00993476.

[158] Andreas Stolcke and Stephen Omohundro. Inducing probabilistic grammars by Bayesian model merging. In *Grammatical Inference and Applications (ICGI-94)*, volume 862 of *Lecture Notes in Computer Science*, pages 106–118. Springer, 1994. doi: 10.1007/3-540-58473-0_141. arXiv:cmp-lg/9409010.

[159] Ajay Subbaroyan, Olivier C. Martin, and Areejit Samal. Minimum complexity drives regulatory logic in Boolean models of living systems. *PNAS Nexus*, 1(1):pgac017, 2022. doi: 10.1093/pnasnexus/pgac017.

[160] Paul E. Utgoff. Shift of bias for inductive concept learning. In Ryszard S. Michalski, Jaime G. Carbonell, and Tom M. Mitchell, editors, *Machine Learning: An Artificial Intelligence Approach*, volume II, pages 107–148. Morgan Kaufmann, Los Altos, CA, 1986.

[161] Sahar Vahdati, Andrei Aioanei, Haridhra Suresh, and Jens Lehmann. The ARC of progress towards AGI: A living survey of abstraction and reasoning, 2026. URL https://arxiv.org/abs/2603.13372.

[162] Ruchika Verma, Neeraj Kumar, Abhijeet Patil, et al. MoNuSAC2020: A multi-organ nuclei segmentation and classification challenge. *IEEE Transactions on Medical Imaging*, 40(12):3413–3423, 2021. doi: 10.1109/TMI.2021.3085712.

[163] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model, 2025. URL https://arxiv.org/abs/2506.21734.

[164] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

[165] Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, Heng Liao, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/85dca46374dc0f27b4bb5f265b3d17f0-Paper-Conference.pdf. arXiv:2310.00656.

[166] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired Open-Ended Trailblazer (POET): Endlessly generating increasingly complex and diverse learning environments and their solutions. arXiv:1901.01753, 2019. URL https://arxiv.org/abs/1901.01753.

[167] Ruocheng Wang, Eric Zelikman, Gabriel Poesia, Yewen Pu, Nick Haber, and Noah D. Goodman. Hypothesis search: Inductive reasoning with language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.05660.

[168] Xinyu Wang, Isil Dillig, and Rishabh Singh. Program synthesis using abstraction refinement. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–30, 2018. doi: 10.1145/3158151.

[169] Johan S. Wind. DSL solution to the ARC challenge. https://github.com/top-quarks/ARC-solution, 2020. First place, Kaggle Abstraction and Reasoning Challenge 2020.

[170] Yudong Xu, Elias B. Khalil, and Scott Sanner. Graphs, constraints, and search for the Abstraction and Reasoning Corpus. In *AAAI Conference on Artificial Intelligence*, 2023. URL https://arxiv.org/abs/2210.09880.

[171] Yudong Xu, Wenhao Li, Pashootan Vaezipoor, Scott Sanner, and Elias B. Khalil. LLMs and the Abstraction and Reasoning Corpus: Successes, failures, and the importance of object-based representations. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2305.18354. Introduces 1D-ARC; data: https://github.com/khalil-research/1D-ARC.

[172] Yongho Yoon, Woosuk Lee, and Kwangkeun Yi. Inductive program synthesis via iterative forward-backward abstract interpretation. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1657–1681, 2023. doi: 10.1145/3591288.

[173] Kunbo Zhang, Lei Fu, Zeyu Wang, Zijing Liu, and Kejian Tong. ARCANA: A reflective multi-agent program synthesis framework for ARC-AGI-2 reasoning. In *2026 11th International Symposium on Advances in Electrical, Electronics and Computer Engineering (ISAEECE)*, pages 890–896. IEEE, 2026. doi: 10.1109/ISAEECE70502.2026.11635197. arXiv:2607.09059.